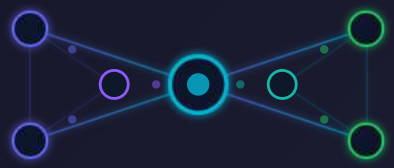
System Design Questions

INTERVIEWERS ACTUALLY ASK



Real World System Design Problems.
The patterns that solve them.

@theskilledcoder

Sold to
kumar.dodda@gmail.com

Why This Book Exists

I still remember the exact moment I realized I didn't understand system design.

It wasn't during a production outage. It wasn't while debugging a cascading failure at 3 AM. It was in a quiet conference room, whiteboard marker in hand, in what I thought was going well.

I had prepared for weeks. Read the blogs. Watched the videos. I could draw load balancers, explain CAP theorem, and debate SQL vs NoSQL with confidence.

And for the first 20 minutes, everything went perfectly.

Then came the follow-up question that changed everything.

✦ ✦ ✦

"Design a 'Pay Now' button for an e-commerce checkout."

I was amazed. A button? I'd built dozens of these.

```
Click "Pay" → API Call → Charge Card → Success. Easy.
```

The interviewer nodded. Then she asked:

> "Scenario: User's on a spotty 3G connection. They tap 'Pay Now'. Nothing happens. Just a spinner. Frustrated, they tap it three more times.
>
> **Did you just charge them 4 times?**"

Easy - I disable the button after the first click.

She continued:

Sold to
kumar.dodda@gmail.com

> "They rage-refresh the page. The button lights up again.
> Or maybe they open a new tab to check if it went through. Or their phone auto-retries the POST request.
>
> **Now what?**"

I started talking about database locks. She cut me off:

> "The request times out. Your server thinks it failed and shows 'Please Try Again'.
> But Stripe *did* get the money - it just took 31 seconds.
>
> User sees the error. Pays again. **Boom. Double charge.**
> How do you make this impossible?"

I stared at my whiteboard. My beautiful boxes and arrows had nothing to say about this.

✦ ✦ ✦

That's the moment I realized:

The question wasn't about payments.
It was about distributed systems, idempotency, and exactly-once semantics -
hiding inside something that looked like a simple button.

> The scariest system design problems don't look scary.
> They look like features you've built a hundred times.
> Until someone asks the follow-up question.

This book is about those follow-ups.

What Most Resources Get Wrong

Here's the thing about system design resources:

Sold to
kumar.dodda@gmail.com

Most of them show you the *final answer*. The perfect architecture diagram. Load balancer, microservices, cache, database. Clean arrows. Labeled boxes.

And you think: *"Great, I'll memorize this."*

But when you're in an interview - or worse, in a production incident - you realize you memorized **what** to build, not **why** it works.

You don't know which trade-offs matter. You don't know what breaks first. You don't know why the "obvious" solution fails at scale - because nobody showed you the obvious solution failing.

> 💡 **This book is different.**
>
> Every chapter starts with a story. A feature that seems simple. A solution that seems obvious. **And then we break it.**

How This Book Actually Works

Each of the 35 chapters follows a specific pattern:

The Situation

A real-world scenario you can picture. Taylor Swift tickets going live. A celebrity posting to 50 million followers. Two users editing the same Google Doc.

The First Attempt (That Fails)

The solution most people think of first - and exactly why it breaks at scale. You'll see the failure mode clearly, not just be told "it doesn't work."

The "Aha" Insight

The key realization that unlocks the correct design. The thing that separates senior engineers from everyone else - explained so it clicks.

The Architecture

The actual design - with diagrams, code snippets, and explanations of what each component does and *why it's there*.

The Interview Answer

Exactly how to explain this in an interview - structured, clear, and confident. Plus the follow-up questions interviewers love to ask.

Sold to
[redacted]@gmail.com

By the end of each chapter, you won't just know the architecture - **you'll understand why every other approach fails.**

What You'll Actually Learn

35 problems. Each one teaches a different pattern:

- 🔔 **Notification systems** - fan-out, push vs pull, handling celebrity users
- 📊 **Real-time analytics** - streaming data, approximate counting, time windows
- 💳 **Payment processing** - idempotency, exactly-once, distributed transactions
- 🎟️ **Booking systems** - race conditions, optimistic vs pessimistic locking, hold patterns
- 🗄️ **Caching** - invalidation strategies, thundering herd, cache-aside vs write-through
- ⚡ **Rate limiting** - token bucket, sliding window, distributed rate limits
- 🔍 **Search** - autocomplete, geospatial queries, relevance ranking
- 📹 **Video/file uploads** - chunking, resumable uploads, processing pipelines
- 💬 **Real-time collaboration** - CRDTs, operational transforms, presence
- 📈 **Scaling patterns** - sharding, replication, connection pooling, backpressure

These aren't theoretical exercises. They're the same problems engineers solve at Netflix, Uber, Stripe, Discord, and every company that's survived their first scaling crisis.

What You'll Walk Away With

After working through these 35 problems, you'll be able to:

- Stop guessing. Start designing with intent.
- Spot the invisible bottlenecks before you write a line of code.
- Survive the interview pressure - and the production spikes.
- Turn "it depends" into "here are the trade-offs."

Sold to
kumar.dodda@gmail.com

Table of Contents

Introduction

01 How would you design a system to send 100,000 notifications?

02 How do you log 1M events/second without slowing the app?

03 How would you compute trending posts every minute?

04 How to handle 10,000 concurrent users?

05 How would you design search autocomplete for 100M queries/day?

06 How do you handle 10,000 concurrent database connections?

07 How do you ensure exactly-once payment processing?

08 How would you design a recommendation engine for millions of users?

09 How would you design a key-value store for 100M reads/second?

10 How do you migrate 10TB of data with zero downtime?

11 How do you build a live streaming platform for millions of viewers?

12 How would you design a real-time leaderboard for 100K players?

13 How would you design a URL shortener for 500M redirects/day?

Sold to
kumar.dodda@gmail.com

14 How do you aggregate logs from 10,000 servers without data loss?

15 How would you design a tagging system for 100 million posts?

16 How would you store and display edit history for millions of posts?

17 How would you design Instagram Stories that expire after 24 hours?

18 How would you design a ride-sharing matching system like Uber?

19 How would you design a load balancer for 1M requests/second?

20 How would you design a real-time collaborative editor like Google Docs?

21 How would you design delayed scheduling for 200K precise reminders?

22 How would you design a news feed for 1 billion users?

23 How would you design a web crawler for 1 billion pages?

24 How would you build a rate limiter for 1M requests/second?

25 How would you design file sync for 100M files across devices?

26 How would you design a ticket booking system for 100K concurrent users?

27 How would you design proximity search for 100M points of interest?

28 How would you design a video upload pipeline for 500 hours/minute?

29 How would you design a distributed cache for 1M hits/second?

30 How would you design a fraud detection system for 10K transactions/second?

Sold to
kumar.dodda@gmail.com

31 How do you design systems knowing requirements will change?

32 How do you design systems where data must disappear everywhere?

33 How do you debug a bug that only appears at peak traffic?

34 How do you design systems where correctness is optional but cost is not?

35 How would you design a stock trading system?

🎁 BONUS CONTENT

🎯 10 Questions to Ask Before Designing Any System

⚡ Backend Tech Concepts - One-Liner Explanations

🔥 25 Concepts You'll Only Learn After a Year in Production

📊 System Design Patterns - Quick Reference

35 chapters + Bonus Content covering notifications, caching, search, real-time systems, payments, architecture patterns, and more.

Sold to
kumar.dodda@gmail.com

How would you design a system to send 100,000 notifications?

Designing a Notification System That Actually Scales

Every modern product - whether it's a banking app, a ride-sharing platform, or a social network - depends on one invisible yet powerful system: **notifications**.

They're the tiny nudges that keep your product alive and your users informed.

- A push when your cab is 2 minutes away
- An SMS with your OTP
- An email confirming your order

Seems simple enough, right? Just send a message.

But here's where the trap lies -

> ⚠️ **The Scale Trap**
>
> At small scale, everything feels easy. At scale, everything breaks.

For a prototype, you can literally loop over all users and call your provider's API. Works fine for a few hundred users. But when you need to send **100,000+ messages**, "just send" quickly turns into "why is nothing working?"

Let's unpack why.

Why This Problem Is Deceptively Hard

Imagine a banking app where 100,000 transactions fail due to a temporary outage. You need to alert all affected users - and fast.

What looks like a simple "send notifications" job now becomes a **distributed system problem**.

You're suddenly fighting on multiple fronts:

Sold to
@theskilledc
kumar.dodda@gmail.com

- **Throughput** - Can you send thousands per second, not per hour?
- **Reliability** - What if FCM or SMTP is slow or returns 5xx errors?
- **Idempotency** - If your service crashes mid-way, can you retry safely without duplicates?
- **Scalability** - Will it still hold if you jump from 100k → 1M tomorrow?
- **Observability** - How do you know what succeeded, failed, or got delayed?

These five are the **pillars** every resilient notification system must solve.

⚡ The Engineering Complexity

At 100 notifications, you're calling an API. At 100,000, you're fighting physics:

🔥 Throughput Pressure

100K notifications in minutes. That's 1,600+ messages per second through external APIs that rate-limit you.

🔄 Retry Complexity

FCM returns 500? Retry. But how many times? How long to wait? What if you're already retrying 10K others?

📋 Delivery Guarantees

At-least-once or exactly-once? Duplicates annoy users. Missed notifications lose revenue.

📊 Multi-Channel Chaos

Push, SMS, Email - each has different latency, cost, and failure modes. Orchestrating them is hard.

First Attempts (and Why They Fail)

Approach 1: Loop and Send Directly

Idea: Just iterate through all users and send via Email/SMS/Push provider.

```python
for user in users:
    sendNotification(user)
```

Why it feels right:

Simple. Easy to code. Works perfectly in dev.

Why it fails:

Sold to @theskilledc kumar.dodda@gmail.com

- Each call might take ~300ms
- 100k × 300ms = 30,000 seconds → **8+ hours!**

You can try threading, but you'll hit:

- Provider rate limits
- Connection errors
- Duplicate sends on retries

If your process crashes mid-way, you've no clue who got what.

> 💡 **Lesson**
>
> Code simplicity ≠ system simplicity. Throughput and fault tolerance don't come free.

Approach 2: Use Database as a Queue

Idea: Insert all notifications in a table:

```sql
INSERT INTO notifications (user_id, status)
VALUES (user_id, 'PENDING')
```

A worker picks up `PENDING`, sends it, then marks `SENT`.

Why it feels right:
SQL is reliable. Transactions give confidence. You can `count(*)` to see progress.

Why it fails:
At 100k+, your DB becomes the bottleneck:

- Hot rows & lock contention
- Slow vacuuming
- Replication lag

DBs are great at **state**, not streaming events. You're forcing it to do something it wasn't built for.

> 💡 **Lesson**
>
> Don't turn your source of truth into a message queue.

Approach 3: Cron Job Batches

Sold to
@theskilledc
kumar.dodda@gmail.com

Idea: Run a job every minute that sends pending messages.

Why it feels right:

Scheduled jobs = easy mental model. "Send everything due now."

Why it fails:

- At the top of the minute → a burst of 100k requests
- Providers respond with 429 (rate-limit) or block your IP
- Your "real-time" notification now has up to 59s latency

> 💡 **Lesson**
>
> Batching helps organization, not elasticity.

The Scalable Approach

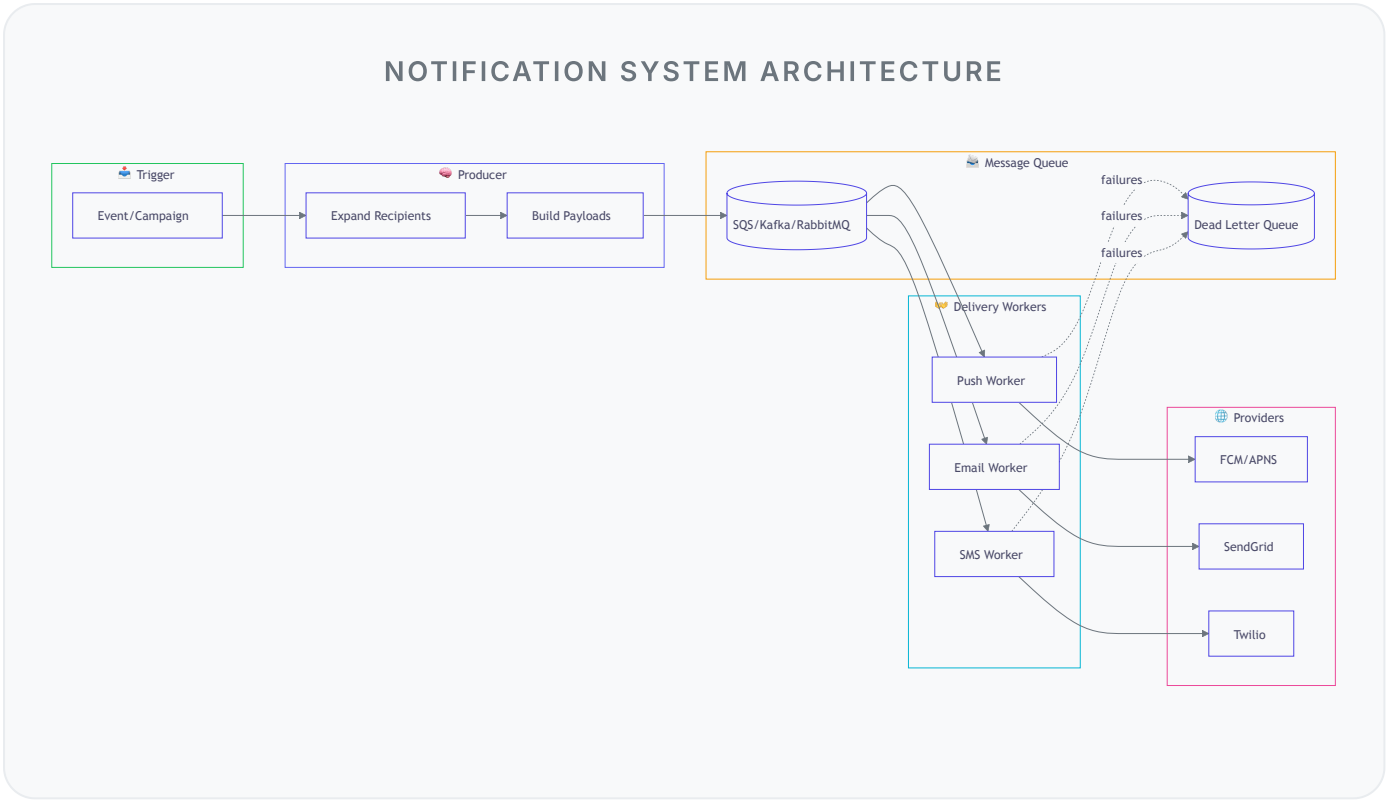
To scale gracefully, separate concerns:

> **"Deciding what to send" ≠ "Actually sending it"**

Think of it as an assembly line:

1. A **producer** decides what needs sending
2. A **queue** manages the flow
3. **Workers** deliver at safe speed
4. **Metrics** keep you informed

Sold to
@theskilledc
kumar.dodda@gmail.com



Let's break it down.

1. Producer Service (Brain)

- Figures out *who* to notify (recipients)
- Prepares personalized payloads (Hello, Tom 👋)
- Publishes one message per user into a queue

It's **stateless**: you can run multiple producers in parallel. It doesn't care *when* it's delivered - just that it's enqueued.

2. Queue or Stream (Buffer)

Use something like **SQS**, **RabbitMQ**, **Kafka**, or **Pub/Sub**.

Why?

- **Absorbs bursts** - workers can pull at their own pace
- **Handles retries** gracefully
- Offers **DLQ** (Dead Letter Queue) for poison messages
- Enables **backpressure** - if providers slow down, queue length grows, not your CPU usage

Think of it as a *shock absorber* in your system.

3. Delivery Workers (Hands)

Each channel (push, email, SMS) has its own workers.

Sold to
@theskilledc
kumar.dodda@gmail.com

They:

- Pull messages at a **controlled rate**
- Enforce per-provider rate limits (avoid bans)
- Use **idempotency keys** (`campaign_id + user_id + channel`)
- Retry transient failures (e.g. network blips) with **exponential backoff**
- Send unrecoverable ones to DLQ for manual retry/inspection

Parallelize with more workers. They can scale **horizontally**.

4. Observability (Eyes)

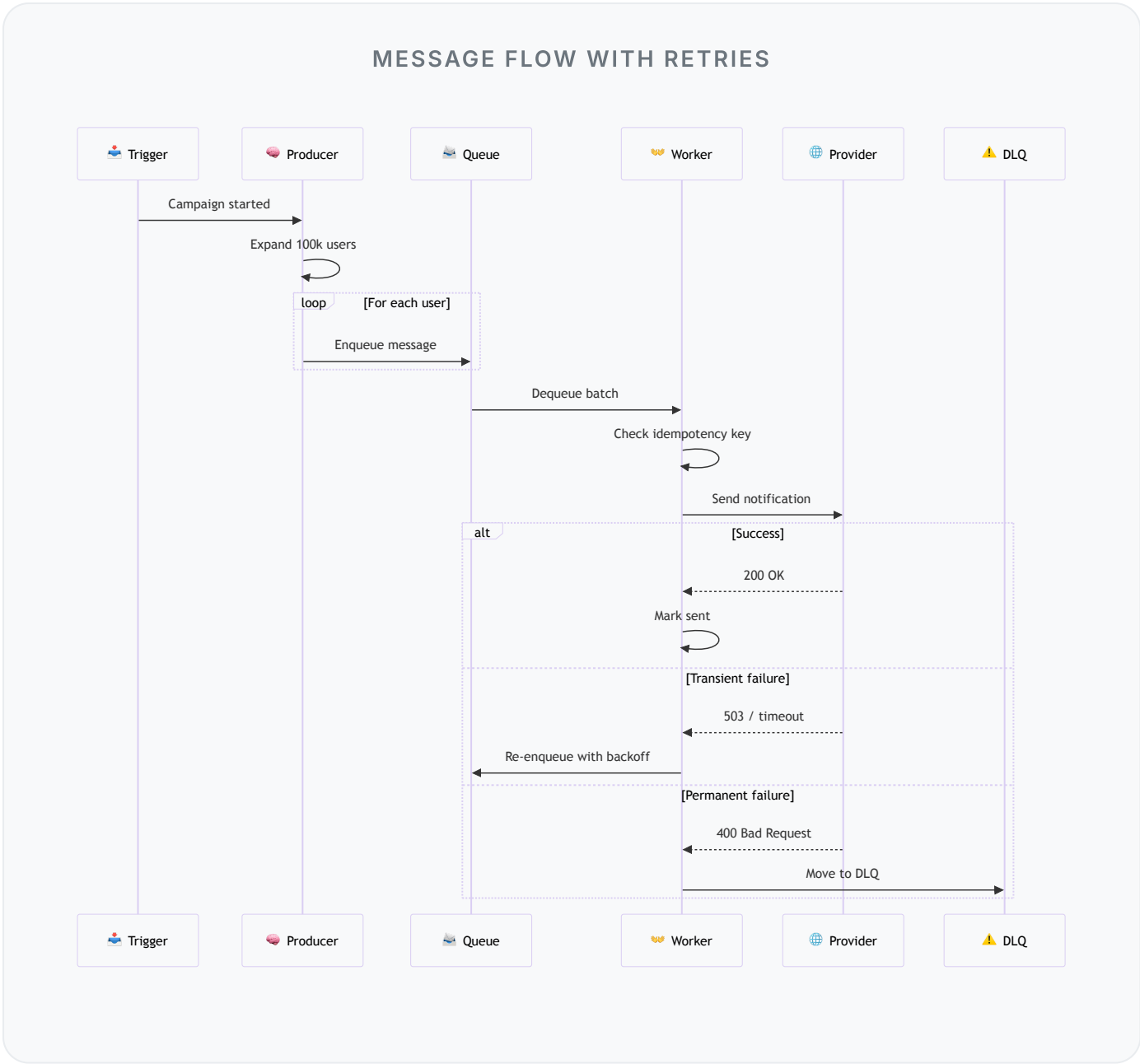
You can't fix what you can't see.

Track:

- **Metrics:** enqueue/sent/failed/sec, retries, p95 latency
- **Logs:** idempotency key + provider response codes
- **Alerts:** spikes in DLQ, latency breaches, 5xx rates

Use tools like **Prometheus + Grafana** or **Datadog**. Even a simple dashboard helps detect silent failures early.

Sold to
@theskilledc
kumar.dodda@gmail.com



Why This Architecture Works

ASPECT	HOW IT'S SOLVED
Throughput	50 workers × 35 req/s = ~1,750 req/s → 100k in ~1 minute
Reliability	Queue buffers bursts, retries are idempotent
Scalability	Add workers; partition queues, scale independently
Correctness	Idempotency prevents duplicates, DLQ captures bad events
Visibility	Metrics & alerts show real-time health

This is first-principles engineering:

- Decouple producers & consumers

Sold to
@theskilledc
kumar.dodda@gmail.com

- Isolate failures
- Scale horizontally
- Design for retries & observability

Interview Pivot (How to Explain It)

If asked in an interview:

"How would you design a system to send 100,000 notifications?"

You say:

> "Naively, I'd loop through all users and call the provider directly - but that couples expansion and delivery, and breaks with retries or rate limits.
>
> Instead, I'd split concerns:
>
> - A **producer** expands recipients and enqueues messages
> - **Workers** consume from the queue at a safe pace, enforce per-provider rate limits, use idempotency keys to prevent duplicates, and push failed messages to a DLQ
>
> That gives me high throughput, safety, and visibility - 100k notifications in minutes, and scalable to millions."

That's not just an answer - that's a **design mindset**.

What's Next?

We've laid the foundation - a robust, scalable notification system. But there's more to explore if you really want to think like an architect:

How do we ensure each notification is sent exactly once? ?

> Use **idempotency keys** (e.g., `campaign_id + user_id + channel`) stored in a fast lookup like Redis. Before sending, check if the key exists. If yes, skip. If no, send and mark it. This turns "at-least-once" into "exactly-once" semantics.

Sold to
@theskilledc
kumar.dodda@gmail.com

How do we handle sudden spikes, like 1 million events per second?

> **Partition your queues** by user segment, region, or channel. Use auto-scaling worker pools that spin up based on queue depth. Add a **rate limiter** at ingestion to smooth out bursts and protect downstream providers from getting overwhelmed.

What if iOS consumers stop working and fail to receive notifications?

> Implement **channel-specific DLQs** and health checks. If APNS returns persistent failures, route to a fallback channel (email/SMS). Track delivery receipts where possible, and alert when success rates drop below a threshold.

How do we alert engineers automatically when something breaks?

> Set up **metric-based alerts**: DLQ size > threshold, error rate > 5%, latency p99 > SLA. Use PagerDuty or Opsgenie for on-call routing. Add **runbooks** linked to each alert so engineers know exactly what to check first.

Here's the thing about notifications: users rarely notice when they work perfectly - but they *always* notice when they don't. A missed OTP means a failed login. A delayed alert means money lost. The systems you build here aren't just engineering puzzles - they're the invisible trust layer between your product and your users.

Build it right, and no one will know you exist. That's the goal.

Sold to
@theskilledc
kumar.dodda@gmail.com

Same Principles Apply To

- **Email marketing systems** - same queue-based delivery with rate limiting
- **Webhook delivery services** - same retry logic with exponential backoff
- **Alert management systems (PagerDuty)** - same multi-channel routing
- **Order status updates (e-commerce)** - same event-driven notification flow

Sold to
@theskilledc
kumar.dodda@gmail.com

How to Log 1M Events/Sec Without Slowing Down Your System

Why Logging Isn't as Simple as It Looks

Every modern system - whether it's a fintech app processing payments, an ad-tech platform tracking impressions, or a multiplayer game updating player actions - relies on one silent backbone:

Logs

They're the breadcrumbs of truth.

- A debit failure in banking
- A player teleporting in a game
- A "Buy Now" button click in e-commerce
- A leaderboard refresh in a fantasy sports app

Individually, they look tiny. Collectively, they become **millions per second**.

The issue is -

> ⚠️ **The Logging Paradox**
>
> Logging feels trivial when traffic is small. At scale, logging becomes one of the most expensive operations in your system.

Because logging is not "just write a line."

It is **I/O + storage + latency + durability + retention + indexing + retrieval + analytics**.

And once your traffic crosses a certain threshold, the naïve approach falls apart.

Let's break down why logging at massive scale is hard, and how to do it right.

Sold to
@theskilledc
kumar.dodda@gmail.com

⚡ The Engineering Complexity

Logging feels like printf debugging until you're drowning in a million lines per second:

📝 Volume vs Speed

1M logs/sec × 1KB = 1GB/sec. Your logging pipeline can't slow down the actual request.

💾 Storage Explosion

Logs are append-only. 1 week of data = 600TB. Where does it go? How long do you keep it?

🔍 Needle in Haystack

Finding one error in billions of logs. Full-text search on petabytes isn't trivial.

⏱️ Latency Budget

Logging is a side-effect. It gets zero tolerance for adding latency to your main path.

First Attempt: Log Directly Inside App Code

Most beginners start here:

```java
log.info("User " + userId + " clicked Buy Now");
```

Why it feels right:

- Easy to write
- Works in dev
- Shows logs instantly

Why it fails at scale:

- Logging is **blocking IO**
- Every write competes with CPU cycles meant for business logic
- If logs go to disk → disk bandwidth becomes bottleneck
- If logs go to external system → network latency kills throughput
- If logs are synchronous → one slow write pauses the entire request

At 1M events/sec, this model collapses.

Sold to @theskilledc kumar.dodda@gmail.com

Even a 1ms logging cost means:

```
1,000,000 events × 1ms = 1,000,000ms = 1000 seconds latency → impossible.
```
MATH

> 💡 **Lesson**
>
> Direct synchronous logging couples application performance with logging performance - and that's dangerous.

Attempt 2: Batch Logs and Write Periodically

Okay, so developers try batching:

```python
buffer.append(event)
if buffer.size ⩾ 1000:
    writeToDisk(buffer)
    buffer.clear()
```
PYTHON

Why it feels right:

- Fewer writes = fewer syscalls
- Better throughput
- Common in logging libraries like Logback, Bunyan, Serilog etc.

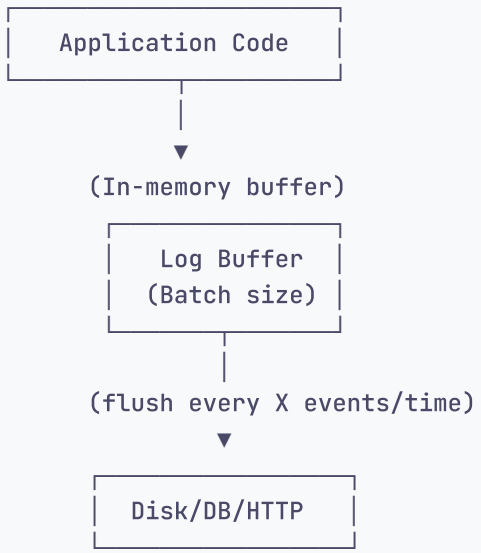
Why it still breaks:

- If your process crashes → buffered logs vanish
- If bursts exceed buffer capacity → logs drop
- Writing periodically still requires locking, causing contention across threads
- If storage slows down, buffers back up → memory explodes

At 1M/sec, even buffering isn't enough.

> 💡 **Lesson**
>
> Batching helps, but doesn't decouple logging from request handling.

Sold to
@theskilledc
kumar.dodda@gmail.com



BATCHED LOGGING FLOW

Attempt 3: Log to Database

Idea: store logs in SQL or NoSQL since it's durable and queryable.

Sounds nice. Until things break:

- DB inserts become bottleneck
- Locks + indexes kill write throughput
- Querying logs affects real app data
- Storage cost skyrockets
- Retention becomes a nightmare

A typical database can handle:

DATABASE	WRITE THROUGHPUT
MySQL	~10k writes/sec (with tuning)
MongoDB	50k–200k writes/sec (clustered)
Cassandra	High, but expensive at scale

1M/sec destroys it - financially and operationally.

Sold to
@theskilledc
kumar.dodda@gmail.com

> 💡 **Lesson**
>
> Databases are for structured data, not firehose-scale telemetry.

Attempt 4: Send Logs Directly to Elasticsearch / OpenSearch

This is where many companies fail.

Elasticsearch gives:

- Searchability
- Visual dashboards
- Indexing
- Alerting

Feels perfect. Until **indexing overhead hits**.

Indexing every event means:

- Tokenization
- Sharding
- Replication
- Compression
- Persistence

Even a large ES/OpenSearch cluster struggles beyond **200k–300k events/sec** without:

- Massive hardware
- Lots of tuning
- Big maintenance overhead

> ⚠️ **Lesson**
>
> Elasticsearch/OpenSearch is great for *querying* logs - not ingesting raw firehose scale directly from apps.

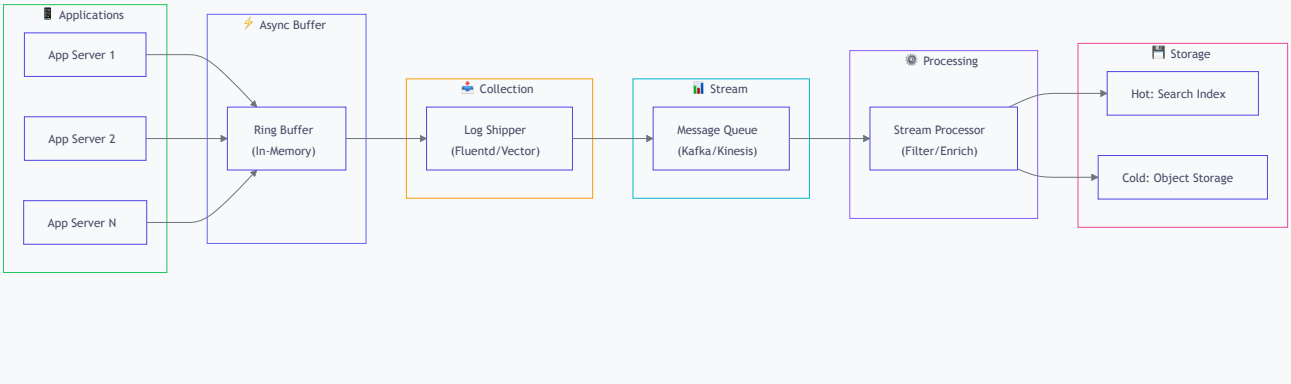
The Scalable Architecture

To log 1M+ events/sec without hurting latency, you need one principle:

Sold to @theskilledc kumar.dodda@gmail.com

Logging must be asynchronous, streamed, and decoupled from the application.

Here's the architecture that works:



1. Application Layer: Non-Blocking Logging

- Use async log appenders
- Logs go to in-memory ring buffers
- Logging never blocks request path

Libraries:

LANGUAGE	LIBRARY
Java	Logback AsyncAppender, Disruptor logging
Go	Zerolog
Node	Pino
Rust	Tracing layer

No waiting. No flushing. No disk writes on main thread.

2. Queue or Stream: The Shock Absorber

This absorbs bursts and smooths throughput.

Use:

- **Kafka**
- **Pulsar**

Sold to
@theskilledc
kumar.dodda@gmail.com

- Kinesis
- Redpanda
- Google Pub/Sub

Benefits:

- Producers write at **memory speed**
- Consumers scale independently
- Backpressure is handled automatically
- Replay and retention available

Kafka alone can handle millions/sec with proper partitions.

3. Storage Layer: Hot + Warm + Cold

Different logs need different storage.

TIER	PURPOSE	STORAGE	RETENTION
🔥 Hot	Real-time search & alerts	ClickHouse, Elasticsearch	7–30 days
🌤️ Warm	Analytics, ad-hoc queries	S3 + Parquet, Athena/Spark	30–90 days
❄️ Cold	Compliance, long retention	Glacier, Tape	Years

You don't index everything - you promote only what's useful.

4. Observability: Metrics Over Logs (When Possible)

Not every event needs a log line.

Sometimes **counters, histograms, and traces** are better.

Use:

- Prometheus
- OpenTelemetry
- Grafana Tempo

The mindset:

> **"Log what matters. Aggregate what repeats."**

Sold to
@theskilledc
kumar.dodda@gmail.com

Why This Architecture Works

This architecture works because **logging no longer stands in the way of real work**.

The app writes once and moves on - no waiting, no blocking. As traffic grows, you don't fight the system, you just add more consumers and partitions.

Storage becomes smart instead of expensive: fast where needed, cheap everywhere else.

Even if parts fail, the system keeps absorbing events without dropping them. In the end, logs flow like a river - not a traffic jam.

Interview Pivot (How to Explain It)

If asked:

"How would you log 1M events/sec?"

You respond:

> "I wouldn't log directly. I'd log asynchronously, push events into an in-memory buffer, stream them through Kafka, and then store them in tiers: hot for recent searchable data, warm for analytics, and cold for long retention.
>
> That way logging never slows requests, stays durable, and scales without rewriting the system."

What's Next?

We've covered the architecture. But there's more depth to explore:

How to avoid log duplication? ?

> Use **idempotent consumers** with message deduplication at the stream level. Kafka supports exactly-once semantics with transaction IDs. Alternatively, add unique event IDs and dedupe at the storage layer before indexing.

Sold to
@theskilledc
kumar.dodda@gmail.com

How to detect ingestion lag / remove backpressure?

> Monitor **consumer lag metrics** (Kafka consumer group lag). Set up alerts when lag exceeds thresholds. Auto-scale consumers based on partition depth. Use Kafka's pause/resume API to apply graceful backpressure.

What if the Kafka cluster goes down?

> Design for **local disk fallback** - if Kafka is unreachable, buffer logs locally and replay when connection restores. Use multi-region Kafka replication (MirrorMaker 2) for critical logs.

How to avoid logging sensitive/PII data?

> Implement **log scrubbing** at the appender level - mask or hash fields like emails, IPs, tokens before they leave the app. Use allow-lists for what CAN be logged rather than deny-lists.

Here's the irony of logging: the better your system runs, the less you look at logs. But when things go wrong at 3 AM, those logs are the difference between a 5-minute fix and a 5-hour outage. Build the pipeline right, and logs become your silent guardian - always there when you need them, never in the way when you don't.

The best logging system is one you forget exists.

Sold to
@theskilledc
kumar.dodda@gmail.com

Same Principles Apply To

- **Metrics collection (Prometheus, Datadog)** - same async, buffered ingestion
- **Clickstream analytics (Google Analytics)** - same tiered storage architecture
- **Audit trail systems** - same append-only, high-throughput design
- **IoT telemetry ingestion** - same Kafka-based streaming pipeline

Sold to
@theskilledc
kumar.dodda@gmail.com

How Would You Compute Trending Posts Every Minute

Computing "Trending Posts" Is Harder Than It Looks

Every social platform has it. A **Trending** section.

The posts everyone seems to be talking about *right now* - not yesterday, not last week, but this minute.

At first glance, it sounds easy. Just count likes and sort, right?

That illusion lasts until traffic grows, events start arriving out of order, and one viral post suddenly floods your system. What felt like a simple ranking problem quickly turns into a **real-time data challenge**.

⚡ The Engineering Complexity

'Just count likes and sort' works until a Beyoncé post breaks your whole pipeline:

⏰ Time Decay

A post from 5 minutes ago vs 5 hours ago. Same likes, different relevance. Recency matters exponentially.

📊 Counting at Scale

Millions of likes/minute. You can't recount everything every minute. Incremental updates are tricky.

🎯 Definition Problem

'Trending' isn't just 'most likes.' It's velocity of engagement. That's a derivative, not a count.

⚡ Real-Time Expectation

Users expect 'trending' to change in real-time. Batch jobs every hour won't cut it.

Sold to
@theskilledc
kumar.dodda@gmail.com

📋 **In This Chapter**

- The solutions that look correct but break at scale
- Why "trending" is not a batch problem
- How real systems compute it continuously using streaming data

We will solve it with first-principles thinking.

First Attempts (and Why They Break)

Attempt 1: Recompute Everything Every Minute

The most obvious idea is this:

1. Every minute, scan all recent likes, comments, shares, and views
2. Group them by post
3. Sort by count
4. Pick the top 10

Simple SQL or Spark job. Done.

Why it feels right:

- Easy to reason about
- Works fine when you have 10k posts
- "Trending every minute" sounds like a batch problem

Why it fails:

- You're reprocessing the same data again and again
- At scale, each minute means scanning millions of events
- Latency creeps up → your "every minute" job takes longer than a minute
- Spikes (viral posts) make jobs miss their SLA

What looked like a clean batch job quietly turns into a runaway compute bill.

💡 **Lesson**

Recomputing from scratch doesn't scale with velocity.

Sold to
@theskilledc
kumar.dodda@gmail.com

Attempt 2: Maintain Counters in the Database

Okay, let's be smarter.

Whenever a user likes or comments on a post: **Increment a counter in the posts table**. Use that counter to decide what's trending.

Why it feels right:

- Constant-time updates
- Data is always "fresh"
- Easy queries like `ORDER BY score DESC`

Why it fails:

- High write contention on hot posts
- Viral posts become bottlenecks
- Locks, retries, and replication lag start showing up
- Your primary DB is now handling analytics traffic

Suddenly, your transactional database is doing real-time ranking. That never ends well.

> 💡 **Lesson**
>
> Databases are great at state, not high-frequency event aggregation.

Attempt 3: Cache Trending Posts in Redis

Fine. Move fast stuff to Redis.

- Increment counters in Redis
- Every minute, read top keys
- Redis is fast, right?

Why it feels right:

- In-memory speed
- No DB locks
- Widely used pattern

What's the problem:

- Redis becomes a single hot spot

Sold to
@theskilledc
kumar.dodda@gmail.com

- You lose ordering guarantees under heavy concurrency
- Crash or restart = counters gone (unless heavily persisted)
- Cross-region or multi-shard ranking gets messy

Redis helps performance, but it doesn't solve time-windowed aggregation cleanly.

> 💡 **Lesson**
>
> Speed alone doesn't solve correctness at scale.

Attempt 4: Trigger Jobs with Cron

Another classic move:

- Run a cron job every minute
- Compute trending posts for the last 60 seconds
- Publish the result

Why it feels right:

- Simple mental model
- Clear time boundaries
- Easy to debug

What's the issue:

- All computation spikes at the same second
- Late events get dropped or miscounted
- Real-time systems don't respect clock boundaries
- Users see inconsistent "trending" lists

Trending is a **flowing signal**, not a punctual snapshot.

> 💡 **Lesson**
>
> Time-based cron jobs don't mix well with real-time streams.

Attempt 5: "Just Use Machine Learning"

Someone always suggests this. "Why not train a model to predict trending posts?"

Sold to @theskilledc kumar.dodda@gmail.com

Why it feels right:

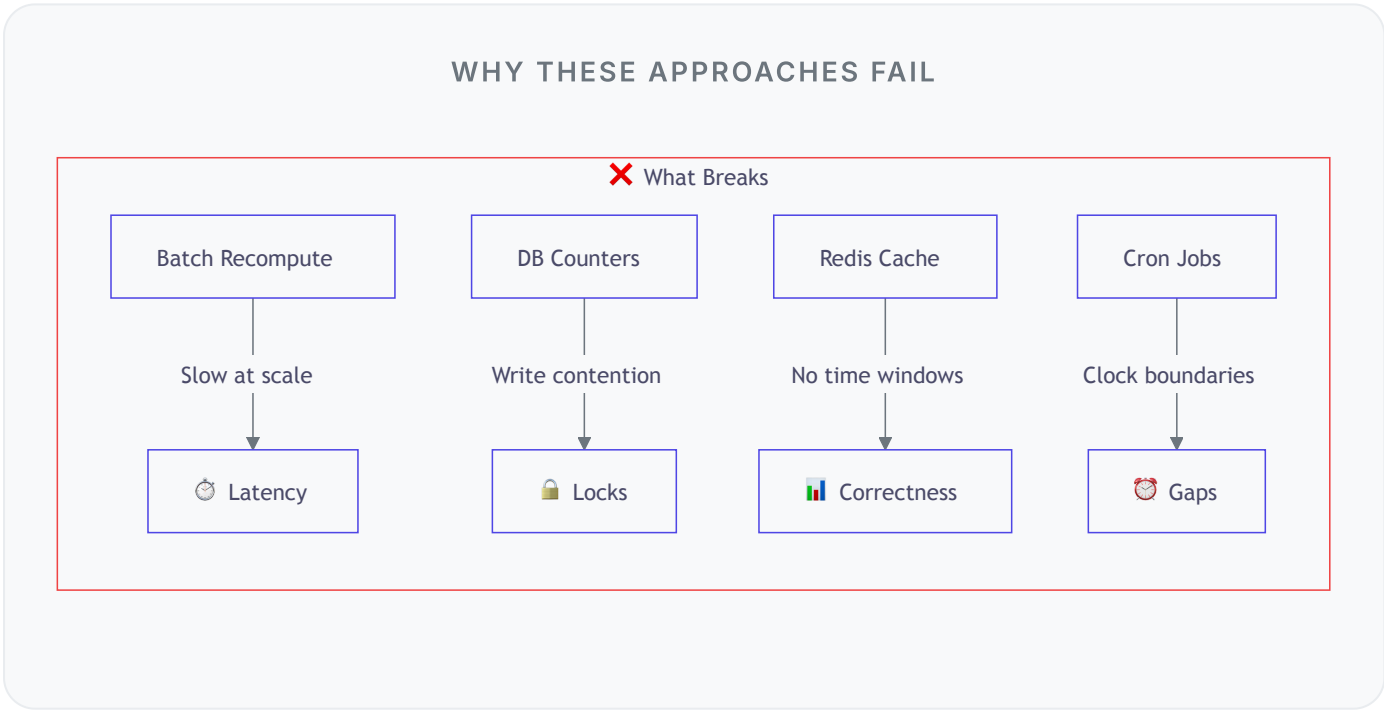
- Sounds sophisticated
- Great for long-term ranking

Why it fails (for this problem):

- ML still needs clean, real-time features
- You still haven't solved streaming aggregation
- Latency requirements make inference tricky
- Overkill for minute-level trends

ML improves ranking. It doesn't replace real-time computation.

> ⚠️ **Lesson**
>
> Intelligence doesn't fix broken pipelines.



The Pattern Emerging

All these approaches fail for the same reason:

They treat trending as a static calculation.
In reality, trending is a moving window over a live stream of events.

Sold to
@theskilledc
kumar.dodda@gmail.com

To solve it properly, we need to stop thinking in **rows** and start thinking in **flows**.

The Right Way to Think About Trending

Once you step back, the mistake in all the earlier attempts becomes obvious.

> 🎯 **Trending is not a number.** It's a signal changing over time.
>
> A post doesn't become trending because it crossed a fixed count. It becomes trending because activity around it is *rising faster than others right now*.

That immediately tells us two things:

1. We must process events **as they happen**
2. We must reason in **sliding time windows**, not batches or cron jobs

This is a **streaming problem**, not a database one.

The Mental Model Shift

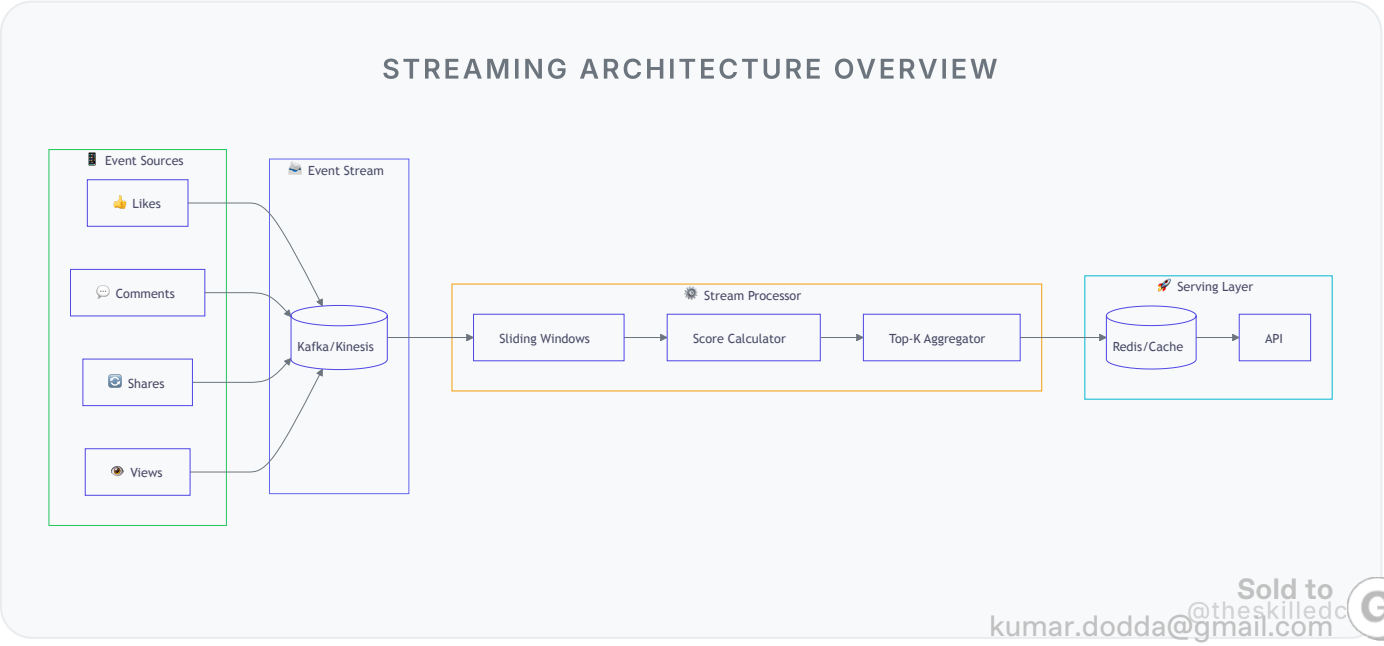
Instead of asking:

"Every minute, what are the top posts?"

We ask:

"As events flow in, how does each post's momentum change?"

Once you adopt this mindset, the architecture almost designs itself.



Sold to
@theskilledc
kumar.dodda@gmail.com

The Architecture: Step by Step

Step 1: Treat Every Interaction as an Event

Likes, comments, shares, views - they are all signals. Each interaction becomes a small event:

```json
{
  "post_id": "abc123",
  "event_type": "like",
  "timestamp": 1702999200
}
```

Nothing is aggregated yet. Nothing is ranked yet. We simply emit events and move on.

This keeps the write path extremely fast and lets the system absorb sudden spikes when something goes viral.

Step 2: Stream, Don't Store First

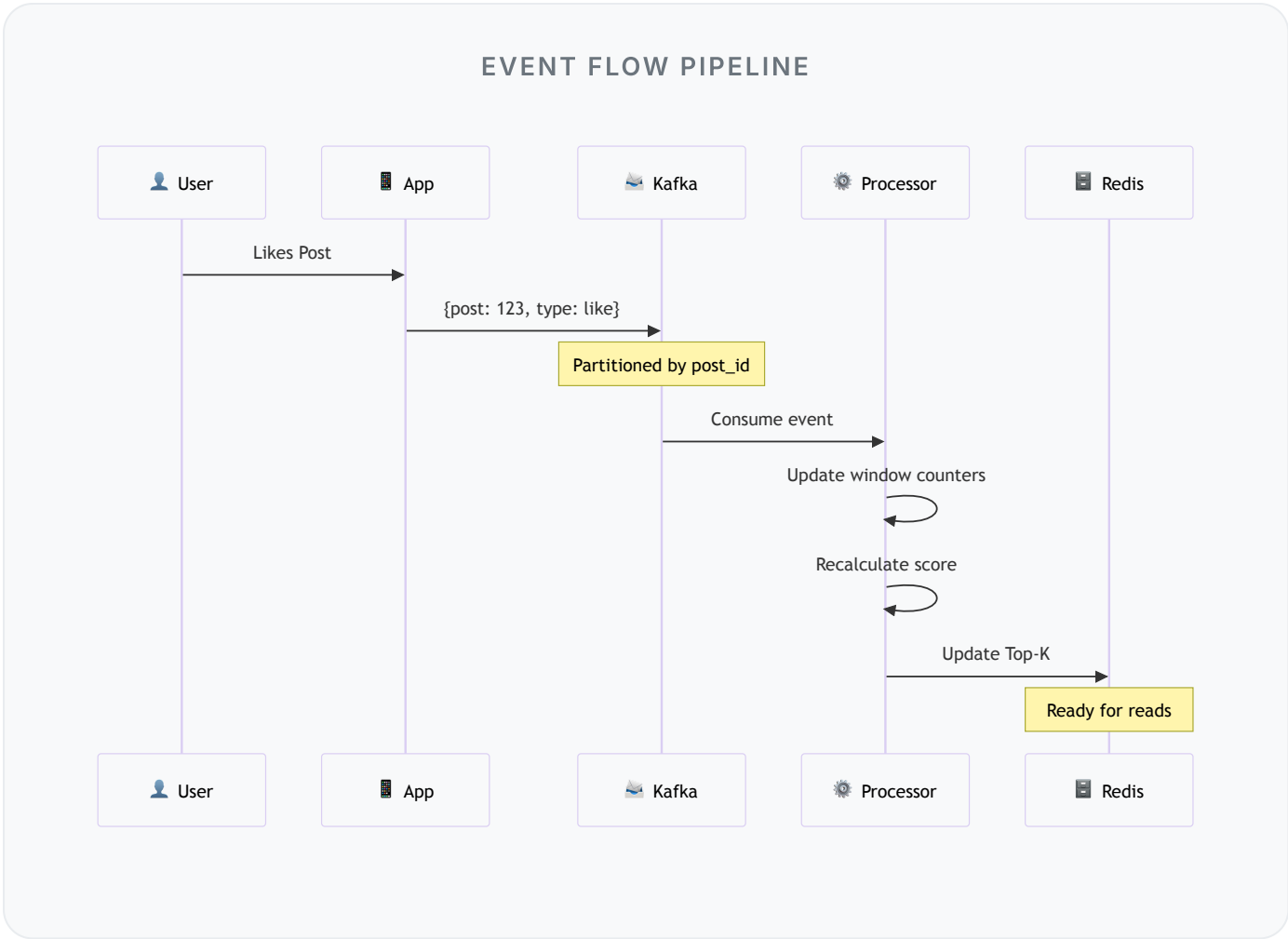
These events flow into a stream - **Kafka**, **Pulsar**, **Kinesis**, **Pub/Sub**.

Why a stream? Because streams:

- Preserve order within partitions
- Handle burst traffic naturally
- Allow multiple consumers to compute different views

At this point, the system isn't "computing trending" yet. It's just **recording reality as it unfolds**.

Sold to
@theskilledc
kumar.dodda@gmail.com



Step 3: Compute Trends Using Sliding Windows

Now comes the core idea.

Instead of recomputing everything every minute, we maintain **rolling windows**:

- Last 1 minute
- Last 5 minutes
- Last 15 minutes

As events flow in, stream processors continuously update counters per post per window.

This is not batch. This is incremental math.

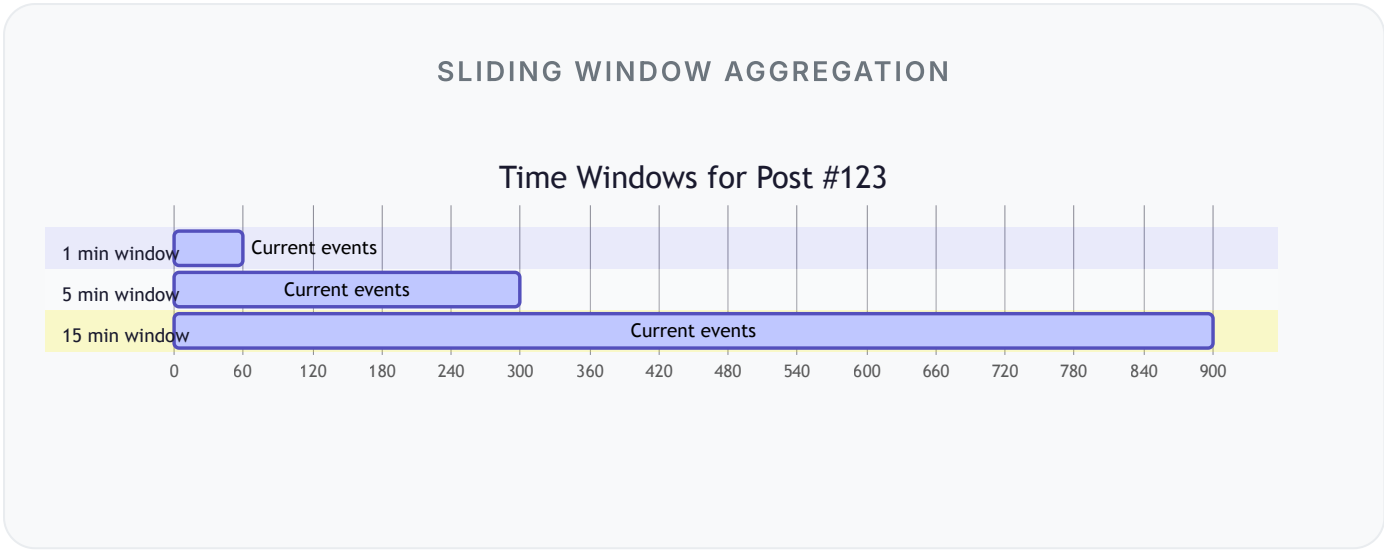
When a new like arrives:

1. Add +1 to that post's current window
2. Expire events that fall out of the window
3. Update the score instantly

No rescans. No cron jobs. No waiting.

Trending becomes a continuously updated result.

Sold to
@theskilledc
kumar.dodda@gmail.com



Step 4: Separate Scoring from Ranking

Raw counts alone don't tell the full story.

A post with 10 likes in 1 minute might be more interesting than one with 1,000 likes over a day.

So instead of just counting, we compute a **score**:

- Weight recent events higher than older ones
- Give comments more weight than likes
- Normalize by post age

This scoring logic lives inside the stream processor.

The output is simple:

```json
{
  "post_id": "abc123",
  "trend_score": 847.5,
  "window": "5min"
}
```

The heavy thinking happens once, in the stream - not on every read.

Step 5: Maintain a Live Top-K Structure

From these scores, we maintain a **Top-K list** (say top 100 posts per region or category).

This structure updates incrementally:

- When a score changes, we adjust its position
- No full sorting required
- No global locks

Sold to
@theskilledc
kumar.dodda@gmail.com

The result is always ready.

When the UI asks for "trending posts", it's a simple read - not a computation.

Step 6: Serve Reads Without Touching the Stream

The final trending list is pushed to a fast store:

- Redis
- DynamoDB
- In-memory cache

Reads are cheap. Writes are controlled. The stream keeps flowing independently.

If the UI goes down, streaming continues. If the stream lags, the UI still serves the last known good state.

Failures don't cascade.

Why This Works (At Scale)

This architecture works because **nothing waits**.

- Events flow in once
- Computation happens incrementally
- Ranking updates continuously
- Reads never trigger heavy work

You're no longer fighting time or traffic spikes.

Trending stops being a periodic job and becomes a living signal.

Interview Pivot (How to Explain It)

If someone asks:

"How would you compute trending posts every minute?"

> "I wouldn't recompute every minute. I'd treat user interactions as a stream, aggregate them in sliding windows, compute a rolling trend score per post, and maintain a live top-K list.
>
> The UI simply reads the latest result, while the stream updates continuously in the background."

Sold to
@theskilledc
kumar.dodda@gmail.com

That answer shows you understand **data flow**, not just data storage.

What's Next?

Trending isn't something you calculate on a schedule. It's something you **observe as it emerges**.

This design opens the door to deeper questions:

How do you handle late or out-of-order events?

> Use **watermarks** in stream processors (Flink, Spark Streaming). Define allowed lateness, buffer late events, and update windows retroactively. For very late events, decide whether to drop or recompute.

How do you avoid one viral post dominating forever?

> Apply **time decay** to scores - older events contribute less. Cap maximum score per window. Consider velocity (rate of change) instead of just absolute counts.

How do you scale this across regions?

> **Partition by region** at the stream level. Compute regional trending independently. For global trending, merge top-K lists from each region with weighted aggregation.

How do you debug wrong trending results?

> Log **intermediate scores** and window boundaries. Store event samples per post. Build dashboards showing score evolution over time. Compare expected vs actual rankings.

Sold to
@theskilledc
kumar.dodda@gmail.com

The key insight: Trending isn't about counting - it's about capturing momentum. The systems that get this right don't fight the stream of events. They ride it.

Think in flows, not snapshots. That's how real-time systems scale.

Same Principles Apply To

- **Trending hashtags (Twitter/X)** - same sliding window aggregation
- **Hot products (Amazon, eBay)** - same velocity-based ranking
- **Breaking news detection** - same spike detection algorithms
- **Viral content identification (TikTok)** - same time-decay scoring

Sold to
@theskilledc
kumar.dodda@gmail.com

How to Handle 10,000 Concurrent Users

Every system eventually hits this moment.

Traffic grows. Marketing works. A feature goes viral. Suddenly someone asks:

> **"Can this handle 10,000 concurrent users?"**

At first, it sounds like a **capacity question**.

In reality, it's a **systems thinking question**.

Let's start with the approaches that seem correct - and why they fail.

⚡ The Engineering Complexity

Adding servers is easy. The hard part is everything those servers share:

🔗 Connection Limits

10K users × 1 DB connection each = 10K connections. Most databases choke at 500.

🧵 Thread Exhaustion

Each request holds a thread. Slow downstream = threads pile up = new requests queue = timeout cascade.

📈 Non-Linear Scaling

2x users ≠ 2x resources. Lock contention, cache invalidation, and coordination overhead grow non-linearly.

⚡ Latency Amplification

At 10K concurrent, a 100ms call becomes 10K × 100ms of CPU time waiting on I/O.

Sold to
@theskilledc
kumar.dodda@gmail.com

First Attempts (and Why They Break)

Attempt 1: Just Add More Servers

The most common instinct. Traffic increases → add more app servers → problem solved.

Why it feels right:

- Horizontal scaling is industry standard
- Cloud makes it easy
- Works at low to medium traffic

Why it fails:

- Shared bottlenecks don't scale automatically
- Database, cache, auth service, third-party APIs all become choke points
- One slow downstream service can stall thousands of requests

You scaled the frontend, not the system.

> 💡 **Lesson**
>
> Concurrency pressure usually breaks dependencies, not CPU.

Attempt 2: Increase Thread Pool Size

Another classic fix. "Let's increase the thread pool from 200 to 2,000."

Why it feels right:

- More threads = more users handled
- Easy config change
- Looks like progress

Why it fails:

- Threads consume memory and context-switching time
- CPU spends more time scheduling than doing work
- Latency gets worse, not better
- JVM / runtime starts thrashing

Concurrency is not free just because threads exist.

Sold to @theskilledc
kumar.dodda@gmail.com

> 💡 **Lesson**
>
> More threads don't mean more throughput.

Attempt 3: Cache Everything

Okay, let's cache aggressively. Cache DB responses, API calls, user sessions.

Why it feels right:

- Reduces load
- Improves latency
- Caching works - until it doesn't

Why it fails:

- Cache stampedes under concurrent misses
- Hot keys get hammered
- Cache invalidation logic becomes fragile
- Memory pressure increases fast

Caching helps load, but it doesn't control concurrency.

> 💡 **Lesson**
>
> Cache reduces work; it doesn't manage access.

Attempt 4: Let Requests Queue Up

Another tempting idea: "If users come faster than we can handle, let them wait."

Why it feels right:

- No requests dropped
- Everything is eventually processed

Why it fails:

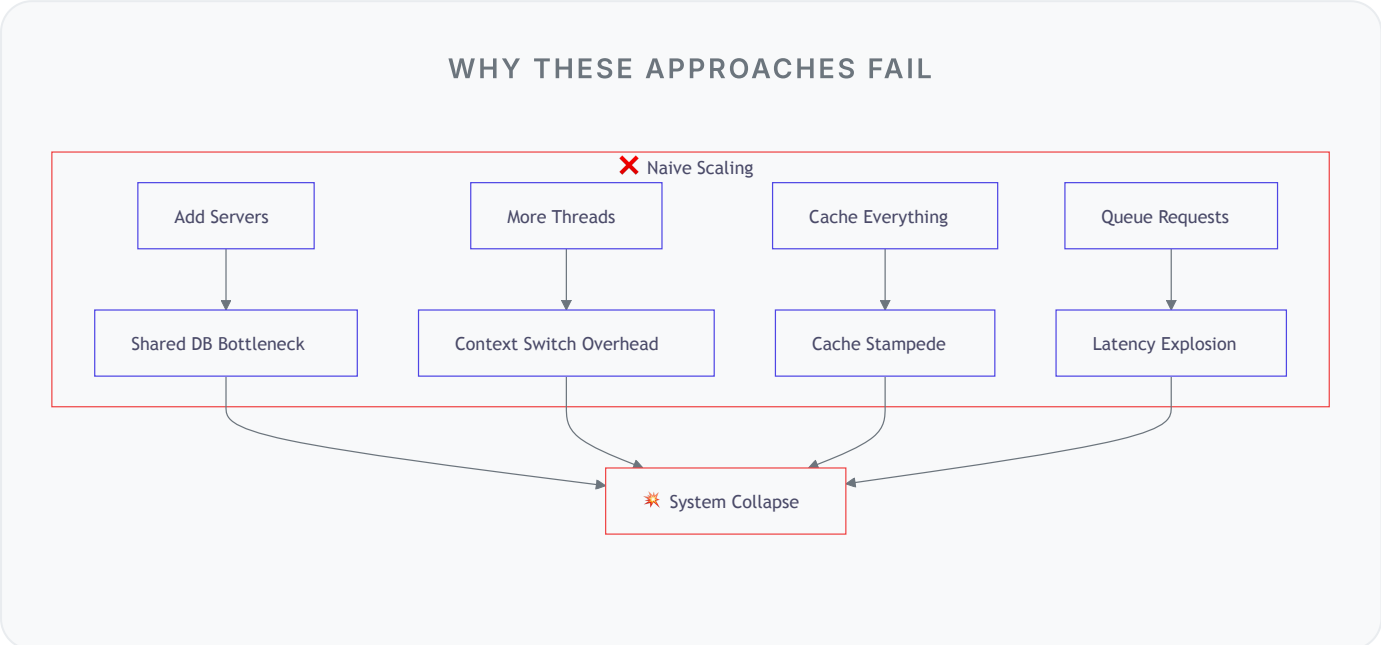
- Queues grow without bound
- Latency explodes
- Timeouts cascade across services

Sold to
@theskilledc
kumar.dodda@gmail.com

- Users retry → doubling traffic

Unbounded queues turn traffic spikes into outages.

> ⚠️ **Lesson**
>
> Waiting is not a strategy.



The Real Problem (Reframed)

10,000 concurrent users is not about *users*.

It's about **how many things your system can do at the same time without stepping on itself**.

The real questions are:

- Which operations are **blocking**?
- Which resources are **shared**?
- Which paths must be **fast**, and which can be **delayed**?

Once you answer those, the solution becomes clearer.

The Correct Architecture (Mindset First)

1. Make the Request Path Thin

The request thread should do **as little as possible**.

Sold to
@theskilledc
kumar.dodda@gmail.com

- Validate input
- Authorize
- Enqueue work
- Return response

Anything slow or non-critical moves out of band.

Concurrency survives when requests finish fast.

2. Control Concurrency Explicitly

Instead of letting everything run freely:

- Limit concurrent DB calls
- Limit outbound API calls
- Apply bulkheads per dependency

When limits are hit:

- Fail fast
- Return degraded responses
- Don't let pressure spread

This protects the system from itself.

3. Separate Immediate Work from Deferred Work

Not everything needs to happen now.

- Emails
- Notifications
- Analytics
- Logging
- Recommendations

Push these to queues and process asynchronously.

Users care about response time, not background perfection.

4. Scale State, Not Just Compute

Stateless app servers scale easily. State doesn't.

- Partition data

Sold to
@theskilledc
kumar.dodda@gmail.com

- Reduce shared locks
- Avoid global coordination
- Prefer per-user or per-shard isolation

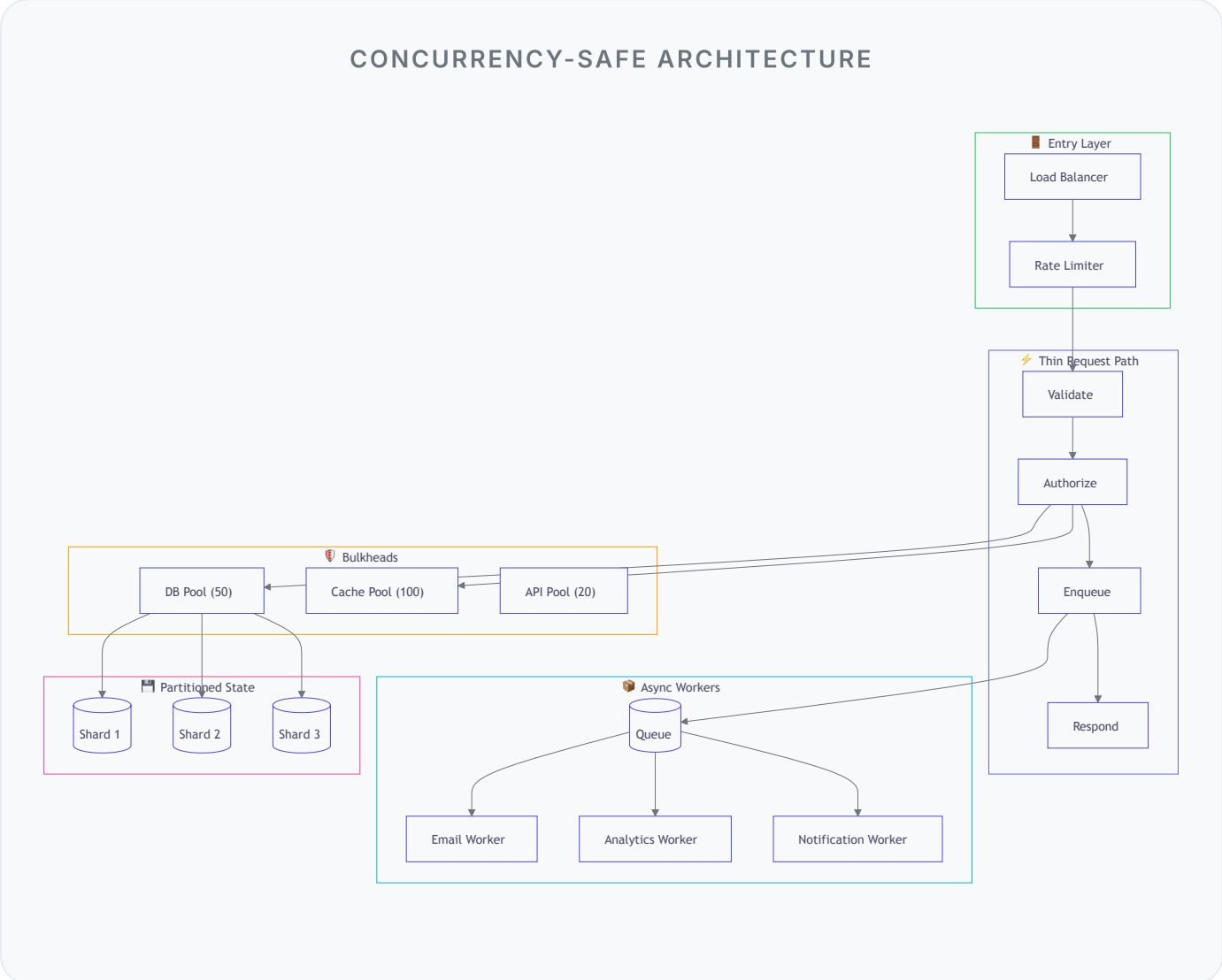
Concurrency collapses when everyone waits on the same thing.

5. Apply Backpressure Early

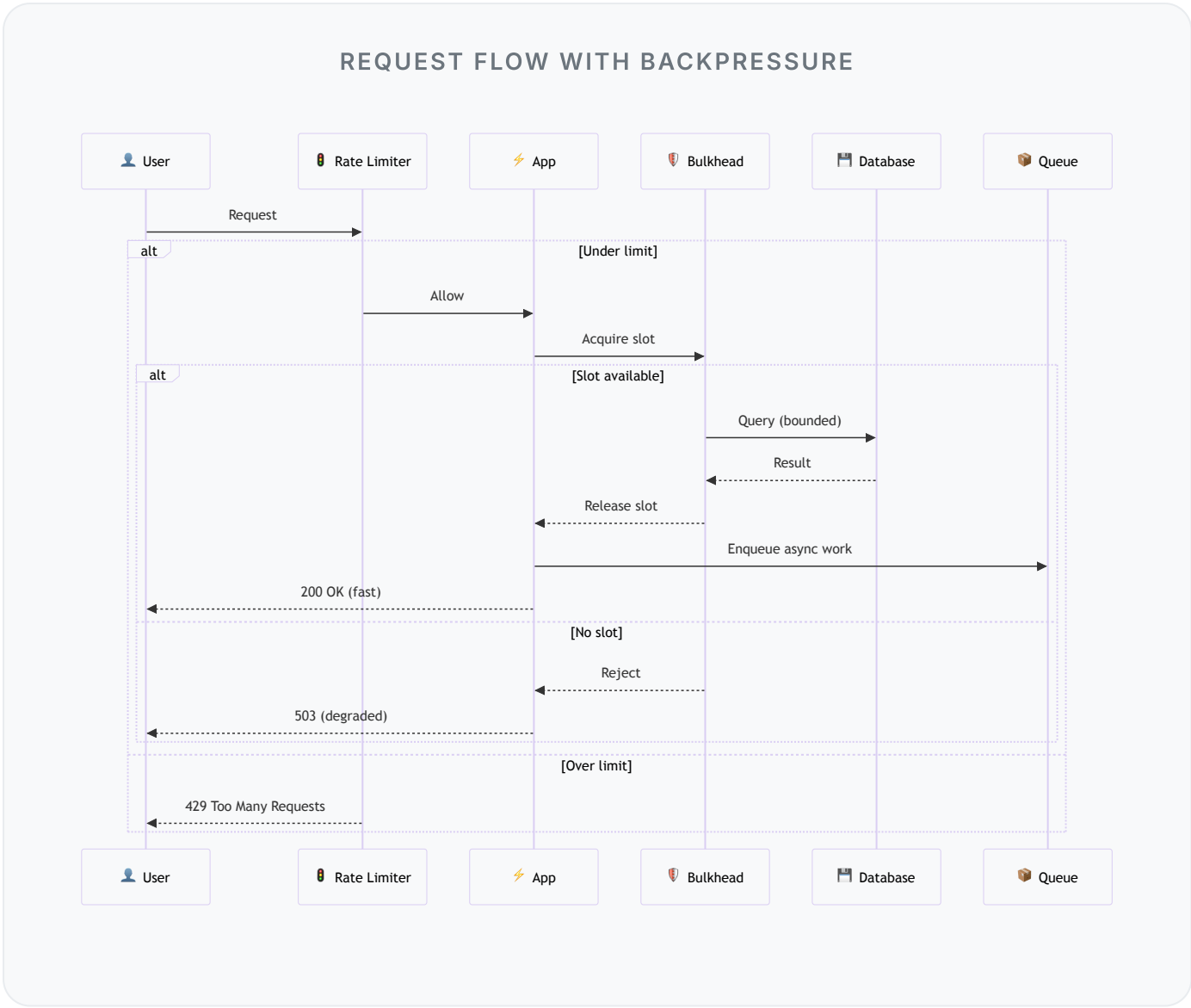
When the system is under stress:

- Reject requests early
- Return 429s
- Degrade features
- Shed load intentionally

A fast rejection is better than a slow failure.



Sold to
@theskilledc
kumar.dodda@gmail.com



Why This Works

This approach works because it accepts a hard truth:

> **You can't handle infinite concurrency - but you can control it.**

- Requests finish quickly
- Slow work happens elsewhere
- Failures stay contained
- Traffic spikes don't turn into outages

10,000 concurrent users stop being scary. They become just another number.

Interview Pivot (How to Explain It)

Sold to
@theskilledc
kumar.dodda@gmail.com

If asked:

"How would you handle 10,000 concurrent users?"

> "I'd keep the request path thin, control concurrency to shared dependencies with bulkheads, move slow work async, apply backpressure early, and scale state carefully.
>
> That way concurrency increases without collapsing latency."

That answer shows you understand **system behavior**, not just scaling slogans.

What's Next?

Real systems bring follow-ups:

What happens when one dependency slows down?

> Use **circuit breakers** to detect slow dependencies. When latency crosses thresholds, open the circuit and fail fast. Prevent cascading failures by isolating the slow path.

How do you prevent retry storms?

> Implement **exponential backoff with jitter**. Add client-side rate limiting. Use idempotency keys to safely handle retries. Cap maximum retry attempts.

How do you degrade gracefully?

> Define **feature tiers**: essential vs nice-to-have. When under pressure, disable recommendations, defer analytics, serve cached content. Keep core flows working.

Sold to
@theskilledc
kumar.dodda@gmail.com

How do you observe concurrency bottlenecks?

> Track **queue depths, pool utilization, p99 latency** per dependency. Set up dashboards showing concurrent connections vs capacity. Alert on saturation, not just errors.

The insight: Concurrency isn't about how many users you can accept - it's about how many operations you can complete without blocking each other.

Control the flow. Protect the bottlenecks. Let the rest scale.

Same Principles Apply To

- **Flash sales (Black Friday)** - same rate limiting and queue-based admission
- **Video conferencing (Zoom joins)** - same bulkhead isolation patterns
- **API rate limiting** - same backpressure and graceful degradation
- **Game server lobbies** - same thin request path with async processing

Sold to
@theskilledc
kumar.dodda@gmail.com

How Would You Design a Search Autocomplete for 100M Queries/Day

Autocomplete Looks Simple - Until You Build It

You've used it a thousand times. Type "how to" into Google, and before you finish - suggestions appear.

- how to tie a tie
- how to screenshot on mac
- how to lose weight

Instant. Relevant. Feels like magic.

But behind that simple dropdown is a system handling:

- **100M+ queries/day** - that's 1,000+ requests per second
- **Sub-100ms latency** - users expect instant feedback
- **Personalization** - different users, different suggestions
- **Real-time updates** - trending queries change by the hour

Let's break down why the obvious approaches fail - and what actually works.

⚡ The Engineering Complexity

Users type faster than you can query. Here's why milliseconds matter:

⌨️ **Keystroke Explosion**

User types 10 chars = 10 requests. 100M users = 1B+ autocomplete requests/day.

⏱️ **100ms Budget**

Suggestions must appear before user finishes typing. That's sub-100ms end-to-end.

Sold to @theskilledc kumar.dodda@gmail.com

🌐 Prefix Matching

'how to' matches millions of queries. You need the TOP 10, not all matches.

📈 Freshness vs Stability

Trending queries change hourly. But you can't rebuild your index every keystroke.

First Attempts (and Why They Break)

Attempt 1: Query the Database on Every Keystroke

The most intuitive approach:

```sql
SELECT query FROM search_history
WHERE query LIKE 'how to%'
ORDER BY frequency DESC
LIMIT 10;
```

Why it feels right:

- Simple to implement
- Leverages existing infrastructure
- SQL is familiar

Why it fails:

- `LIKE 'prefix%'` requires index scans
- At 1,000 QPS × 5 keystrokes = 5,000 DB queries/sec
- Database becomes the bottleneck
- Latency spikes under load

💡 **Lesson**

Databases aren't designed for prefix-matching at high concurrency.

Attempt 2: Cache Popular Queries

Okay, let's cache the top queries in Redis.

Sold to @theskilledc
kumar.dodda@gmail.com

```python
results = redis.get("autocomplete:how to")
if not results:
    results = db.query(...)
    redis.set("autocomplete:how to", results)
```

Why it feels right:

- Reduces DB load
- Faster reads
- Common pattern

Why it fails:

- Prefix space is **exponentially large**
- "a", "ab", "abc", "abcd"... millions of prefix combinations
- Cache miss rate is high for long-tail queries
- Memory explodes trying to cache everything

> 💡 **Lesson**
>
> You can't cache your way out of a prefix-matching problem.

Attempt 3: Full-Text Search (Elasticsearch)

"Let's use Elasticsearch - it's built for search!"

Why it feels right:

- Powerful query capabilities
- Handles large datasets
- Industry standard for search

Why it fails (for autocomplete):

- ES is optimized for **full-text search**, not prefix matching
- Prefix queries are expensive - they scan inverted indices
- Completion suggesters exist but have limitations
- Overhead for simple prefix lookups

Sold to
@theskilledc
kumar.dodda@gmail.com

⚠️ **Lesson**

Elasticsearch is great for search results - not ideal for real-time autocomplete.

Attempt 4: Compute Suggestions on the Client

"What if we send all popular queries to the client and filter locally?"

Why it feels right:

- Zero server load for filtering
- Instant local matching
- Works offline

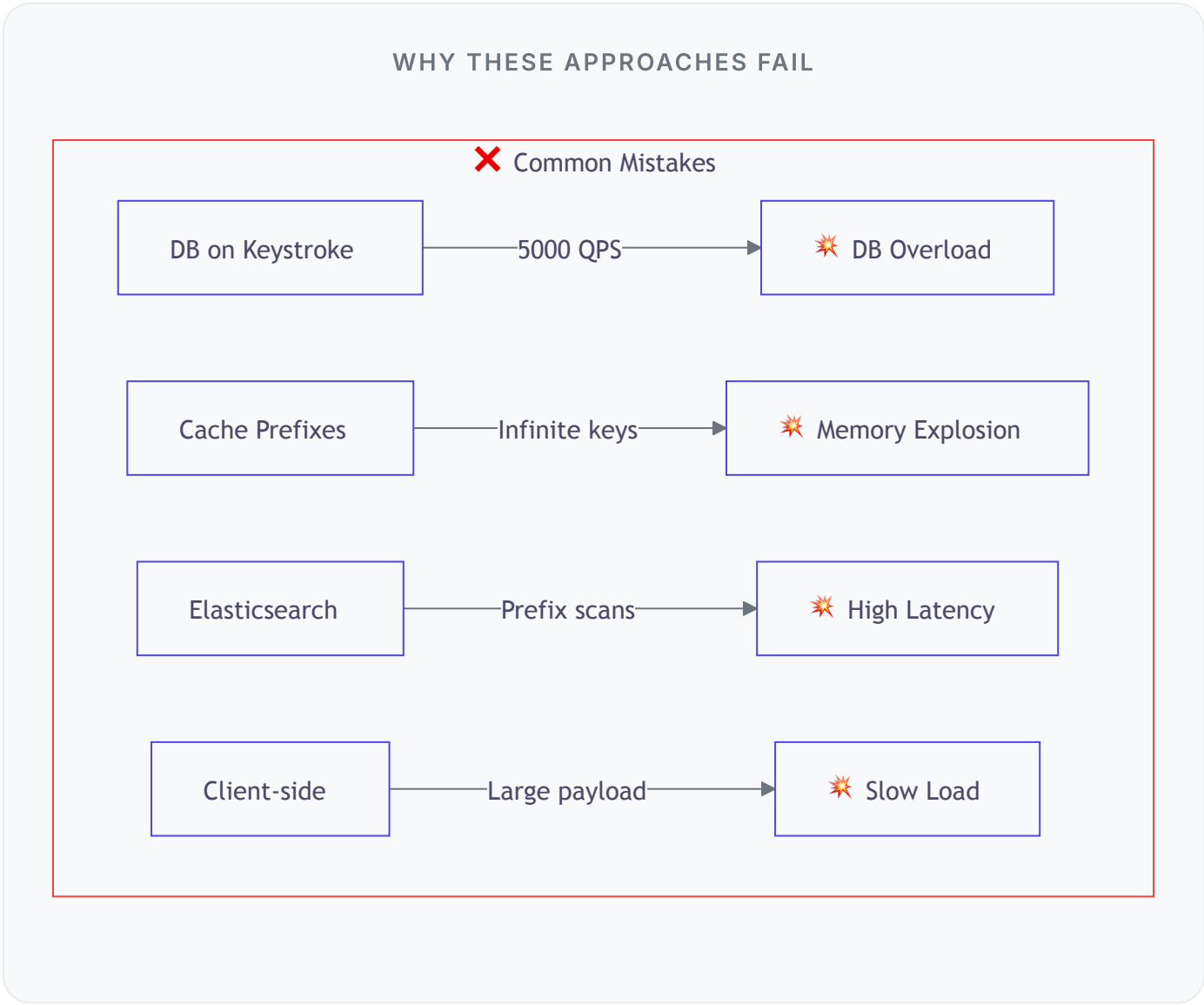
Why it fails:

- Millions of queries = massive payload
- Can't personalize per user
- Stale data until next sync
- Mobile users hate large downloads

💡 **Lesson**

Client-side filtering works for small datasets, not web-scale autocomplete.

Sold to
@theskilledc
kumar.dodda@gmail.com



The Right Data Structure: Tries

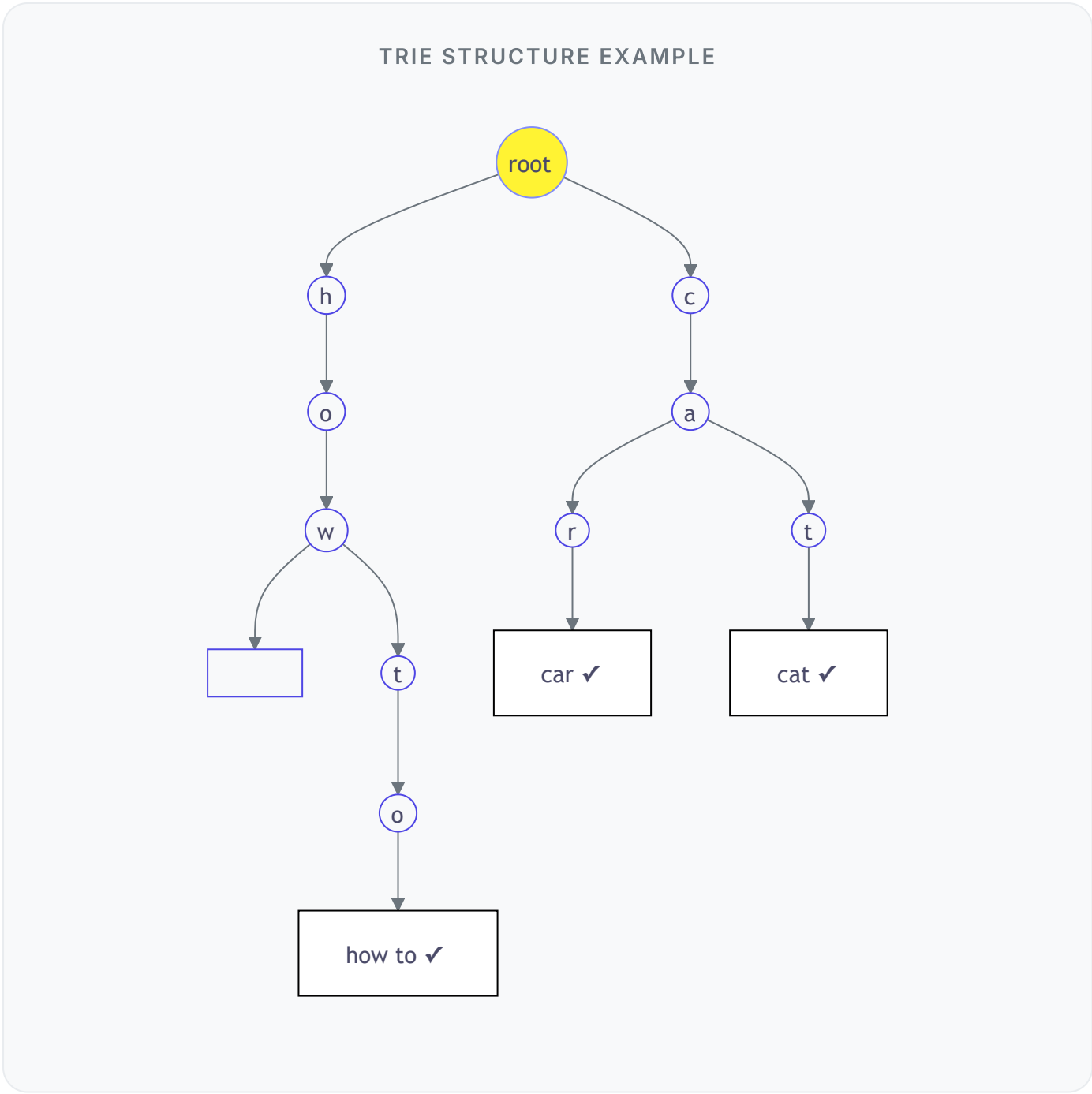
The key insight is this:

Autocomplete is a prefix-matching problem.
And the data structure built for prefix-matching is a Trie.

What is a Trie?

A Trie (prefix tree) stores strings character by character, with shared prefixes stored once.

Sold to
@theskilledc
kumar.dodda@gmail.com



With a Trie:

- **Lookup is O(prefix length)** - not O(total queries)
- **Common prefixes share nodes** - memory efficient
- **All completions are in the subtree** - fast retrieval

Enhanced Trie: Store Scores

For autocomplete, each node stores:

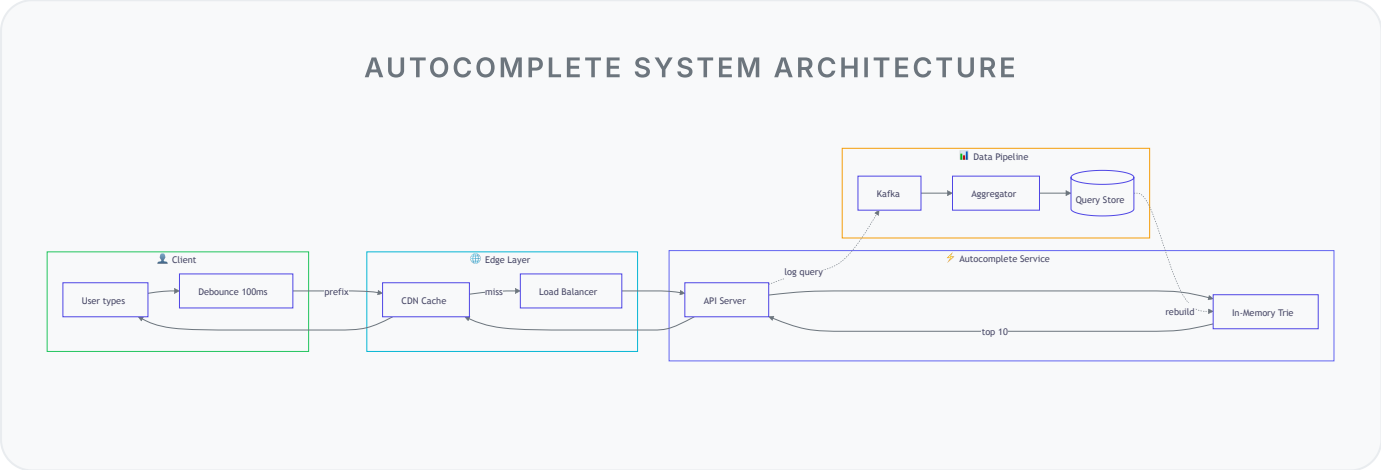
- Children (next characters)
- Top-K completions for this prefix (pre-computed)
- Scores (frequency, recency, personalization)

Sold to
@theskilledc
kumar.dodda@gmail.com

```python
class TrieNode:
    children: Dict[str, TrieNode]
    top_suggestions: List[Tuple[str, int]]  # (query, score)
    is_end: bool
```

Now lookup is just: **traverse to prefix node → return top_suggestions**.

The Complete Architecture



1. Client: Debounce Requests

Don't send a request on every keystroke. Wait 100-200ms after the user stops typing.

```javascript
const debounce = (fn, delay) => {
    let timer;
    return (...args) => {
        clearTimeout(timer);
        timer = setTimeout(() => fn(...args), delay);
    };
};
```

This alone reduces requests by **70-80%**.

2. Edge: CDN Caching

Popular prefixes ("how to", "what is", "best") are highly cacheable.

- Cache responses at CDN edge
- TTL of 5-15 minutes for trending queries
- Longer TTL for stable queries

CDN handles 80%+ of traffic - only cache misses hit origin.

Sold to
@theskilledc
kumar.dodda@gmail.com

3. Service: In-Memory Trie

The autocomplete service holds the Trie entirely in memory.

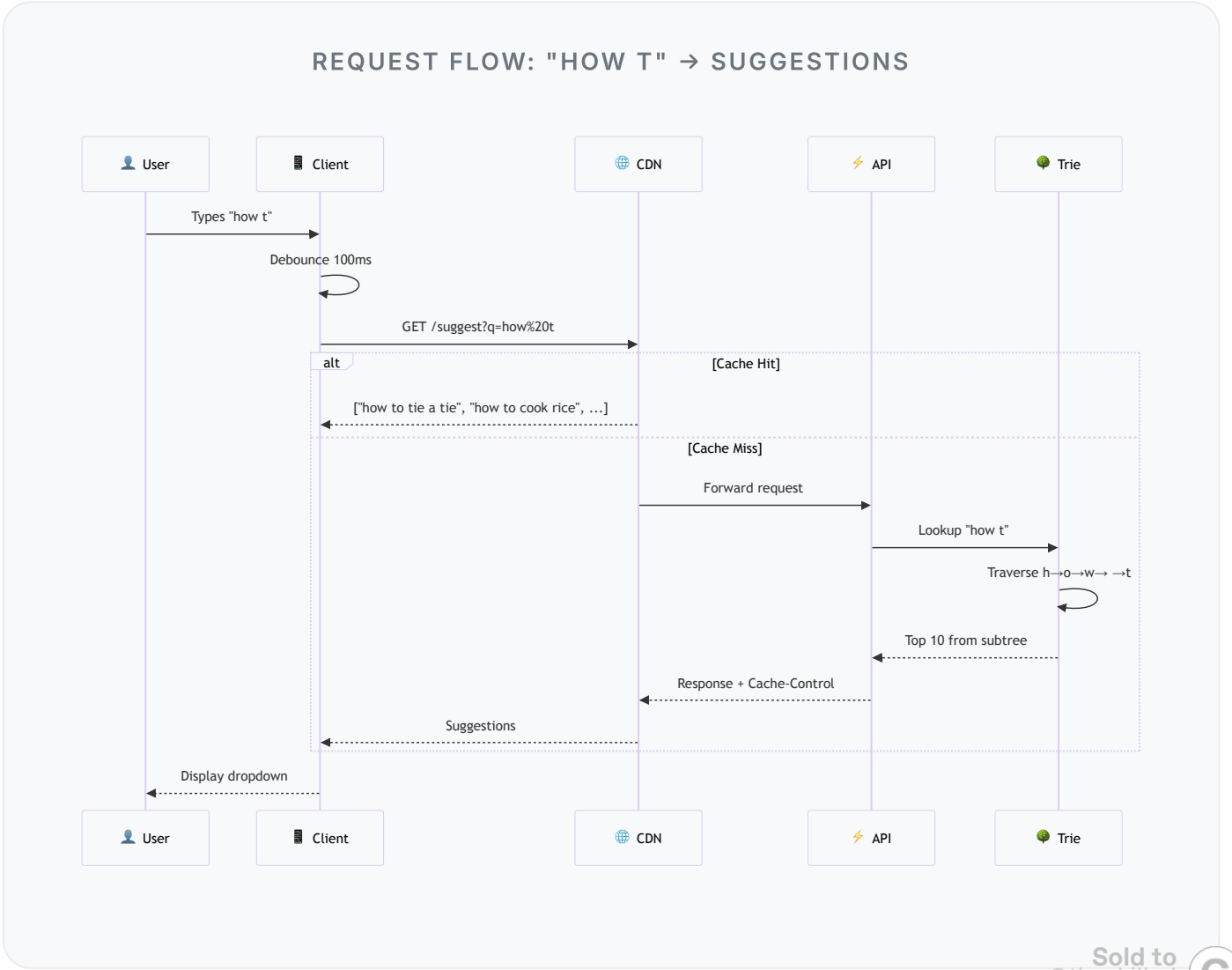
- **Read latency: <10ms**
- No database queries on the hot path
- Stateless - any replica can serve any request
- Replicate across multiple servers for redundancy

4. Data Pipeline: Continuous Updates

Queries flow through a pipeline:

1. **Log**: Every search query is logged to Kafka
2. **Aggregate**: Count query frequency in time windows
3. **Score**: Combine frequency, recency, and trends
4. **Rebuild**: Periodically rebuild the Trie with fresh data

The Trie is rebuilt every **1-15 minutes** and hot-swapped into memory.



Sold to
@theskilledc
kumar.dodda@gmail.com

Key Optimizations

Pre-compute Top-K at Each Node

Instead of traversing the entire subtree on each request, store the top 10-20 completions at every Trie node.

Lookup becomes: **O(prefix length)** instead of **O(subtree size)**.

Shard by Prefix

For massive scale, partition the Trie:

- Shard A: prefixes starting with a-f
- Shard B: prefixes starting with g-m
- Shard C: prefixes starting with n-z

Route requests based on first character.

Personalization Layer

Blend global suggestions with user-specific history:

```python
final_suggestions = (
    0.7 * global_top_k +
    0.3 * user_recent_queries
)
```

User history can be fetched from a fast KV store (Redis) in parallel.

Interview Pivot (How to Explain It)

If asked:

"How would you design search autocomplete for 100M queries/day?"

> "I'd use a Trie data structure with pre-computed top-K suggestions at each node, held entirely in memory. Clients debounce requests, CDN caches popular prefixes, and a data pipeline continuously updates the Trie from query logs.
>
> This gives sub-10ms latency, handles millions of requests, and stays fresh with trending queries."

Sold to
@theskilledc
kumar.dodda@gmail.com

What's Next?

Autocomplete opens up deeper questions:

How do you handle typos and fuzzy matching?

> Use **edit distance algorithms** (Levenshtein) or phonetic matching (Soundex). Build a secondary index of common misspellings. Pre-compute corrections for frequent typos.

How do you handle multiple languages?

> Build **separate Tries per language**. Detect language from user locale or input characters. Handle transliteration for non-Latin scripts.

How do you prevent offensive suggestions?

> Maintain a **blocklist** of offensive terms. Filter suggestions before returning. Use ML classifiers for edge cases. Human review for flagged content.

How do you update the Trie without downtime?

> **Build new Trie in background**, then atomic swap. Use versioned Tries with blue-green deployment. Old version serves traffic while new one loads.

The insight: Autocomplete isn't a search problem - it's a prefix-matching problem. Once you choose the right data structure, the architecture becomes obvious.

Tries for structure. Caching for speed. Pipelines for freshness.

Sold to
@theskilledc
kumar.dodda@gmail.com

Same Principles Apply To

- **Email address suggestions** - same Trie-based prefix matching
- **IDE code completion** - same pre-computed suggestions at nodes
- **Command palette (Spotlight, VS Code)** - same in-memory Trie with scoring
- **Product search (e-commerce)** - same debounce + CDN caching pattern

Sold to
@theskilledc
kumar.dodda@gmail.com

How Do You Handle 10,000 Concurrent Database Connections

The Database Is Almost Always the Bottleneck

You've scaled your app servers. Load balancer is humming. CDN is caching static assets.

Then traffic spikes - and everything freezes.

The culprit? **The database.**

Every request needs data. Every request opens a connection. And databases have hard limits.

DATABASE	DEFAULT MAX CONNECTIONS
PostgreSQL	100
MySQL	151
MongoDB	~65,000 (but memory-bound)

You can increase these limits - but **more connections ≠ more throughput**.

Let's explore why the obvious fixes fail, and what actually works.

⚡ The Engineering Complexity

Your database has limits your code doesn't respect. Here's what breaks first:

🔌 Connection Overhead

Each PostgreSQL connection = 2-10MB RAM. 10K connections = 20-100GB just for connections.

🚤 Pool Contention

100 connections, 10K requests. 99% of requests wait in line. That wait time adds up.

Sold to
@theskilledc
kumar.dodda@gmail.com

🔄 Connection Lifecycle

Open, query, close. Repeat 10K times/sec. TCP handshakes alone become a bottleneck.

💀 Thundering Herd

Database restarts. 10K clients reconnect simultaneously. Database dies again.

First Attempts (and Why They Break)

Attempt 1: Just Increase max_connections

The obvious fix:

```sql
-- PostgreSQL
ALTER SYSTEM SET max_connections = 10000;
```

Why it feels right:

- Simple config change
- More connections = more capacity, right?
- Works in small tests

Why it fails:

- Each connection consumes **5-10 MB of RAM**
- 10,000 connections = 50-100 GB just for connection overhead
- Context switching between connections kills CPU
- Query performance degrades for *everyone*

⚠️ **The Math**

10,000 connections × 10 MB = **100 GB RAM** just for connection state - before any actual queries run.

💡 **Lesson**

Databases are optimized for fewer, busier connections - not thousands of idle ones.

Sold to @theskilledc kumar.dodda@gmail.com

Attempt 2: One Connection Per Request

Many frameworks default to this: open connection, run query, close connection.

Why it feels right:

- Clean resource management
- No connection leaks
- Simple mental model

Why it fails:

- Connection setup takes **2-10ms** (TCP handshake + auth)
- At 1,000 QPS, you're spending 2-10 seconds/sec just connecting
- Connection storms under load
- Database spends more time accepting connections than running queries

> 💡 **Lesson**
>
> Opening connections is expensive. Reusing them is cheap.

Attempt 3: Connection Pool Per App Instance

Better! Use a connection pool in each app server.

```python
# Each app server has its own pool
pool = ConnectionPool(min=5, max=20)
```

Why it feels right:

- Reuses connections
- Limits per-instance usage
- Industry standard

Why it fails at scale:

- 50 app servers × 20 connections = **1,000 connections**
- Auto-scale to 500 servers = **10,000 connections**
- Each scale event compounds the problem
- Database doesn't know which connections are actually busy

Sold to
@theskilledc
kumar.dodda@gmail.com

> **Lesson**
>
> Per-app pools help, but don't solve the aggregate problem.

Attempt 4: Just Add Read Replicas

"If one database can't handle it, use more!"

Why it feels right:

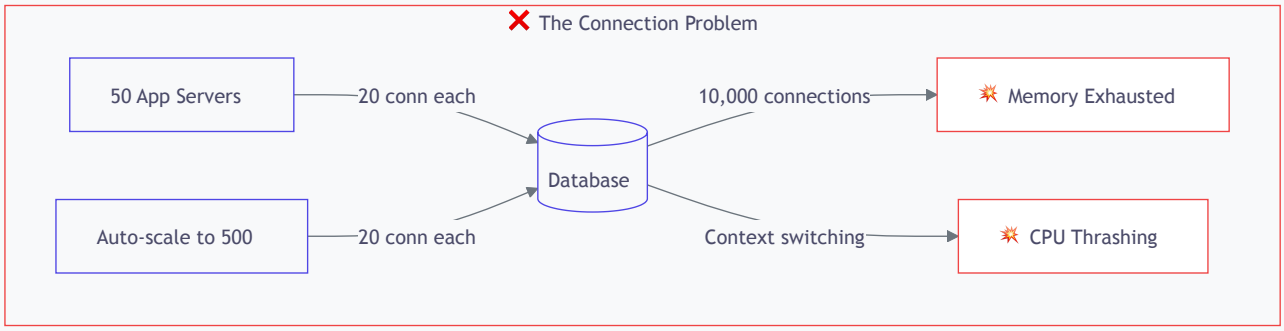
- Distributes read load
- More capacity
- Works for read-heavy workloads

Why it's incomplete:

- Writes still go to primary - single bottleneck
- Each replica needs its own connection pool
- Replication lag causes stale reads
- Doesn't reduce total connection count

> **Lesson**
>
> Read replicas help throughput, not connection management.

WHY THESE APPROACHES FAIL



The Real Problem (Reframed)

Sold to
@theskilledc
kumar.dodda@gmail.com

The issue isn't "how do we allow 10,000 connections."

> **The issue is: "How do we serve 10,000 concurrent requests with far fewer database connections?"**

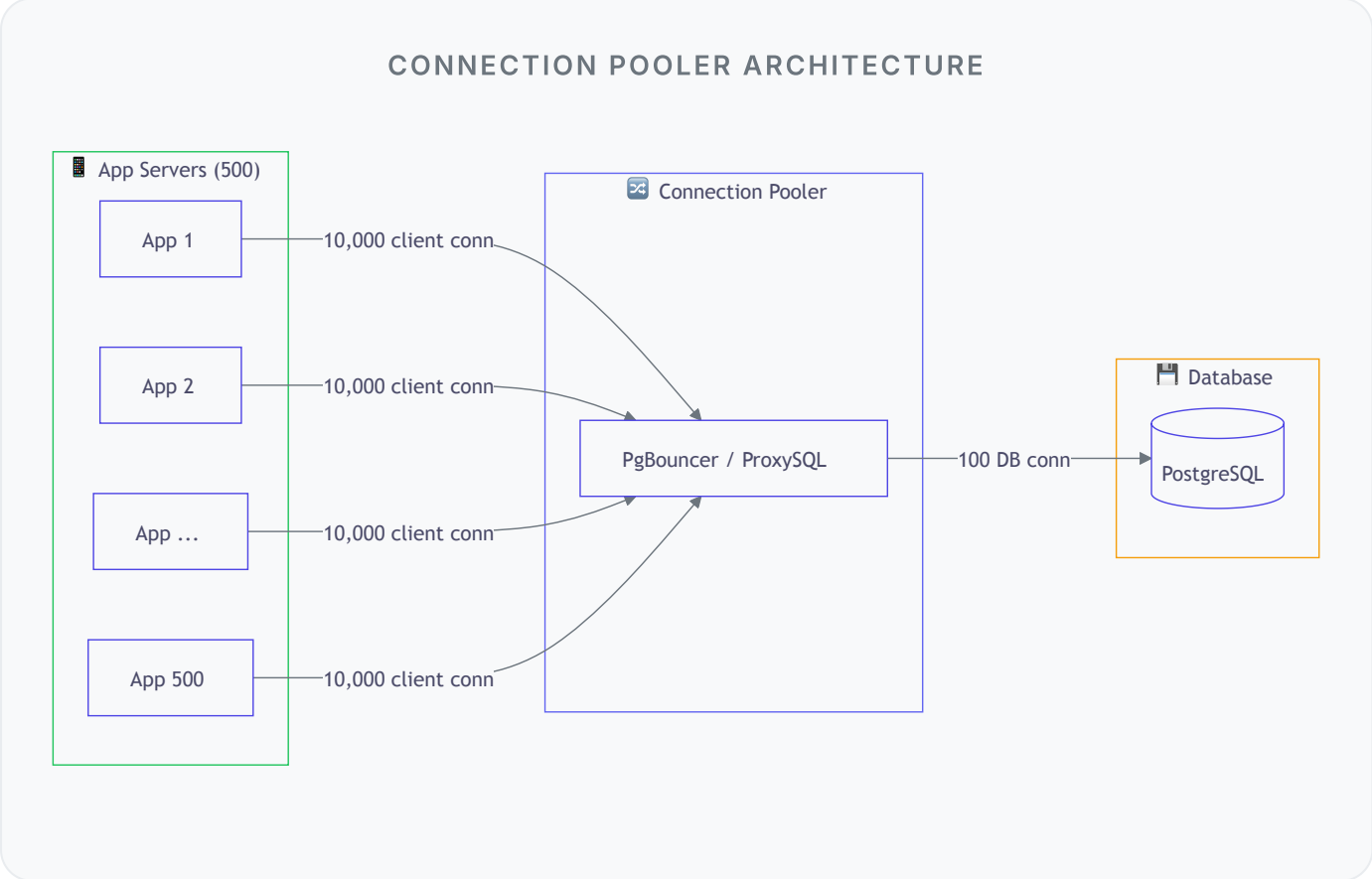
The answer involves three strategies:

1. **Connection Pooling** - at the right layer
2. **Caching** - avoid the database entirely
3. **Query Optimization** - do more with each connection

The Correct Architecture

1. Centralized Connection Pooler (PgBouncer / ProxySQL)

Instead of each app managing its own pool, use a **dedicated connection pooler**.



How it works:

- Apps connect to the pooler (lightweight connections)
- Pooler maintains a small set of real DB connections (e.g., 100)
- Queries are multiplexed across these connections

Sold to
@theskilledc
kumar.dodda@gmail.com

- Apps wait briefly if all connections are busy

POOLER	BEST FOR	MODE
PgBouncer	PostgreSQL	Transaction / Session
ProxySQL	MySQL	Query routing + pooling
Amazon RDS Proxy	AWS RDS	Managed pooling

2. Cache Aggressively

The best database query is the one you **don't make**.

- **Application cache**: In-memory (local) for hot data
- **Distributed cache**: Redis/Memcached for shared data
- **Query result cache**: Cache entire query results

PYTHON

```python
def get_user(user_id):
    # Check cache first
    cached = redis.get(f"user:{user_id}")
    if cached:
        return cached

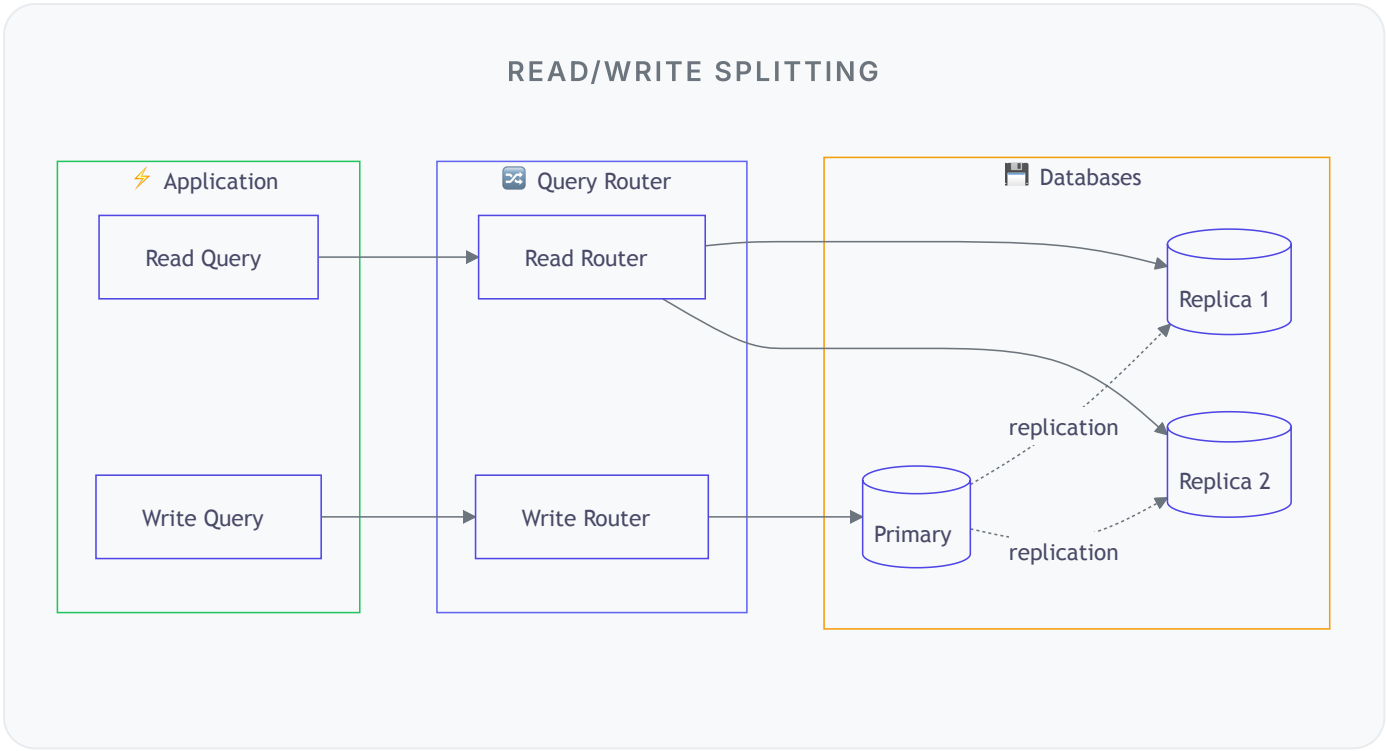
    # Only hit DB on cache miss
    user = db.query("SELECT * FROM users WHERE id = ?", user_id)
    redis.setex(f"user:{user_id}", 300, user)  # 5 min TTL
    return user
```

With good caching, **80-95% of reads never touch the database**.

3. Read/Write Splitting

Route reads to replicas, writes to primary.

Sold to
@theskilledc
kumar.dodda@gmail.com



Benefits:

- Reads scale horizontally with replicas
- Write load isolated to primary
- Each replica has its own connection pool

4. Optimize Queries

A connection used for 100ms is 10x worse than one used for 10ms.

- **Add indexes** for common query patterns
- **Avoid N+1 queries** - batch or join instead
- **Paginate** large result sets
- **Use EXPLAIN ANALYZE** to find slow queries

```sql
-- Bad: N+1 queries
SELECT * FROM orders WHERE user_id = 1;
SELECT * FROM orders WHERE user_id = 2;
-- ... 1000 more queries

-- Good: Single batch query
SELECT * FROM orders WHERE user_id IN (1, 2, 3, ... 1000);
```

5. Async & Queue Heavy Operations

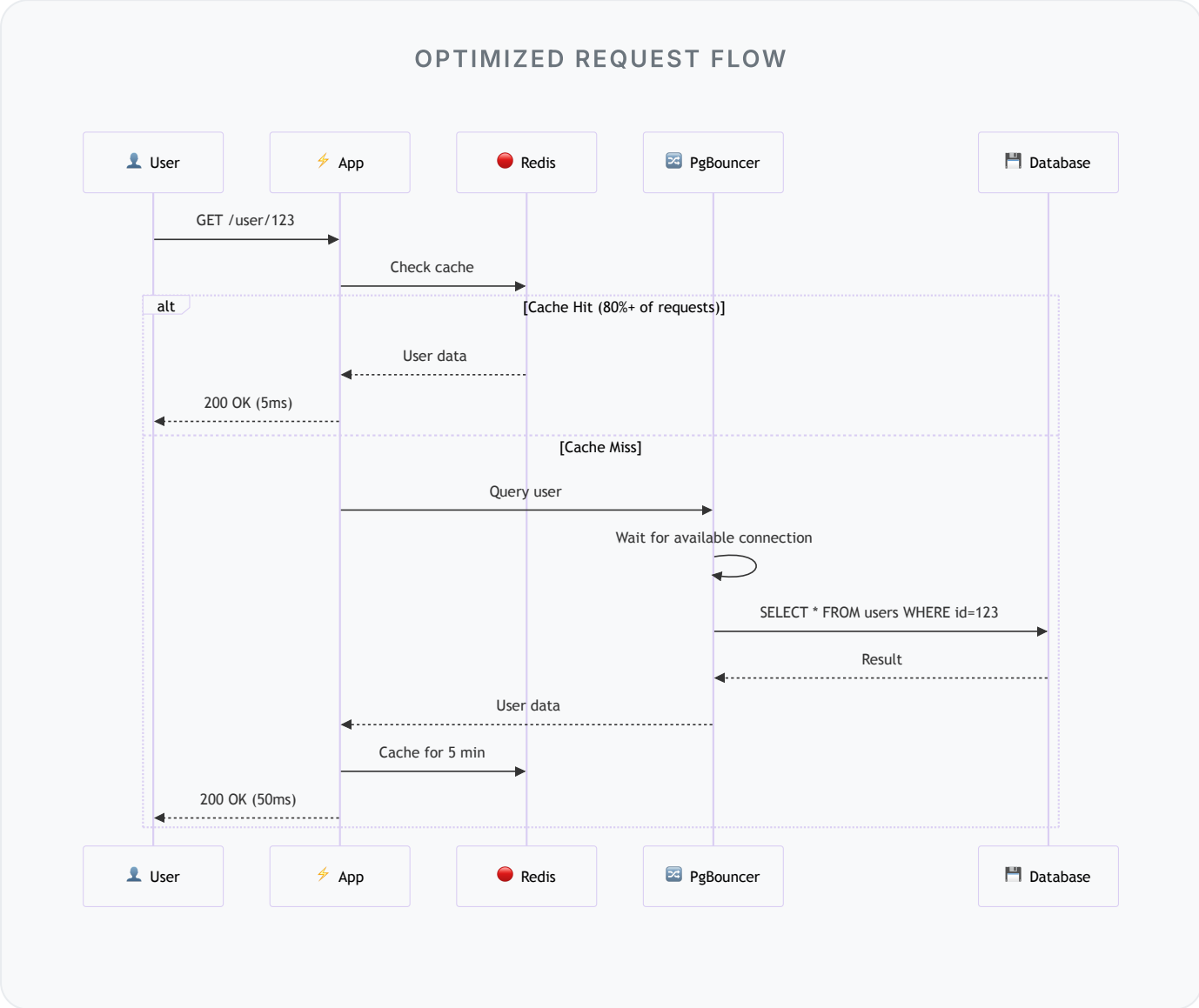
Not every request needs an immediate database response.

- Analytics writes → queue to Kafka, batch insert

Sold to
@theskilledc
kumar.dodda@gmail.com

- Notifications → async workers
- Reports → pre-compute, cache results

Move slow writes off the hot path.



The Complete Strategy

STRATEGY	IMPACT	IMPLEMENTATION
Connection Pooler	10,000 → 100 DB connections	PgBouncer, ProxySQL, RDS Proxy
Caching	80-95% fewer DB reads	Redis, Memcached
Read Replicas	Horizontal read scaling	Primary + N replicas
Query Optimization	Faster queries = faster releases	Indexes, batching, pagination
Async Writes	Decouple heavy operations	Kafka, queues, batch inserts

Sold to
@theskilledc
kumar.dodda@gmail.com

Interview Pivot (How to Explain It)

If asked:

"How would you handle 10,000 concurrent database connections?"

> "I wouldn't try to handle 10,000 connections directly - databases aren't designed for that. Instead, I'd reduce the connection count with a connection pooler like PgBouncer, add caching to avoid unnecessary queries, split reads to replicas, and optimize queries to release connections faster.
>
> The goal is serving 10,000 users with 100 database connections."

What's Next?

Database scaling leads to deeper questions:

How do you handle connection pool exhaustion?

> Set **connection timeouts** so requests don't wait forever. Return 503 when pool is empty. Monitor pool utilization and alert before saturation.

How do you deal with replication lag?

> Use **read-your-writes consistency** - route to primary after writes. Monitor lag metrics. For critical reads, always use primary.

What about connection storms during deployments?

> Use **graceful shutdown** - drain connections before stopping. Rolling deployments to avoid all instances reconnecting at once. Connection pooler absorbs the spike.

Sold to
@theskilledc
kumar.dodda@gmail.com

When do you need to shard the database?

> When **write load exceeds single-node capacity**. When data size exceeds single-node storage. Shard by tenant, user ID, or geographic region.

The insight: Database connections are precious resources, not free commodities. The systems that scale don't create more connections - they use fewer connections more efficiently.

Pool. Cache. Optimize. That's how you serve 10,000 users without melting the database.

Same Principles Apply To

- **HTTP client connection pools** - same pooling principles for outbound calls
- **Thread pool management** - same bounded resource allocation
- **File descriptor limits** - same resource exhaustion patterns
- **Redis connection management** - same pooler-based multiplexing

Sold to
@theskilledc
kumar.dodda@gmail.com

How Do You Ensure Exactly-Once Payment Processing

The Hardest Problem in Payments: Doing It Only Once

A user clicks "Pay Now." The request goes through. The payment gateway responds.

Except - the response never arrives. The connection times out.

The user refreshes. Clicks "Pay Now" again.

Did the first payment go through? Did they just pay twice?

This single scenario has caused:

- Duplicate charges that destroy user trust
- Refund operations that cost money and time
- Customer support tickets that never end
- Legal and compliance nightmares

> 💰 **The Stakes**
>
> In payments, a bug isn't just a bug - it's someone's money. There's no "undo" button for a double charge.

Why This Isn't as Simple as It Sounds

You might think: *"Just check if the payment exists before processing."*

That logic works on paper. In reality, distributed systems break every assumption.

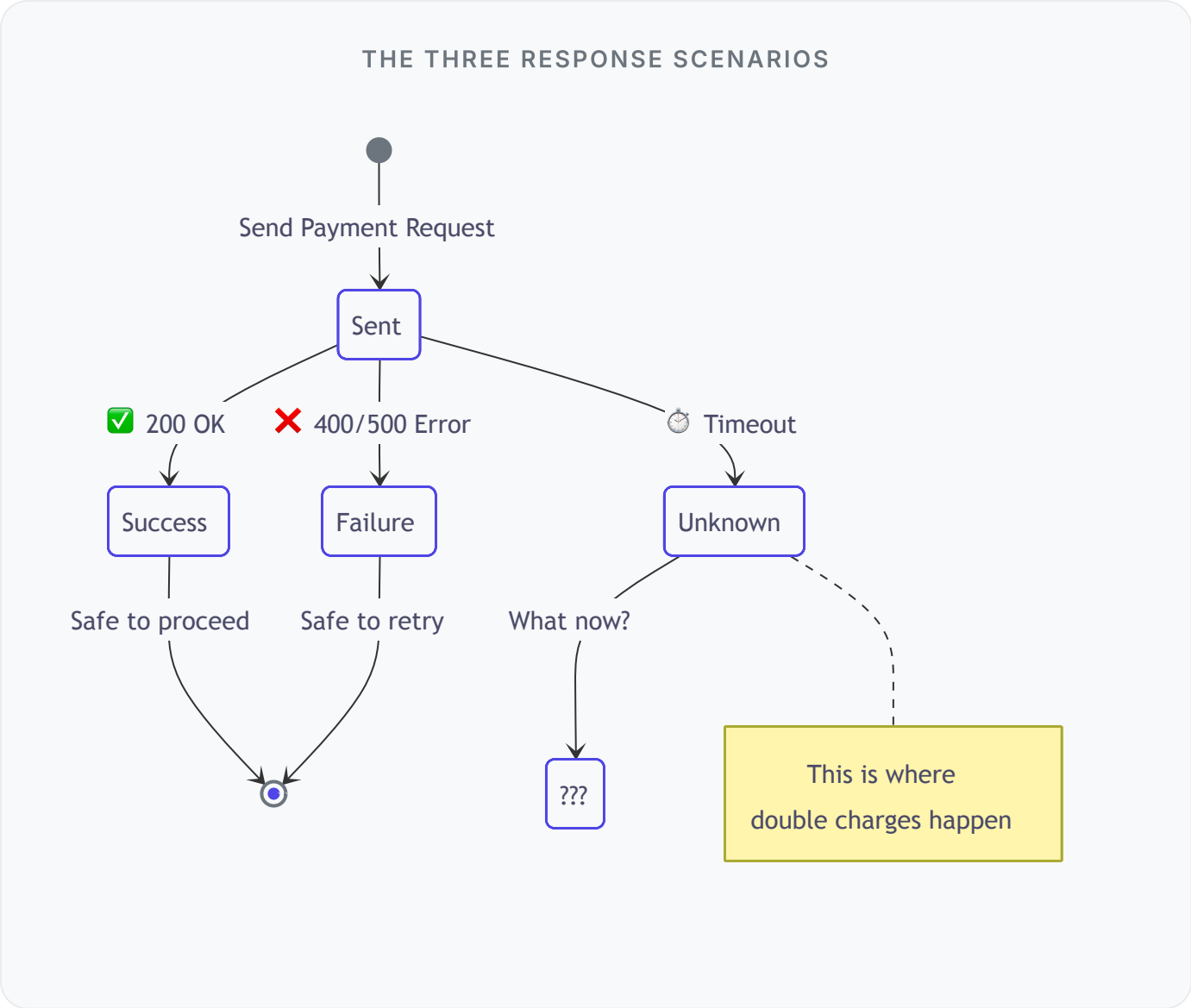
The Network Is Unreliable

When you call a payment gateway, three things can happen:

Sold to
kumar.dodda@gmail.com

1. **Success** - you get a response, payment went through
2. **Failure** - you get an error, payment didn't go through
3. **Timeout** - you get *nothing*

The third case is the killer. Did the payment succeed or fail? **You don't know.**



Retries Are Dangerous

When a request times out, the natural instinct is to retry. But:

- The first request might have *succeeded* - just the response was lost
- Retrying creates a second payment
- Now you've charged the customer twice

Users Are Impatient

Users don't wait. They:

Sold to
kumar.dodda@gmail.com

- Refresh the page
- Click "Pay" again
- Open multiple tabs
- Blame you when charged twice

Concurrency Creates Race Conditions

If two requests arrive at the same millisecond:

```text
Request A: Check if payment exists → No → Process payment
Request B: Check if payment exists → No → Process payment
                                          ↓
                                Both payments go through 💥
```

The "check-then-act" pattern is fundamentally broken for payments.

> 🎯 **The Core Truth**
>
> You cannot prevent duplicates by checking first. You must make the operation **inherently safe to repeat**.

Intuitive Solutions (That Don't Work)

Solution A: "Check Before Processing"

```python
if not payment_exists(order_id):
    process_payment(order_id)  # Race condition here!
```

Why it fails: Race condition. Two checks can pass before either insert completes.

Solution B: "Use a Transaction Lock"

```sql
BEGIN;
SELECT * FROM payments WHERE order_id = 123 FOR UPDATE;
-- Process payment
COMMIT;
```

Why it fails: The lock is on *your* database. The payment gateway is external - you can't lock it.

Sold to
kumar.dodda@gmail.com

Solution C: "Just Make It Fast"

"If we process quickly enough, duplicates won't happen."

Why it fails: Networks are slow. Users are fast. Speed doesn't prevent concurrency.

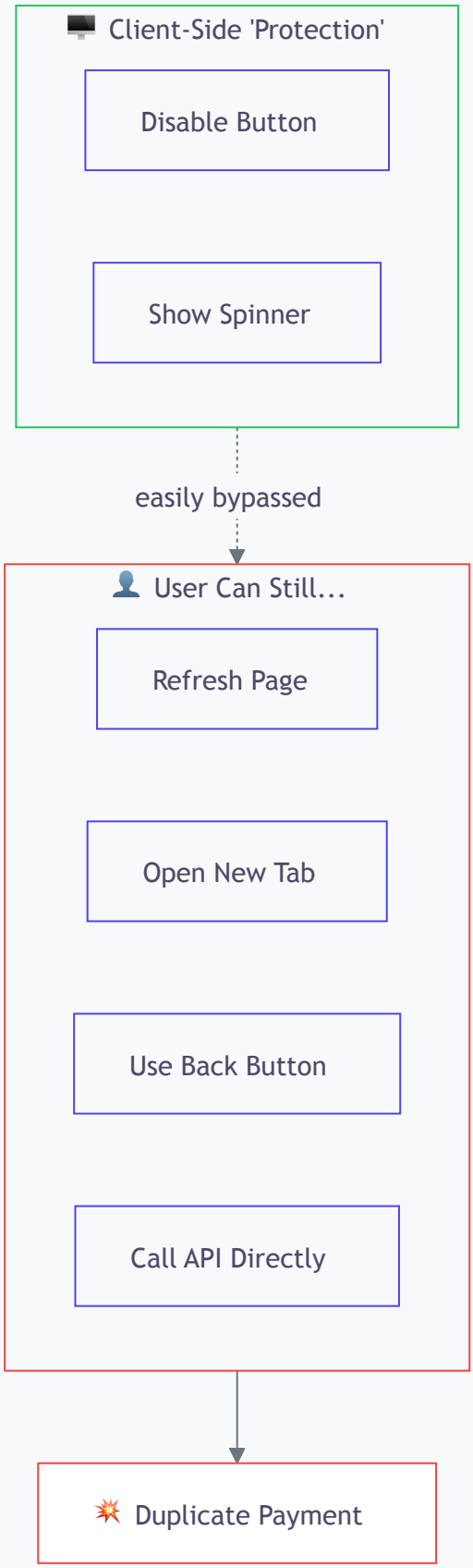
Solution D: "Disable the Button"

```javascript
button.disabled = true;
await processPayment();
button.disabled = false;
```

Why it fails: Users can refresh, use multiple tabs, or bypass client-side controls.

Sold to
kumar.dodda@gmail.com

WHY CLIENT-SIDE CONTROLS FAIL



Sold to
kumar.dodda@gmail.com

The Real Solution: Idempotency

The answer isn't to prevent retries - it's to make retries **safe**.

> **An operation is idempotent if doing it twice produces the same result as doing it once.**

- `GET /user/123` → Idempotent (reading doesn't change anything)
- `DELETE /user/123` → Idempotent (deleting twice = still deleted)
- `POST /charge` → **NOT** idempotent (charging twice = double charge)

We need to make `POST /charge` idempotent.

The Idempotency Key Pattern

Every payment request includes a unique key - the **idempotency key**.

```http
POST /v1/charges
Idempotency-Key: order_123_user_456_abc789
Content-Type: application/json

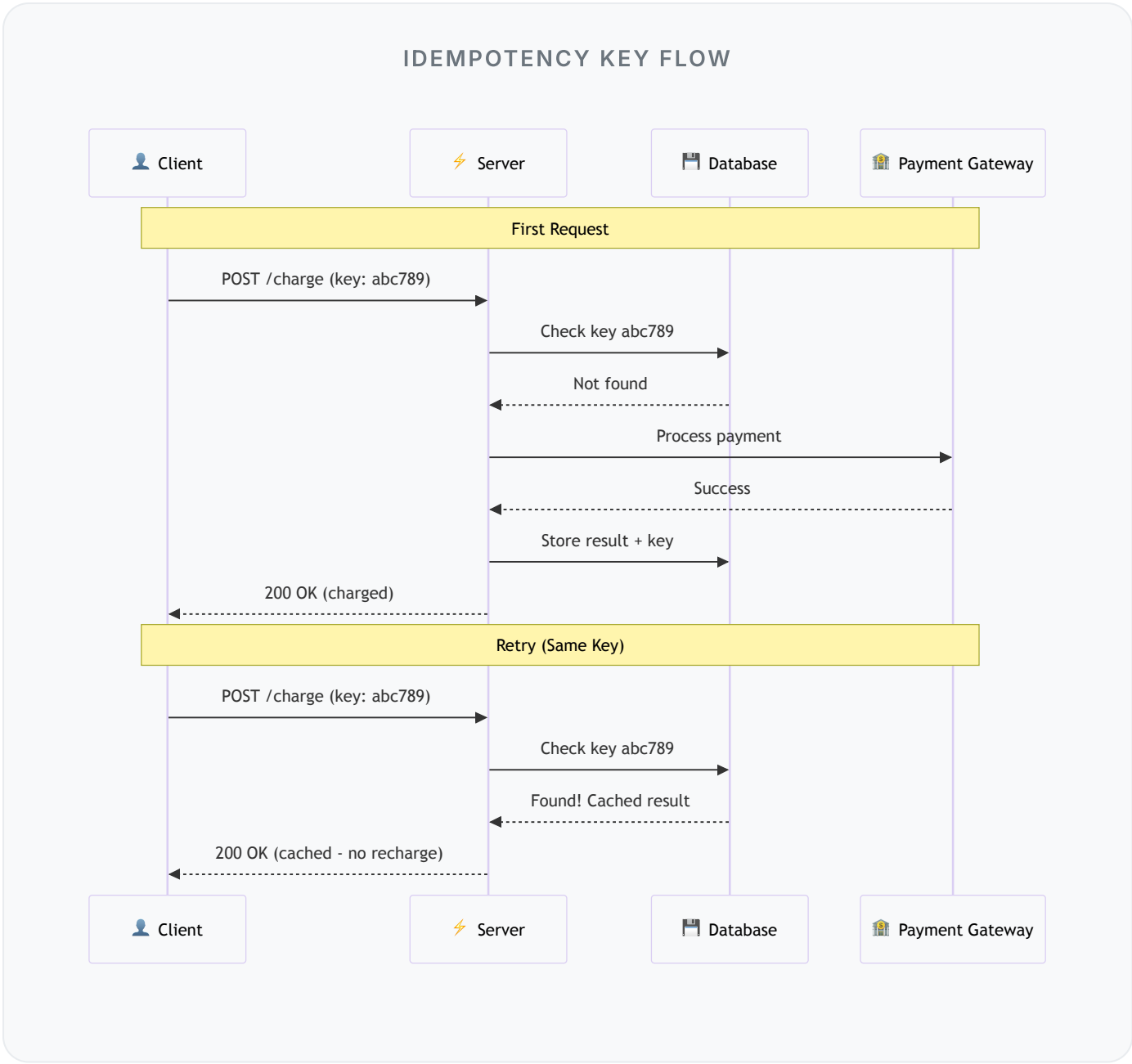
{
  "amount": 9999,
  "currency": "usd",
  "customer": "cus_456"
}
```

How it works:

1. First request with key `abc789` → process payment, store result
2. Second request with same key → return stored result (no reprocessing)
3. Third request with same key → return stored result

No matter how many times the request comes, the payment happens **exactly once**.

Sold to
kumar.dodda@gmail.com



Implementation Deep Dive

Generating Idempotency Keys

The key must be:

- **Unique per logical operation** - same order shouldn't have different keys
- **Deterministic** - retries must use the same key
- **Client-generated** - server can't know if it's a retry

Good patterns:

Sold to
kumar.dodda@gmail.com

```python
# Pattern 1: Order-based
idempotency_key = f"order_{order_id}_charge"

# Pattern 2: User action-based
idempotency_key = f"user_{user_id}_order_{order_id}_{action}"

# Pattern 3: UUID for user's session
idempotency_key = str(uuid.uuid4())  # Generated once, stored in form
```

Server-Side Storage

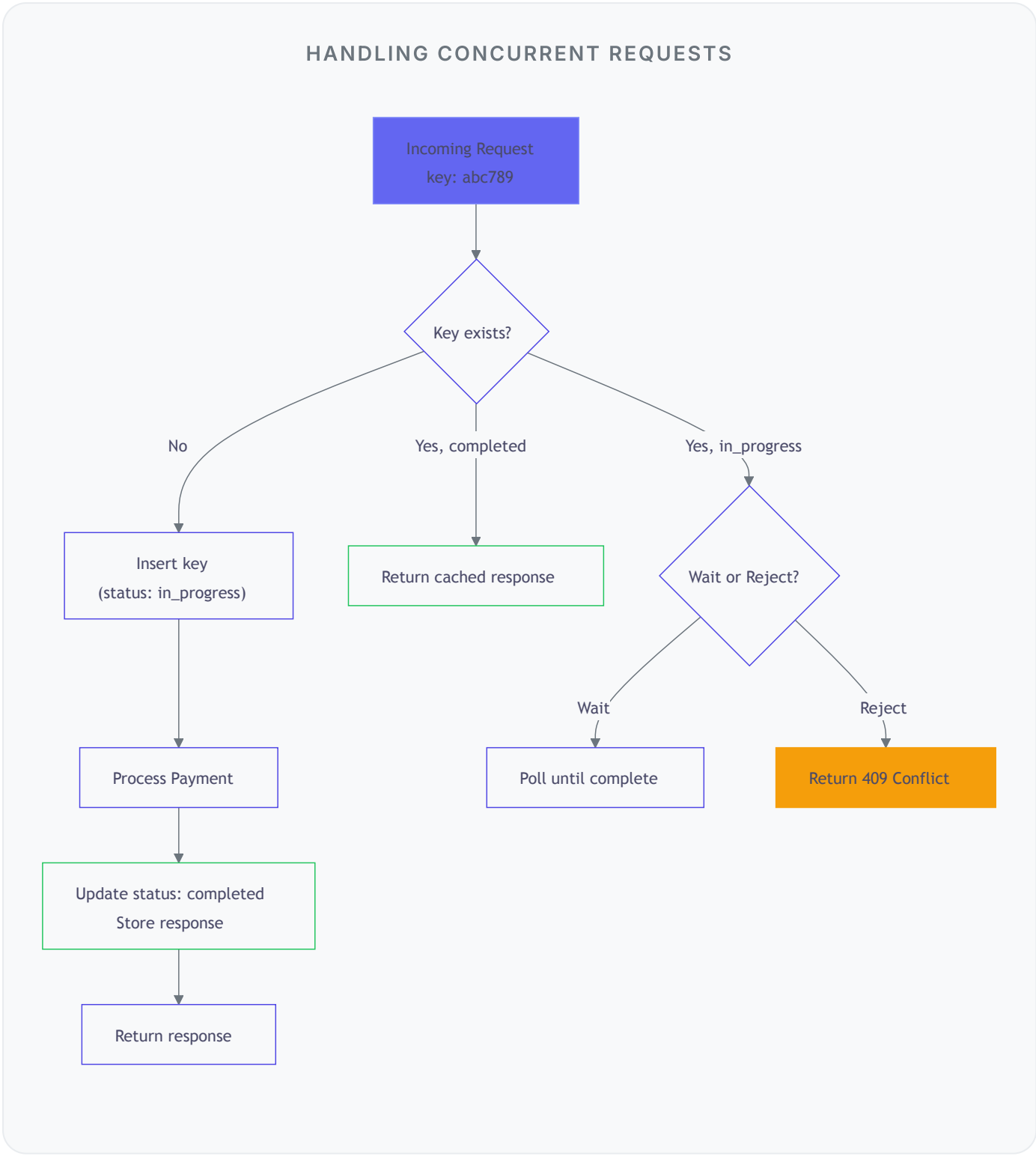
Store idempotency keys in a fast, durable store:

IDEMPOTENCY_KEY	STATUS	RESPONSE	CREATED_AT
order_123_charge	completed	{"payment_id": "py_789"}	2024-01-15 12:00:00
order_456_charge	in_progress	null	2024-01-15 12:01:00

Handling the In-Progress State

What if a request is *still processing* when a retry arrives?

Sold to
kumar.dodda@gmail.com



Using Database Constraints

The ultimate safety net - make duplicates **impossible** at the database level.

Sold to
kumar.dodda@gmail.com

```sql
CREATE TABLE payments (
    id SERIAL PRIMARY KEY,
    idempotency_key VARCHAR(255) UNIQUE NOT NULL,
    order_id INT NOT NULL,
    amount INT NOT NULL,
    status VARCHAR(50),
    gateway_response JSONB,
    created_at TIMESTAMP DEFAULT NOW()
);

-- This INSERT will fail on duplicate key
INSERT INTO payments (idempotency_key, order_id, amount)
VALUES ('order_123_charge', 123, 9999);
```

Even with race conditions, the database ensures only one row is inserted.

Working with Payment Gateways

Major payment providers support idempotency natively:

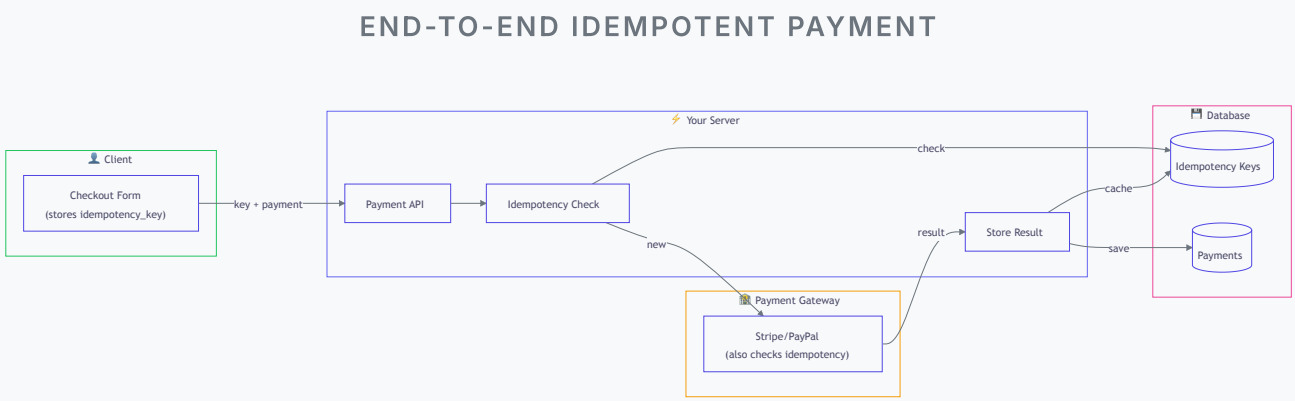
PROVIDER	HEADER	KEY LIFETIME
Stripe	`Idempotency-Key`	24 hours
PayPal	`PayPal-Request-Id`	72 hours
Square	`Idempotency-Key`	24 hours
Adyen	`reference` field	Indefinite

```python
# Stripe example
import stripe

charge = stripe.Charge.create(
    amount=9999,
    currency="usd",
    customer="cus_456",
    idempotency_key="order_123_charge"  # Safe to retry!
)
```

Sold to
kumar.dodda@gmail.com

The Complete Payment Flow



How to Explain It in an Interview

If asked:

"How do you ensure exactly-once payment processing?"

"You can't prevent retries - networks fail, users are impatient. So instead of preventing duplicates, I make the operation idempotent using an idempotency key.

Every payment request includes a unique key. Before processing, I check if that key exists. If it does, I return the cached result. If not, I process the payment and store the result with the key.

Combined with database UNIQUE constraints and gateway-level idempotency, this guarantees exactly-once semantics."

Further Considerations

Payment idempotency opens up related challenges:

Sold to
kumar.dodda@gmail.com

How long should idempotency keys be stored?

> **24-72 hours minimum** for retries. Keep completed payment records indefinitely for auditing. Prune in-progress keys that never completed after timeout.

What if the payment succeeds but your database fails?

> Use **webhooks** as the source of truth. Payment gateways send confirmation webhooks. Reconcile periodically by comparing your records with gateway records.

How do you handle refunds idempotently?

> Same pattern: **refund idempotency key** per refund request. Link to original payment. Ensure refund ≤ original amount.

What about distributed transactions across services?

> Use the **Saga pattern** with compensating transactions. Each step is idempotent. Failed steps trigger rollback actions.

The key insight: In distributed systems, you cannot guarantee a request runs exactly once. But you *can* guarantee that running it multiple times produces the same result.

Idempotency doesn't prevent retries - it makes retries harmless.

Sold to
kumar.dodda@gmail.com

Same Principles Apply To

- **Order creation (e-commerce)** - same idempotency key pattern
- **Booking systems (flights, hotels)** - same reservation ID as idempotency key
- **Bank transfers** - same transaction ID for deduplication
- **Subscription management** - same idempotent charge/cancel/upgrade operations

Sold to
kumar.dodda@gmail.com

How Would You Design a Recommendation Engine for Millions of Users

Personalization at Scale Is a Different Game

Open Netflix. Spotify. Amazon. YouTube.

Within milliseconds, you see content *curated for you*. Not random. Not the same for everyone. Personalized.

- "Because you watched..."
- "Customers who bought this also bought..."
- "Made for you playlist"

Behind this magic is a **recommendation engine** - a system that predicts what you'll like before you know you want it.

Sounds simple? It's not.



The Scale

Netflix serves 200M+ users. Each user sees personalized recommendations in under 200ms. That's billions of predictions per day.

Why This Problem Is Deceptively Complex

You might think: *"Just find similar users and show what they liked."*

That intuition is correct - but implementing it at scale breaks in surprising ways.

The Data Is Massive

- Millions of users
- Millions of items (products, videos, songs)
- Billions of interactions (views, clicks, purchases, ratings)

Sold to @theskilledc
kumar.dodda@gmail.com

Finding patterns in this data requires techniques that don't fit in memory or run in real-time.

Preferences Change

A user's taste at 9 AM (news) differs from 9 PM (entertainment). Last month's interests may be irrelevant today. Recommendations must adapt - fast.

Cold Start Problem

New user? No history. New item? No ratings. How do you recommend something when you know nothing?

Latency Matters

Users expect results in **100-200ms**. You can't compute complex ML models on every request.



First Attempts (and Why They Struggle)

Approach 1: Show Popular Items

The simplest approach: show everyone the most popular items.

```sql
SELECT * FROM products
ORDER BY total_purchases DESC
LIMIT 20;
```

Why it feels right:

- Simple to implement
- Popular items are popular for a reason
- Works for new users

Why it fails:

Sold to
@theskilledc
kumar.dodda@gmail.com

- Not personalized - everyone sees the same thing
- Creates filter bubbles around already-popular items
- New items never get discovered
- Doesn't leverage user behavior

> 💡 **Lesson**
>
> Popularity is a fallback, not a strategy.

Approach 2: User-Based Collaborative Filtering

Find users similar to you, recommend what they liked.

Why it feels right:

- Intuitive: "People like you liked this"
- No need to understand item content
- Classic recommendation approach

Why it struggles at scale:

- Comparing every user pair is $O(n^2)$
- 10M users = 100 trillion comparisons
- User preferences change faster than similarity can be recomputed
- Sparse data: most users rate few items

> 💡 **Lesson**
>
> User similarity doesn't scale without smart indexing.

Approach 3: Item-Based Collaborative Filtering

Pre-compute item-to-item similarity. "Users who bought X also bought Y."

Why it's better:

- Items change less frequently than user preferences
- Similarity can be pre-computed offline
- Amazon's famous approach

Sold to
@theskilledc
kumar.dodda@gmail.com

Why it's still limited:

- Only works for items with interaction overlap
- New items have no similarity data
- Doesn't capture *why* items are similar

> 💡 **Lesson**
>
> Pre-computation helps, but content-blind approaches hit a ceiling.

Approach 4: Real-Time Computation

"Let's compute personalized recommendations on every request using ML."

Why it fails:

- ML inference at scale is expensive
- Complex models can't run in <200ms
- Every request = GPU/CPU cost
- Latency spikes kill user experience

> ⚠️ **Lesson**
>
> Real-time ML is a luxury. Batch + cache is the reality.

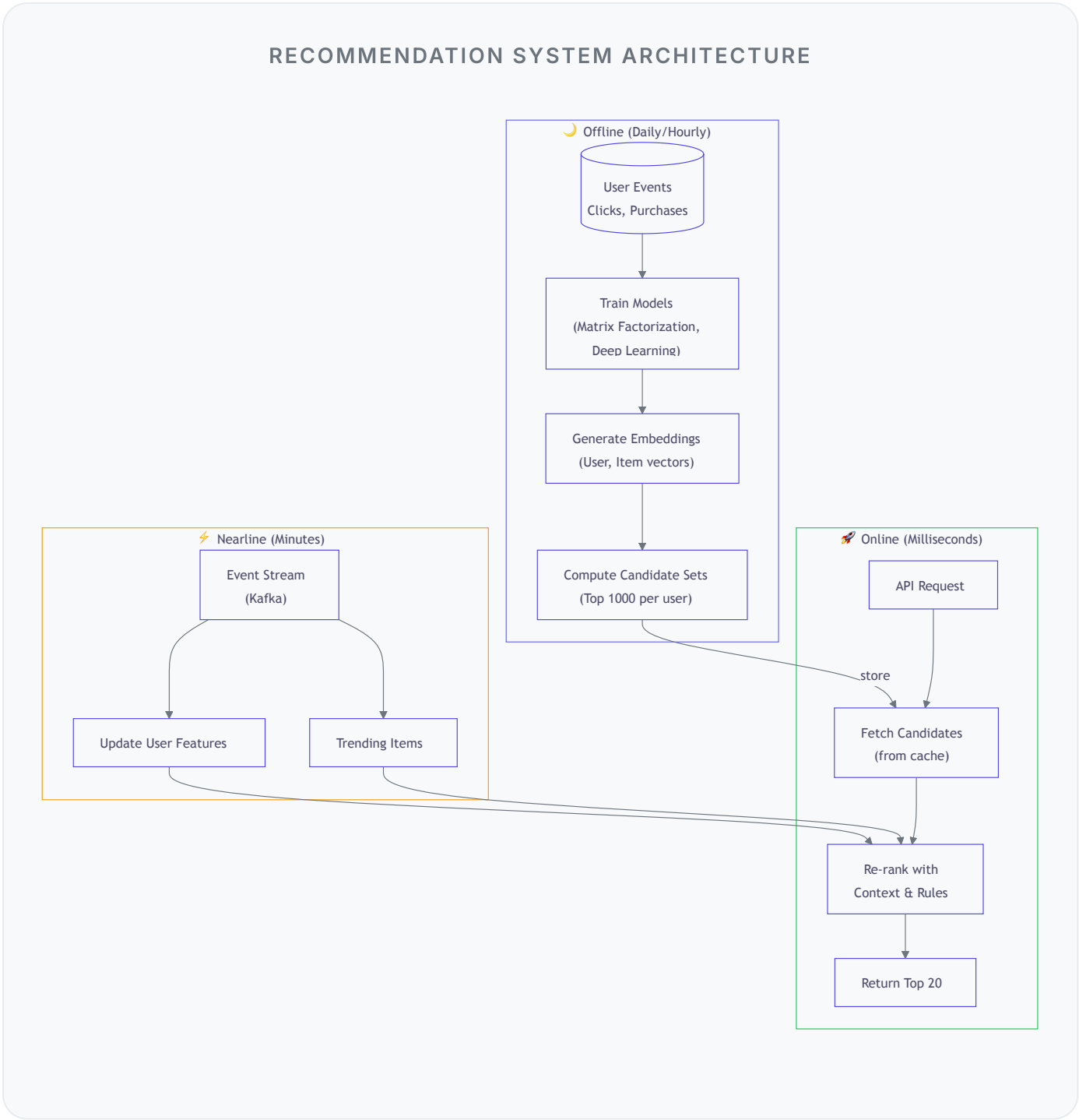
The Scalable Architecture

Production recommendation systems separate concerns:

1. **Offline**: Heavy ML training and pre-computation
2. **Nearline**: Incremental updates as events happen
3. **Online**: Fast lookup and light re-ranking

Train models offline. Compute candidates in batches. Serve from cache. Re-rank lightly online.

Sold to
@theskilledc
kumar.dodda@gmail.com



1. Offline: Train Models & Generate Candidates

Heavy computation happens in batch jobs (daily or hourly):

- **Matrix Factorization**: Decompose user-item matrix into embeddings
- **Deep Learning**: Neural networks for complex patterns
- **Content Features**: Extract item attributes (genre, category, tags)

Output: **User embeddings** and **Item embeddings** in the same vector space.

Sold to
@theskilledc
kumar.dodda@gmail.com

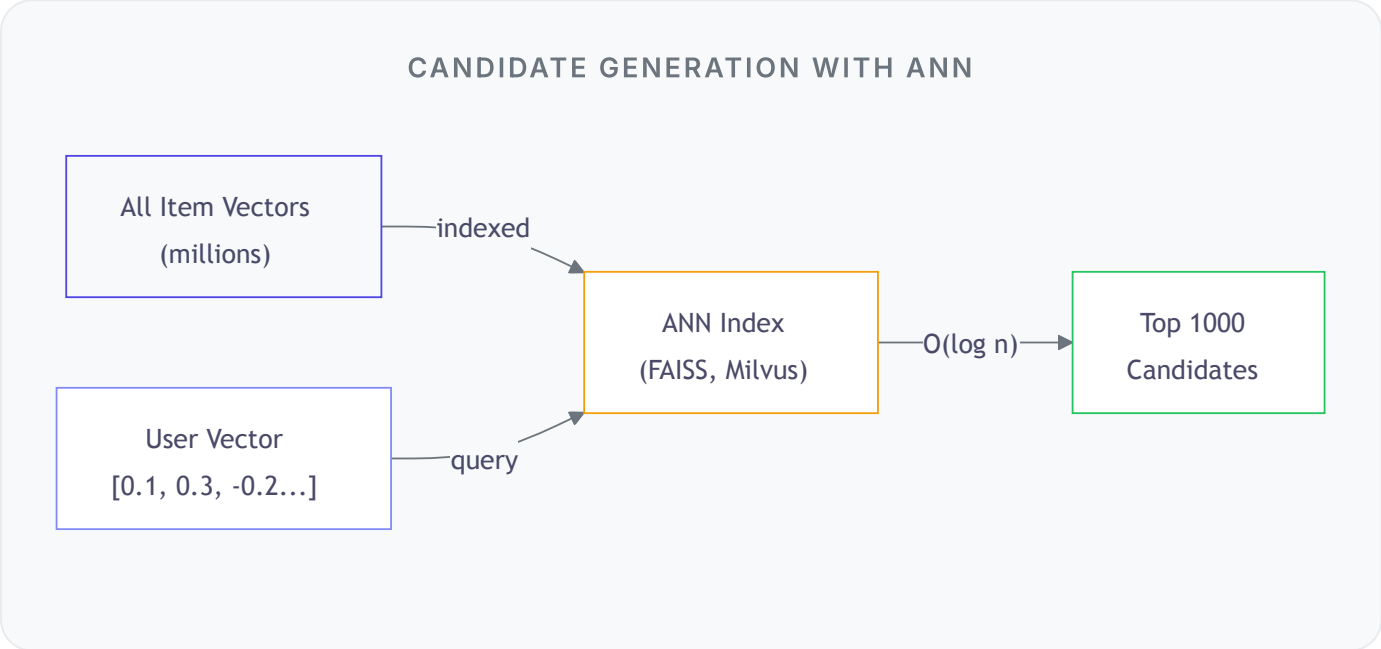
```python
# Simplified: User and Item as 128-dim vectors
user_embedding = model.get_user_vector(user_id)  # [0.1, 0.3, -0.2, ...]
item_embedding = model.get_item_vector(item_id)  # [0.2, 0.1, -0.3, ...]

# Similarity = dot product
score = np.dot(user_embedding, item_embedding)
```

2. Candidate Generation: Narrow the Search

You can't score all items for every user. Instead:

1. Pre-compute top 1000 candidates per user
2. Use **Approximate Nearest Neighbor (ANN)** search
3. Store in Redis or a vector database



ANN libraries: FAISS (Facebook), Milvus, Pinecone, Weaviate

3. Nearline: React to Recent Behavior

Batch models go stale. Nearline updates keep recommendations fresh:

- Stream user events (clicks, add-to-cart) via Kafka
- Update user feature store within minutes
- Track trending items in sliding windows

Sold to
@theskilledc
kumar.dodda@gmail.com

```python
# On each user event
def on_event(user_id, item_id, event_type):
    # Update recent activity
    redis.lpush(f"user:{user_id}:recent", item_id)
    redis.ltrim(f"user:{user_id}:recent", 0, 99)

    # Boost item in trending
    redis.zincrby("trending:1h", 1, item_id)
```

4. Online: Fast Lookup + Light Re-ranking

When a user opens the app:

1. Fetch pre-computed candidates from cache
2. Apply business rules (filter out-of-stock, adult content)
3. Boost based on recent activity and context (time, device)
4. Return top 20 in <100ms

```python
def get_recommendations(user_id, context):
    # 1. Fetch candidates (cached)
    candidates = redis.get(f"user:{user_id}:candidates")

    # 2. Filter
    candidates = [c for c in candidates if is_available(c)]

    # 3. Re-rank with context
    for c in candidates:
        c.score += time_decay(c) + trending_boost(c) + category_match(c, context)

    # 4. Return top results
    return sorted(candidates, key=lambda x: x.score, reverse=True)[:20]
```

Solving the Cold Start Problem

New Users

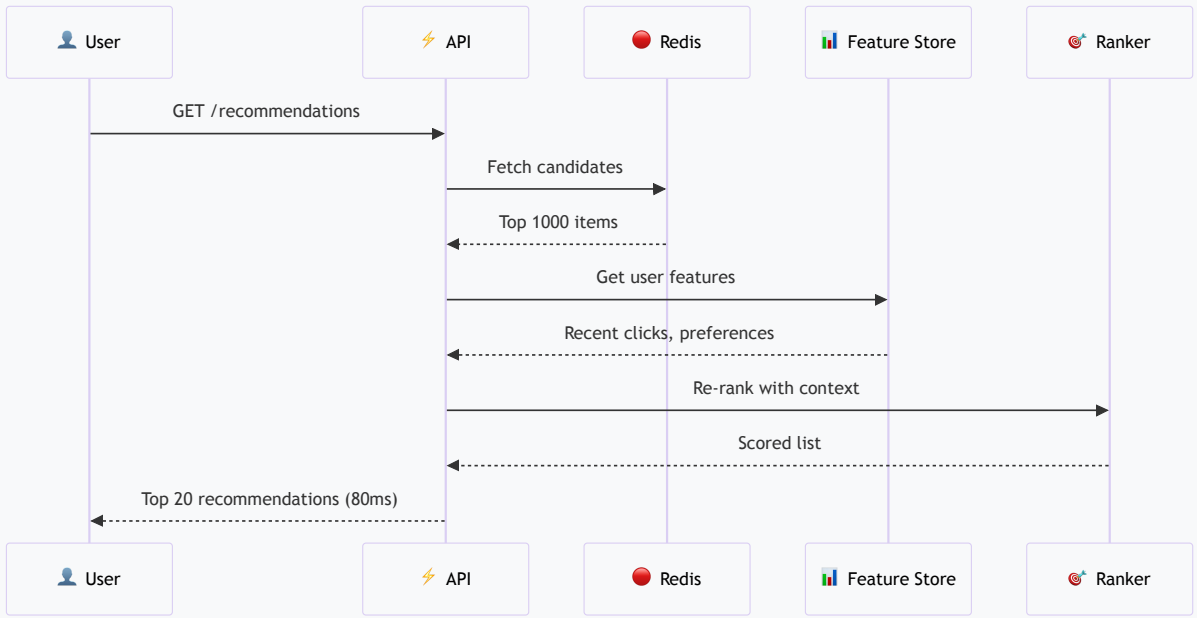
STRATEGY	HOW IT WORKS
Popularity	Show globally popular items as fallback
Onboarding	Ask preferences upfront (genres, categories)
Demographics	Use age, location as initial signals
Session-based	Recommend based on current session clicks

Sold to @theskilledc
kumar.dodda@gmail.com

New Items

STRATEGY	HOW IT WORKS
Content-based	Use item attributes to find similar items
Exploration	Show new items to random users, collect feedback
Bandit algorithms	Balance explore vs. exploit automatically

FULL REQUEST FLOW



Interview Pivot (How to Explain It)

If asked:

"How would you design a recommendation engine for millions of users?"

"I'd separate the system into three layers: offline for heavy ML training, nearline for real-time feature updates, and online for fast serving.

Offline, I'd train embedding models and pre-compute top candidates per user using ANN search. Nearline, I'd stream user events to update features and track trending items. Online, I'd fetch cached candidates, apply filters and context-based re-ranking, and return results in under 100ms.

For cold start, I'd fall back to popularity, content similarity, and onboarding preferences."

Sold to
@theskilledc
kumar.dodda@gmail.com

What's Next?

Recommendation engines lead to deeper challenges:

How do you measure recommendation quality?

> **Offline metrics**: Precision, Recall, NDCG. **Online metrics**: CTR, conversion rate, engagement time. Run A/B tests to compare models.

How do you avoid filter bubbles?

> Inject **exploration** into recommendations. Use multi-armed bandits. Diversify results by category. Track and limit content homogeneity.

How do you handle real-time personalization?

> Re-rank using **session context**: current page, recent clicks, time of day. Keep re-ranking model lightweight (gradient-boosted trees, not deep learning).

How do you update models without downtime?

> **Shadow deployment**: new model scores alongside old. Compare results. Blue-green switch when validated. Rollback if metrics drop.

The insight: Recommendation engines are not pure ML problems - they're systems problems. The best models mean nothing if you can't serve them in 100ms to millions of users.

Sold to
@theskilledc
kumar.dodda@gmail.com

Batch for training. Cache for speed. Stream for freshness. That's the formula.

Same Principles Apply To

- **Search ranking (Google, Bing)** - same batch indexing + real-time re-ranking
- **Ad serving systems** - same candidate generation + fast scoring
- **Feed ranking (Facebook, LinkedIn)** - same personalization pipeline
- **Match-making (dating apps, gaming)** - same embedding similarity search

Sold to
@theskilledc
kumar.dodda@gmail.com

How Would You Design a Key-Value Store for 100 Million Reads per Second

The Foundation of High-Performance Systems

Key-value stores are the backbone of modern infrastructure:

- **Redis** - Cache, sessions, leaderboards
- **Memcached** - Pure in-memory caching
- **DynamoDB** - Managed NoSQL at AWS scale
- **Cassandra** - Wide-column distributed storage
- **etcd** - Configuration and service discovery

The interface is deceptively simple:

- `GET(key)` → value
- `PUT(key, value)`
- `DELETE(key)`

But at **100 million reads per second**, the design choices matter enormously.

The Scale

100M reads/second = 1.2M reads/second per server (assuming 80 servers). That's 1.2 million hash lookups, network round trips, and responses every second per machine.

⚡ The Engineering Complexity

Simple key-value? Sure. Until you need consistency, partitioning, and replication:

🔄 Consistency Trade-offs

Strong consistency = slow writes. Eventual consistency = stale reads. Pick your poison.

📦 Partitioning

1B keys across 100 nodes. Which key goes where? What when a node dies?

Sold to @theskilledc kumar.dodda@gmail.com

🔥 Hot Keys

One viral post = one key = one node = that node dies. Hotspots break even distribution.

💾 Memory vs Disk

All in memory = fast but expensive. Disk-backed = cheap but slow. Hybrid is complex.

First Attempts (and Why They Break)

Attempt 1: Just Use a Hash Map in Memory

Store everything in an in-memory hash table. O(1) lookups.

Why it feels right: Fastest possible reads. Simple implementation. No disk I/O.

Why it fails: Memory is finite. A single server might have 256GB RAM - that's 256 billion bytes. If each key-value pair is 1KB, that's 256 million entries max. And when the server restarts? **Everything is gone.**

💡 **Lesson**

In-memory is fast but not durable. Good for cache, not for primary storage.

Attempt 2: Store on Disk with B-Tree Index

Use a traditional database with B-tree indexes. Proven technology.

Why it feels right: Durable. Handles more data than RAM. Well-understood.

Why it fails at scale: B-trees require random disk I/O for writes. Each write may update multiple tree nodes. At high write rates, disk becomes the bottleneck. SSDs help, but writes still amplify.

💡 **Lesson**

B-trees are read-optimized. High write rates break them.

Attempt 3: Single Server, Replicated

One primary server handles writes. Replicas handle reads.

Sold to
@theskilledc
kumar.dodda@gmail.com

Why it feels right: Simple mental model. Read traffic distributed.

Why it fails at scale: Writes still bottleneck on one server. Replication lag means reads might be stale. Primary failure = downtime or data loss.

> 💡 **Lesson**
>
> Single-primary doesn't scale writes. You need distribution.

Attempt 4: Consistent Hashing + Multiple Nodes

Hash keys to a ring. Each node owns a range of the ring. Data is partitioned.

Why it's better: Writes scale horizontally. No single bottleneck. Adding nodes is smooth.

Why it's still not complete: Node failure loses data. How do you handle network partitions? What consistency guarantees do you provide?

> ⚠️ **Lesson**
>
> Distribution is necessary but not sufficient. You need replication + consistency.

The Real Nature of the Problem

Designing a key-value store is really about choosing which guarantees matter:

DIMENSION	OPTION A	OPTION B
Storage	In-memory (fast, volatile)	On-disk (slow, durable)
Consistency	Strong (slower, simpler)	Eventual (faster, complex)
Availability	Prioritize uptime	Prioritize correctness
Partitioning	Hash-based (even spread)	Range-based (locality)

The CAP theorem says you can't have everything. During network partitions, you must choose between consistency and availability.

This tells us the design principles:

Sold to @theskilledc kumar.dodda@gmail.com

1. **Know your access patterns** - Read-heavy vs write-heavy changes everything
2. **Accept trade-offs explicitly** - Strong consistency costs latency
3. **Replicate for durability** - Data must survive failures
4. **Partition for scale** - One machine can't handle 100M reads/sec

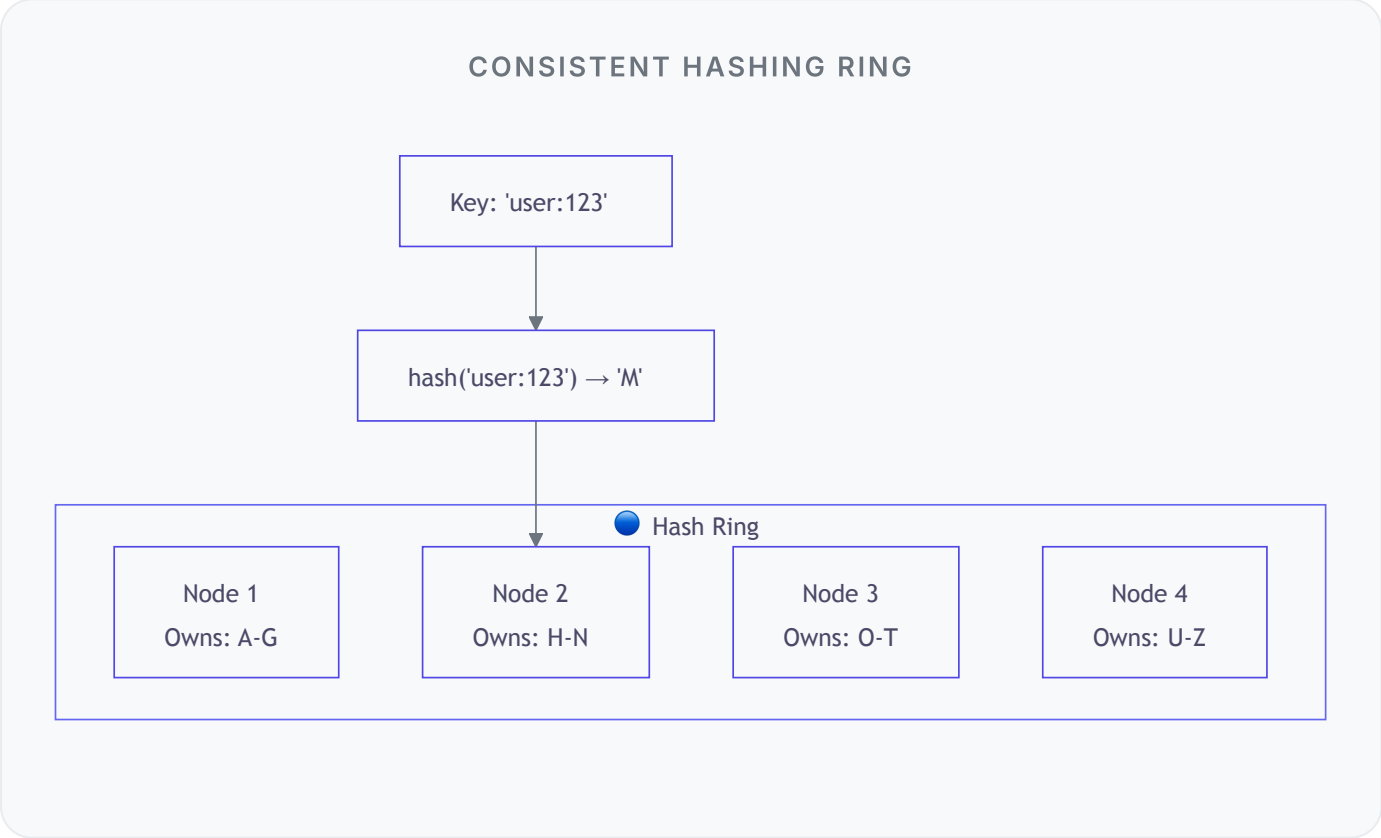
The Correct Architecture

Principle 1: Partition Data Across Nodes (Sharding)

100M reads/second is too much for one machine. Distribute the data:

- Hash the key to determine which partition (shard) owns it
- Each partition handles a subset of keys
- Consistent hashing ensures adding/removing nodes is smooth

Why consistent hashing? When you add a node, only ~1/N of keys need to move (N = number of nodes). With naive hashing, nearly all keys would need to move.



Principle 2: Replicate for Availability

Each partition is replicated to multiple nodes. Common factor: **3 replicas**.

- One node fails? Two others have the data.
- Reads can go to any replica (load distribution).

Sold to
@theskilledc
kumar.dodda@gmail.com

- Writes must coordinate between replicas.

Replication strategy matters:

STRATEGY	CONSISTENCY	LATENCY
Write to all, wait for all	Strong	High (slowest replica)
Write to all, wait for majority (quorum)	Strong	Medium
Write to one, async replication	Eventual	Low

Principle 3: Choose Your Consistency Model

Strong consistency: Every read sees the most recent write. Requires coordination (Paxos, Raft). Higher latency, simpler application logic.

Eventual consistency: Reads might see stale data temporarily. No coordination needed. Lower latency, application must handle conflicts.

Tunable consistency (DynamoDB approach): Let the client choose per-request:

- **Read quorum = (N/2)+1**: Guarantees latest if write quorum used
- **Read one**: Fastest, possibly stale
- **Write quorum = (N/2)+1**: Survives minority failure

Principle 4: Use LSM Trees for Write-Heavy Workloads

B-trees are read-optimized. For write-heavy workloads, use **Log-Structured Merge Trees**:

- Writes go to an in-memory buffer (memtable)
- When full, flush to disk as immutable sorted file
- Background compaction merges files
- Reads check memtable, then disk files

Why LSM? Writes are sequential (fast), not random. Perfect for SSDs. Used by Cassandra, LevelDB, RocksDB.

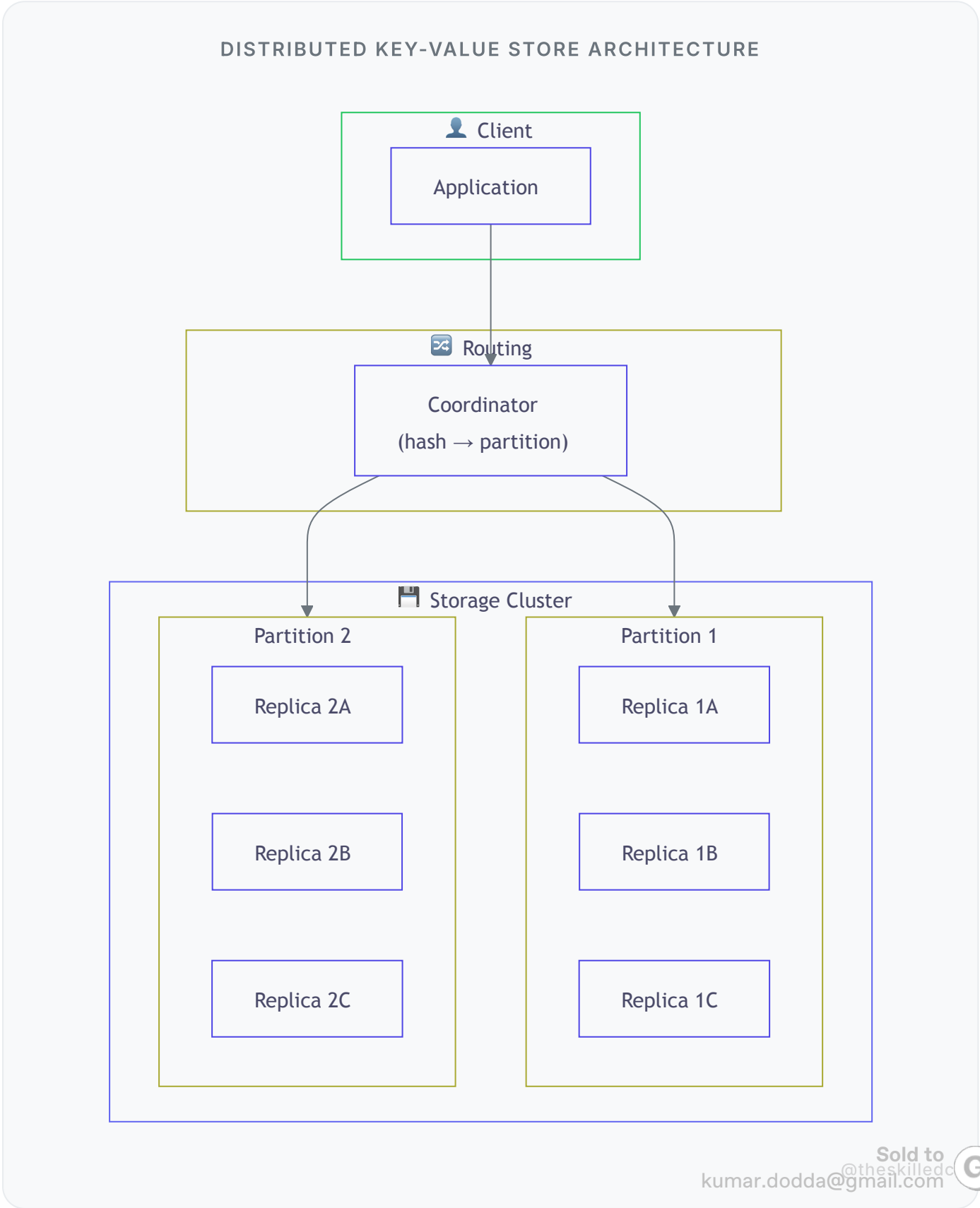
Principle 5: Layer Memory + Disk

The fastest systems combine both:

Sold to
@theskilledc
kumar.dodda@gmail.com

LAYER	WHAT	PURPOSE
L1	In-memory cache (Redis)	Hot data, <1ms latency
L2	SSD-backed store	Warm data, <10ms latency
L3	Disk/Object storage	Cold data, durable archive

This is the pattern behind systems like Facebook's TAO and Google's Spanner.



Sold to
@theskilledc
kumar.dodda@gmail.com

Handling Failures

At scale, failures are constant. Here's how to handle them:

Node Failure

- **Detection:** Heartbeats every few seconds. After N misses, node is marked down.
- **Recovery:** Replicas serve requests. Failed node's data is re-replicated to healthy nodes.
- **Hinted handoff:** If a replica is down, another node temporarily stores its writes.

Network Partition

- **If prioritizing consistency:** Minority partition stops accepting writes.
- **If prioritizing availability:** Both sides accept writes, merge later (conflict resolution).

Conflict Resolution

With eventual consistency, concurrent writes can conflict. Strategies:

- **Last-write-wins (LWW):** Timestamp determines winner. Simple but lossy.
- **Vector clocks:** Track causality. Detect conflicts for application resolution.
- **CRDTs:** Data structures that merge automatically without conflicts.

Interview Pivot (How to Explain It)

If asked:

"How would you design a key-value store for 100 million reads per second?"

> "First, I'd recognize that one machine can't handle this. I need to partition data using consistent hashing across many nodes.
>
> Each partition would be replicated 3x for availability. For reads, I'd use quorum reads for consistency or single-replica reads for speed - depending on requirements.
>
> For storage, if write-heavy, I'd use LSM trees. If read-heavy with durability needs, I'd layer an in-memory cache over an SSD-backed store.
>
> The key design decision is the consistency model - strong requires coordination and adds latency. Eventual is faster but applications must handle stale reads."

What's Next?

Sold to
@theskilledc
kumar.dodda@gmail.com

Key-value store design leads to deeper questions:

How do you handle hot keys?

> **Replicate hot keys** across more nodes. Or cache at the client. Or use a dedicated caching layer. Detect hot keys via access logging.

How do you handle range queries?

> **Range-based partitioning** instead of hash-based. Sorted storage (LSM or B-tree). Trade-off: hot ranges can overload partitions.

How do you handle schema changes?

> **Schema-less design:** Values are opaque bytes. Application handles serialization (Protobuf, JSON). Versioning inside the value.

How do you handle cross-key transactions?

> **Most don't.** Key-value stores sacrifice transactions for scale. If needed: two-phase commit or Spanner-style TrueTime.

The insight: A key-value store is a distributed system with explicit trade-offs. You choose between consistency and availability, between memory and disk, between simplicity and performance.

Partition for scale. Replicate for availability. Choose your consistency model.

Sold to
@theskilledc
kumar.dodda@gmail.com

Same Principles Apply To

- **Session storage** - same partitioning and caching patterns
- **Configuration stores (etcd, Consul)** - same replication and consistency choices
- **Feature flag systems** - same read-heavy, distributed architecture
- **Rate limiter counters** - same in-memory + persistence tradeoffs

Sold to
@theskilledc
kumar.dodda@gmail.com

How Do You Migrate 10TB of Data with Zero Downtime

The Most Stressful Operation in Production

Sooner or later, every growing system faces this moment:

- Moving from MySQL to PostgreSQL
- Migrating from on-prem to cloud
- Splitting a monolith database into microservice databases
- Changing the schema of a billion-row table

The data is massive. The system is live. Users are active. And someone says:

> **"We can't afford any downtime."**

That's when the real engineering begins.

> ⚠️ **The Stakes**
>
> A failed migration can corrupt data, break applications, and cost millions. There's no "undo" button for 10TB of production data.

Why Zero-Downtime Migration Is Hard

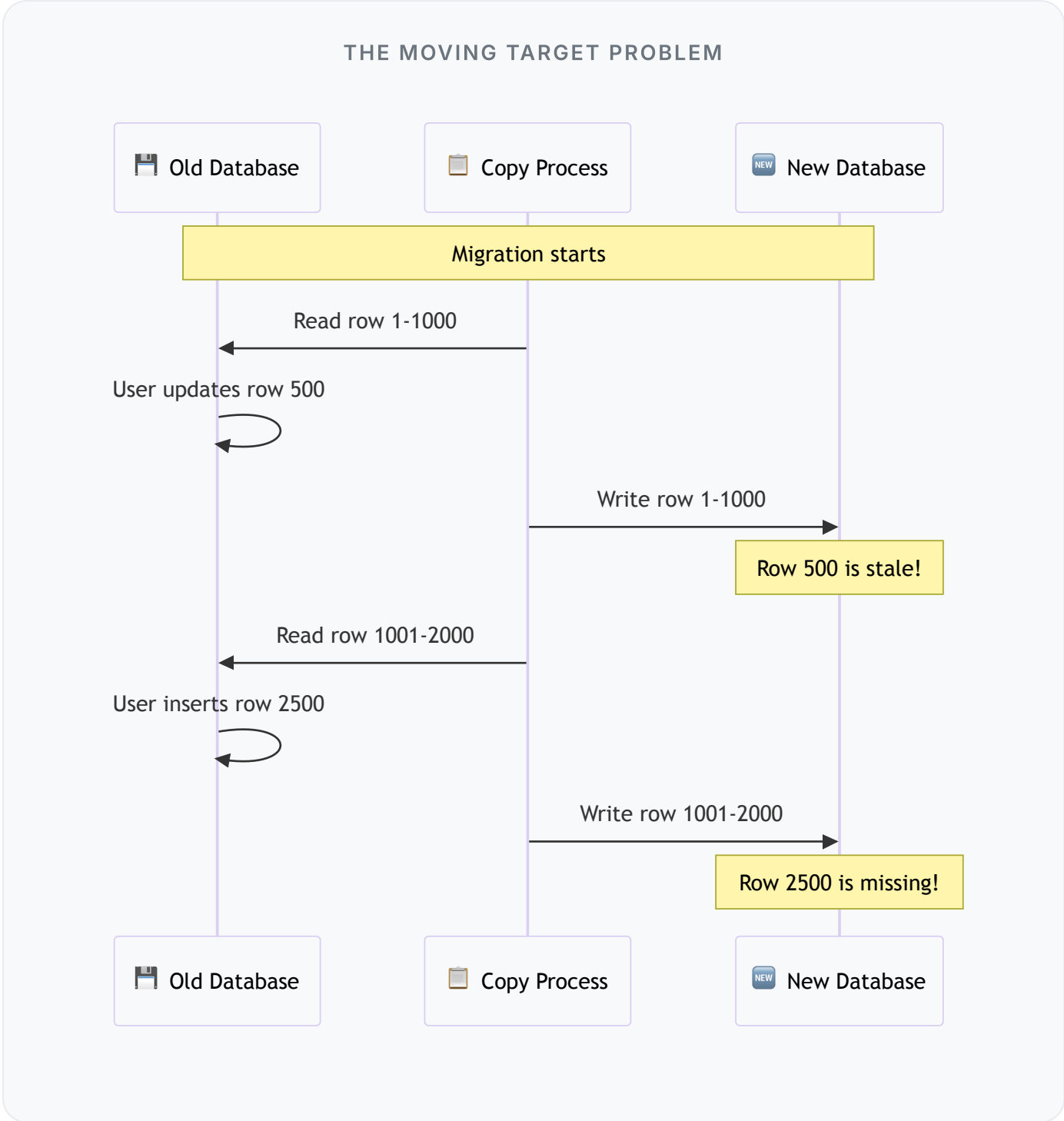
You might think: *"Just copy the data and switch."*

But while you're copying:

- New data is being written
- Existing data is being updated
- Users are actively querying
- The copy takes hours or days

Sold to @theskilledc
kumar.dodda@gmail.com

By the time you finish copying, your copy is already **stale**.



⚡ The Engineering Complexity

Moving data sounds easy until you realize the system can't stop while you do it.

Sold to @theskilledc kumar.dodda@gmail.com

⏱️ Zero Downtime

Users don't stop. You're changing the engine while the car is moving at 100mph.

🔄 Dual Writes

Write to old AND new during migration. What if one fails? Data diverges.

📊 Data Validation

1B rows migrated. How do you verify every single one matches?

↩️ Rollback Strategy

New system fails after 80% migration. Rolling back 800M rows isn't trivial.

First Attempts (and Why They Break)

Attempt 1: Scheduled Maintenance Window

The traditional approach: announce downtime, stop the system, migrate, restart.

Why it feels right:

- Simple to execute
- No race conditions
- Clean cutover

Why it fails:

- 10TB at 100MB/s = **28 hours** of downtime
- Global users mean there's no good maintenance window
- Revenue loss, SLA violations, user churn
- What if migration fails? Rollback takes another 28 hours

💡 **Lesson**

Downtime doesn't scale with data size.

Attempt 2: mysqldump / pg_dump and Restore

Export the database to a file, import into the new system.

Sold to @theskilledc kumar.dodda@gmail.com

```bash
# Export
mysqldump --single-transaction mydb > backup.sql

# Import
psql newdb < backup.sql
```

Why it feels right:

- Built-in tools
- Well-documented
- Works for small databases

Why it fails:

- Dump is a point-in-time snapshot - immediately stale
- Schema differences require manual transformation
- No way to capture ongoing changes
- Restore can take longer than dump

> 💡 **Lesson**
>
> Snapshots don't work for live data.

Attempt 3: Stop Writes, Copy, Resume

Keep reads running, pause writes, copy data, resume writes.

Why it feels right:

- Users can still read during migration
- Data doesn't change while copying
- Smaller downtime window

Why it fails:

- Many applications can't function without writes
- E-commerce: no orders, no payments, no cart updates
- Social: no posts, no likes, no messages
- Users notice immediately

Sold to
@theskilledc
kumar.dodda@gmail.com

> 💡 **Lesson**
>
> "Read-only mode" is often indistinguishable from downtime.

Attempt 4: Replicate at the Database Level

Use native replication (MySQL → MySQL, Postgres → Postgres).

Why it feels right:

- Built-in, battle-tested
- Continuous sync
- Near-zero lag

Why it's limited:

- Only works for same database type
- Can't transform schema during migration
- Can't migrate to different storage (MySQL → MongoDB, SQL → NoSQL)
- Vendor lock-in

> ⚠️ **Lesson**
>
> Native replication is for scaling, not for cross-platform migration.

The Right Approach: Dual-Write + Backfill

Zero-downtime migration requires a different mental model:

> **Don't move data in one shot. Run both systems in parallel, gradually shift traffic, then retire the old one.**

This is the **Dual-Write + Backfill** pattern.

Sold to
@theskilledc
kumar.dodda@gmail.com



Phase 1: Dual Write

Modify your application to write to **both** databases.

```python
def save_user(user):
    # Write to old database (primary)
    old_db.save(user)

    # Write to new database (async, non-blocking)
    try:
        new_db.save(transform(user))
    except Exception as e:
        log_error(e)  # Don't fail the request
```

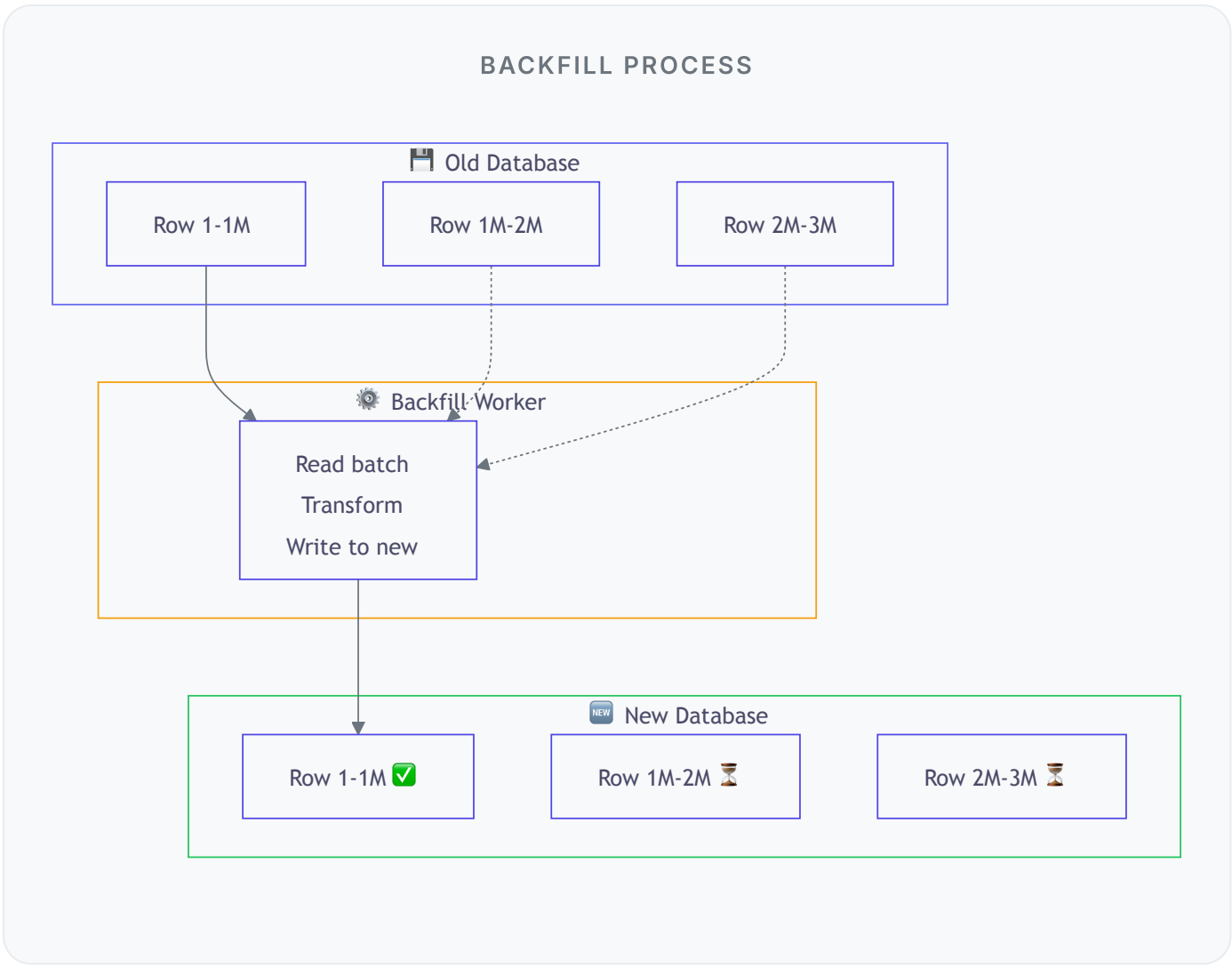
Key points:

- Old database is still the source of truth
- New database failures don't break the application
- All new data lands in both systems

Phase 2: Backfill Historical Data

While dual-write handles new data, copy existing data in the background.

Sold to
@theskilledc
kumar.dodda@gmail.com



PYTHON

```python
def backfill():
    last_id = 0
    batch_size = 1000

    while True:
        # Fetch batch from old database
        rows = old_db.query(
            "SELECT * FROM users WHERE id > ? ORDER BY id LIMIT ?",
            last_id, batch_size
        )

        if not rows:
            break

        # Transform and write to new database
        for row in rows:
            new_db.upsert(transform(row))

        last_id = rows[-1].id
        time.sleep(0.1)  # Don't overload production
```

Key points:

- Use `UPSERT` to handle conflicts with dual-writes

Sold to
@theskilledc
kumar.dodda@gmail.com

- Process in batches to avoid memory issues
- Add throttling to not impact production
- Track progress for resumability

Phase 3: Verify Data Integrity

Before switching reads, verify the data matches.

- **Row counts**: Same number of records?
- **Checksums**: Same content?
- **Shadow reads**: Read from both, compare results

```python
def get_user(user_id):
    old_result = old_db.get(user_id)
    new_result = new_db.get(user_id)

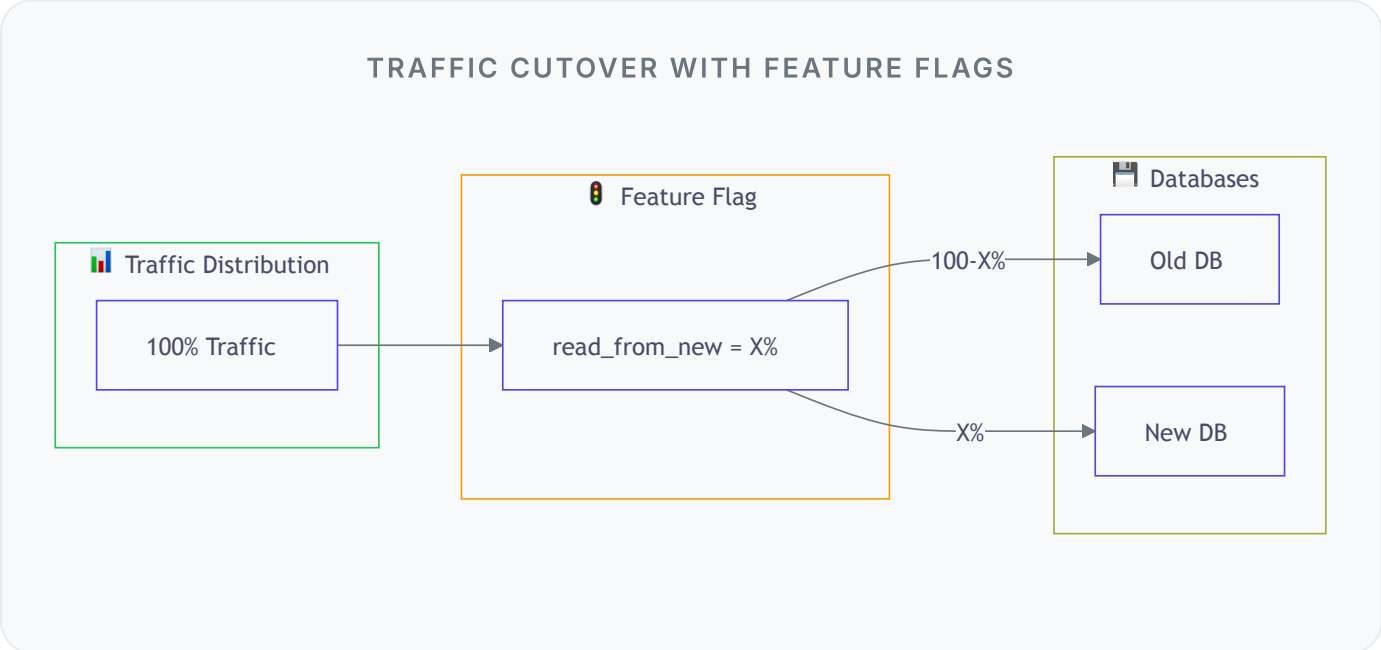
    if old_result ≠ transform_back(new_result):
        log_mismatch(user_id, old_result, new_result)

    return old_result  # Still return from old
```

Run shadow mode for days. Fix discrepancies. Build confidence.

Phase 4: Cutover

Gradually shift traffic to the new database.



Sold to
@theskilledc
kumar.dodda@gmail.com

```python
def get_user(user_id):
    if feature_flag("read_from_new", percentage=10):
        return new_db.get(user_id)
    else:
        return old_db.get(user_id)
```

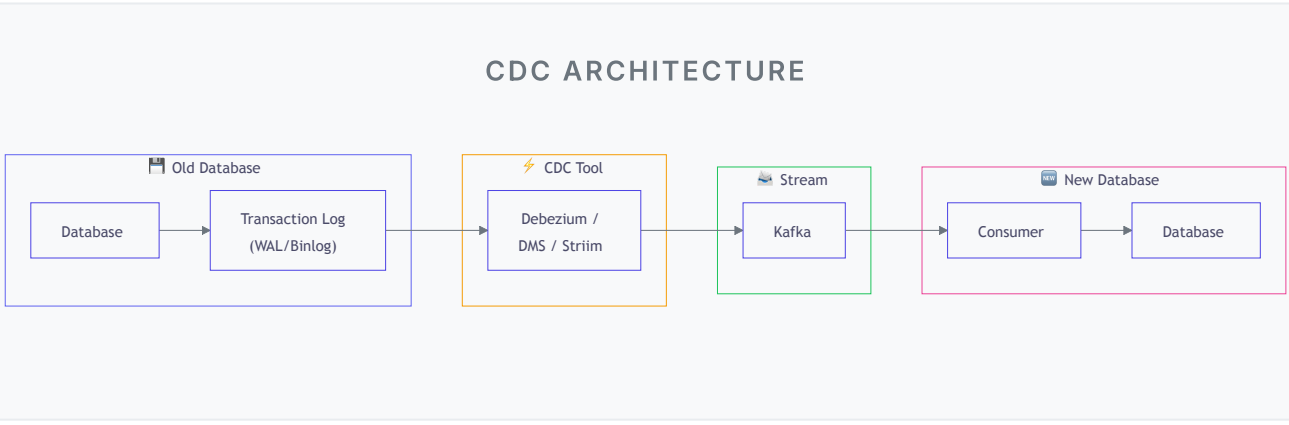
Rollout stages:

STAGE	% TO NEW DB	DURATION	WHAT TO WATCH
Canary	1%	1 day	Errors, latency, data issues
Early adopters	10%	3 days	Edge cases, load patterns
Half traffic	50%	1 week	Performance under load
Full migration	100%	-	Retire old database

Alternative: Change Data Capture (CDC)

For complex migrations, use **CDC** instead of dual-write.

CDC reads the database's transaction log and streams changes to the new system.



Benefits of CDC:

- No application code changes needed
- Captures all changes (including direct DB updates)
- Handles deletes properly
- Works across different database types

Tools: Debezium, AWS DMS, Google Datastream, Striim

Sold to
@theskilledc
kumar.dodda@gmail.com

Interview Pivot (How to Explain It)

If asked:

"How would you migrate 10TB of data with zero downtime?"

> "I'd use a dual-write pattern. First, update the application to write to both old and new databases. Then, backfill historical data in batches while dual-write captures ongoing changes.
>
> After backfill, I'd verify data integrity using shadow reads. Finally, I'd gradually shift read traffic using feature flags - 1%, 10%, 50%, 100% - watching for errors at each stage.
>
> For complex cases, I'd use CDC tools like Debezium to stream changes from the transaction log without modifying application code."

What's Next?

Data migrations lead to deeper challenges:

How do you handle schema changes during migration?

> Use a **transform layer** during dual-write. Old schema writes are transformed to new schema before writing to new DB. Reverse transform for reads.

What if dual-write causes inconsistencies?

> Always treat **old DB as source of truth** until cutover. Use eventual consistency. Run reconciliation jobs to fix drift.

How do you roll back a failed migration?

> **Keep dual-write active** until fully confident. Roll back = flip feature flag. Old DB has all the data. Never delete old DB until weeks after successful migration.

Sold to
@theskilledc
kumar.dodda@gmail.com

How do you handle foreign keys and relationships?

> **Migrate in dependency order**: parent tables first, then children. Disable FK constraints during backfill, re-enable after. Validate relationships at the end.

The insight: Zero-downtime migration isn't about moving data faster - it's about running two systems in parallel until you're confident enough to switch.

Dual-write for new data. Backfill for old. Verify before switching. Rollback if needed.

Same Principles Apply To

- **Cloud migrations (AWS to GCP)** - same dual-write + gradual cutover
- **Microservice extraction** - same strangler fig pattern
- **Schema evolution (large tables)** - same backfill + shadow read verification
- **API versioning transitions** - same parallel run with traffic shifting

Sold to
@theskilledc
kumar.dodda@gmail.com

How Do You Build a Live Streaming Platform for Millions of Viewers

Real-Time Video at Scale Is a Different Beast

A creator goes live. Within seconds, millions of viewers tune in simultaneously.

- A major sporting event
- A product launch
- A viral Twitch stream
- Breaking news coverage

Each viewer expects: low latency, high quality, zero buffering, instant start. And all of this while handling **millions of concurrent connections**.



The Scale

A single 1080p stream at 5 Mbps × 1 million viewers = 5 Petabits per second. That's 5,000 Terabits. No single server can handle that.

Why Live Streaming Is Uniquely Challenging

Unlike video-on-demand (Netflix, YouTube), live streaming has constraints that change everything:

CHALLENGE	VOD (NETFLIX)	LIVE STREAMING
Content	Pre-encoded, cached everywhere	Being created in real-time
Latency	Minutes of buffering OK	Seconds matter
Traffic	Gradual, predictable	Sudden spikes (millions in seconds)
Caching	Cache everything	Content expires in seconds

Sold to
@theskilledc
kumar.dodda@gmail.com

You can't pre-compute live content. You can't cache it for long. And millions of viewers all want the *same content at the same time*.

⚡ The Engineering Complexity

One camera, one million screens. The math doesn't add up without serious engineering:

📡 One-to-Many Fan-out

1 streamer → 1M viewers. Each viewer needs the same video, but each connection is unique.

⏱️ Latency vs Quality

Buffer more = higher quality, more delay. Buffer less = faster but choppy.

🌐 Global Distribution

Viewer in Tokyo, streamer in New York. 200ms network latency minimum.

📈 Spike Handling

Celebrity goes live. 0 to 500K viewers in 30 seconds. Scale or crash.

First Attempts (and Why They Break)

Attempt 1: Direct Connection from Broadcaster

The simplest model: viewers connect directly to the broadcaster's stream.

Why it feels right: Simple architecture. Lowest possible latency. No middlemen.

Why it fails: Broadcaster's upload bandwidth is limited (10-100 Mbps typical). At 5 Mbps per viewer, that's **2 viewers max**. The broadcaster's machine can't handle thousands of connections.

💡 **Lesson**

Broadcasters produce content. They can't distribute it.

Attempt 2: Single Media Server

Broadcaster sends to a server. Server distributes to all viewers.

Sold to
@theskilledc
kumar.dodda@gmail.com

Why it feels right: Broadcaster only needs one outbound connection. Server has more bandwidth.

Why it fails: One server = one location = high latency for distant viewers. Even a powerful server: 10 Gbps ÷ 5 Mbps = **2,000 viewers max**. Single point of failure.

> 💡 **Lesson**
>
> A single server can't serve millions. You need distributed infrastructure.

Attempt 3: Traditional CDN

Use a CDN to cache and distribute video files.

Why it feels right: CDNs are built for scale. Edge servers close to viewers. Works great for Netflix.

Why it's not enough for live: Traditional CDNs cache static files for hours. Live content changes every second. Cache-miss storms on every new segment.

> 💡 **Lesson**
>
> Static CDNs cache content. Live CDNs must relay content in real-time.

Attempt 4: WebRTC for Everything

"WebRTC enables peer-to-peer real-time video. Let's use it!"

Why it feels right: Sub-second latency. Built into browsers. Works for video calls.

Why it fails at scale: P2P mesh doesn't scale beyond ~50 participants. Signaling becomes a bottleneck. NAT traversal issues multiply at scale.

> ⚠️ **Lesson**
>
> WebRTC is for real-time communication, not broadcast to millions.

The Real Nature of the Problem

Live streaming has a magical property:

Sold to
@theskilledc
kumar.dodda@gmail.com

All million viewers want the same content at the same time.

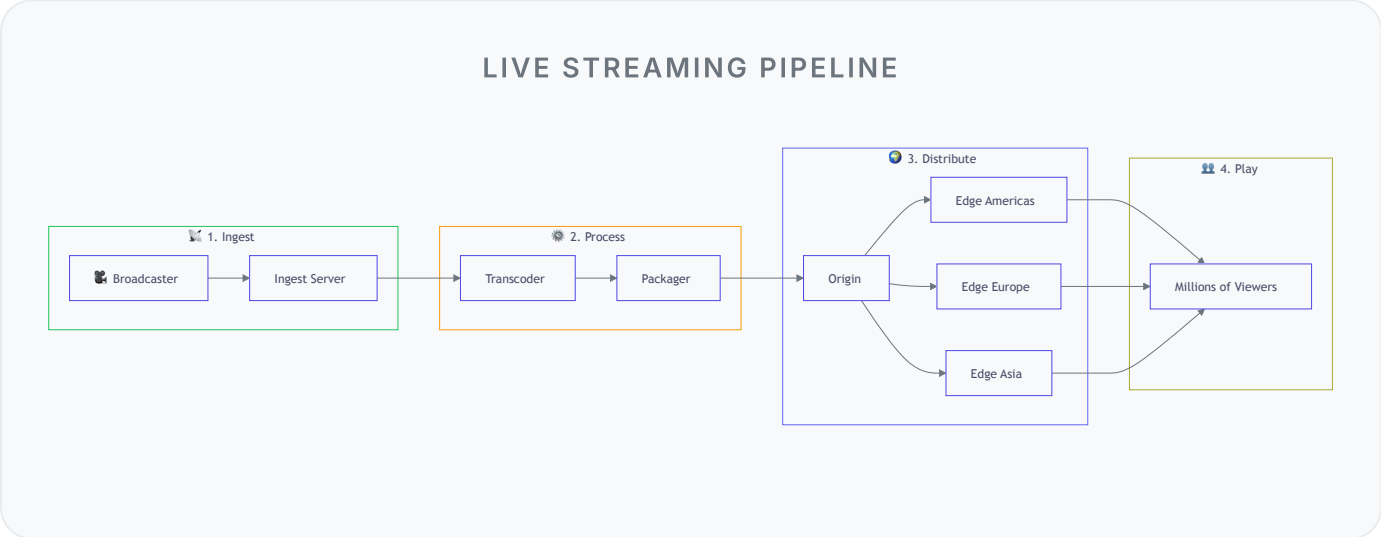
This is what makes it possible. Unlike video calls (everyone sees different content), live streaming is **one-to-many broadcast**. The same bytes go to everyone.

This tells us the design principles:

1. **Segment the stream** - Chop it into small, cacheable pieces
2. **Cache briefly but aggressively** - Even 2 seconds of caching collapses millions of requests
3. **Distribute globally** - Edge servers close to viewers
4. **Adapt to bandwidth** - Multiple quality levels for different connections

The Correct Architecture

A production live streaming platform has distinct stages:



Stage 1: Ingest - Receive the Stream

Broadcasters push their stream to an ingest server using:

- **RTMP** - Real-Time Messaging Protocol (legacy but ubiquitous)
- **SRT** - Secure Reliable Transport (better for unreliable networks)
- **WebRTC** - For browser-based broadcasting

Ingest servers are deployed in multiple regions. Broadcasters connect to the nearest one.

Stage 2: Transcode - Multiple Quality Levels

Not all viewers have the same bandwidth. Transcode into multiple bitrates:

Sold to
@theskilledc
kumar.dodda@gmail.com

QUALITY	RESOLUTION	BITRATE	TARGET
Source	1080p60	6 Mbps	Fiber/5G
High	720p30	2.5 Mbps	WiFi
Medium	480p30	1 Mbps	4G
Low	360p30	0.5 Mbps	Congested

This is called the **Adaptive Bitrate (ABR) ladder**. The player switches quality based on available bandwidth.

Stage 3: Package - Segment for HTTP Delivery

Video is chopped into small segments (2-6 seconds each) for HTTP delivery:

- **HLS** (HTTP Live Streaming) - Apple's protocol, most widely supported
- **DASH** (Dynamic Adaptive Streaming over HTTP) - Open standard

A manifest file tells the player which segments are available and at what quality. The player fetches segments one at a time, never downloading the whole stream.

Stage 4: Distribute - Global Edge Network

Segments are distributed to edge servers worldwide. Two strategies:

- **Push**: Origin actively sends to edges (lower latency)
- **Pull**: Edges fetch on first viewer request (more efficient)

Key insight: All million viewers requesting segment #42 get the *same bytes*. Even a 2-second cache at the edge collapses millions of requests into one origin fetch.

Stage 5: Playback - Adaptive Bitrate in the Player

The player monitors bandwidth and switches quality in real-time. If your connection degrades, it drops to 480p. If it improves, it upgrades to 1080p.

This happens seamlessly, mid-stream, without buffering.

The Latency-Scale Tradeoff

Different use cases need different latency:

Sold to
@theskilledc
kumar.dodda@gmail.com

USE CASE	ACCEPTABLE LATENCY	TECHNOLOGY
Video call	<200ms	WebRTC
Live auction / betting	<1 second	WebRTC / LL-HLS
Gaming (interactive)	2-5 seconds	Low-Latency HLS
Sports broadcast	5-15 seconds	Standard HLS
Entertainment	15-30 seconds	Standard HLS

Lower latency = smaller segments = more requests = harder to cache = more expensive.

Handling Millions of Concurrent Viewers

When a stream goes viral, viewer count can 10x in minutes. Here's how to handle it:

Request Coalescing

When 10,000 viewers request the same segment simultaneously:

- First request goes to origin
- Other 9,999 wait for that response
- All 10,000 get the same cached response

This is done at the edge. The origin sees one request, not 10,000.

Multi-CDN Strategy

Don't rely on one CDN. Use multiple for redundancy and capacity:

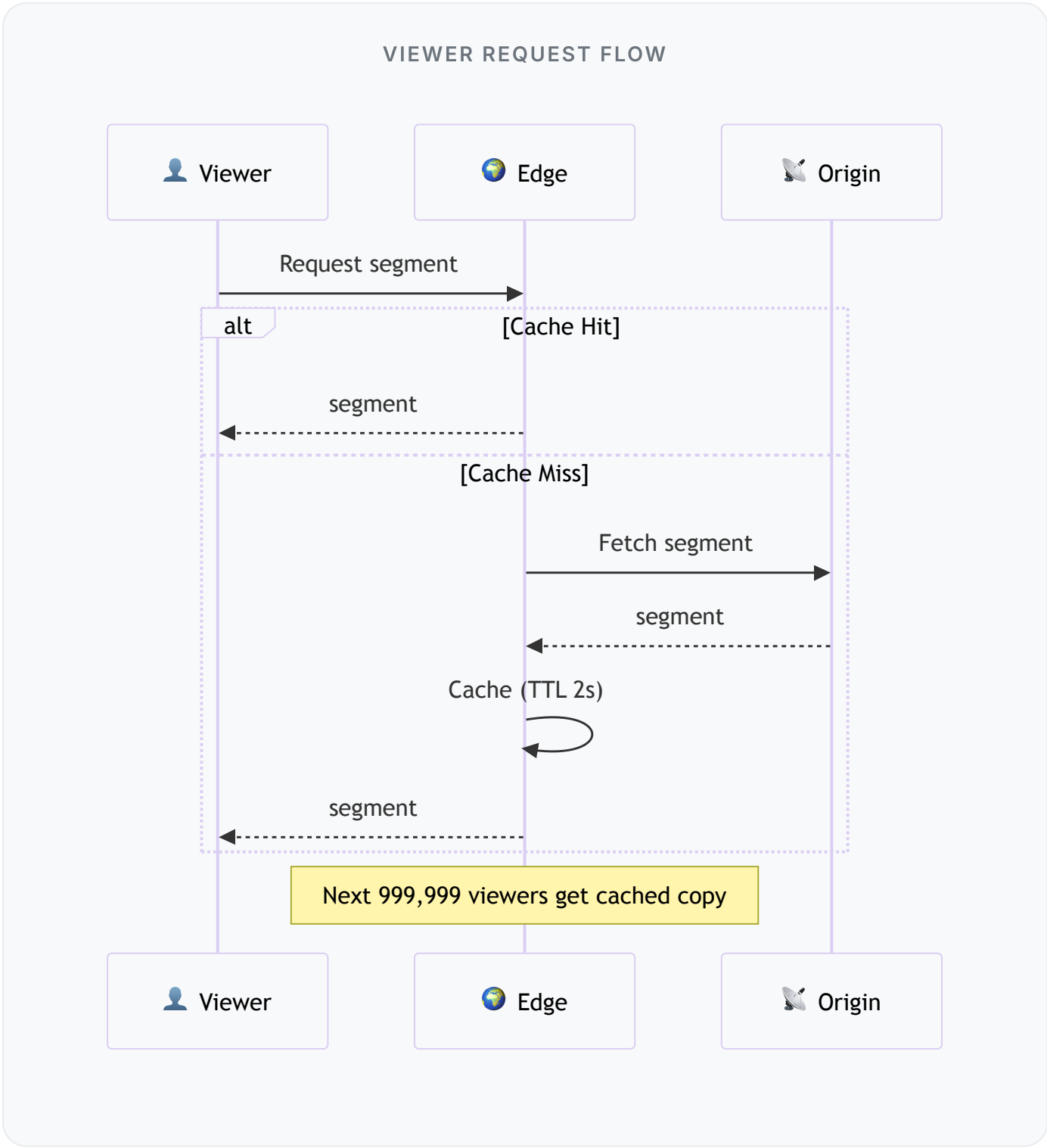
- CloudFront + Akamai + Fastly
- DNS or client-side load balancing
- Failover if one CDN has issues

Pre-warming for Scheduled Events

For scheduled events (Super Bowl, product launch):

- Pre-provision CDN capacity
- Warm up edge caches before the event
- Traffic engineering on standby

Sold to
@theskilledc
kumar.dodda@gmail.com



Interview Pivot (How to Explain It)

If asked:

"How would you design a live streaming platform for millions of viewers?"

> "The key insight is that all viewers want the same content at the same time. This makes caching possible even for live content.
>
> I'd design a pipeline: ingest via RTMP, transcode to multiple bitrates for adaptive streaming, segment into HLS chunks, and distribute via a global CDN. Even with 2-second TTLs, edge caching collapses millions of requests.

Sold to kumar.dodda@gmail.com

For scale, I'd use request coalescing at the edge and multiple CDNs for redundancy. The latency-scale tradeoff is explicit - smaller segments for lower latency, but harder to cache."

What's Next?

Live streaming leads to deeper challenges:

How do you implement live chat alongside the stream?

> **Separate system**: WebSocket connections to chat servers. Shard by stream ID. Rate limit messages. Delay chat to sync with video latency.

How do you enable DVR (rewind live streams)?

> **Persist segments** to object storage as they're created. Extend the manifest to include past segments. Set a DVR window (e.g., last 2 hours).

How do you handle broadcaster disconnection?

> **SRT protocol** for resilience. Show "reconnecting" slate. Buffer at origin. Allow seamless reconnection without new stream.

How do you insert ads in live streams?

> **Server-side ad insertion (SSAI)**: Splice ads at the origin. All viewers see ads. SCTE-35 markers signal ad breaks.

Sold to
@theskilledc
kumar.dodda@gmail.com

The insight: Live streaming at scale works because everyone wants the same content at the same time. This shared demand is what makes caching possible - even with content that expires in seconds.

Ingest. Transcode. Segment. Cache at the edge. That's how millions watch together.

🔁 **Same Principles Apply To**

- **Webinar platforms** - same segmentation and CDN distribution
- **Live sports betting** - same latency-scale tradeoffs
- **Live audio streaming (podcasts, radio)** - simpler encoding, same distribution
- **Software update distribution (OS patches)** - same CDN and edge caching patterns

Sold to
@theskilledc
kumar.dodda@gmail.com

How Would You Design a Real-Time Leaderboard for 100K Players

Rankings That Update Instantly - Without Breaking Your Database

Every competitive system needs a leaderboard:

- **Games** - Who's in the top 100?
- **Fitness apps** - Weekly step challenges
- **E-learning** - Quiz score rankings
- **Sales dashboards** - Rep performance

Users expect to see their rank update **instantly** when their score changes. They want to know:

- What's my current rank?
- Who's above me? Who's below?
- Who's in the top 10?

Simple for 100 players. Challenging for 100,000. Brutal for millions.

⚡ The Engineering Complexity

Sorting is O(n log n). Now do it a million times per second:

📊 Ranking at Scale

100M players. Each score update = re-rank everyone below. O(n) per update is deadly.

⚡ Real-Time Updates

Score changes every second. Users refresh constantly. Stale by 5 seconds = complaints.

🎮 Concurrent Writes

1000 players score simultaneously. Lock contention on the leaderboard kills throughput.

📈 Range Queries

'Show ranks 500-510' - that's not just top-K. It's arbitrary range on sorted data.

Sold to @theskilledc... kumar.dodda@gmail.com

First Attempts (and Why They Break)

Attempt 1: Store Scores in a Database and Sort

The most obvious idea.

- Store `(player_id, score)` in a table
- Query with `ORDER BY score DESC LIMIT 100`

```sql
SELECT player_id, score
FROM leaderboard
ORDER BY score DESC
LIMIT 100;
```

Why it feels right:

- Simple schema
- Easy to query
- Works perfectly for a few thousand players

Why it fails:

- Every score update is a write
- Every leaderboard view is a sort
- At high update rates, the DB becomes the bottleneck
- Sorting under concurrent writes kills latency

The database turns into a **hot contention point**.

> 💡 **Lesson**
>
> Databases are not built for high-frequency ranking.

Attempt 2: Recompute Leaderboard Every Few Seconds

Okay, don't query on every read.

- Every 5 seconds, recompute rankings
- Cache the top 100

Sold to @theskilledc kumar.dodda@gmail.com

- Serve reads from cache

Why it feels right:

- Fewer queries
- Predictable updates
- Easy to reason about

Why it fails:

- You recompute even if nothing changed
- Spikes cause recompute lag
- Rankings feel "stale" under fast gameplay
- High write amplification during bursts

Players expect **live feedback**, not delayed snapshots.

> 💡 **Lesson**
>
> Periodic recomputation doesn't scale with event velocity.

Attempt 3: Use Redis Sorted Sets

Now we're talking.

- Store players in a Redis ZSET
- Score = ZSET score
- Fetch top K instantly

```bash
# Add/update score
ZADD leaderboard 1500 "player:123"

# Get top 10
ZREVRANGE leaderboard 0 9 WITHSCORES
```

Why it feels right:

- O(log N) updates
- O(K) reads
- Fast and battle-tested

Why it fails at 100K+ with high churn:

Sold to
@theskilledc
kumar.dodda@gmail.com

- Hot shards when many players update frequently
- Single Redis instance becomes a bottleneck
- Cross-region or global leaderboards get messy
- Persistence and recovery are tricky

Redis helps, but it's **not the full solution by itself**.

> 💡 **Lesson**
>
> Fast data structures don't eliminate architectural limits.

Attempt 4: Just Scale Redis Bigger

Add more memory. Bigger boxes. Replicas.

Why it feels right:

- Hardware solves problems... until it doesn't
- Easy short-term win

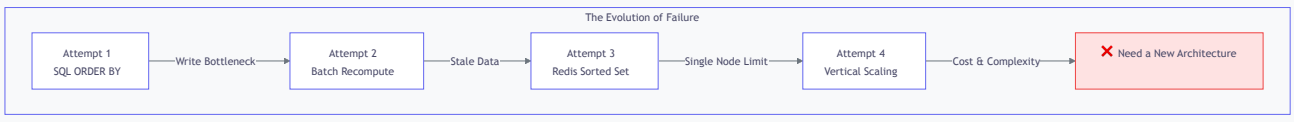
Why it fails:

- Writes still need coordination
- Sharding ranked data is non-trivial
- Failures become more expensive
- Costs grow faster than traffic

You're **buying time**, not fixing the design.

> ⚠️ **Lesson**
>
> Vertical scaling postpones the problem.

WHY EACH APPROACH FALLS SHORT



Sold to
@theskilledc
kumar.dodda@gmail.com

The Real Nature of the Problem

A real-time leaderboard is **not about sorting**.

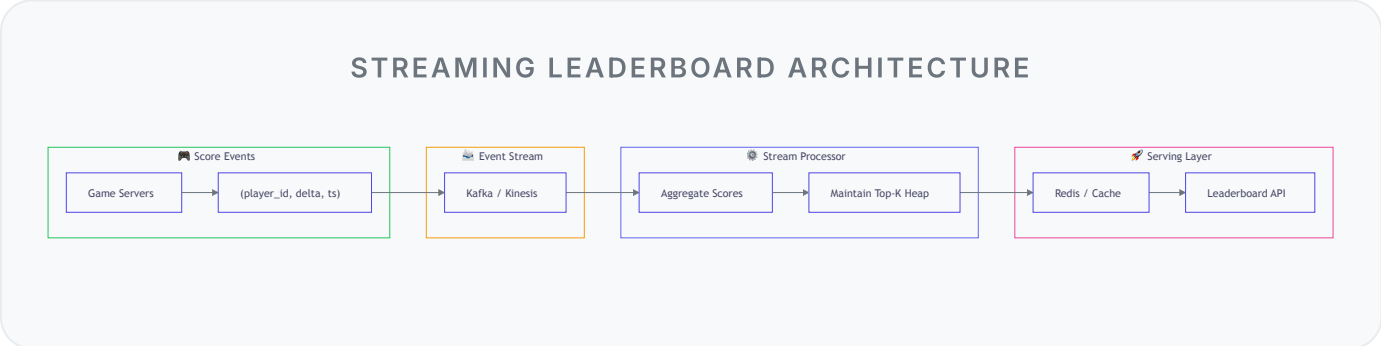
> **It's about handling a continuous stream of score updates and maintaining a Top-K view efficiently.**

That tells us two things:

1. Scores change frequently
2. Only a small fraction of players actually matter for the leaderboard

So we design around **events**, not tables.

The Correct Architecture (Streaming Mindset)



Step 1: Treat Score Updates as Events

Every action that affects score emits an event:

```json
{
    "player_id": "player_123",
    "delta": +50,
    "timestamp": 1703012345678
}
```

- No immediate ranking
- No global locks
- Just record the change

This keeps the **write path fast and scalable**.

Step 2: Stream Score Updates

Sold to @theskilledc kumar.dodda@gmail.com

Send these events into a stream:

- Kafka
- Pulsar
- Kinesis

Why?

- Handles bursts naturally
- Preserves order per player
- Allows parallel processing

At this stage, we are **capturing reality**, not computing rankings yet.

Step 3: Maintain Incremental Scores

A stream processor consumes events and maintains:

- Current score per player
- In-memory state or fast KV store

Instead of recalculating everything, it:

- Adds deltas
- Updates only affected players

```python
# Stream processor logic
def process_event(event):
    player_id = event["player_id"]
    delta = event["delta"]

    # Increment score (not replace)
    current = scores.get(player_id, 0)
    new_score = current + delta
    scores[player_id] = new_score

    # Update Top-K if necessary
    update_topk(player_id, new_score)
```

No full scans. No re-sorting.

Step 4: Maintain a Live Top-K Structure

Here's the key insight:

Sold to
@theskilledc
kumar.dodda@gmail.com

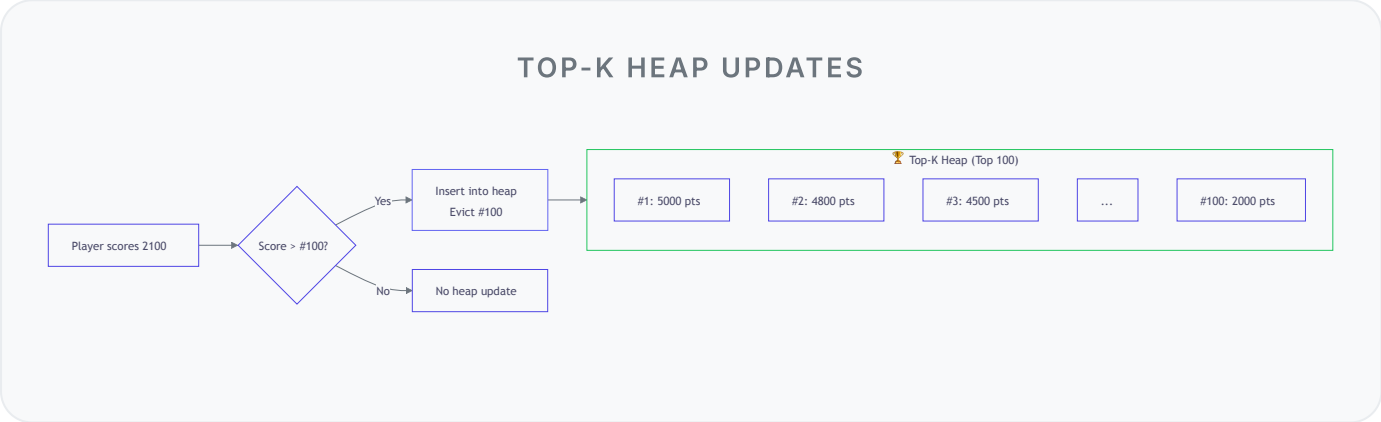
You do not need to rank all 100K players all the time.

You maintain:

- A Top-K heap (say top 100 or 1,000)
- Update it only when a score change crosses thresholds

Most players never touch the leaderboard.

This keeps updates cheap and predictable.



Step 5: Serve Reads from Cache

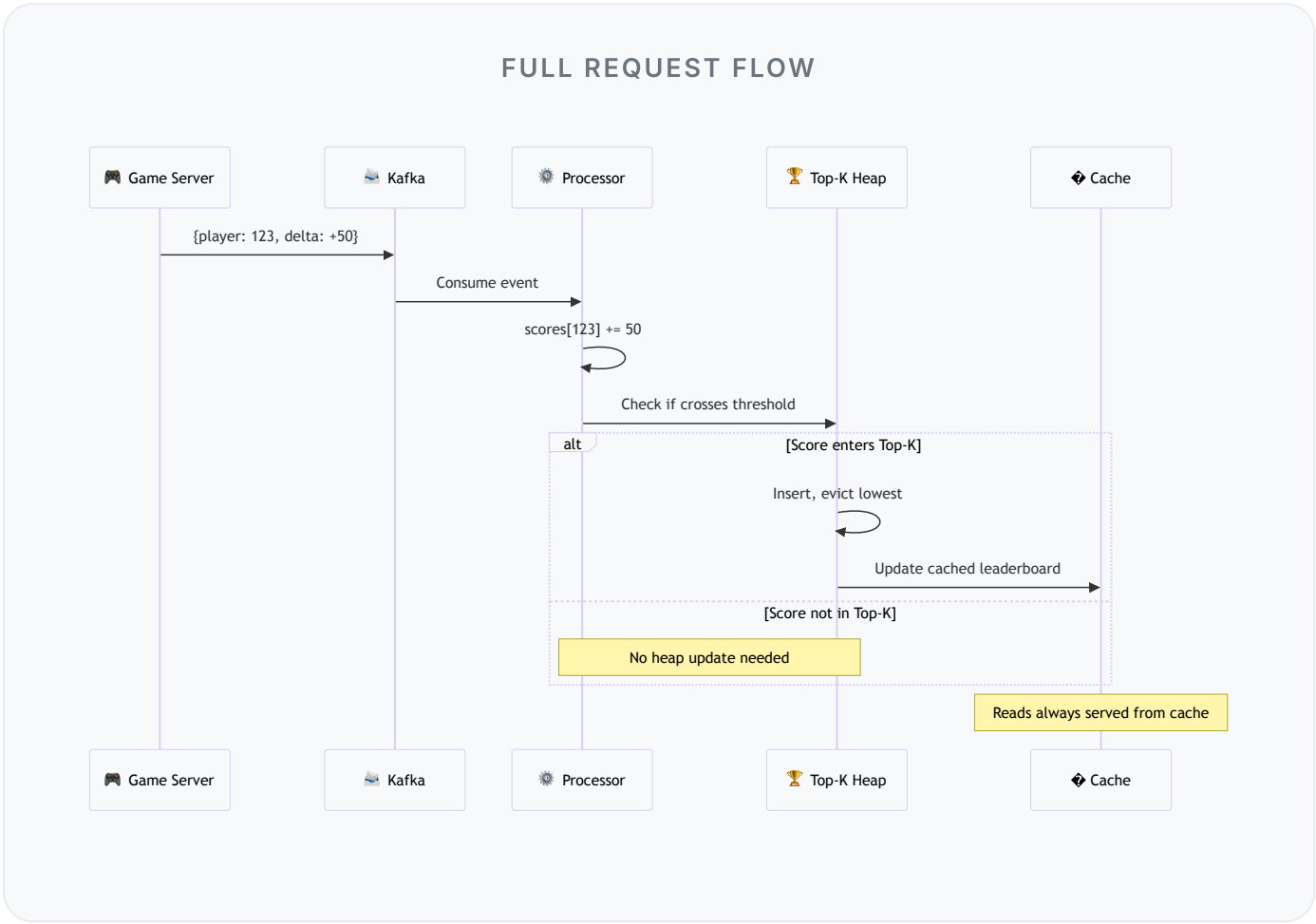
The current leaderboard is stored in:

- Redis
- In-memory cache
- Fast KV store

Reads are constant-time. No computation. No locks.

Even if the stream lags briefly, users see the last consistent leaderboard.

Sold to
@theskilledc
kumar.dodda@gmail.com



Why This Works at Scale

This design works because:

PROPERTY	BENEFIT
Score updates are processed once	No redundant computation
Ranking is incremental, not global	O(log K) not O(N log N)
Writes don't block reads	Consistent read latency
Only top players incur ranking cost	Work proportional to output

The system stays fast even when gameplay explodes.

100K players stop being scary.

Interview Pivot (How to Explain It)

If asked:

"How would you design a real-time leaderboard for 100K players?"

Sold to
@theskilledc
kumar.dodda@gmail.com

> "I'd treat score updates as a stream, aggregate them incrementally, maintain a Top-K structure instead of global sorting, and serve reads from cache. That avoids recomputation and scales with update rate."

That answer shows **systems thinking**, not tool memorization.

What's Next?

Real leaderboards raise follow-ups:

How do you prevent cheating?

> **Server-side validation**: Never trust client-submitted scores. Anomaly detection for impossible scores. Manual review for top positions. Easy rollback with score history.

How do you handle ties?

> **Composite score**: Include timestamp as tiebreaker - earlier achievement ranks higher. Or use secondary metrics like games played.

How do you shard by region?

> **Separate streams per region**. Regional leaderboards are independent. Global leaderboard merges regional Top-Ks periodically.

How do you reset seasons safely?

> **Archive old leaderboard** before reset. Use versioned keys (leaderboard:season:3). Clear stream offsets. Announce scheduled maintenance.

Sold to
@theskilledc
kumar.dodda@gmail.com

The insight: A real-time leaderboard isn't about sorting - it's about processing a stream of events and maintaining just enough state to answer Top-K queries.

Stream the updates. Aggregate incrementally. Serve from cache. That's the formula.

Same Principles Apply To

- **Fantasy sports rankings** - same streaming aggregation with Top-K
- **Sales dashboards** - same real-time metric computation
- **Contest leaderboards (coding, trivia)** - same event-driven score updates
- **Achievement systems (badges, XP)** - same incremental state management

Sold to
@theskilledc
kumar.dodda@gmail.com

How Would You Design a URL Shortener That Handles 500 Million Redirects per Day

The Classic System Design Interview - Done Right

URL shorteners are everywhere:

- **bit.ly** - The OG
- **t.co** - Twitter's shortener
- **tinyurl.com** - Still kicking
- **Your company's internal link tracker**

The concept is simple: turn a long URL into a short one. But at **500 million redirects per day**, the design becomes interesting.

⚡ The Engineering Complexity

Seven characters. Billions of URLs. Zero collisions. Here's the math problem:

🔢 ID Collision

7-char code = 62^7 = 3.5 trillion combos. But random generation can still collide.

📖 Read-Heavy

1 write creates a short URL. Then 10,000 reads. 99.99% read workload.

🌐 Global Redirects

User in India clicks a link. Redirect should be <50ms. Can't call New York every time.

📊 Analytics Overhead

Track every click: who, when, where, device. That's metadata on every redirect.

Sold to
@theskilledc
kumar.dodda@gmail.com

📊 **The Numbers**

500M redirects/day = ~5,800 redirects/second average, ~30,000/second at peak. Reads vastly outnumber writes (100:1 ratio typical).

First Attempts (and Why They Break)

Attempt 1: Auto-Increment ID + Base62 Encoding

The simplest approach: use the database's auto-increment ID, convert to Base62.

Why it feels right: Simple, no collisions, sequential IDs are efficient.

Why it fails: IDs are **predictable**. Users can enumerate all URLs by incrementing the short code. Security risk - competitors can scrape your entire link database. Single database = single point of failure, can't scale writes.

💡 **Lesson**

Sequential IDs leak information and create scaling bottlenecks.

Attempt 2: Hash the Long URL

Hash the URL with MD5 or SHA, take the first N characters.

Why it feels right: Deterministic - same URL always gets same short code. No database needed for generation. Feels random.

Why it fails: **Collisions**. 7 hex characters = 268 million possibilities. With 500 million URLs, collisions are guaranteed (birthday paradox). Same URL → same code, but what if users want different short links for tracking?

💡 **Lesson**

Hashing without collision resolution doesn't scale.

Attempt 3: Random String + Collision Check

Generate random strings, check if they exist in the database.

Why it feels right: Unpredictable codes. 7 Base62 characters = 3.5 trillion possibilities.

Sold to @theskilledc kunalsodde@gmail.com

Why it fails at scale: Database read before every write. Race conditions - two servers generate the same code simultaneously. As database fills, collision retries increase.

> 💡 **Lesson**
>
> Random generation with collision checks creates coordination overhead.

Attempt 4: Distributed Counter + Range Allocation

Pre-allocate ID ranges to each server. Server 1 gets 1-1M, Server 2 gets 1M-2M, etc.

Why it's better: No coordination during normal operation. Each server generates unique IDs independently.

Why it's still limited: Range exhaustion requires coordination. Server failure wastes allocated range. IDs still somewhat predictable within ranges.

> ⚠️ **Lesson**
>
> Range allocation works but adds centralized dependency for range management.

The Real Nature of the Problem

A URL shortener has a fundamental asymmetry:

OPERATION	FREQUENCY	LATENCY REQUIREMENT
Create short URL	~5M/day (1%)	Can be slower
Redirect to long URL	~500M/day (99%)	Must be instant

The system is overwhelmingly read-heavy. Optimize for reads.

This tells us the design principles:

1. **Writes can be slower** - users wait a moment to create a link
2. **Reads must be instant** - redirects happen millions of times
3. **Cache aggressively** - short codes are immutable after creation
4. **ID generation must be distributed** - no single point of failure

Sold to @theskilledc kumar.dodda@gmail.com

The Correct Architecture

Principle 1: Distributed ID Generation (Snowflake)

Use Twitter's Snowflake algorithm to generate unique IDs without coordination.

A Snowflake ID is 64 bits:

- **41 bits**: Timestamp (milliseconds) - 69 years of unique IDs
- **5 bits**: Datacenter ID - 32 datacenters
- **5 bits**: Worker ID - 32 machines per datacenter
- **12 bits**: Sequence - 4,096 IDs per millisecond per worker

Each server generates unique IDs independently. **No central coordination.** IDs are roughly time-ordered but not predictable.

Principle 2: Base62 Encoding for Short Codes

Convert the numeric ID to a URL-safe string using Base62 (a-z, A-Z, 0-9).

CODE LENGTH	POSSIBLE CODES	ENOUGH FOR
6 characters	56 billion	Most use cases
7 characters	3.5 trillion	Large scale
8 characters	218 trillion	Enterprise

7 characters is plenty for most systems. Short, readable, and collision-free when backed by Snowflake IDs.

Principle 3: Aggressive Caching

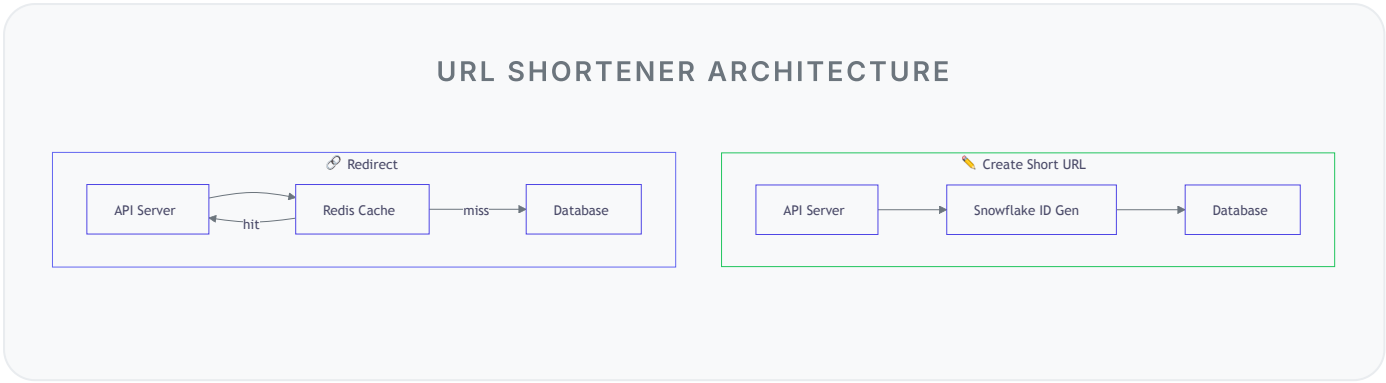
With 90% cache hit rate, the database sees 10x less traffic.

Why caching works so well for URL shorteners:

- Short codes are **immutable** - once created, they never change
- Popular links are clicked repeatedly
- Recent links are more likely to be clicked
- Long tail of old links rarely accessed

Use Redis with 24-hour TTL. Cache miss → fetch from DB → populate cache → return.

Sold to
@theskilledc
kumar.dodda@gmail.com



Principle 4: Database Sharding

Shard by short code hash for even distribution. Each shard handles a subset of codes.

Why shard by short code (not by long URL)?

- Redirects are by short code - that's the hot path
- Even distribution across shards
- No need to know which shard at creation time

Adding Analytics Without Slowing Redirects

URL shorteners often track clicks. But at 500M/day, you can't write to the database on every click.

Solution: Fire-and-forget to a queue

- Redirect happens immediately - user isn't blocked
- Click event goes to Kafka asynchronously
- Consumers aggregate in batch (hourly/daily counts)
- Store aggregates in analytics database

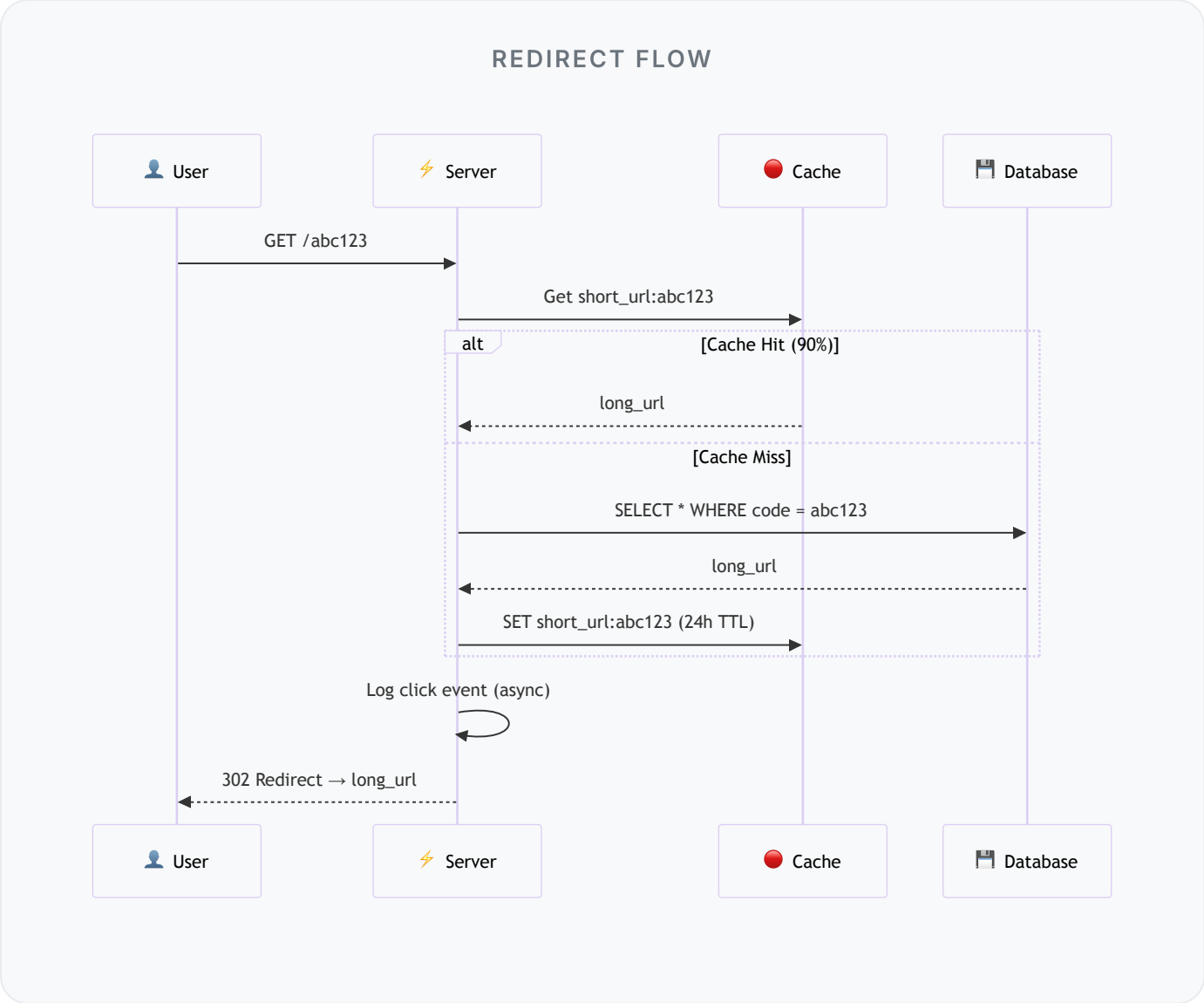
The redirect path stays fast. Analytics processing happens separately.

301 vs 302: A Critical Decision

STATUS	MEANING	BROWSER BEHAVIOR	USE WHEN
301	Permanent Redirect	Caches redirect, bypasses server next time	SEO, permanent links
302	Temporary Redirect	Always hits server	Analytics, A/B testing

Sold to
@theskilledc
kumar.dodda@gmail.com

For analytics, use 302. With 301, the browser caches the redirect and bypasses your server entirely after the first click. You lose visibility.



Interview Pivot (How to Explain It)

If asked:

"How would you design a URL shortener for 500 million redirects per day?"

> "First, I'd recognize this is a read-heavy system - 99% redirects, 1% creates. So I'd optimize for reads.
>
> For ID generation, I'd use Snowflake to generate unique IDs without coordination, then Base62 encode for short codes. This avoids collisions and single points of failure.
>
> For redirects, I'd cache aggressively in Redis. Short codes are immutable, so caching is perfect. With 90% cache hit rate, the database sees 10x less load.

Sold to @theskilledc kumar.dodda@gmail.com

> Analytics would be fire-and-forget to Kafka, processed async. I'd use 302 redirects to ensure every click hits the server for tracking."

What's Next?

URL shorteners lead to deeper questions:

How do you handle custom short codes?

> **Check availability** before allowing. Reserve popular words. Store in same table with a "custom" flag.

How do you prevent abuse (spam, phishing)?

> **Rate limiting** per IP/user. URL reputation checks (Google Safe Browsing). Report/block functionality. Expiration for untrusted sources.

How do you handle URL expiration?

> **TTL field** in database. Background job cleans expired URLs. Return 410 Gone for expired links. Recycle short codes after grace period.

How do you ensure high availability?

> **Multi-region deployment**. DNS-based load balancing. Database replication. Graceful degradation - serve redirects even if analytics fail.

Sold to
@theskilledc
kumar.dodda@gmail.com

The insight: A URL shortener is a key-value store with a smart key generation strategy. The challenge isn't the shortening - it's handling 500M reads efficiently with caching and sharding.

Snowflake for IDs. Base62 for encoding. Redis for caching. Async for analytics.

Same Principles Apply To

- **Paste/Code sharing services (Pastebin)** - same ID generation and caching
- **CDN URL signing** - same read-heavy, immutable content pattern
- **QR code generators** - same short code + redirect architecture
- **Deep linking services** - same redirect with analytics pattern

Sold to
@theskilledc
kumar.dodda@gmail.com

How Do You Aggregate Logs from 10,000 Servers Without Data Loss

Building Reliable Observability Infrastructure

Your infrastructure has grown. You now have:

- 10,000 servers across multiple datacenters
- Hundreds of microservices
- Each service logging 1,000+ events per second
- Critical debugging data you *cannot* afford to lose

When an incident happens at 3 AM, you need those logs. All of them.

> 📊 **The Numbers**
>
> 10,000 servers × 1,000 logs/sec = 10 million logs per second. At 1KB per log, that's 10 GB/second, 864 TB/day.

The question isn't "how do we collect logs?" - it's "how do we collect logs **without losing any**, even when things go wrong?"

⚡ The Engineering Complexity

1000 servers. 1 million logs/second. One search box. The plumbing is the product:

🌊 Ingestion Rate

1000 servers × 1000 logs/sec = 1M logs/sec. Your pipeline is the bottleneck.

⏱️ Time Ordering

Logs arrive out of order. Server A's log at t=5 arrives after Server B's log at t=10.

Sold to @theskilledc kumar.dodda@gmail.com

🔍 Query Patterns

'Show me all errors in the last hour' - that's 3.6B logs to scan.

💾 Retention Cost

Keep logs 90 days for compliance. 90 days × 86TB/day = 7.7PB. That's $$$.

First Attempts (and Why They Break)

Attempt 1: Direct HTTP to Centralized Logger

Each application sends logs directly to a logging service via HTTP.

Why it feels right: Simple to implement. Centralized control. Real-time ingestion.

Why it fails: HTTP in the hot path slows down the application. If the logging service is down, logs are **dropped**. 10,000 servers hammering one endpoint = overload.

> 💡 **Lesson**
>
> Synchronous logging creates coupling that breaks under load.

Attempt 2: Write to Local Files, Tail with Agent

Applications write to local files. A log agent (Filebeat, Fluentd) tails and ships them.

Why it feels right: Decoupled - app doesn't know about log shipping. Buffering on local disk. Standard pattern, well-tooled.

Why it still loses data: Agent crashes → unread logs pile up, may be rotated away. Disk fills up → new logs can't be written. Network partition → buffer overflows. No acknowledgment - agent doesn't know if backend received the log.

> 💡 **Lesson**
>
> Local buffering helps, but without acknowledgments, you're guessing.

Attempt 3: Push to Kafka, Then Process

Use Kafka as a durable buffer. Agents push to Kafka, consumers index to Elasticsearch.

Sold to @theskilledc
kumar.dodda@gmail.com

Why it's better: Kafka provides durability and replay. Decouples ingestion from processing. Can handle bursts.

Why it still loses data: If the agent crashes before Kafka acks, logs are lost. Memory buffers in the agent can overflow. Consumer crashes mean you need careful offset tracking.

> ⚠️ **Lesson**
>
> Kafka helps, but data loss happens at the edges - between app and Kafka.

Attempt 4: At-Least-Once with Acknowledgments

Configure everything for at-least-once: `acks=all` in Kafka, persistent queues in agents.

Why it's better: Logs confirmed before deletion from buffer. Kafka replication ensures durability.

Why it's still not enough: Still vulnerable at the "first mile" - app to disk. Unflushed OS buffers lost on crash. Need end-to-end acknowledgment, not just Kafka ack.

> 💡 **Lesson**
>
> At-least-once is necessary but not sufficient. You need defense in depth.

The Real Nature of the Problem

Log aggregation has a cruel irony:

> **Logs are most valuable during incidents - exactly when they're most likely to be lost.**

During normal operation, everything works. But when an incident happens:

- Log volume spikes 10-100x
- Services are dying (including logging infrastructure)
- Networks are congested
- Disks are filling up

This tells us the design principles:

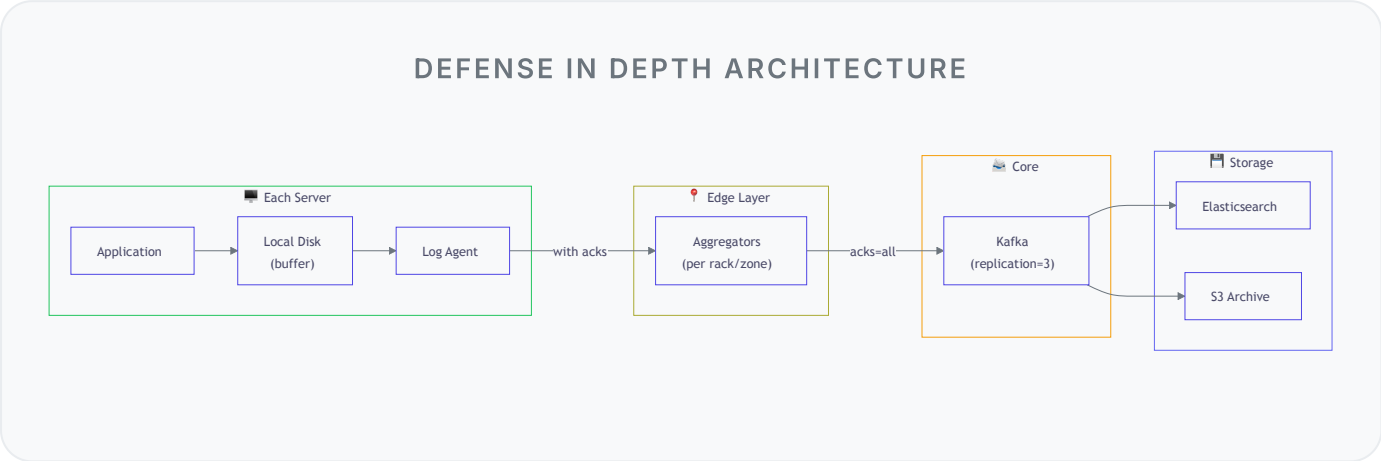
1. **Every layer must buffer** - absorb bursts at each stage

Sold to @theskilledc kumar.dodda@gmail.com

2. **Acknowledge before delete** - only forget data when next layer confirms receipt
3. **Handle 10x normal load** - because that's when you need logs most
4. **Fail independently** - one component failure shouldn't lose logs

The Correct Architecture: Defense in Depth

Zero data loss requires protection at **every stage** of the pipeline.



Layer 1: Application to Disk (The First Mile)

The most vulnerable point. Logs must hit disk before the app moves on.

- **Write to local disk, never directly over network**
- **Use append-only log files** - fast sequential writes
- **For critical logs, fsync** - force OS buffer to disk
- **Rotate by size, retain enough for agent to catch up**

The application's job is to get logs onto local disk as fast as possible. Everything else is the agent's problem.

Layer 2: Agent with Persistent Disk Buffer

The agent reads from log files and maintains its own durable buffer.

- **Disk-backed buffer** - survives agent restarts
- **Track file position** - never re-read or skip lines
- **Retry with backoff** - if downstream is unavailable
- **Only delete after ack** - downstream must confirm receipt

Popular agents like Fluent Bit and Vector support persistent disk buffers. This is the difference between "works most of the time" and "zero data loss."

Sold to
@theskilledc
kumar.dodda@gmail.com

Layer 3: Edge Aggregators (Reduce Fan-in)

10,000 servers can't all connect directly to Kafka. The connection count alone kills it.

Instead, deploy **aggregators per rack or zone**:

- 100 servers → 1 aggregator
- 100 aggregators → Kafka
- Aggregators batch, compress, and buffer
- Local to the servers they collect from

This reduces Kafka connections from 10,000 to 100, and adds another buffer layer.

Layer 4: Kafka with Strong Durability

Kafka is the **durable buffer** at the core. Configure for maximum reliability:

- **acks=all** - wait for all in-sync replicas before acknowledging
- **min.insync.replicas=2** - at least 2 copies before ack
- **replication.factor=3** - survive 2 broker failures
- **enable.idempotence=true** - prevent duplicates from retries

With this configuration, a log is only acknowledged when it exists in at least 2 replicas.

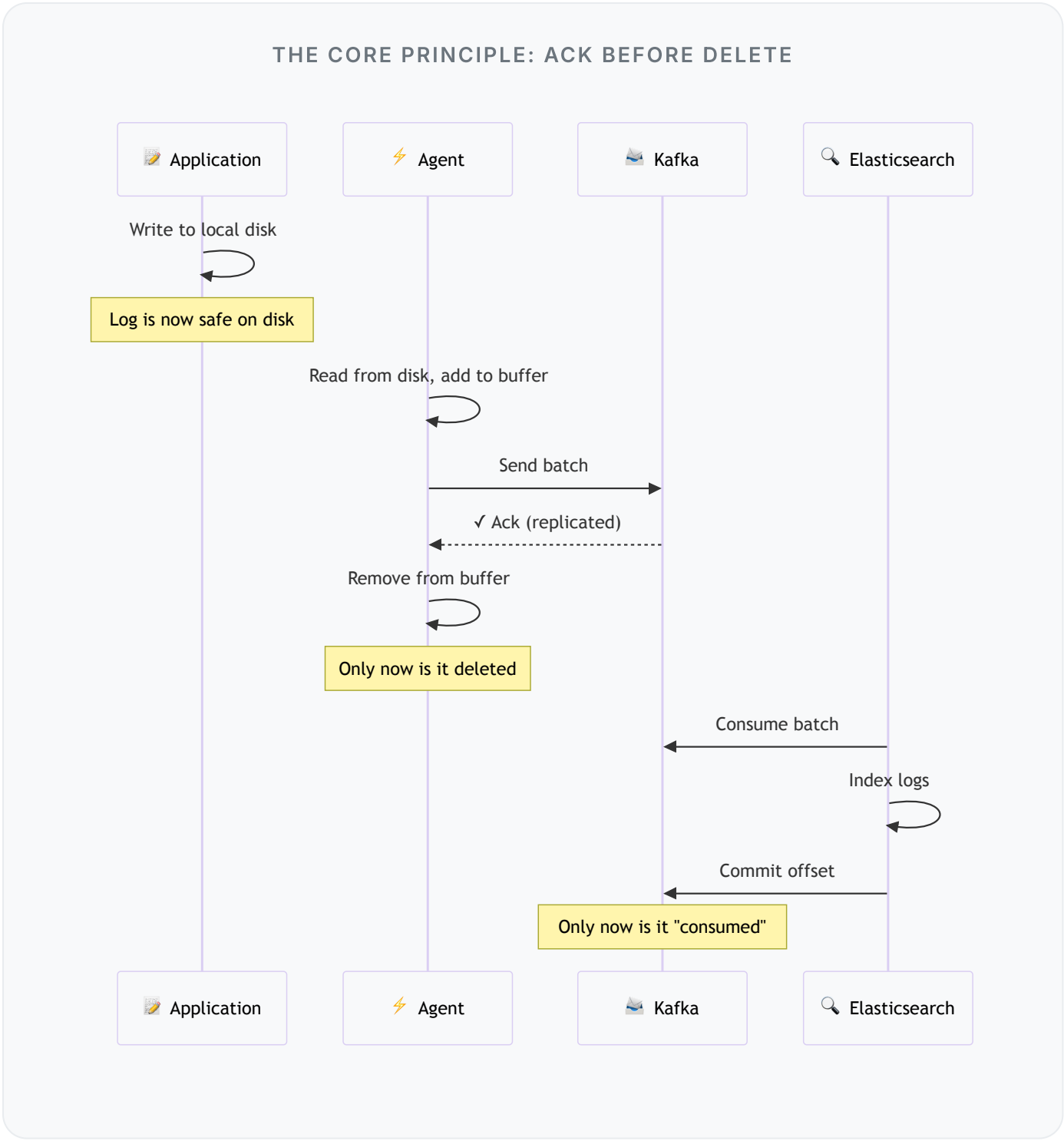
Layer 5: Consumer with Checkpointing

Consumers pull from Kafka and write to Elasticsearch (or S3, etc.).

- **Manual offset commits** - only commit after successful processing
- **Idempotent writes** - handle duplicates gracefully
- **Dead letter queue** - for poison messages that can't be processed

If the consumer crashes, it restarts from the last committed offset. No data lost.

Sold to
@theskilledc
kumar.dodda@gmail.com



Handling Traffic Bursts

During incidents, log volume can spike 10-100x. Every layer must absorb this:

Sold to
@theskilledc
kumar.dodda@gmail.com

LAYER	BURST PROTECTION
Application	Sample debug logs during overload, always keep errors
Agent	Large disk buffer (hours of capacity)
Aggregator	Spill to local disk when downstream is slow
Kafka	Provisioned for peak, retention for replay
Consumer	Auto-scale, can catch up after burst subsides

Every layer should be able to absorb 10 minutes of peak traffic in its buffer.

Interview Pivot (How to Explain It)

If asked:

"How would you aggregate logs from 10,000 servers without data loss?"

> "The key insight is that logs are most valuable during incidents - exactly when systems are under stress. So every layer must handle 10x normal load.
>
> I'd use defense in depth: apps write to local disk first. Agents with persistent disk buffers ship to edge aggregators, which batch and forward to Kafka with acks=all and replication.
>
> The core principle is 'ack before delete' - every layer only removes data after the next layer confirms receipt. This creates an end-to-end guarantee."

What's Next?

Log aggregation leads to deeper questions:

How do you handle multi-line logs (stack traces)? ?

> **Multiline parsing** in the agent. Match patterns like "starts with timestamp" to group lines. Or use structured logging (JSON) to avoid the problem.

Sold to @theskilledc kumar.dodda@gmail.com

How do you manage log retention cost-effectively?

> **Tiered storage**: Hot (Elasticsearch, 7 days), Warm (S3, 90 days), Cold (Glacier, years). Aggregate older logs to reduce volume.

How do you correlate logs across services?

> **Trace IDs**: Propagate correlation ID in every request. Include in all log entries. Query by trace ID to see full request path.

How do you handle sensitive data in logs?

> **Redaction at the edge**: Scrub PII in the agent before shipping. Use patterns to detect credit cards, emails, etc.

The insight: Zero data loss isn't about one magic component - it's about acknowledgments at every stage. Each layer only forgets data after the next layer confirms receipt.

Write to disk. Buffer locally. Ack before delete. Handle 10x bursts.

Same Principles Apply To

- **Metrics collection (Prometheus, Datadog)** - same buffering and ack patterns
- **Event streaming pipelines** - same defense-in-depth architecture
- **Audit log systems** - same zero-loss requirements
- **Distributed tracing (Jaeger, Zipkin)** - same agent-to-collector pattern

Sold to
@theskilledc
kumar.dodda@gmail.com

How Would You Design a Tagging System That Handles 100 Million Tagged Posts

Many-to-Many Relationships at Scale

Tags are everywhere:

- **Social media** - #travel, #photography, #foodie
- **E-commerce** - Filter by "wireless", "bluetooth", "noise-cancelling"
- **Content platforms** - Categories, topics, labels
- **Internal tools** - Jira labels, GitHub issues

The operations look simple:

- Add tags to a post
- Get all posts with a specific tag
- Get all tags for a post
- Find posts matching multiple tags (AND/OR)

But at **100 million posts** with an average of 5 tags each, that's 500 million tag associations. The naive approaches collapse.



The Scale

Popular tags like #love have 2 billion+ posts on Instagram. Querying "all posts with #travel" can't scan billions of rows.

First Attempts (and Why They Break)

Attempt 1: Comma-Separated Tags in a Column

Store tags as a string: `"travel,beach,sunset"` in a column on the posts table.

Why it feels right: Simple schema, no joins, easy to read and update.

Sold to @theskilledc kumar.dodda@gmail.com

Why it fails: Querying "all posts with #travel" requires `LIKE '%travel%'` - a full table scan. No index can help. Updating a tag name means rewriting millions of rows.

> 💡 **Lesson**
>
> Denormalized strings prevent efficient querying.

Attempt 2: Classic Many-to-Many Join Table

The textbook approach: a junction table with `(post_id, tag_id)` pairs.

Why it feels right: Normalized, no redundancy, tags are entities, easy to rename.

Why it struggles: "Get all posts with #travel" returns 10 million rows to join. Pagination with `ORDER BY created_at` becomes expensive. Multi-tag queries (AND/OR) require complex joins.

> 💡 **Lesson**
>
> Joins work, but large result sets kill performance.

Attempt 3: Inverted Index in the Database

Store post IDs per tag as an array: `travel → [1, 2, 45, 892, ...]`

Why it feels right: One row per tag, fast lookup, similar to search engines.

Why it fails: Popular tags have arrays with millions of IDs. Updating the array is expensive (read-modify-write). Can't paginate or sort within the array. Databases aren't optimized for huge array operations.

> 💡 **Lesson**
>
> Inverted indexes belong in search engines, not SQL columns.

Attempt 4: Just Use Elasticsearch

Index posts in Elasticsearch with tags as a field. Query with term filters.

Why it's better: Built for this - inverted indexes, fast filtering, scales horizontally. Multi-tag queries are natural.

Sold to
@theskilledc
kumar.dodda@gmail.com

Why it's not complete: Elasticsearch is eventually consistent - not ideal as primary storage. Expensive to update documents frequently. You still need a source of truth.

> ⚠️ **Lesson**
>
> Elasticsearch is a secondary index, not a database replacement.

⚡ The Engineering Complexity

#TaylorSwift has 10M posts. #MyLunch has 3. Same system, wildly different problems:

🔥 Hot Tags

#WorldCup gets 10M posts. #MyRandomTag gets 3. Same table, wildly different access patterns.

🔍 Tag Search

'Show all posts with #travel' - that's a join across millions of posts.

📝 Many-to-Many

Each post has N tags. Each tag has M posts. M × N relationships explode fast.

⚡ Real-Time Feed

'New posts with #breaking' - subscription to a firehose of updates.

The Real Nature of the Problem

Tagging has a fundamental asymmetry:

- **Few tags per post** - typically 5-10
- **Many posts per tag** - popular tags have millions

And tags follow a **power law**:

Sold to @theskilledc kumar.dodda@gmail.com

TAG TYPE	POST COUNT	FREQUENCY
Hot (#love, #photography)	10M+	Few tags
Warm (#coffeelover)	10K-10M	Some tags
Cold (#mycat123)	<10K< /td>	Most tags

A few tags contain most of the posts. You can't treat all tags the same.

This tells us the design principles:

1. **Optimize both directions** - posts→tags and tags→posts need different indexes
2. **Treat hot tags specially** - precompute feeds for popular tags
3. **Use the right tool for each query** - DB for simple, search engine for complex
4. **Cache aggressively** - tag queries are highly cacheable

The Correct Architecture

Principle 1: Two Tables for Two Directions

"Get all tags for post X" and "Get all posts for tag Y" are fundamentally different queries. Optimize with **two tables**:

- **post_tags**: Clustered by `post_id` → fast "get tags for this post"
- **tag_posts**: Clustered by `tag_id + timestamp` → fast "get recent posts for this tag"

When a post is tagged, write to both tables. Read from whichever matches your query direction. This is **denormalization for read performance**.

Principle 2: Precomputed Feeds for Hot Tags

For a tag with 10 million posts, even an indexed query is expensive. Instead:

- Maintain a **precomputed feed** in Redis for hot tags
- When a post is tagged with #travel, push its ID to the Redis list
- Keep only the most recent 10,000 posts (who scrolls further?)
- Reads are O(1) - just read from the list

This is the same pattern used for Twitter timelines and Instagram feeds.

Sold to
@theskilledc
kumar.dodda@gmail.com

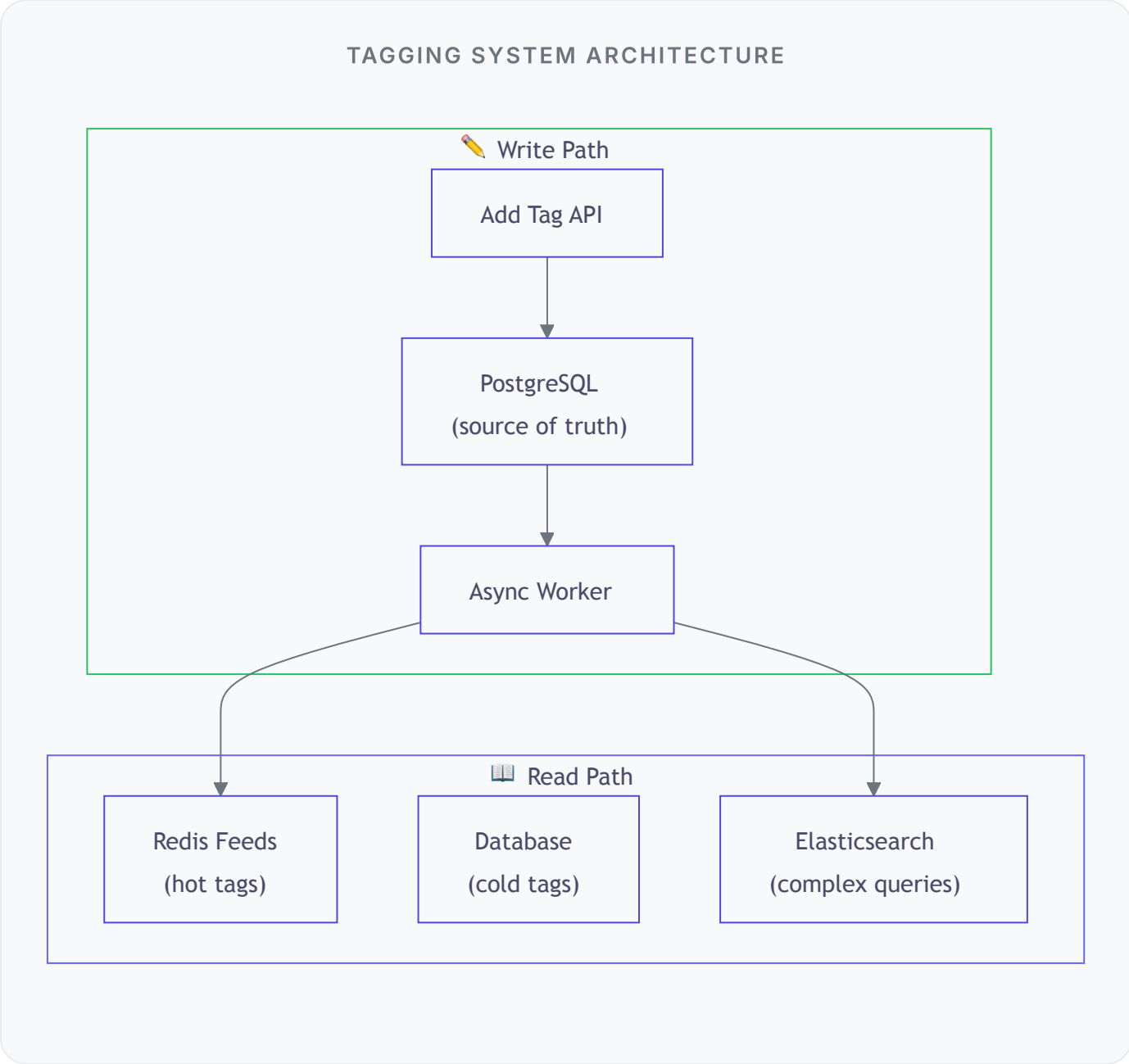
Principle 3: Elasticsearch for Complex Queries

"Show me posts with #travel AND #photography from last week" is painful in SQL.

Elasticsearch handles this naturally:

- Posts are indexed with their tags as a field
- Term filters on tags, date range filters, scoring - all efficient
- Sync from the database asynchronously

The database is the source of truth. Elasticsearch is a read replica optimized for querying.



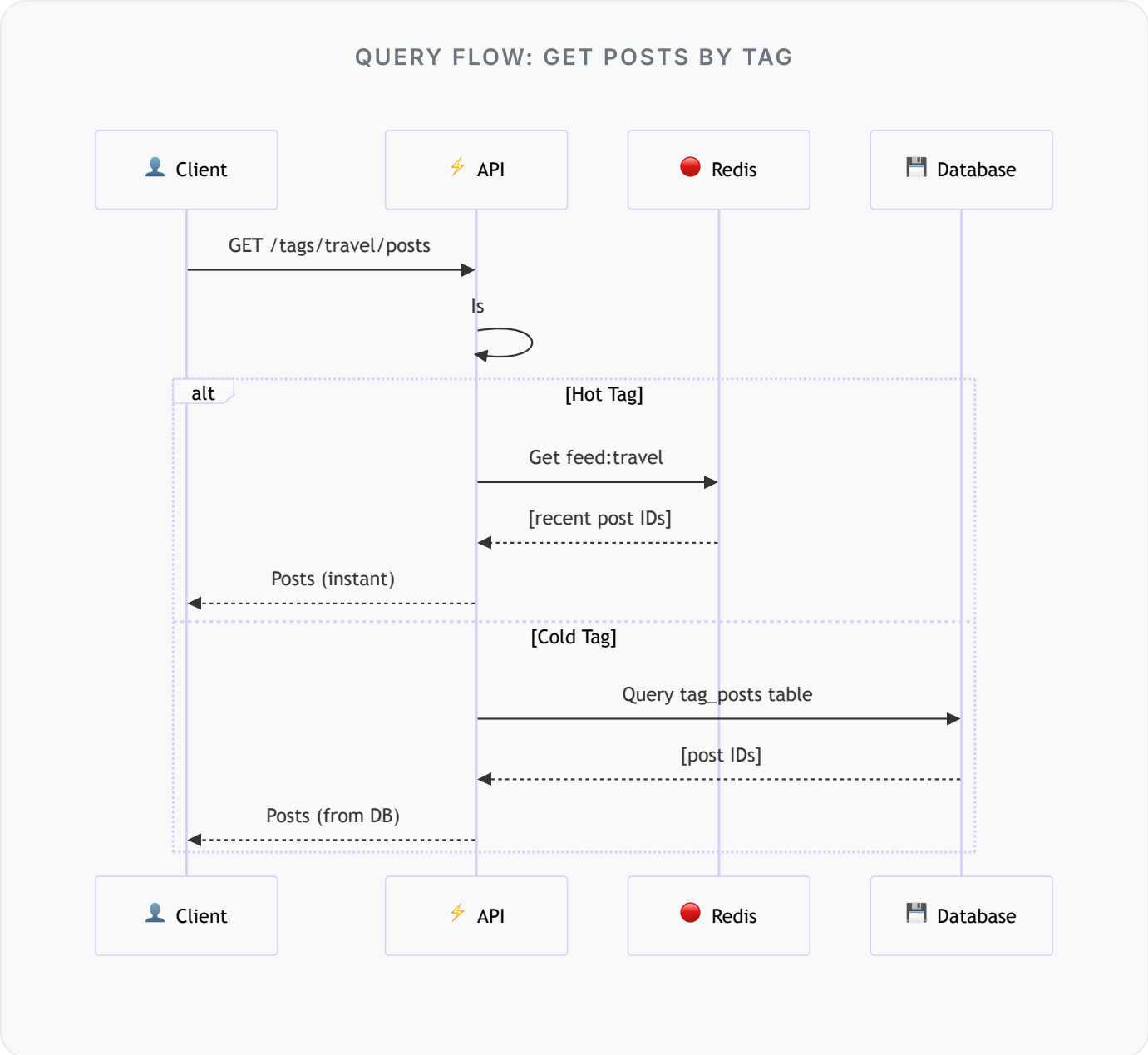
Principle 4: Normalize on Write, Cache on Read

Users type #Travel, #TRAVEL, #travel - these should be the same tag.

Sold to
@theskilledc
kumar.dodda@gmail.com

Normalize on write: Convert to lowercase, strip whitespace, limit length. Store the canonical form. Display the original if needed.

Cache on read: Tag queries are highly repetitive. Cache results aggressively with short TTLs (seconds to minutes).



Interview Pivot (How to Explain It)

If asked:

"How would you design a tagging system for 100 million posts?"

> "First, I'd recognize that tags follow a power law - a few tags have millions of posts, most have few. You can't treat them the same.

Sold to
@theskilledc
kumar.dodda@gmail.com

For storage, I'd use two tables optimized for different access patterns: one clustered by post for 'get tags', one clustered by tag for 'get posts'. Denormalization for read performance.

For hot tags, I'd maintain precomputed feeds in Redis - just a list of recent post IDs. For complex queries like multi-tag AND/OR, I'd use Elasticsearch as a secondary index.

The database remains the source of truth. Everything else is derived for query optimization."

What's Next?

Tagging systems lead to deeper questions:

How do you handle tag suggestions/autocomplete?

> **Prefix trie** or Elasticsearch prefix query. Cache popular tags. Rank by usage count. Personalize based on user history.

How do you detect and handle spam tags?

> **Rate limiting** per user. Flag unusual tag velocity. Blocklist known spam patterns. Human review for new trending tags.

How do you implement tag hierarchies?

> **Closure table** pattern: store all ancestor-descendant pairs. Or path-based: "electronics/phones/iphone". Query by prefix.

How do you show trending tags?

> **Sliding window counts**: Track tag usage in last hour/day. Compare to baseline. Spike = trending. Redis sorted sets for real-time ranking.

Sold to
@theskilledc
kumar.dodda@gmail.com

The insight: Tagging is a many-to-many relationship with extreme skew. A few tags contain most of the posts. Design differently for hot vs cold tags, and use the right tool for each query pattern.

Two tables for two directions. Precompute hot paths. Search engine for complex queries.

🔁 **Same Principles Apply To**

- **E-commerce product filters** - same power-law distribution, same hot/cold optimization
- **Email labels (Gmail)** - same many-to-many with precomputed feeds
- **Bookmark/favorites systems** - same bidirectional query needs
- **Skills/expertise tagging (LinkedIn)** - same normalization and search patterns

Sold to
@theskilledc
kumar.dodda@gmail.com

How Would You Store and Display Edit History for Millions of Posts

Version Control for User Content

Edit history is everywhere:

- **Wikipedia** - Every edit, forever
- **Google Docs** - Revision history with restore
- **Twitter/X** - "Edited" indicator with history
- **Notion, Confluence** - Page version history
- **Git** - The gold standard

Users expect:

- See what changed and when
- Know who made each change
- Restore previous versions
- Compare any two versions (diff)

At **millions of posts** with frequent edits, storage and retrieval become interesting challenges.



The Scale

Wikipedia has 60M+ articles with 1B+ total revisions. Storing full content for each revision would be 100+ petabytes.

📍 THE INTERVIEW ROOM

Interviewer: "Design edit history for Wikipedia-scale content."

Junior Dev: "Easy! Store a copy of the content every time someone edits."

Interviewer: "A 10KB article. Edited 500 times. That's 5MB per article."

Sold to @theskilledc kunal.kadam@gmail.com

```
Junior Dev: "Okay... just store the diffs then!"

Interviewer: "Nice. Now show me version 347."

Junior Dev: "Sure, let me apply 346 diffs sequentially..."

Interviewer: "The user is waiting."

Junior Dev: "..."

Interviewer: "Git solved this 20 years ago." 😉
```

⚡ The Engineering Complexity

Version history sounds simple until you realize every design choice has painful trade-offs:

💾 Storage Explosion

Full snapshots = 100x redundant data. That 60M articles × 500 avg edits = petabytes of mostly identical bytes.

🔄 Reconstruction Cost

Pure diffs = O(n) reconstruction. Version 500 means applying 499 patches. That's seconds, not milliseconds.

🧩 Diff Complexity

Text diffs are easy. Rich text? HTML? Images embedded in docs? Each needs different algorithms.

📊 Access Pattern Mismatch

99% of reads want the current version. 1% want history. Optimizing for both simultaneously is tricky.

First Attempts (and Why They Break)

Attempt 1: Store Full Content on Every Edit

The simplest approach: every time someone edits, insert a new row with the complete content.

Why it feels right: Simple to implement. Easy to retrieve any version. No complex diff logic.

Why it fails: A 10KB post edited 100 times becomes 1MB. Multiply by millions of posts, and you're storing 100TB of mostly **duplicate data**. Most edits change only a few words.

Sold to @theskilledc kumar.dodda@gmail.com

> 💡 **Lesson**
>
> Full snapshots waste storage on unchanged content.

Attempt 2: Store Diffs Only

Instead of full content, store only what changed between versions - the delta.

Why it feels right: Massive storage savings (often 90%+). Efficient for small edits. Git uses this.

Why it's tricky: To reconstruct version 100, you need to start from version 1 and apply 99 diffs in sequence. That's slow. Worse: if any diff is corrupted, all subsequent versions break.

> 💡 **Lesson**
>
> Pure diffs save storage but make reads expensive.

Attempt 3: Reverse Diffs from Current

Store the current version in full. To get older versions, store reverse diffs that undo each change.

Why it's better: The most common case (view current) is instant - O(1). Older versions are still reconstructable by walking backwards.

Why it's still limited: Viewing old versions still requires walking the chain. Every new edit means updating the reverse diff chain. Complex for binary or rich content.

> 💡 **Lesson**
>
> Optimize for the common case (current version), but history access still suffers.

Attempt 4: Snapshot + Diffs Hybrid

Store a full snapshot every N versions (say, every 10). Between snapshots, store diffs.

Why it's smarter: Bounded reconstruction - at most 9 diffs to apply. Balance between storage and read speed. If one segment corrupts, only that segment is affected.

This is the foundation of the correct approach.

The Real Nature of the Problem

Sold to
@theskilledc
kumar.dodda@gmail.com

Edit history has two very different workloads:

- **Writes** - frequent, small, incremental
- **Reads** - infrequent, but expensive when they happen

Most users:

- Read the latest version
- Rarely open edit history

Moderators:

- Inspect diffs
- Roll back versions

> **The system must: keep the latest read path fast, store history cheaply, and make history readable when needed - not always.**

This tells us the design principles:

1. **Current version must be O(1)** - no reconstruction, store it separately
2. **History storage should be compact** - diffs dominate, snapshots are checkpoints
3. **History retrieval can be slower** - it's a rare path
4. **Older history can be even slower** - tier to cheaper storage

The Correct Architecture

A production edit history system is built on these principles:

Principle 1: Separate Current from History

The current version lives in the main `posts` table. It's always the latest, always complete, always fast to read.

History lives in a separate `post_history` table. When a user edits, we:

1. Save the old version to history (as snapshot or diff)
2. Update the main table with the new content

This separation means reading the current post never touches history at all.

Principle 2: Periodic Snapshots with Diffs Between

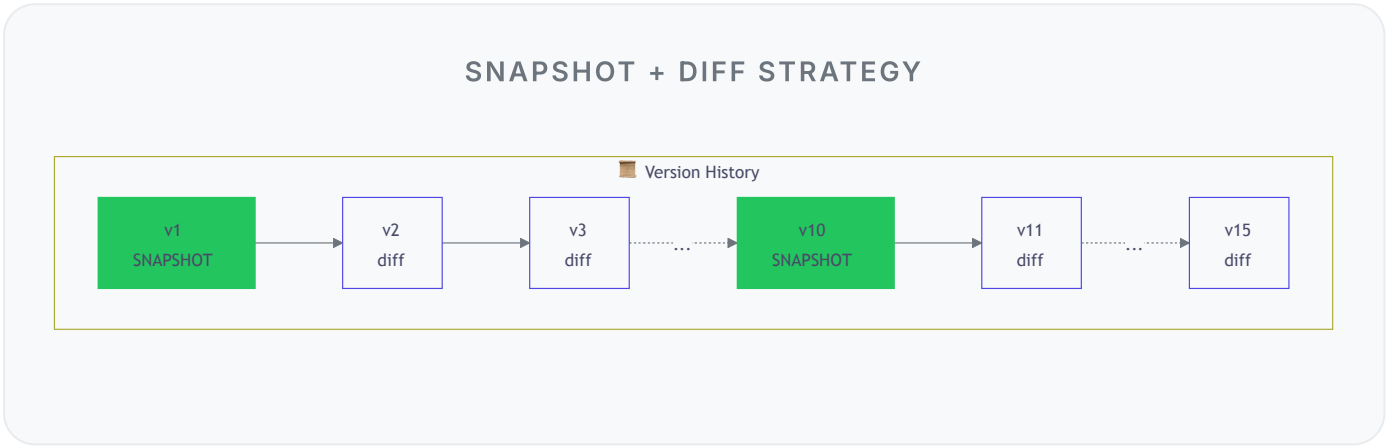
Sold to @theskilledc kumar.dodda@gmail.com

Every Nth version (e.g., 10), store a full snapshot. Between snapshots, store only diffs.

To reconstruct any version:

1. Find the nearest snapshot at or before that version
2. Apply diffs forward to reach the target version

Example: To get version 15, load snapshot at v10, then apply diffs for v11, v12, v13, v14, v15. Five operations max, not 15.



Principle 3: Tier Storage by Age

Not all history is equally valuable:

TIER	AGE	STORAGE	ACCESS SPEED
Hot	Last 30 days	Primary database	Fast (milliseconds)
Warm	30-365 days	S3 with DB metadata	Medium (seconds)
Cold	1+ years	S3 Glacier	Slow (minutes)

A background job migrates old history to cheaper tiers. When someone requests ancient history, we fetch from cold storage - slower, but rare.

Principle 4: Choose Diff Algorithm by Content Type

Different content needs different diff strategies:

- **Plain text** → Line-based diff (Myers algorithm, like Git)
- **Rich text / HTML** → DOM tree diff
- **JSON / Structured data** → JSON Patch (RFC 6902)
- **Binary files** → Full snapshots (or specialized tools like xdelta)

Sold to
@theskilledc
kumar.dodda@gmail.com

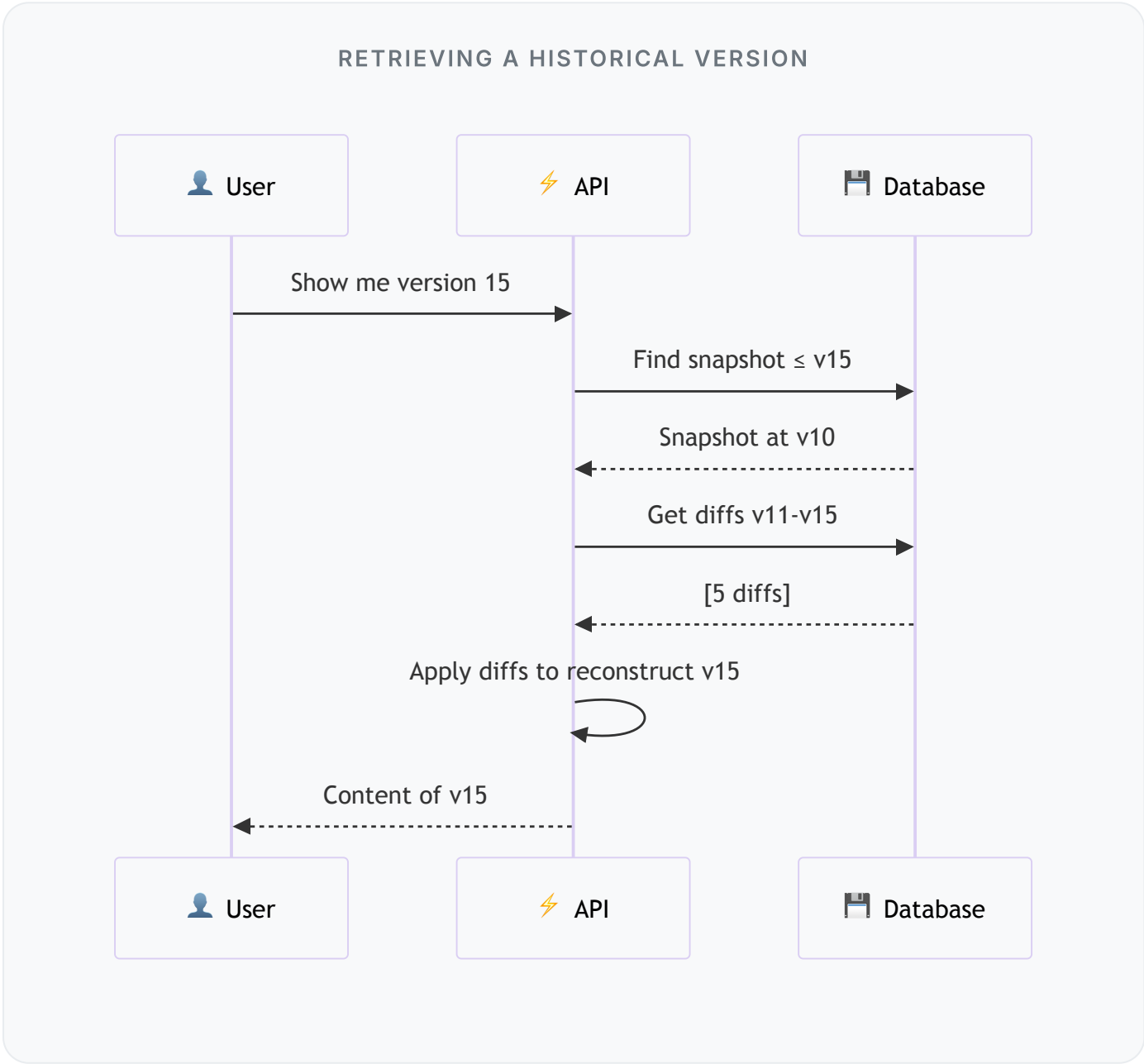
Wikipedia uses line-based diffs. Google Docs uses operational transforms on rich text. Choose based on your content type.

Displaying History to Users

Users don't want to see raw diffs. They want visual comparisons:

- **Inline diff** - Strikethrough deleted text, highlight added text
- **Side-by-side** - Old on left, new on right
- **Unified** - Single view with +/- markers

Key insight: Compute visual diffs on-demand when the user requests them. Don't store rendered HTML diffs - they're expensive to store and may need to change if the display format changes.



Interview Pivot (How to Explain It)

Sold to @theskilledc kumar.dodda@gmail.com

If asked:

"How would you store and display edit history for millions of posts?"

> "First, I'd recognize that edit history has asymmetric workloads - writes are frequent but reads are rare. Most users only see the current version.
>
> So I'd separate current content (main table, always fast) from history (separate table, compact storage). For history, I'd use a snapshot + diff hybrid: full snapshot every N versions, diffs in between. This bounds reconstruction cost while saving 90%+ storage.
>
> For old history, I'd tier to cold storage since it's rarely accessed. Visual diffs are computed on-demand, not stored."

What's Next?

Edit history leads to deeper questions:

How do you handle concurrent edits? ?

> **Optimistic locking**: Include version in update, reject if stale. Or **Operational Transform / CRDTs** for real-time collaboration.

How do you handle rollback? ?

> **Restore creates a new version**, don't delete history. Reconstruct the old content and save it as a new edit. Audit trail preserved.

How do you handle embedded media? ?

> **Separate versioning** for images/attachments. Store references, not copies. Link post version to asset versions.

Sold to @theskilledc
kumar.dodda@gmail.com

How do you comply with "right to be forgotten"?

?

> **Redaction, not deletion**: Replace PII with placeholders in all versions. Maintain audit log. Legal retention may require keeping non-PII metadata.

The insight: Edit history is about access patterns. Current content is read constantly; history is read rarely. Design for both - fast current reads, compact history storage, lazy reconstruction.

Separate current from history. Snapshot periodically. Diff in between. Tier by age.

Same Principles Apply To

- **Wiki revision history (Wikipedia)** - same snapshot + diff approach
- **Document version control (Google Docs)** - same lazy reconstruction
- **Database audit logs** - same tiered storage for compliance
- **Configuration change tracking** - same append-only history pattern

Sold to
@theskilledc
kumar.dodda@gmail.com

How Would You Design Instagram Stories That Expire After 24 Hours

Ephemeral Content at Massive Scale

Stories changed social media:

- **Instagram Stories** - 500+ million daily users
- **Snapchat** - The originator
- **WhatsApp Status** - 450+ million daily users
- **LinkedIn, Twitter, YouTube** - Everyone copied it

The core mechanics:

- User posts a photo or video
- Visible to followers for exactly 24 hours
- Then it disappears forever
- Viewers see a chronological tray of active stories

Sounds simple. But at **500 million daily active users**, each following hundreds of accounts, the system must handle billions of story views per day.



The Scale

500M users × avg 20 stories viewed/day = 10 billion story views/day. Millions of stories expire every hour and must be cleaned up.



THE INTERVIEW ROOM

Interviewer: "Design Instagram Stories."

Junior Dev: "Easy! Just add a timestamp and delete after 24 hours."

Interviewer: "Kylie Jenner posts a story. She has 400 million followers."

Sold to @theskilledc
kumar.dodda@gmail.com

Junior Dev: "Okay, so we push to all followers..."

Interviewer: "400 million writes. Per story. She posts 10 stories a day."

Junior Dev: "Um... pull on read then?"

Interviewer: "User follows 500 accounts. Check each one on app open?"

Junior Dev: "..."

Interviewer: "Now you see the problem." 😏

⚡ The Engineering Complexity

Building ephemeral content at this scale isn't just "add a TTL and you're done." Here's what makes it genuinely hard:

📤 Massive Fan-Out

One celebrity posts → 100M followers need to see it. Push to all? Impossible. Pull by all? Database dies.

⏰ Time-Based Expiration

Millions of stories expire every hour. Deleting them all without impacting reads is a scheduling nightmare.

📖 Read-Heavy Workload

10 billion views/day. Every user opens the app and expects their story tray instantly. That's ~115K reads/second.

👥 Celebrity Problem

You can't treat @KylieJenner the same as @RandomUser123. The asymmetry breaks every naive approach.

First Attempts (and Why They Break)

Attempt 1: Query All Followed Users on Every Request

When a user opens the app, query the database: "For each of my 500 followees, do they have a story posted in the last 24 hours?"

Why it feels right: Always fresh data. Simple logic. No precomputation.

Sold to
@theskilledc
kumar.dodda@gmail.com

Why it fails: User follows 500 people → 500 queries (or one massive join). 500M users opening the app → billions of queries hitting the database. **Latency explodes.** Database melts.

> 💡 **Lesson**
>
> Fan-out on read doesn't scale for high-traffic, many-to-many relationships.

Attempt 2: Precompute Story Tray for Each User

When someone posts a story, push it to all their followers' precomputed trays.

Why it feels right: Reads are instant - just fetch the precomputed tray. No complex queries at read time.

Why it fails for stories: A celebrity posts → push to 100 million followers. That's 100 million writes for one story. Now multiply by all the stories posted every second. **Write amplification is catastrophic.**

> 💡 **Lesson**
>
> Fan-out on write doesn't scale for high-follower accounts.

Attempt 3: Just Use TTL in the Database

Store stories with a 24-hour TTL. Let the database auto-delete expired entries.

Why it feels right: Built-in expiration. No cleanup jobs. Simple.

Why it's incomplete: TTL handles deletion, but not the read path. You still need to efficiently answer: "Which of my 500 followees have active stories right now?"

> 💡 **Lesson**
>
> TTL solves storage cleanup, not query efficiency.

Attempt 4: Hybrid - Push for Normal, Pull for Celebrities

Push stories to followers for normal users. For celebrities (>1M followers), don't push - pull at read time.

Why it's better: Avoids write amplification for celebrities. Most users still get fast reads.

Sold to @theskilledc
kumar.dodda@gmail.com

This is the foundation of the correct approach.

The Real Nature of the Problem

Stories have unique properties that define the architecture:

PROPERTY	IMPLICATION
24-hour lifespan	No need for permanent storage. Heavy churn.
Time-ordered	Most recent first. Simple ordering.
All-or-nothing visibility	Either you see it or you don't. No partial access.
Heavy reads, light writes	1 post → millions of views. Optimize for reads.
Follower graph asymmetry	Celebrities have millions of followers. Handle differently.

Stories are ephemeral, read-heavy, and have extreme fan-out asymmetry. You can't treat all users the same.

This tells us the design principles:

1. **Separate storage from feed generation** - Store stories simply, generate feeds smartly
2. **Hybrid fan-out** - Push for normal users, pull for celebrities
3. **TTL everywhere** - Stories are temporary by design
4. **Cache the story tray** - Most users check stories often, cache is very effective

The Correct Architecture

Principle 1: Simple Story Storage

Stories are stored in a straightforward table:

- **story_id** - Unique identifier
- **user_id** - Who posted it
- **media_url** - Pointer to image/video in object storage
- **created_at** - When posted (determines expiration)
- **expires_at** - created_at + 24 hours (indexed for cleanup)

Sold to
@theskilledc
kumar.dodda@gmail.com

Media (images, videos) goes to object storage (S3) with CDN in front of it. Metadata goes to the database with TTL.

Principle 2: Maintain an "Active Stories" Index

Instead of querying for each followee, maintain a fast index of **users who currently have active stories**.

- When a user posts their first story (in 24h), add them to the index
- When their last story expires, remove them from the index
- To build the story tray: intersect "users you follow" with "users with active stories"

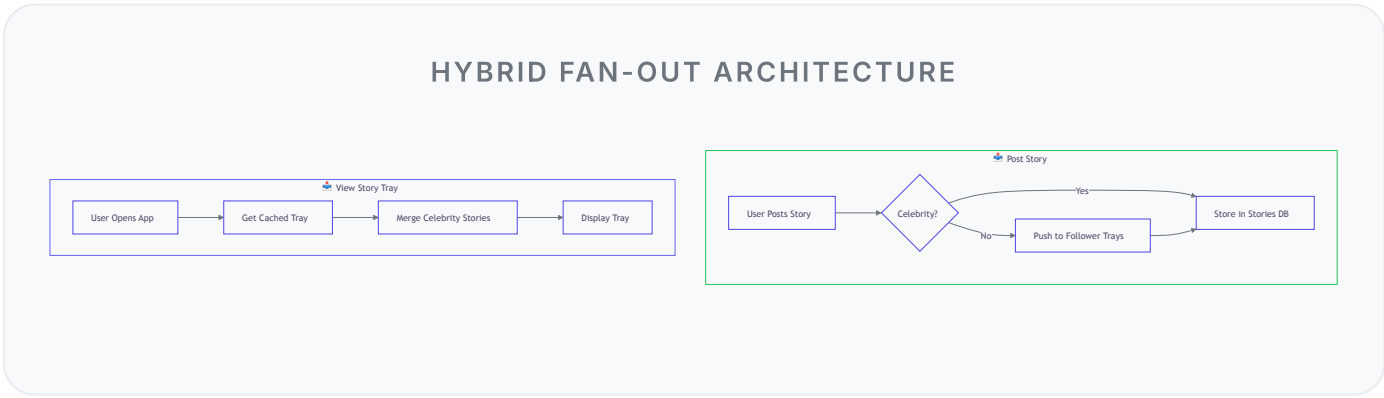
This reduces the query from "check 500 followees" to "intersect two sets."

Principle 3: Hybrid Fan-Out Strategy

Different users need different strategies:

USER TYPE	FOLLOWERS	STRATEGY
Normal	<10K	**Push:** Add to followers' cached trays
Influencer	10K - 1M	**Hybrid:** Push to active followers only
Celebrity	>1M	**Pull:** Merge at read time

Why this works: Most users follow mostly normal accounts, so their tray is precomputed. Celebrities are pulled at read time and merged - but there are only a few celebrities to check.



Principle 4: Aggressive Caching with Short TTL

The story tray changes slowly (only when someone posts or a story expires). Cache it aggressively:

- **Cache key:** user:{user_id}:story_tray
- **TTL:** 5 minutes (balance freshness vs load)

Sold to
@theskilledc
kumar.dodda@gmail.com

- **Invalidation:** When a followed user posts or expires

Most users check stories multiple times per hour. Cache hit rates are very high.

Principle 5: Efficient Expiration and Cleanup

Millions of stories expire every hour. Cleanup must be efficient:

- **Database TTL:** Cassandra/DynamoDB can auto-expire rows
- **Object storage lifecycle:** S3 can auto-delete after 24 hours
- **Background job:** For non-TTL databases, batch delete expired stories

Don't delete synchronously on read - that adds latency. Let background processes handle cleanup.

Viewing and Engagement

View Tracking

Users see who viewed their story. This requires tracking views efficiently:

- Writes are append-only (user X viewed story Y at time T)
- Store in a time-series or append-optimized store
- Don't block the viewing experience on write success
- Views can be eventually consistent - slight delay is acceptable

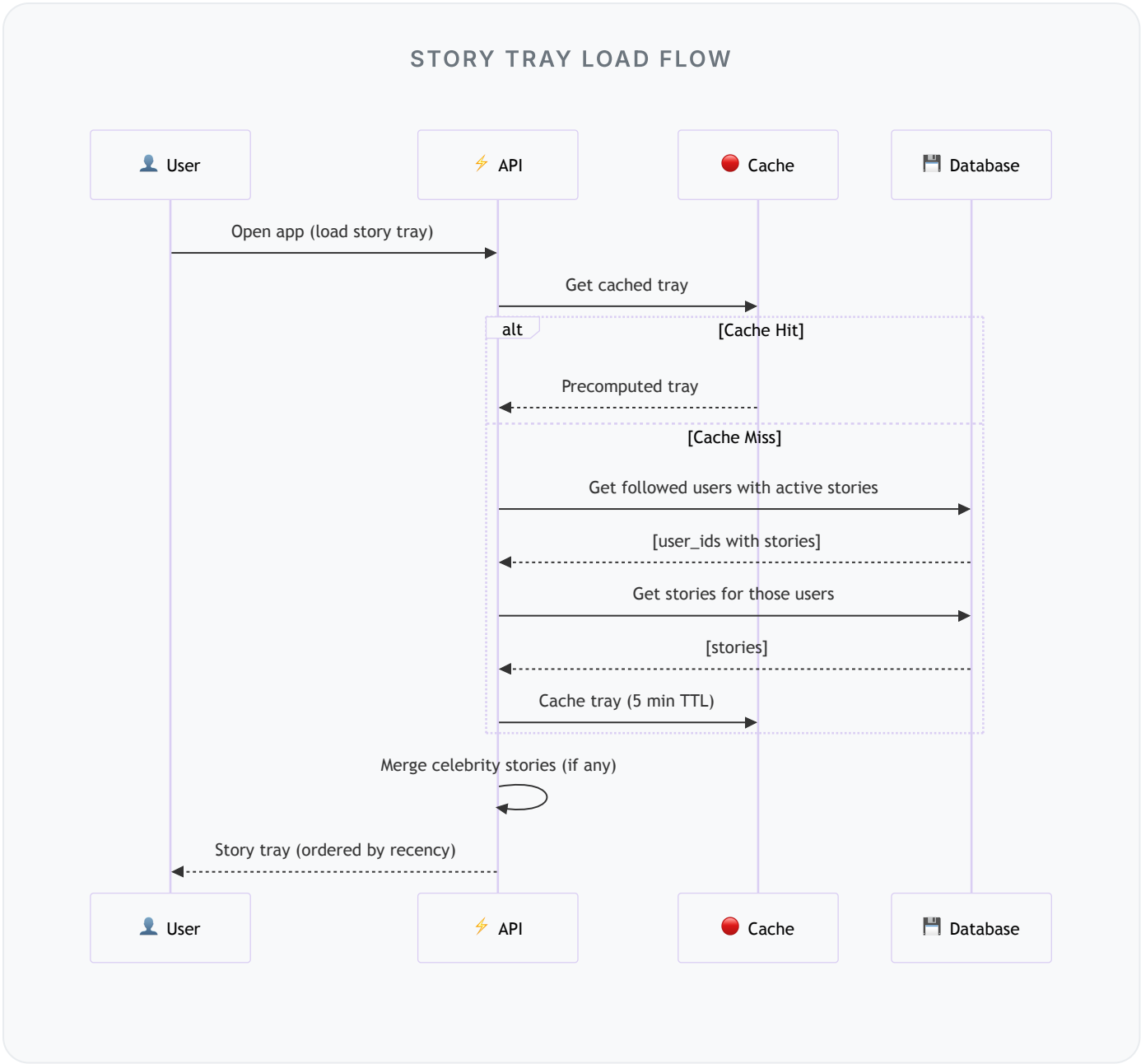
Read Receipts and Ring Indicator

The colored ring around a profile indicates unseen stories:

- Track "last seen story timestamp" per user-pair
- If story.created_at > last_seen → show colored ring
- Update last_seen when user views the story

This is a lightweight lookup that can be cached per session.

Sold to
@theskilledc
kumar.dodda@gmail.com



Interview Pivot (How to Explain It)

If asked:

"How would you design Instagram Stories that expire after 24 hours?"

"Stories have extreme fan-out asymmetry - celebrities have millions of followers, so you can't push-to-all. I'd use hybrid fan-out: push for normal users (whose followers get a precomputed tray), pull for celebrities (merged at read time).

I'd maintain an 'active stories' index - users who currently have unexpired stories. To build the tray, I intersect 'who I follow' with 'who has active stories' instead of checking each followee.

Expiration is handled by TTL in the database and S3 lifecycle policies - no complex cleanup jobs needed.

Sold to
@theskilledc
kumar.dodda@gmail.com

Caching is very effective because story trays change slowly. 5-minute TTL gives high hit rates while keeping content fresh."

What's Next?

Stories lead to deeper questions:

How do you handle "Close Friends" stories?

> **Visibility flag** on the story. Check if viewer is in the poster's close friends list. Filter at read time. Separate tray or tag in same tray.

How do you handle story replies?

> **Direct messages** with story reference. The reply goes to DM, not stored with the story. Include story preview in the message.

How do you handle story highlights (permanent saves)?

> **Copy to permanent storage** before expiration. Highlights are a different data model - grouped collections on the profile. Don't delete highlighted stories.

How do you handle stories in different time zones?

> **UTC everywhere.** Store created_at in UTC. Calculate expiration as created_at + 24 hours (absolute). Client displays relative time.

Sold to
@theskilledc
kumar.dodda@gmail.com

The insight: Stories are ephemeral content with extreme asymmetry. Fan-out on write fails for celebrities. Fan-out on read fails for everyone. The answer is hybrid: push for normal users, pull for celebrities, cache aggressively.

Maintain an active stories index. Use TTL for automatic cleanup. Cache the tray.

🔁 **Same Principles Apply To**

- **Twitter Fleets / LinkedIn Stories** - same hybrid fan-out
- **Disappearing messages (WhatsApp)** - same TTL-based expiration
- **Activity feeds** - same celebrity vs normal user handling
- **Real-time status updates** - same "active users" index pattern

Sold to
@theskilledc
kumar.dodda@gmail.com

How Would You Design a Ride-Sharing Matching System for 100K Requests per Minute

Real-Time Geospatial Matching at Scale

Ride-sharing is a massive coordination problem:

- **Uber** - 100+ million rides per month globally
- **Lyft** - Billions of driver location updates per day
- **Didi** - Peak of 30 million rides in a single day

When you tap "Request Ride":

- The system finds available drivers near you
- Selects the best match (ETA, rating, car type)
- Notifies the driver
- Driver accepts or declines in seconds
- Match confirmed, ride begins

All of this happens in **under 10 seconds**, while handling 100,000+ concurrent ride requests per minute.

> 📊 **The Scale**
>
> 100K requests/min = 1,600 matches/second. Millions of drivers sending location updates every 3-5 seconds. A geospatial matching problem in real-time.

📍 THE WHITEBOARD MOMENT

Interviewer: "Design Uber's driver matching."

Candidate: "Easy. Find the closest driver and assign them."

Interviewer: "The closest driver is stuck in traffic. ETA: 15 minutes."

Candidate: "Okay, optimize for fastest ETA then."

Sold to @theskilledc kumar.dodda@gmail.com

Interviewer: "Driver with best ETA has a 2.1 rating. User will cancel."

Candidate: "Fine, I'll score them on all factors..."

Interviewer: "While you computed the score, the driver moved 3 blocks. Recalculate?"

Candidate: "..."

Interviewer: "Now do this 1,600 times per second." 😉

⚡ The Engineering Complexity

Drivers move. Riders wait. The optimal match changes every second:

📍 Moving Targets

Drivers move. Riders wait. The optimal match at t=0 is wrong by t=5.

🎯 Multi-Objective

Closest driver? Fastest arrival? Best rating? Optimize all three? Pick one?

⚡ Time Pressure

User opens app. 30 seconds to show ETA. Searching 10K drivers in 30 seconds.

🔄 State Explosion

Driver accepts, rider cancels. Driver en route, rider moves. Every combination is an edge case.

First Attempts (and Why They Break)

Attempt 1: Query All Drivers Within Radius

When a rider requests, query the database: "Find all drivers within 5km of this location."

Why it feels right: Simple SQL with spatial extension. Get all candidates, pick the best one.

Why it fails: In a dense city, there might be 10,000 drivers within 5km. Scanning all of them for every request is expensive. 1,600 requests/second, each scanning thousands of drivers = **database collapse**.

Sold to
@theskilledc
kumar.dodda@gmail.com

> 💡 **Lesson**
>
> Brute-force radius queries don't scale in dense urban areas.

Attempt 2: Partition by City

Shard drivers by city. Each city has its own matching service.

Why it feels right: Reduces the search space. NYC matches only check NYC drivers.

Why it fails: Cities are too coarse. Manhattan alone has tens of thousands of active drivers. City boundaries are messy - what about suburbs? Cross-boundary trips become complicated.

> 💡 **Lesson**
>
> Cities are too large. You need finer-grained spatial partitioning.

Attempt 3: Static Grid Cells

Divide the world into a grid. Each cell is ~1km². Only search drivers in nearby cells.

Why it's better: Much smaller search space. Only check 9 cells (center + 8 neighbors). Predictable, simple to implement.

Why it's still limited: Fixed-size cells are inefficient. In dense Manhattan, a 1km² cell has 1,000 drivers. In rural Texas, it has 0. You're either over-partitioned or under-partitioned.

> 💡 **Lesson**
>
> Static grids don't adapt to varying driver density.

Attempt 4: Geohash Indexing

Use geohashes - a hierarchical encoding of location into a string. Nearby locations share prefixes. Variable precision based on string length.

Why it's smarter: Hierarchical - zoom in or out by prefix length. String comparisons are fast. Built-in spatial indexing.

This is the foundation of the correct approach.

Sold to
@theskilledc
kumar.dodda@gmail.com

The Real Nature of the Problem

Ride matching has unique constraints:

CONSTRAINT	IMPLICATION
Real-time	Match must happen in seconds, not minutes
Location changes constantly	Drivers move. Index must update continuously
Asymmetric density	10,000 drivers in Manhattan, 10 in rural Kansas
Mutual acceptance	Both rider and driver must agree
No double-booking	A driver can't be matched to two riders simultaneously

This is a real-time, continuously-updating, geospatial matching problem with hard consistency requirements (no double-booking).

This tells us the design principles:

1. **Efficient geospatial indexing** - Find nearby drivers in O(1) or O(log n)
2. **Continuous location updates** - Handle millions of updates per second
3. **Fast matching algorithm** - Select best driver quickly
4. **Atomic state transitions** - Prevent double-booking
5. **Timeout and retry** - Handle driver non-response gracefully

The Correct Architecture

Principle 1: Geospatial Indexing with Geohash or S2

Use a hierarchical spatial index:

- **Geohash:** Encodes lat/lng as a string. Longer = more precise. "drt2ym" covers ~1km². Nearby points share prefixes.
- **S2 (Google):** Projects Earth onto a cube, then uses a space-filling curve. More uniform cell sizes, better for global coverage.

How it works:

- Driver at (40.7128, -74.0060) → geohash "dr5ru7"

Sold to
@theskilledc
kumar.dodda@gmail.com

- Store in index: dr5ru7 → [driver_1, driver_2, ...]
- Rider requests from same area → look up "dr5ru7" + neighboring cells
- Instant candidate list without scanning all drivers

Principle 2: In-Memory Location Index

Driver locations change every 3-5 seconds. You can't write to disk that fast.

- Keep the location index **in memory** (Redis, custom data structure)
- Drivers send location updates to a **location service**
- Location service updates the geospatial index
- Matching service queries the in-memory index

Why in-memory? Millions of updates per second. Disk-based databases can't keep up. Even with SSDs, the write amplification kills performance.

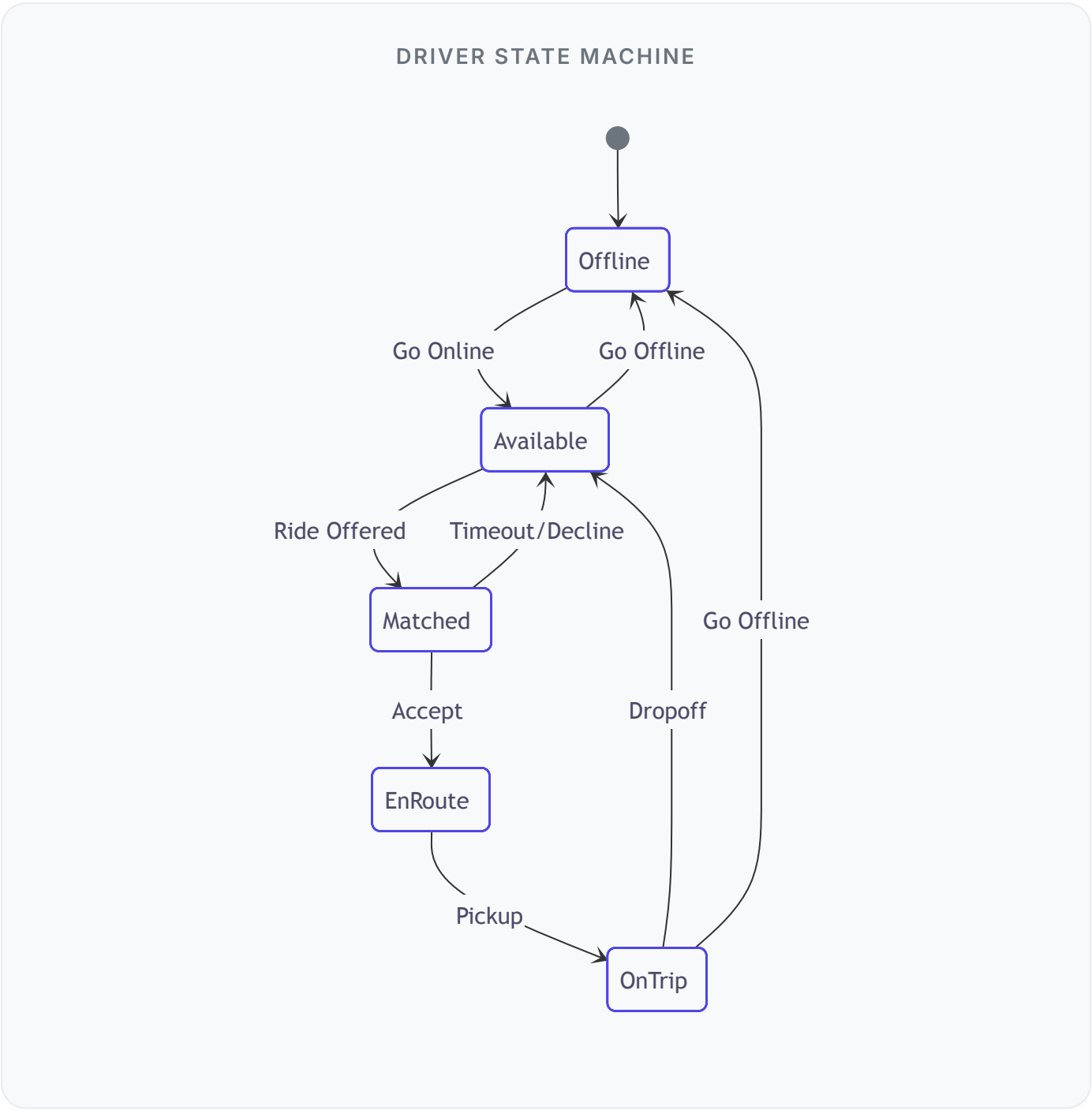
Principle 3: Driver State Machine

Drivers have explicit states to prevent double-booking:

- **Offline** - Not accepting rides
- **Available** - Can be matched
- **Matched** - Offered a ride, waiting for response
- **En Route** - Heading to pickup
- **On Trip** - Rider in car

Critical rule: Only "Available" drivers appear in the matching index. State transitions are **atomic** - if two ride requests try to match the same driver, only one succeeds.

Sold to
@theskilledc
kumar.dodda@gmail.com



Principle 4: Matching Algorithm

Finding the "best" driver is a scoring problem:

- **ETA** - How fast can they get there? (primary factor)
- **Rating** - Driver quality
- **Car type match** - Rider requested UberX, driver has UberX
- **Historical accept rate** - Will they accept?
- **Fairness** - How long since their last ride?

Simple approach: Score = weighted sum of factors. Pick highest score among candidates.

Advanced approach: Batch matching - collect requests for 2 seconds, then globally optimize all matches to minimize total wait time. Used by Uber in high-density areas.

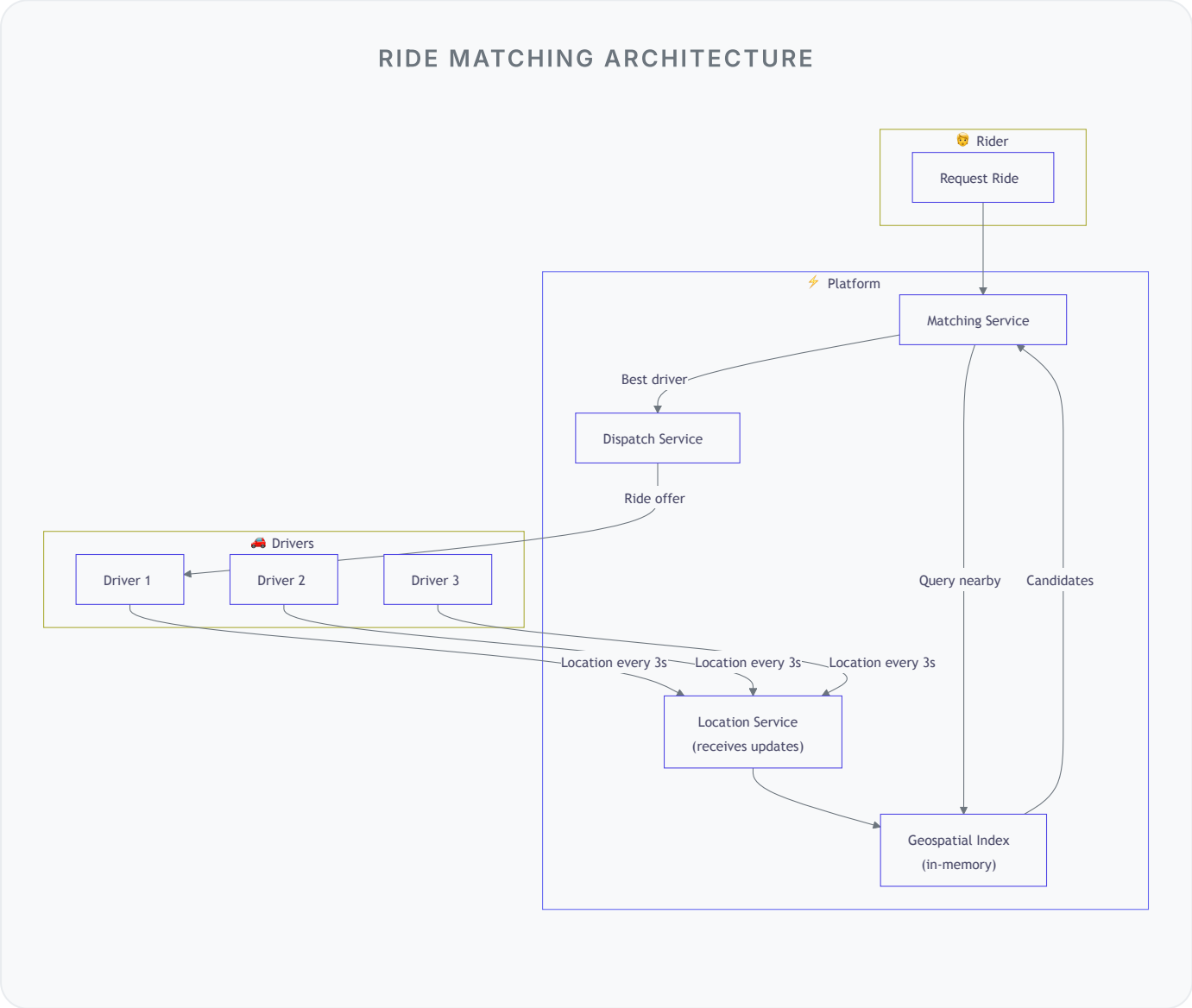
Sold to @theskilledc kumar.dodda@gmail.com

Principle 5: Offer, Timeout, Retry

Matching isn't complete until the driver accepts:

1. Send push notification to matched driver
2. Driver has 15 seconds to accept
3. If no response → move to next best driver
4. Repeat until someone accepts or rider cancels

Optimistic vs pessimistic: Uber uses optimistic matching - assume the driver will accept, start calculating route. If they decline, rollback and match again. This reduces perceived latency.



Handling Supply-Demand Imbalance

What happens when there are more riders than drivers?

Surge Pricing

When demand exceeds supply in an area:

Sold to
@theskilledc
kumar.dodda@gmail.com

- Increase prices (multiplier: 1.5x, 2x, 3x)
- Some riders drop out (reduced demand)
- More drivers attracted to the area (increased supply)
- Equilibrium restored

Surge is calculated per geohash cell, updated every few minutes based on request/supply ratio.

Driver Positioning

Proactively guide drivers to high-demand areas:

- Show "heat map" of predicted demand
- Incentivize drivers with bonuses for specific zones
- Predict demand using historical patterns (concerts, sports events, weather)

ETA Calculation

ETA is crucial for matching and rider experience:

- **Road network routing:** Not straight-line distance - actual driving time
- **Real-time traffic:** Congestion changes ETA significantly
- **Historical patterns:** This intersection is slow at 5 PM on Fridays
- **Precomputed estimates:** For common routes, cache the result

ETA calculation is often a separate service (or powered by Google Maps / OSRM).

Interview Pivot (How to Explain It)

If asked:

"How would you design a ride-sharing matching system like Uber?"

> "The core challenge is real-time geospatial matching. I'd use geohash or S2 to index driver locations - this gives O(1) lookup for 'nearby drivers' without scanning all drivers.
>
> The location index must be in-memory because drivers update every 3 seconds. Millions of updates per second can't go to disk.
>
> For matching, I'd score candidates on ETA, rating, and car type. The driver state machine is critical - only 'Available' drivers can be matched, and state transitions are atomic to prevent double-booking.

Sold to
@theskilledc
kumar.dodda@gmail.com

The offer flow has timeouts: if a driver doesn't respond in 15 seconds, move to the next candidate. Surge pricing handles supply-demand imbalance."

What's Next?

Ride matching leads to deeper questions:

How do you handle ride pools (shared rides)?

> **Combinatorial optimization:** Match multiple riders to one driver if routes overlap. Score based on total detour time. More complex matching graph.

How do you handle scheduled rides?

> **Pre-matching:** Match 15 minutes before pickup. Reserve driver capacity. Remind driver. Fall back to on-demand if driver cancels.

How do you prevent fraud (fake locations, GPS spoofing)?

> **Cross-check signals:** GPS + cell tower triangulation + speed consistency. Flag anomalies. Block repeat offenders.

How do you handle driver offline mid-trip?

> **Heartbeat monitoring:** If driver stops sending updates, alert safety team. Store last known location. Contact rider.

Sold to
@theskilledc
kumar.dodda@gmail.com

The insight: Ride matching is a real-time geospatial problem with strict consistency requirements. Geohash gives you fast spatial queries. In-memory indexing handles update velocity. State machines prevent double-booking.

Index by location. Match by score. Transition atomically. Handle timeouts gracefully.

🔁 **Same Principles Apply To**

- **Food delivery matching (DoorDash, Uber Eats)** - same geospatial + state machine
- **Dating app matching (proximity-based)** - same geohash indexing
- **Local service marketplaces (TaskRabbit)** - same location + availability matching
- **Ambulance/emergency dispatch** - same real-time ETA-based matching

Sold to
@theskilledc
kumar.dodda@gmail.com

How Would You Design a Load Balancer for 1 Million Requests per Second

The Gateway to Scale

Load balancers are the unsung heroes of scalable systems:

- **AWS ELB** - Handles trillions of requests for AWS customers
- **Cloudflare** - 46+ million HTTP requests per second
- **NGINX** - Powers 30%+ of the web
- **HAProxy** - The TCP/HTTP load balancer of choice

A load balancer does three things:

- **Distribute traffic** - Spread requests across servers
- **Health checking** - Only send to healthy servers
- **High availability** - No single point of failure

At **1 million requests per second**, the load balancer itself becomes the scaling challenge.



The Scale

1M req/sec = 60 billion requests/hour. Each request needs routing in microseconds. A single load balancer can handle ~100K-500K req/sec. You need multiple.

🔥 THE OPS ROOM

Alert: Server 3 is down! Traffic spiking to 500K req/sec!

Junior SRE: "No problem. Load balancer will route around it."

Alert: Server 4 timing out! Connection pool exhausted!

Junior SRE: "Wait, why is the LB still sending traffic there?"

Senior SRE: "Health check passed. It returned HTTP 200... 5 seconds ago."

Sold to @theskilledc kumar.dodda@gmail.com

```
Alert: Load balancer at 98% CPU! It's becoming the bottleneck!

Junior SRE: "So who load-balances the load balancer?"

Senior SRE: "Now you're asking the right questions." 😏
```

⚡ The Engineering Complexity

Distributing load is easy. Knowing which servers are actually healthy is the hard part:

📊 Health Checking

100 backends. Check each every second. 1 slow response = 1 second of bad routing?

🎯 Sticky Sessions

User A must always go to Server 1. But Server 1 is overloaded. Now what?

⚡ L4 vs L7

TCP-level is fast but dumb. HTTP-level is smart but slow. Pick based on needs.

🔄 Graceful Draining

Server 1 is going down. 100 active connections. Drop them? Wait forever?

First Attempts (and Why They Break)

Attempt 1: Single Load Balancer (Reverse Proxy)

One NGINX or HAProxy instance in front of all backend servers.

Why it feels right:

- Simple to set up and understand
- Single point of configuration
- Works great for small/medium traffic

Why it fails:

- One server can handle ~100K-500K req/sec max
- At 1M req/sec, the load balancer becomes the bottleneck

Sold to @theskilledc kumar.dodda@gmail.com

- Single point of failure - if it dies, everything dies
- Vertical scaling has limits (more CPU/RAM only goes so far)

> 💡 **Lesson**
>
> A single load balancer can't handle 1M req/sec. You need to scale the load balancer itself.

Attempt 2: Active-Passive Load Balancers

Two load balancers: one active, one standby. If active fails, passive takes over.

Why it feels right:

- Eliminates single point of failure
- Automatic failover with VRRP/keepalived
- Standard HA pattern

Why it fails:

- Still limited to capacity of one server (passive is idle)
- Doesn't solve throughput - only availability
- Wastes 50% of resources (passive just waiting)
- Failover takes seconds - requests dropped during transition

> 💡 **Lesson**
>
> Active-Passive solves availability but not throughput. One server still does all the work.

Attempt 3: Active-Active with Round-Robin DNS

Multiple load balancers, DNS returns all their IPs. Clients pick one.

Why it feels right:

- All load balancers are active (no wasted resources)
- Horizontal scaling - add more load balancers
- Simple, no extra hardware needed

Why it fails:

- DNS TTL caching - clients hold onto stale IPs

Sold to
@theskilledc
kumar.dodda@gmail.com

- No health awareness - DNS doesn't know if a LB is down
- Uneven distribution - clients don't pick evenly
- Failover is slow (depends on DNS TTL expiration)

> 💡 **Lesson**
>
> DNS is not a load balancer. It's a naming system. No health checks, slow failover.

Attempt 4: Hardware Load Balancer (F5, Citrix)

Dedicated hardware appliance designed for high throughput.

Why it feels right:

- Purpose-built for millions of connections
- Hardware acceleration (ASICs, FPGAs)
- Single device handles massive traffic

Why it fails at extreme scale:

- Extremely expensive ($100K-$1M per device)
- Vendor lock-in, proprietary configuration
- Still a single point of failure (need pairs)
- Hard to scale - buy bigger box, not more boxes
- Doesn't fit cloud-native architectures

> ⚠️ **Lesson**
>
> Hardware solves throughput but not cost or flexibility. Cloud needs software.

The Real Nature of the Problem

Load balancing at 1M req/sec has layers:

Sold to
@theskilledc
kumar.dodda@gmail.com

LAYER	QUESTION	CHALLENGE
L4 (Transport)	Which server gets this TCP connection?	Must handle millions of concurrent connections
L7 (Application)	Which server handles this HTTP request?	Must parse headers, route by URL/cookie
Global	Which datacenter handles this user?	Must consider latency, capacity, failures

You need load balancing at multiple layers. No single solution handles everything.

This tells us the design principles:

1. **Layer the load balancing** - DNS → L4 → L7 → backend
2. **Stateless when possible** - Any LB can handle any request
3. **Health checking everywhere** - Remove unhealthy nodes instantly
4. **Scale horizontally** - Add more LBs, not bigger LBs

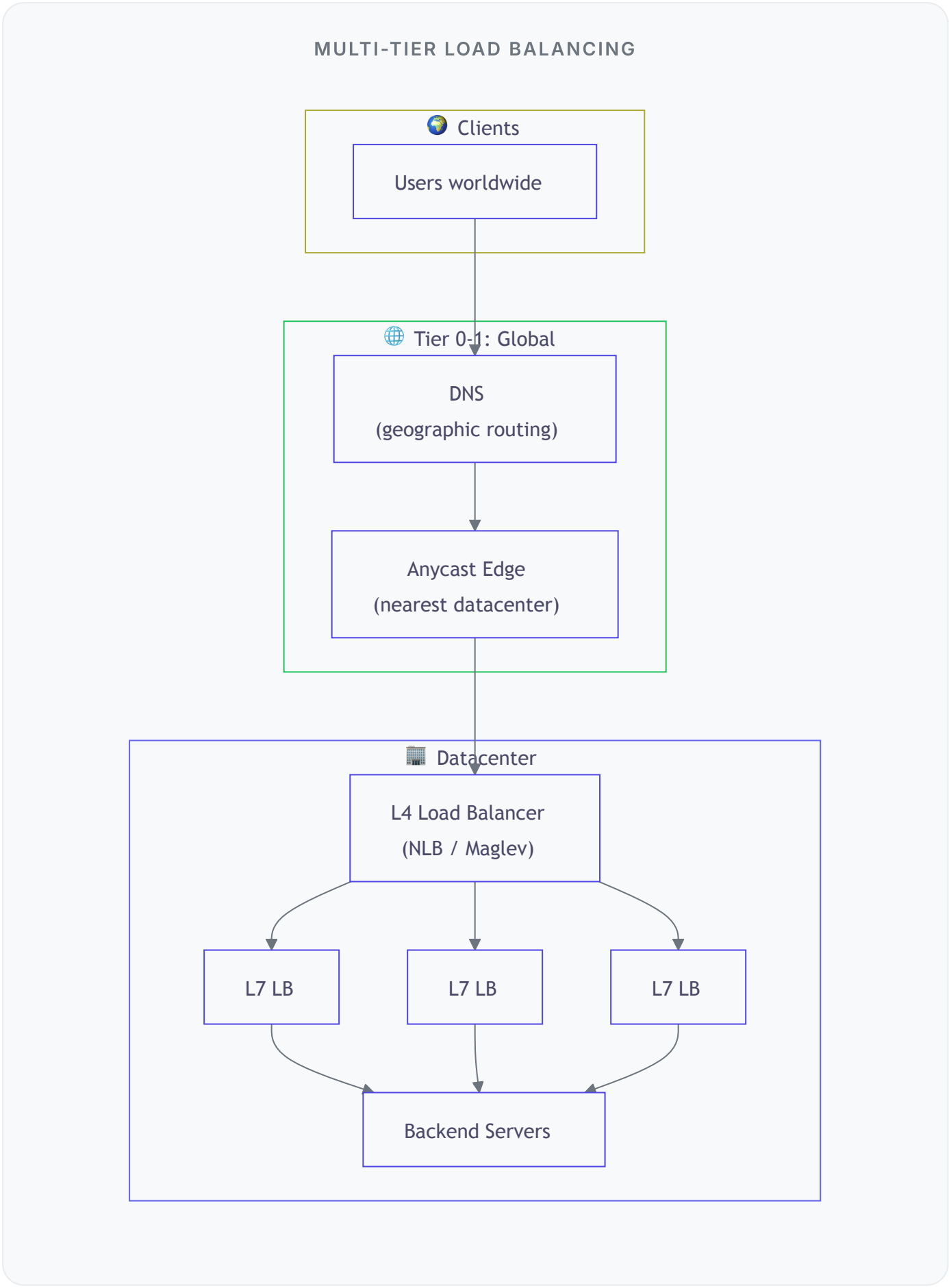
The Correct Architecture

Principle 1: Multi-Tier Load Balancing

At 1M req/sec, you need load balancers in front of load balancers:

TIER	TECHNOLOGY	FUNCTION
Tier 0: DNS	Route 53, Cloudflare DNS	Geographic routing, initial distribution
Tier 1: Edge/Anycast	BGP Anycast, Cloudflare	Route to nearest datacenter
Tier 2: L4 Load Balancer	AWS NLB, Maglev, IPVS	Distribute TCP connections
Tier 3: L7 Load Balancer	NGINX, Envoy, HAProxy	Route by URL, headers, content

Sold to
@theskilledc
kumar.dodda@gmail.com



Principle 2: L4 vs L7 - Know the Difference

L4 (Layer 4 - Transport):

- Works at TCP/UDP level

Sold to
@theskilledc
kumar.dodda@gmail.com

- Sees: source IP, dest IP, ports
- Extremely fast - no payload inspection
- Can handle 10M+ connections
- Used for: initial distribution, SSL passthrough

L7 (Layer 7 - Application):

- Works at HTTP level
- Sees: URL, headers, cookies, body
- Slower - must parse HTTP
- More intelligent routing (by path, host, etc.)
- Used for: API routing, A/B testing, canary deployments

> **Use L4 for raw throughput. Use L7 for smart routing. Combine them.**

Principle 3: Consistent Hashing for Stateful Distribution

Some scenarios need the same client to hit the same server (session affinity):

- WebSocket connections (must maintain connection)
- Session-based apps (shopping cart in server memory)
- Caching (same user → same cache → better hit rate)

Solution: Consistent hashing

- Hash the client IP (or session ID) to a server
- Same client always goes to same server
- If server dies, only that server's clients are rehashed

Principle 4: Health Checking

A load balancer is only as good as its health checking:

- **Passive:** Track failed responses, remove after threshold
- **Active:** Ping health endpoint every N seconds
- **Graceful removal:** Drain connections before removing node

Health check types:

Sold to
@theskilledc
kumar.dodda@gmail.com

TYPE	WHAT IT TESTS	LIMITATION
TCP connect	Port is open	App might be hung
HTTP GET /health	App responds 200	App might be partially broken
Deep health check	App checks dependencies	Might be too slow

Principle 5: Load Balancing Algorithms

How to pick which server gets the next request:

ALGORITHM	HOW IT WORKS	BEST FOR
Round Robin	Server 1, 2, 3, 1, 2, 3...	Identical servers, uniform requests
Weighted Round Robin	Bigger servers get more traffic	Mixed-capacity clusters
Least Connections	Send to server with fewest active connections	Varying request durations
Least Response Time	Send to fastest-responding server	Latency-sensitive apps
Random	Pick randomly	Simple, stateless, surprisingly effective
IP Hash	Hash client IP to server	Session affinity without cookies

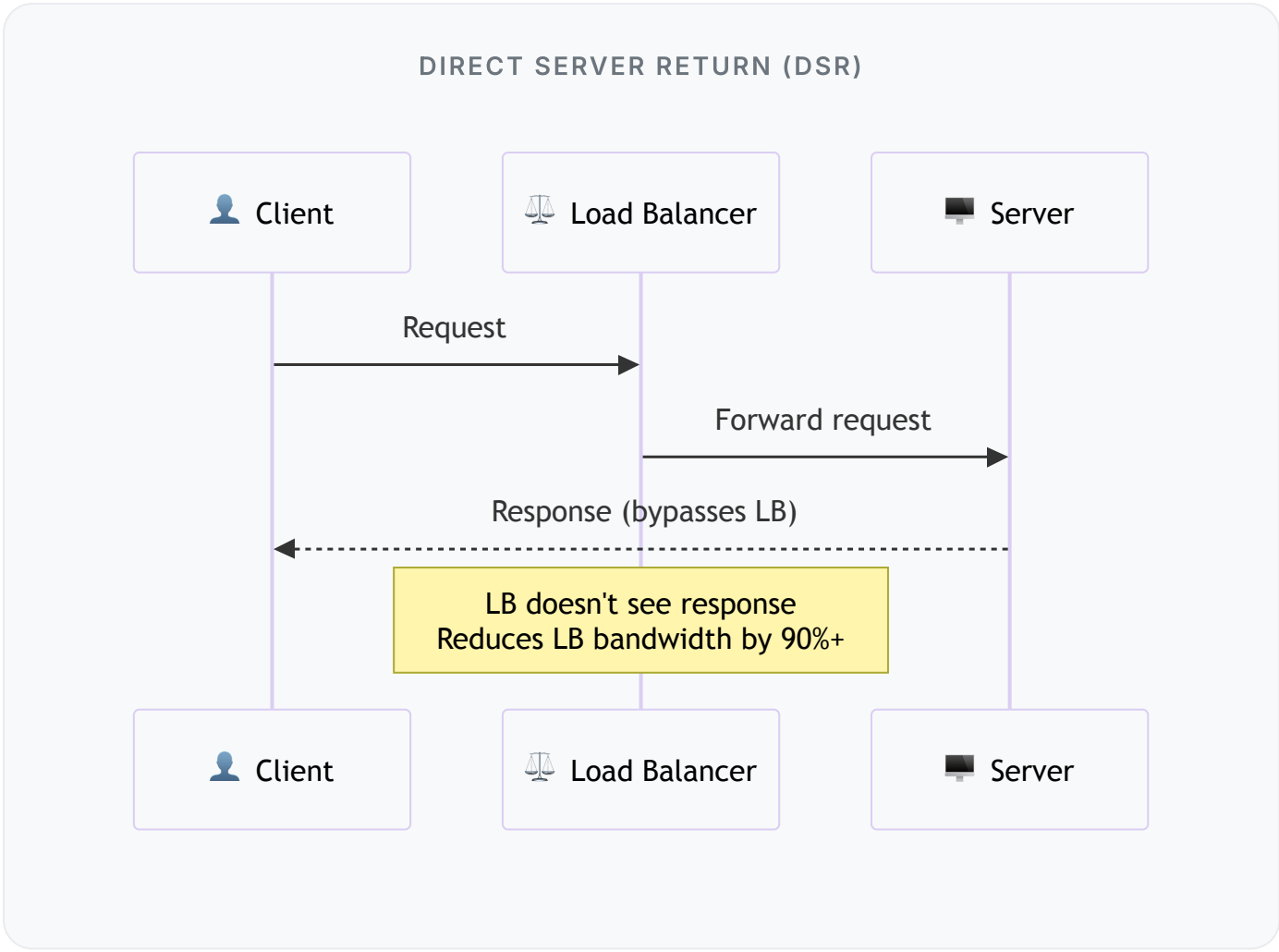
How Google Does It: Maglev

Google's Maglev handles millions of requests per second using:

- **L4 software load balancer** - Runs on commodity Linux servers
- **Consistent hashing** - Minimizes disruption when servers change
- **ECMP (Equal-Cost Multi-Path)** - Routers distribute across Maglev instances
- **Direct Server Return** - Response bypasses LB (reduces bottleneck)

Key insight: **The load balancer doesn't need to see the response.** With Direct Server Return, the backend sends directly to the client. The LB only handles inbound traffic.

Sold to
@theskilledc
kumar.dodda@gmail.com



SSL Termination: Where to Decrypt?

HTTPS traffic must be decrypted somewhere. Options:

OPTION	PROS	CONS
Terminate at L7 LB	Can inspect HTTP, route by path/header	LB must handle CPU-intensive decryption
Terminate at backend	End-to-end encryption	Every backend needs certs, L7 routing limited
Passthrough at L4	LB doesn't touch crypto	Can only route by IP/port, not HTTP

Common pattern: Terminate SSL at L7 load balancer, use plain HTTP internally. Internal network is trusted; saves backend CPU.

Interview Pivot (How to Explain It)

If asked:

"How would you design a load balancer for 1 million requests per second?"

Sold to @theskilledc kumar.dodda@gmail.com

"A single load balancer can't handle 1M req/sec. I'd use multi-tier load balancing: DNS for geographic distribution, Anycast for datacenter routing, L4 load balancers for raw TCP distribution, and L7 load balancers for HTTP routing.

L4 is fast (just routes TCP) and handles millions of connections. L7 is smarter (routes by URL/headers) but slower. Combine them.

For extreme scale, I'd use Direct Server Return - the response bypasses the LB entirely, reducing LB bandwidth by 90%+. Health checking at every tier ensures we only route to healthy servers.

Algorithm choice depends on the use case - Round Robin for uniform traffic, Least Connections for variable request durations, IP Hash for session affinity."

What's Next?

Load balancing leads to deeper questions:

How do you handle sticky sessions without cookies?

> **IP hash** for simple cases. **Token in URL** for mobile apps. Or better: **externalize session state** to Redis so any server works.

How do you do zero-downtime deployments?

> **Graceful drain:** Stop sending new requests to old servers, wait for existing connections to finish, then terminate. LB health check marks them unhealthy first.

How do you handle sudden traffic spikes?

> **Auto-scaling** backends based on LB metrics. **Rate limiting** at the LB to protect backends. **Queue overflow** traffic during spike.

Sold to
@theskilledc
kumar.dodda@gmail.com

How do you implement blue-green or canary deployments?

> **L7 routing rules:** Send 1% of traffic to canary. Split by header, cookie, or random selection. Monitor error rates, gradually shift traffic.

The insight: At 1M req/sec, you can't rely on a single load balancer. Layer them: DNS → Anycast → L4 → L7 → backends. Each layer handles what it's best at.

L4 for speed. L7 for smarts. Health checks everywhere. Direct Server Return for extreme scale.

Same Principles Apply To

- **API Gateways (Kong, Envoy)** - same L4/L7 layering
- **CDN edge routing** - same multi-tier distribution
- **Database connection pooling (PgBouncer)** - same health checking patterns
- **Service mesh (Istio, Linkerd)** - same consistent hashing and L7 routing

Sold to
@theskilledc
kumar.dodda@gmail.com

How Would You Design a Real-Time Collaborative Editor for 10K Concurrent Users

When Everyone Edits at Once

Real-time collaboration is everywhere:

- **Google Docs** - The pioneer of real-time editing
- **Notion** - Blocks, databases, and collaboration
- **Figma** - Real-time design collaboration
- **VS Code Live Share** - Collaborative coding
- **Miro, FigJam** - Whiteboard collaboration

The magic moment:

- User A types "Hello" at position 10
- User B simultaneously types "World" at position 10
- Both see consistent results in milliseconds
- No one's work is lost

At **10,000 concurrent users on a single document**, managing conflicts and broadcasting changes becomes a distributed systems challenge.



The Scale

Google Docs handles 2 billion documents. A popular document might have thousands of concurrent editors. Each keystroke must be synced in under 100ms.



```
THE COLLABORATION NIGHTMARE

Alice (New York): Types "Hello" at position 0

Bob (Tokyo): Types "World" at position 0

Server: Alice's edit arrives first → "Hello"
```

Sold to @theskilledc kumar.dodda@gmail.com

```
Server: Bob's edit arrives → Insert "World" at position 0?

Alice sees: "WorldHello"

Bob sees: "HelloWorld"

Junior Dev: "Just use last-write-wins!"

Senior Dev: "Cool. Alice loses 'Hello'. Bob loses 'World'. Both are
angry."

Junior Dev: "...How does Google Docs work again?" 🤯
```

⚡ The Engineering Complexity

Two people type at the same spot. Whose cursor wins? Distributed consensus in real-time:

✏️ Concurrent Edits

User A types at position 5. User B deletes at position 3. A's position is now wrong.

🔄 Conflict Resolution

Both users type at the same spot. Which one wins? How do you merge?

⏱️ Real-Time Sync

Keystrokes must appear on other screens in <100ms. Anything slower feels broken.

📡 Offline Support

User goes offline, types 500 characters, comes back. Merge with 10 minutes of others' edits.

First Attempts (and Why They Break)

Attempt 1: Lock the Document

Only one user can edit at a time. Others wait for the lock.

Why it feels right:

- Simple to implement - just a mutex
- No conflict resolution needed

Sold to
@theskilledc
kumar.dodda@gmail.com

- Guarantees consistency

Why it fails:

- Completely destroys the collaborative experience
- One user typing = everyone else blocked
- At 10K users, 9,999 are waiting
- This is how 1990s file servers worked - not collaboration

> 💡 **Lesson**
>
> Locking prevents conflicts by preventing collaboration entirely.

Attempt 2: Last-Write-Wins

Everyone edits freely. When syncing, the latest timestamp wins.

Why it feels right:

- No blocking - everyone edits
- Simple conflict resolution
- Works for some systems (DNS, eventual consistency)

Why it fails:

- **Data loss** - User A's edits are silently overwritten
- Users type for 10 seconds, then their work vanishes
- Timestamps don't capture causality
- Unacceptable for document editing

> 💡 **Lesson**
>
> Last-write-wins loses data. Collaborative editing must preserve all edits.

Attempt 3: Diff and Merge on Save

Users edit locally. When they save, compute a diff and merge.

Why it feels right:

- Works for Git (code collaboration)

Sold to
@theskilledc
kumar.dodda@gmail.com

- Well-understood algorithms (3-way merge)
- Handles offline editing

Why it fails for real-time:

- Merge happens on save, not continuously
- Users don't see each other's changes in real-time
- Merge conflicts require manual resolution
- "Your changes conflict with Alice's" - not acceptable for docs

> 💡 **Lesson**
>
> Batch merge is good for code (Git). Real-time editing needs continuous sync.

Attempt 4: Send Every Keystroke to Server

Each keystroke is sent to the server, which broadcasts to all clients.

Why it feels closer:

- Server is the single source of truth
- All clients see the same state
- Simple mental model

Why it still fails:

- User A types at position 5: insert "X"
- User B types at position 5: insert "Y"
- Both operations arrive at server - which goes first?
- If "X" goes first, B's position 5 is now wrong
- **Positions shift - naive approach corrupts the document**

> ⚠️ **Lesson**
>
> Concurrent edits cause position conflicts. You need a transformation algorithm.

The Real Nature of the Problem

Real-time collaboration has a fundamental conflict:

Sold to @theskilledc kumar.dodda@gmail.com

GOAL	CHALLENGE
Instant response	User sees their keystroke immediately (no waiting for server)
Consistency	All users see the same document state
No data loss	Every edit is preserved
Concurrent edits	Multiple users editing simultaneously

Users can't wait for a network round-trip on every keystroke. So they edit locally and sync. But concurrent local edits can conflict.

This tells us the design principles:

1. **Optimistic local editing** - Apply changes locally first, sync later
2. **Conflict-free merge** - Automatically resolve concurrent edits
3. **Eventual consistency** - All clients converge to same state
4. **Intent preservation** - What the user meant should be preserved

The Correct Architecture

Two algorithms dominate real-time collaboration:

APPROACH	USED BY	KEY IDEA
Operational Transform (OT)	Google Docs	Transform operations based on concurrent changes
CRDTs	Figma, Notion	Data structure that merges automatically

Principle 1: Operational Transformation (OT)

Core idea: When you receive a remote operation, *transform* it based on local operations that happened concurrently.

Example:

- Document: "Hello"
- User A: Insert "X" at position 0 → "XHello"
- User B (concurrently): Insert "Y" at position 3 → "HelYlo"

Sold to
@theskilledc
kumar.dodda@gmail.com

Without transformation: B's operation (insert at 3) applied to "XHello" gives "XHeYllo" - wrong position!

With OT: Transform B's operation. A inserted before position 3, so B's position shifts to 4. Result: "XHelYlo" - both intents preserved.

OT requires a **central server** to serialize operations and broadcast transformed versions to all clients.

Principle 2: CRDTs (Conflict-free Replicated Data Types)

Core idea: Design the data structure so that any merge order produces the same result. No central server needed.

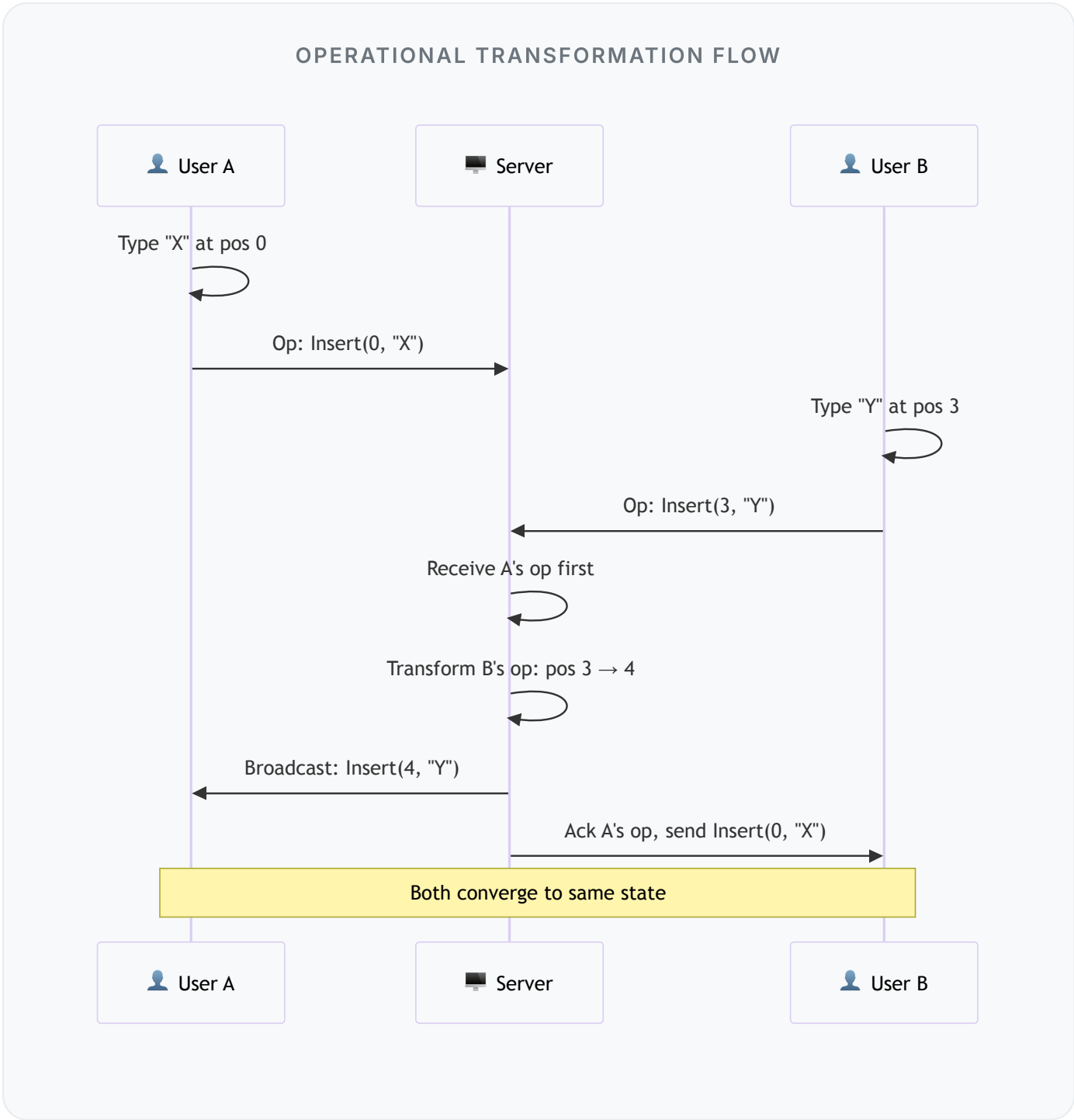
For text, use a structure where each character has a unique, ordered ID:

- Character "H" has ID (user1, seq1)
- Character "e" has ID (user1, seq2)
- When inserting, generate a new unique ID between neighbors
- IDs are totally ordered - merging is just sorting

OT vs CRDT:

ASPECT	OT	CRDT
Server	Required (central)	Optional (peer-to-peer OK)
Complexity	Complex transforms	Complex data structures
Offline	Limited	Excellent
Memory	Low	Higher (stores IDs)

Sold to
@theskilledc
kumar.dodda@gmail.com



Principle 3: Client Architecture

Each client needs:

- **Local document state** - What the user sees
- **Pending operations queue** - Sent to server, waiting for ack
- **Transform engine** - OT or CRDT logic
- **WebSocket connection** - Real-time communication

Client flow:

1. User types → apply locally immediately (optimistic)
2. Add to pending queue → send to server

Sold to
@theskilledc
kumar.dodda@gmail.com

3. Receive remote ops → transform against pending → apply
4. Receive ack → remove from pending queue

Principle 4: Server Architecture

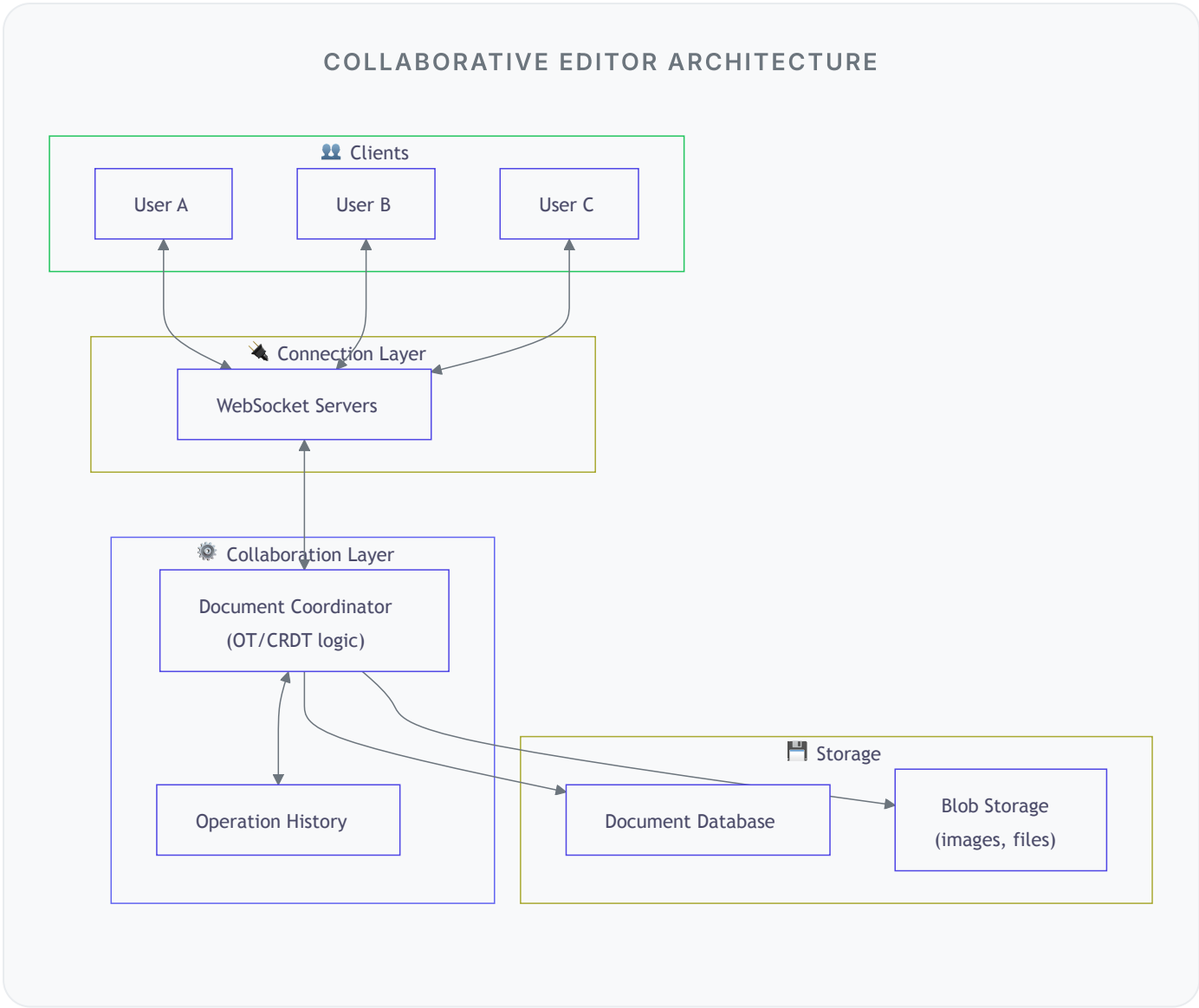
The server must handle:

- **Document sessions** - Track who's editing which document
- **Operation history** - Log all operations for transformation
- **Broadcasting** - Fan-out to all connected clients
- **Persistence** - Save to database periodically

Scaling challenges:

CHALLENGE	SOLUTION
Many documents	Shard by document ID
10K users on one doc	Document-level coordinator + broadcast tree
WebSocket connections	Connection servers separate from logic servers
Persistence	Batch writes, eventual persistence

Sold to
@theskilledc
kumar.dodda@gmail.com



Principle 5: Presence and Cursors

Users need to see where others are editing:

- **Cursor positions** - Show colored cursors for each user
- **Selection highlights** - Show what text others have selected
- **Typing indicators** - Show when someone is active
- **User list** - Who's currently viewing the document

Presence is **ephemeral** - doesn't need persistence. Broadcast via WebSocket, update frequently (every 50-100ms for cursors).

Handling Edge Cases

Offline Editing

- Queue operations locally when offline
- On reconnect, send queued ops to server

Sold to
@theskilledc
kumar.dodda@gmail.com

- Server transforms and merges
- CRDTs handle this more gracefully than OT

Undo/Redo

- Not just "reverse the operation" - other edits may have happened
- Undo = generate an *inverse* operation, apply through OT
- Each user has their own undo stack

Large Documents

- Split into chunks/blocks
- Only sync visible areas initially
- Lazy load distant sections
- Notion does this with blocks

Interview Pivot (How to Explain It)

If asked:

"How would you design a real-time collaborative editor like Google Docs?"

> "The core challenge is concurrent edits. Two users typing at the same position creates a conflict. Naive approaches either block users (locking) or lose data (last-write-wins).
>
> I'd use Operational Transformation or CRDTs. OT transforms operations based on concurrent changes - if User A inserted before my position, my position shifts. CRDTs use unique IDs per character that merge automatically.
>
> Clients edit optimistically - apply locally, then sync. WebSocket connections keep everyone updated in real-time. The server serializes operations for OT or just relays for CRDTs.
>
> For 10K concurrent users, I'd use a document coordinator for OT logic, with WebSocket servers handling connection fanout. Presence (cursors) is broadcast separately, ephemeral."

What's Next?

Collaborative editing leads to deeper questions:

Sold to
@theskilledc
kumar.dodda@gmail.com

How do you handle rich text (bold, links, etc.)?

> **Structured operations:** Not just "insert character" but "apply format" operations. OT transforms format operations too. Or use a tree-based CRDT (Peritext).

How do you handle comments and suggestions?

> **Anchored to positions:** Comments reference a range. When text changes, the anchor must be updated. Similar transformation logic.

How do you implement version history?

> **Periodic snapshots + operation log:** Store snapshots every N operations. To view v5, replay ops from nearest snapshot. Similar to event sourcing.

How do you handle permissions (view-only, comment, edit)?

> **Server-side enforcement:** Validate operations before broadcasting. Reject write ops from view-only users. Client hides UI, but server is the gate.

The insight: Real-time collaboration requires embracing concurrency, not fighting it. Operations will conflict. The algorithm must transform them so everyone converges to the same state, with all intents preserved.

OT for server-centric. CRDTs for peer-to-peer. WebSockets for real-time. Optimistic UI for responsiveness.

Sold to
@theskilledc
kumar.dodda@gmail.com

Same Principles Apply To

- **Collaborative spreadsheets (Google Sheets)** - same OT/CRDT for cells
- **Multiplayer games (state sync)** - same conflict resolution patterns
- **Shared shopping carts** - same concurrent update handling
- **Kanban boards (Trello, Jira)** - same real-time sync with optimistic UI

Sold to
@theskilledc
kumar.dodda@gmail.com

How Would You Design Delayed Scheduling for 200K Reminders That Must Trigger at Precise Times

Time-Based Job Execution at Scale

Delayed scheduling is everywhere:

- **Calendar reminders** - "Remind me at 3 PM"
- **Scheduled notifications** - "Send push at 9 AM user's local time"
- **Subscription billing** - "Charge this card on the 15th"
- **Order timeouts** - "Cancel if not paid in 30 minutes"
- **SLA alerts** - "Escalate if not resolved in 4 hours"

The requirements:

- Schedule a job to run at a specific future time
- Execute with precision (within seconds, not minutes)
- Scale to hundreds of thousands of concurrent scheduled jobs
- Handle spikes (200K reminders all set for 9 AM)
- Survive server restarts and failures



The Scale

200K reminders/minute = 3,300 triggers/second. Many clustered at popular times (9 AM, noon, 5 PM). System must handle 10x spikes without delay.

⚡ The Engineering Complexity

Sold to
@theskilledc
kumar.dodda@gmail.com

'Run this in 5 minutes' sounds trivial. Now schedule 10 million jobs reliably:

⏰ Time Precision

'Run at exactly 3:00:00 PM' - but your clock says 2:59:59, their clock says 3:00:01.

📊 Scale of Jobs

10M scheduled emails. When 3:00 PM hits, all 10M fire together. Spike city.

🔄 Persistence

Scheduler crashes. 1M pending jobs in memory. Gone. Users miss their reminders.

💀 At-Least-Once vs Exactly-Once

Job ran but ack failed. Run again? Now user gets 2 birthday emails.

First Attempts (and Why They Break)

Attempt 1: Cron Jobs

Use cron to run a job every minute that checks for due tasks.

Why it feels right:

- Built into every Unix system
- Simple to set up
- Works for simple scheduled tasks

Why it fails:

- Cron runs at fixed intervals (minute granularity)
- Can't schedule for "3:47:32 PM" - only "3:47 PM"
- Single machine - if it dies, jobs don't run
- Checking 200K jobs every minute is expensive
- No built-in retry, no failure handling

💡 **Lesson**

Cron is for recurring system tasks, not user-scheduled precise triggers.

Sold to @theskilledc kumar.dodda@gmail.com

Attempt 2: Database Polling

Store scheduled jobs in a database. Poll every second for due jobs.

Why it feels right:

- Durable - jobs survive restarts
- Queryable - "SELECT * WHERE trigger_time <= NOW()"
- Can have multiple workers polling

Why it fails:

- Polling every second = 86,400 queries/day per worker
- 200K rows to scan on each poll
- Multiple workers = duplicate execution risk
- Database becomes bottleneck
- Latency: job due at :00.000, poll at :00.500 = 500ms delay

> 💡 **Lesson**
>
> Polling doesn't scale. Every poll scans all pending jobs.

Attempt 3: In-Memory Priority Queue

Load all scheduled jobs into a min-heap sorted by trigger time.

Why it feels right:

- O(log n) insert and extract-min
- No polling - just sleep until next job is due
- Very fast execution

Why it fails:

- 200K jobs × 1KB each = 200MB in memory (manageable)
- But: server restarts = all jobs lost
- Single machine = single point of failure
- Can't scale horizontally
- Adding jobs requires coordination

Sold to
@theskilledc
kumar.dodda@gmail.com

> 💡 **Lesson**
>
> In-memory is fast but not durable. Need persistence + distribution.

Attempt 4: Message Queue with Delay

Use a message queue (RabbitMQ, SQS) with delayed delivery.

Why it's better:

- SQS supports delays up to 15 minutes
- RabbitMQ has delayed message plugins
- Distributed, durable, scales well

Why it's limited:

- SQS max delay: 15 minutes (not enough for "remind me tomorrow")
- Precision varies - not guaranteed sub-second
- Cancellation is hard (message already in queue)
- Modification is impossible (can't change trigger time)

> ⚠️ **Lesson**
>
> Message queues work for short delays. Long delays and modifications need custom design.

The Real Nature of the Problem

Delayed scheduling has competing requirements:

REQUIREMENT	CHALLENGE
Precision	Trigger within seconds of scheduled time
Durability	Jobs survive server failures
Scale	Handle spikes (everyone sets reminder for 9 AM)
Modifiability	Cancel or reschedule before trigger
Exactly-once	Don't send reminder twice

Sold to
@theskilledc
kumar.dodda@gmail.com

You need durability (database) + efficiency (no polling) + distribution (multiple workers).

This tells us the design principles:

1. **Store in database for durability** - Jobs survive failures
2. **Use time buckets** - Avoid scanning all jobs on every tick
3. **Move to hot queue near trigger time** - Precision when it matters
4. **Partition for scale** - Different workers handle different time ranges
5. **Idempotent execution** - Handle duplicate triggers gracefully

The Correct Architecture

Principle 1: Two-Phase Design (Cold → Hot)

Don't treat a job due in 30 days the same as one due in 30 seconds:

PHASE	STORAGE	PURPOSE
Cold	Database (PostgreSQL, DynamoDB)	Long-term storage, queryable, modifiable
Hot	Redis sorted set / in-memory queue	Near-term precision, fast execution

Flow:

1. Job scheduled → store in database (cold)
2. Background worker scans for jobs due in next 5 minutes
3. Move to hot queue (Redis sorted set by trigger time)
4. Executor pulls from hot queue when time is due

Principle 2: Time Bucketing

Instead of scanning all 200K jobs, partition by time window:

- Bucket jobs by minute (or 5-minute window)
- Only scan the current bucket + next bucket
- 200K jobs across 24 hours = ~140 jobs per minute bucket

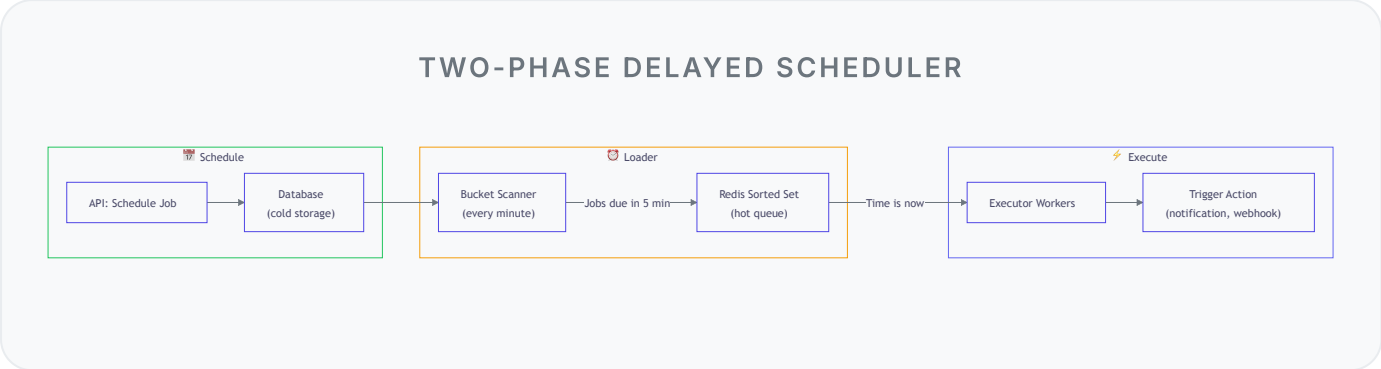
Database schema:

- `trigger_bucket`: "2024-01-15:09:00" (minute granularity)

Sold to @theskilledc kumar.dodda@gmail.com

- Index on `(trigger_bucket, status)`
- Query: "SELECT * WHERE trigger_bucket = '2024-01-15:09:05' AND status = 'pending'"

This turns a full table scan into a precise index lookup.



Principle 3: Redis Sorted Sets for Hot Queue

Redis sorted sets are perfect for this:

- **Score = trigger timestamp**
- **Member = job ID**
- `ZADD hot_queue 1705312800 job:12345`
- `ZRANGEBYSCORE hot_queue 0 NOW LIMIT 100` - get due jobs

Why Redis works:

- O(log n) operations
- Atomic operations (no race conditions)
- Can remove by job ID (for cancellation)
- Sub-millisecond latency

Principle 4: Distributed Execution with Locking

Multiple workers pulling from the same queue risks duplicate execution:

- **Option A: Claim with atomic pop**
  - `ZPOPMIN` - atomically removes and returns the smallest score
  - Only one worker gets each job
- **Option B: Distributed lock**
  - Acquire lock on job ID before execution
  - Release after completion (or timeout)

Sold to
@theskilledc
kumar.dodda@gmail.com

Failure handling: If worker crashes mid-execution, have a reaper process that reclaims stuck jobs after timeout.

Principle 5: Cancellation and Modification

Users change their minds. The system must support:

- **Cancel:** Mark as cancelled in DB, remove from Redis if present
- **Reschedule:** Update trigger time in DB, remove and re-add to Redis

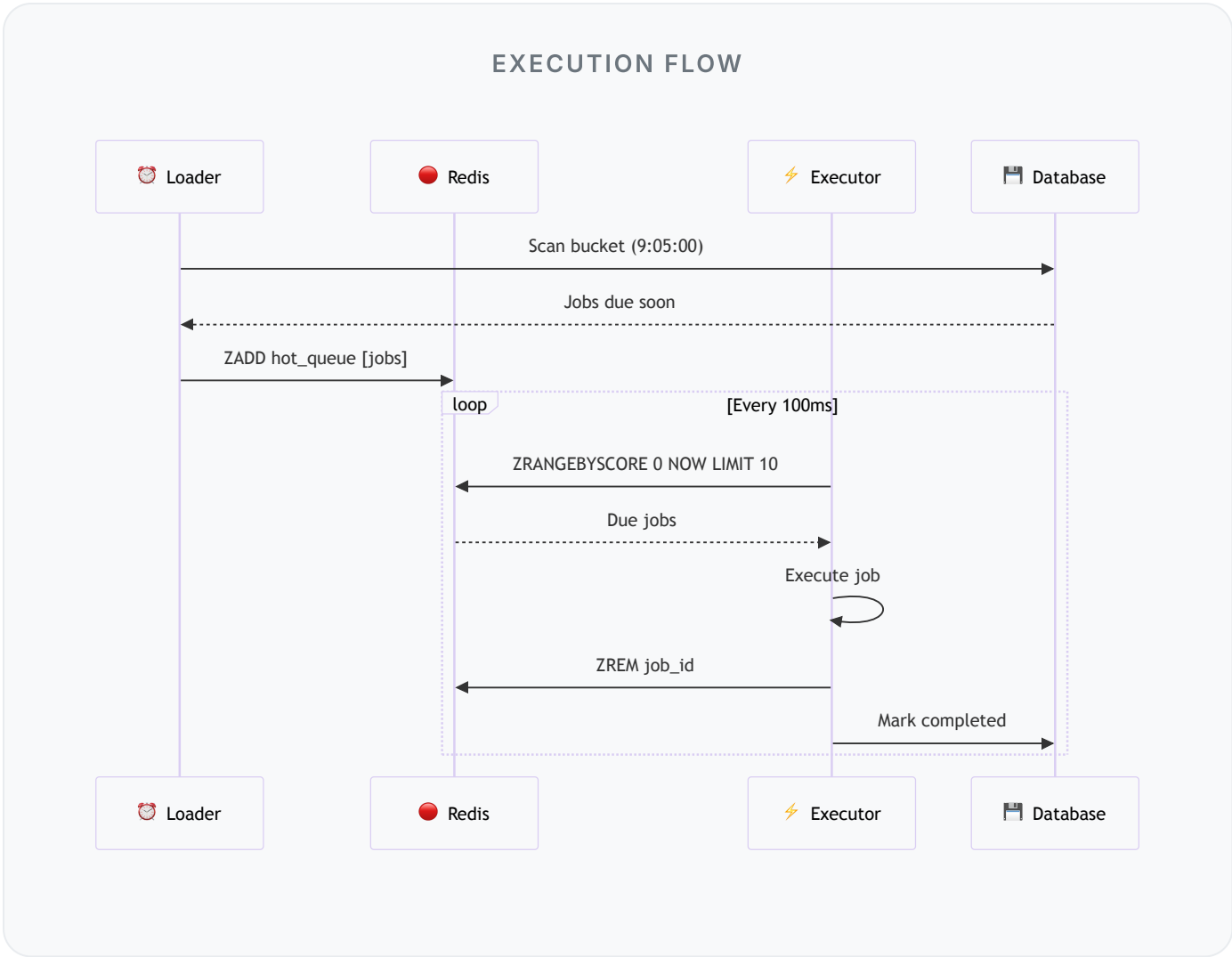
Edge case: Job is being executed while cancel request arrives. Use a status check before executing: "Is this job still pending?"

Handling Time Spikes

Everyone sets reminders for 9:00 AM. How do you handle the spike?

STRATEGY	HOW IT WORKS
Pre-load	Scan the 9:00 AM bucket at 8:55, not 9:00
Auto-scale executors	More workers during predicted peaks
Spread the load	Add small random jitter (±30 sec) if precision isn't critical
Rate limit per action type	Notifications can be slightly delayed, billing cannot

Sold to
@theskilledc
kumar.dodda@gmail.com



Achieving Exactly-Once Execution

Sending a reminder twice is bad. Not sending it is worse. How to guarantee exactly-once?

Reality check: True exactly-once is impossible in distributed systems. Instead, design for **at-least-once with idempotency**:

- **Idempotent actions:** Executing twice has same effect as once
- **Deduplication ID:** Each job has unique ID; downstream checks before acting
- **Status tracking:** Mark job as "executing" before action, "completed" after

If worker crashes after sending notification but before marking complete, the job may be retried. The notification service deduplicates by job ID.

Interview Pivot (How to Explain It)

If asked:

"How would you design delayed scheduling for 200K reminders with precise timing?"

Sold to
@theskilledc
kumar.dodda@gmail.com

"The core challenge is balancing durability with precision. Jobs need to survive failures (database) but execute on time (in-memory efficiency).

I'd use a two-phase design: cold storage in a database with time-bucketed indexing, hot queue in Redis sorted sets for near-term precision. A loader process moves jobs from cold to hot 5 minutes before trigger time.

Executors poll the Redis sorted set for due jobs using ZRANGEBYSCORE. Atomic operations prevent duplicate execution. For spikes at popular times like 9 AM, I'd pre-load the bucket early and auto-scale executors.

Exactly-once is achieved through idempotent downstream actions and deduplication IDs."

What's Next?

Delayed scheduling leads to deeper questions:

How do you handle recurring schedules (every day at 9 AM)?

> **Separate recurrence rule from instances:** Store cron expression or RRULE. Generate next instance after each execution. Don't pre-generate years of instances.

How do you handle timezone-aware scheduling?

> **Store in UTC + user timezone:** Convert at load time. Handle DST transitions (2 AM might not exist or happen twice).

How do you handle very long delays (1 year)?

> **Stay in cold storage:** Only load to hot queue near trigger time. Periodically validate job still exists (user might have deleted account).

Sold to
@theskilledc
kumar.dodda@gmail.com

How do you prioritize different job types?

> **Priority queues:** Critical jobs (billing) get dedicated queue. Or use weighted scoring: trigger_time - (priority × 1000).

The insight: Delayed scheduling is a two-phase problem. Long-term storage needs durability and modifiability (database). Near-term execution needs precision and speed (Redis). Move jobs from cold to hot as trigger time approaches.

Bucket by time. Load to hot queue early. Execute with locks. Handle idempotently.

Same Principles Apply To

- **Email campaign scheduling** - same cold-to-hot queue migration
- **Payment retry systems** - same time-bucketed execution
- **Workflow engines (Temporal, Airflow)** - same delayed task patterns
- **Subscription renewal systems** - same scheduled job with idempotency

Sold to
@theskilledc
kumar.dodda@gmail.com

How Would You Design a News Feed for 1 Billion Users

The Problem That Defined Social Media Architecture

Every major social platform has solved this problem:

- **Facebook** - The original News Feed, 2+ billion daily users
- **Twitter/X** - Timeline for 500M+ users
- **Instagram** - Photo feed for 2B+ users
- **LinkedIn** - Professional feed for 900M+ users
- **TikTok** - For You Page for 1B+ users

The scenario looks simple:

- User A follows 500 people
- User A opens the app
- Show the latest posts from those 500 people, ranked by relevance

At **1 billion users**, each following hundreds of accounts, each posting multiple times per day - this simple scenario becomes an engineering nightmare.

⚡ The Engineering Complexity

You follow 500 people. Loading their posts instantly is a fan-out nightmare:

📥 Fan-Out Problem

User follows 500 people. On load, fetch posts from 500 accounts? That's 500 queries.

📥 Push vs Pull

Push new posts to all followers? Celebrity with 100M followers posts = 100M writes.

Sold to
@theskilledc
kumar.dodda@gmail.com

⏱️ Ranked on the Fly

Chronological is easy. 'Most relevant' requires scoring. Scoring 1000 posts in 100ms.

🔄 Updates After Load

User scrolls. New posts arrive. Inject them? Shuffle the feed? Annoying either way.

📊 **The Scale**

1B users × average 200 follows = 200B follow relationships. 500M posts/day. 10B+ feed requests/day. Each feed must render in under 200ms.

First Attempts (and Why They Break)

Attempt 1: Query on Read (Pull Model)

When user opens feed, query all their followed accounts for recent posts.

```sql
SELECT * FROM posts
WHERE author_id IN (
    SELECT followed_id FROM follows WHERE follower_id = ?
)
ORDER BY created_at DESC
LIMIT 20;
```

Why it feels right:

- Simple to implement
- Always shows the freshest content
- No storage overhead for feeds

Why it fails:

- User follows 500 people → query 500 accounts on every feed load
- Each account might have millions of posts → massive index scans
- At 10B feed requests/day, database collapses
- Latency: 500+ ms just to fetch, then rank, then render

Sold to
@theskilledc
kumar.dodda@gmail.com

> 💡 **Lesson**
>
> Querying at read time doesn't scale. The work is proportional to (follows × posts per account) on every single request.

Attempt 2: Precompute Everything (Push Model / Fan-Out-On-Write)

When a user posts, immediately push that post to all followers' feeds.

```python
def publish_post(author_id, post):
    save_to_posts_table(post)

    # Push to all followers' feeds
    followers = get_followers(author_id)
    for follower in followers:
        append_to_feed(follower, post)
```

Why it feels right:

- Read is instant - feed is precomputed
- Just fetch from user's feed table
- Constant-time reads regardless of follow count

Why it fails:

- **Celebrity problem**: Taylor Swift has 90M followers
- One tweet = 90 million writes
- Write amplification is catastrophic
- Storage: 90M copies of the same post
- Latency: Her followers won't see the post for minutes

> ⚠️ **The Celebrity Problem**
>
> A single post from a celebrity with 100M followers would require 100M writes. At 1KB per entry = 100GB of storage for ONE post. Unacceptable.

> 💡 **Lesson**
>
> Fan-out-on-write works for normal users. Celebrities break it completely.

Sold to
@theskilledc
kumar.dodda@gmail.com

Attempt 3: Cache the Feed

Compute the feed once, cache it in Redis. Serve from cache on subsequent reads.

Why it feels right:

- Fast reads from cache
- Reduces database load
- Standard caching pattern

Why it fails:

- 1B users × 1KB feed = 1TB just for feeds
- Cache invalidation nightmare - when does feed expire?
- Cold start: First load after cache miss is still slow
- Doesn't solve the celebrity problem

> 💡 **Lesson**
>
> Caching helps, but doesn't solve the fundamental architecture problem.

Attempt 4: Shard by User

Partition the posts table by user. Each user's posts live on a dedicated shard.

Why it feels right:

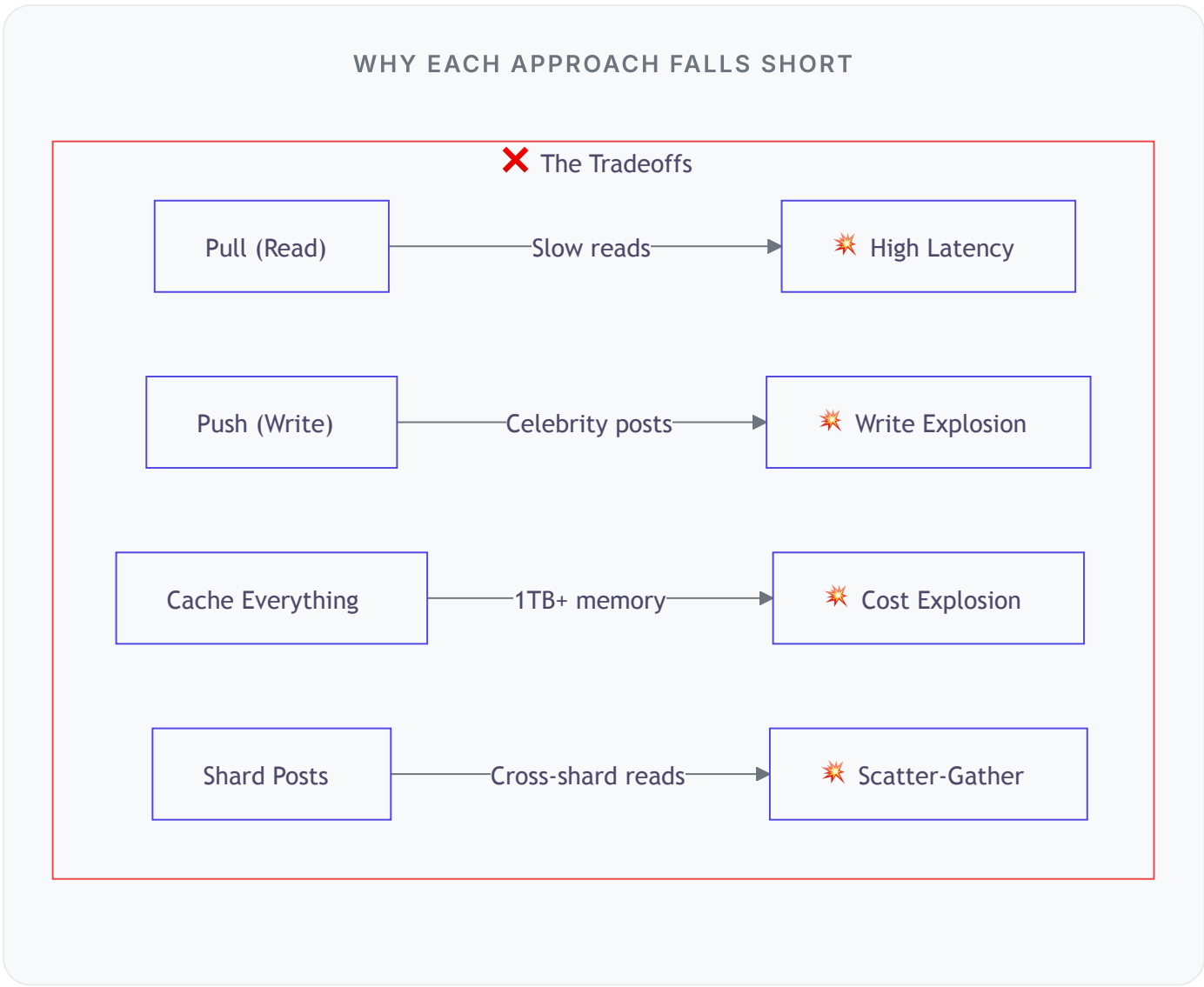
- Horizontal scaling
- Each shard handles less load
- Industry standard

Why it fails for feeds:

- User follows 500 people across 50 different shards
- Feed generation requires querying 50 shards
- Cross-shard queries are slow and complex
- Scatter-gather pattern doesn't scale

> 💡 **Lesson**
>
> Sharding solves write scale, but feed reads span shards. Different problem.

Sold to
@theskilledc
kumar.dodda@gmail.com



The Real Nature of the Problem

News feed design is fundamentally about a tradeoff:

APPROACH	WRITE COST	READ COST	BEST FOR
Pull (Fan-out-on-read)	O(1)	O(follows)	Users who follow few accounts
Push (Fan-out-on-write)	O(followers)	O(1)	Users with few followers

The insight: Use BOTH approaches. Push for normal users. Pull for celebrities.

This tells us the design principles:

1. **Hybrid fan-out** - Push for most, pull for celebrities
2. **Precomputed feeds** - Store feed as a list of post IDs, not full posts
3. **Merge at read time** - Combine precomputed feed + celebrity posts

Sold to @theskilledc kumar.dodda@gmail.com

4. **Rank and filter** - Apply ML-based ranking before serving

The Correct Architecture

Principle 1: Hybrid Fan-Out

Classify users into two categories:

USER TYPE	FOLLOWER COUNT	STRATEGY
Normal User	< 10,000	Fan-out-on-write (push to followers)
Celebrity	> 10,000	Fan-out-on-read (pull at feed generation)

When a normal user posts:

- Push post ID to all followers' feed caches
- Immediate visibility
- Manageable write amplification (10K writes max)

When a celebrity posts:

- Do NOT fan out
- Store in celebrity's posts list
- Fetch at read time, merge with precomputed feed

HYBRID FAN-OUT STRATEGY



Principle 2: Feed as a List of Post IDs

Don't store full posts in the feed. Store only post IDs.

Sold to
@theskilledc
kumar.dodda@gmail.com

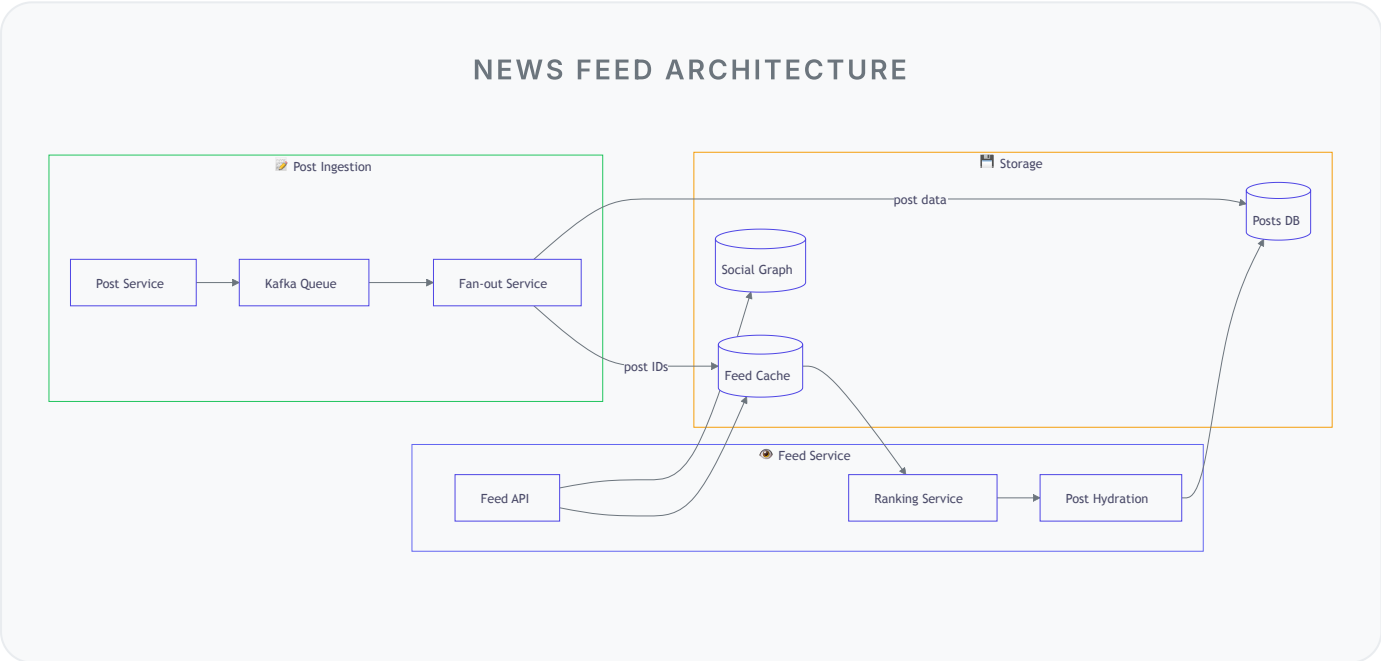
```
// User 123's Feed (in Redis)
feed:123 = [post_id_999, post_id_998, post_id_997, ...]

// Hydrate at read time
posts = fetch_posts([post_id_999, post_id_998, ...])
```

Why this works:

- Post IDs are tiny (8 bytes) vs full posts (1KB+)
- 100x storage reduction
- Posts can be cached separately (deduplication)
- Post updates don't require feed updates

Principle 3: The Feed Generation Pipeline

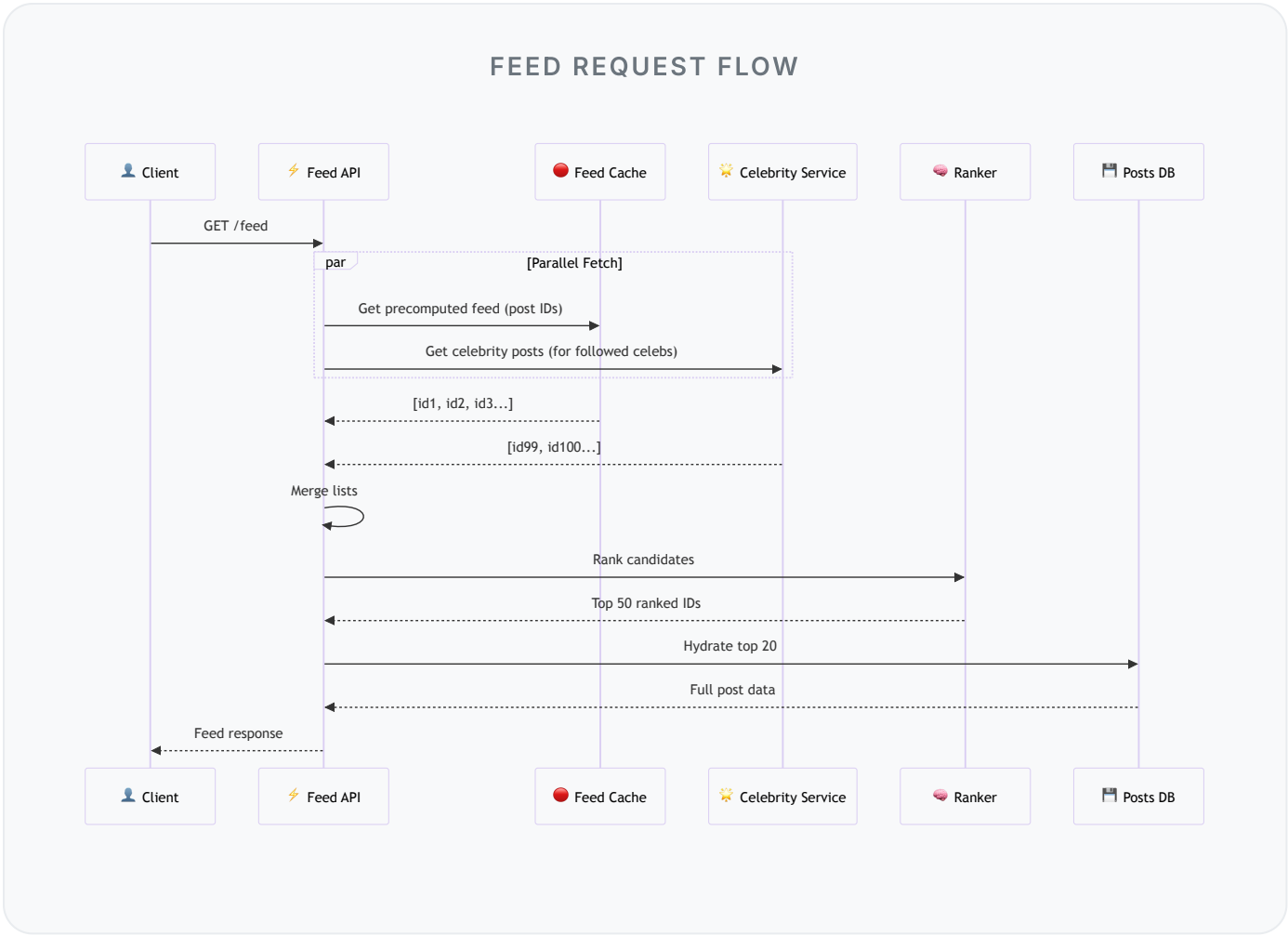


Principle 4: Feed Generation Flow

When User A requests their feed:

1. **Fetch precomputed feed** - Post IDs from fan-out
2. **Fetch celebrity posts** - Query celebrities User A follows
3. **Merge** - Combine both lists
4. **Rank** - Apply ML model (engagement prediction)
5. **Hydrate** - Fetch full post data for top N
6. **Return** - Send to client

Sold to
@theskilledc
kumar.dodda@gmail.com



Principle 5: Ranking the Feed

Chronological feeds are dead. Users expect **relevance-based ranking**.

Ranking signals:

SIGNAL	WHAT IT MEASURES	WEIGHT
Affinity	How often do you interact with this author?	High
Recency	How old is the post?	Medium
Engagement	How many likes/comments/shares?	Medium
Content type	Do you prefer videos, images, or text?	Medium
Predicted engagement	Will YOU engage with this post?	High

Ranking formula (simplified):

TEXT

```
score = (affinity × 0.3) +
        (engagement × 0.2) +
        (recency_decay × 0.2) +
        (predicted_ctr × 0.3)
```

Sold to
@theskilledc
kumar.dodda@gmail.com

Principle 6: Storage Architecture

DATA	STORAGE	WHY
Posts	MySQL/PostgreSQL + Cache	Durability, rich queries
Feed (post IDs)	Redis Sorted Set	Fast reads, sorted by time
Social Graph	Graph DB or adjacency list	Who follows whom
User Features	Feature Store (Redis)	ML features for ranking

Handling Edge Cases

New User (Cold Start)

- No followers yet → empty feed
- **Solution:** Show popular/trending content until graph builds
- Suggest accounts to follow based on interests

Inactive User Returns

- Feed cache may be stale or evicted
- **Solution:** Regenerate feed on first request (pull model)
- Background job to warm up feeds for returning users

Viral Post (Thundering Herd)

- Celebrity post goes viral → millions fetch the same post
- **Solution:** Cache hot posts aggressively at CDN edge
- Use read-through cache with request coalescing

Deleted or Edited Posts

- Post ID is in feed, but post was deleted
- **Solution:** Handle gracefully at hydration time (skip missing)
- Eventual cleanup via background job

Privacy / Blocking

- User blocks someone → their posts shouldn't appear
- **Solution:** Filter at ranking stage, not fan-out

Sold to
@theskilledc
kumar.dodda@gmail.com

- Maintain block list in user's session context

Interview Pivot (How to Explain It)

If asked:

"How would you design a news feed for 1 billion users?"

> "The core challenge is fan-out. Pushing every post to every follower doesn't scale for celebrities with millions of followers. Pulling at read time is too slow for users who follow hundreds of accounts.
>
> I'd use a hybrid approach: fan-out-on-write for normal users (under 10K followers), and fan-out-on-read for celebrities. Store feeds as lists of post IDs in Redis, not full posts. At read time, merge the precomputed feed with celebrity posts, rank using ML signals, then hydrate the top results.
>
> This gives O(1) reads for most content while handling celebrities gracefully. Ranking ensures relevance, and caching at multiple layers handles the load."

What's Next?

News feed design leads to deeper questions:

How do you handle real-time updates (new posts while viewing)?

> **WebSocket for push:** Notify client of new posts. Show "N new posts" indicator. Client can pull updates or full refresh. Don't auto-scroll - it's jarring.

How do you paginate efficiently?

> **Cursor-based pagination:** Use the last post's timestamp or ID as cursor. Avoids offset-based issues. Handle posts inserted between pages gracefully.

Sold to
@theskilledc
kumar.dodda@gmail.com

How do you A/B test different ranking algorithms?

> **Experiment framework:** Assign users to buckets. Each bucket uses a different ranker. Track engagement metrics. Roll out winners gradually.

How do you prevent echo chambers / filter bubbles?

> **Diversity injection:** Intentionally include posts outside the user's usual preferences. Track content diversity as a metric. Balance engagement with exposure to varied viewpoints.

The insight: News feed is a problem of extremes. Most users post rarely and have few followers - fan-out-on-write works. Celebrities have millions of followers - only fan-out-on-read works. The solution is hybrid: treat different users differently.

Push for the many. Pull for the few. Rank for relevance. Cache for speed.

Same Principles Apply To

- **Email inbox (Gmail, Outlook)** - same hybrid push/pull model
- **Activity feeds (GitHub, Slack)** - same fan-out strategies
- **Notification feeds** - same precomputed feed with ranking
- **For You pages (TikTok, YouTube)** - same ML-based ranking pipeline

Sold to
@theskilledc
kumar.dodda@gmail.com

How Would You Design a Web Crawler for 1 Billion Pages

First-Principles Thinking: From Problem to Architecture

Before we dive into solutions, let's understand how to **think about** this problem. System design interviews test your ability to reason from first principles - not just recall architectures you've memorized.

> 🧠 **First-Principles Approach**
>
> Break the problem into fundamental truths. Build up from there. Don't start with "I'll use Kafka and Redis." Start with "What does this system fundamentally need to do?"

Step 1: What Does a Web Crawler Fundamentally Do?

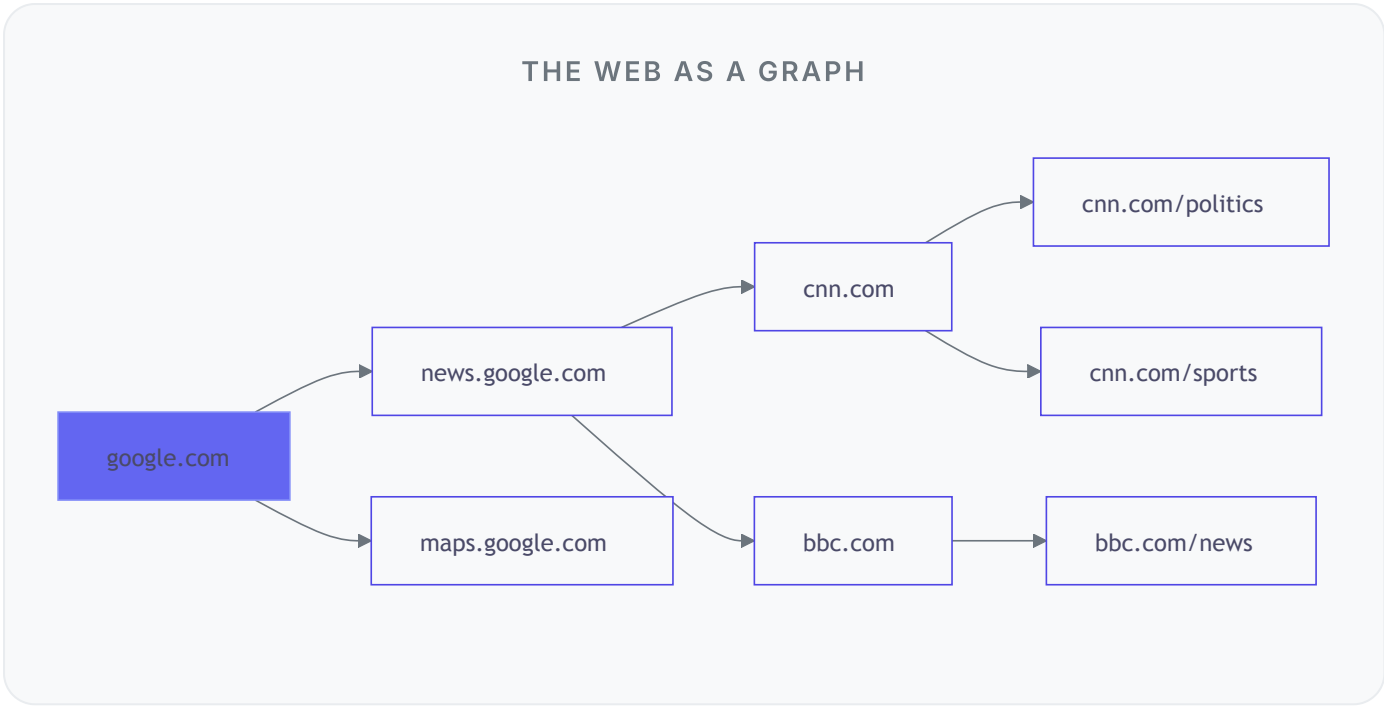
Strip away all complexity. At its core, a web crawler does this:

1. **Start** with a list of URLs (seed URLs)
2. **Fetch** the content of a URL
3. **Extract** new URLs from the content
4. **Store** the content somewhere
5. **Repeat** for the extracted URLs

That's it. A web crawler is essentially a **graph traversal algorithm** where:

- Nodes = web pages
- Edges = hyperlinks
- Traversal = BFS or priority-based

Sold to
@theskilledc
kumar.dodda@gmail.com



Step 2: What Are the Real Constraints?

Now ask: **What makes this hard at scale?**

CONSTRAINT	WHY IT MATTERS	IMPLICATION
1 Billion Pages	Can't fit in one machine's memory	Need distributed crawling
Network latency	Each fetch takes 100ms-2s	Need massive parallelism
Politeness	Can't DDoS websites	Need rate limiting per domain
Duplicate URLs	Same page, many URLs	Need URL normalization + deduplication
Changing content	Pages get updated	Need re-crawl scheduling
Storage	1B pages × 100KB = 100TB	Need distributed storage

The constraints tell us: This is a distributed, parallel, rate-limited graph traversal with deduplication and persistent storage.

Step 3: Do the Math (Back-of-Envelope)

Goal: Crawl 1 billion pages in 1 month.

Sold to
@theskilledc
kumar.dodda@gmail.com

```
1 billion pages ÷ 30 days = 33 million pages/day
33 million ÷ 86,400 seconds = ~380 pages/second

If each fetch takes 500ms average:
  - 1 thread = 2 pages/second
  - Need 380 ÷ 2 = 190 concurrent fetchers

But with network variance and politeness delays:
  - Need 500-1000 concurrent fetchers to be safe
  - Distributed across 50-100 machines
```

TEXT

Storage:

```
1 billion pages × 100KB average = 100 TB raw content
After compression (~5x): ~20 TB
Plus metadata, indexes: ~30-50 TB total
```

TEXT

Bandwidth:

```
380 pages/sec × 100KB = 38 MB/sec = ~300 Mbps sustained
Burst capacity needed: 1-2 Gbps
```

TEXT

> 💡 **Why Math Matters**
>
> Back-of-envelope calculations tell you if your design is feasible. If the math doesn't work, no architecture will save you.

First Attempts (and Why They Break)

Attempt 1: Single-Machine BFS

Sold to
@theskilledc
kumar.dodda@gmail.com

```python
def crawl(seed_urls):
    queue = deque(seed_urls)
    visited = set()

    while queue:
        url = queue.popleft()
        if url in visited:
            continue

        content = fetch(url)
        visited.add(url)
        save(url, content)

        for link in extract_links(content):
            queue.append(link)
```

Why it feels right:

- Simple BFS traversal
- Classic algorithm textbook approach
- Works for small crawls

Why it fails:

- **Sequential fetching** - 500ms per page = 2 pages/sec
- **Memory** - visited set of 1B URLs = 50+ GB RAM
- **Single point of failure** - machine dies, lose everything
- **No politeness** - will get blocked by websites

> 💡 **Lesson**
>
> Sequential algorithms don't scale. Network I/O is the bottleneck, not CPU.

Attempt 2: Multi-threaded Crawler

Add threads to parallelize fetching.

Why it's better:

- 100 threads = 200 pages/sec
- Better utilization of network

Why it still fails:

- All threads hitting same domain = DDoS

Sold to
@theskilledc
kumar.dodda@gmail.com

- Shared queue becomes contention point
- Still single machine - can't scale beyond one box
- No fault tolerance

> 💡 **Lesson**
>
> Parallelism helps, but without politeness you'll get blocked. Without distribution you're limited to one machine.

Attempt 3: Just Distribute the Queue

Put URLs in Kafka/Redis. Multiple workers consume and fetch.

Why it's better:

- Horizontal scaling
- Fault tolerance (workers can restart)

Why it still fails:

- **No coordination** - multiple workers fetch same URL
- **No politeness** - multiple workers hammer same domain
- **Deduplication** - where does visited set live?

> ⚠️ **Lesson**
>
> Distribution without coordination creates chaos. You need to partition work intelligently.

Step 4: Identify the Core Sub-Problems

Step back. What sub-problems must we solve?

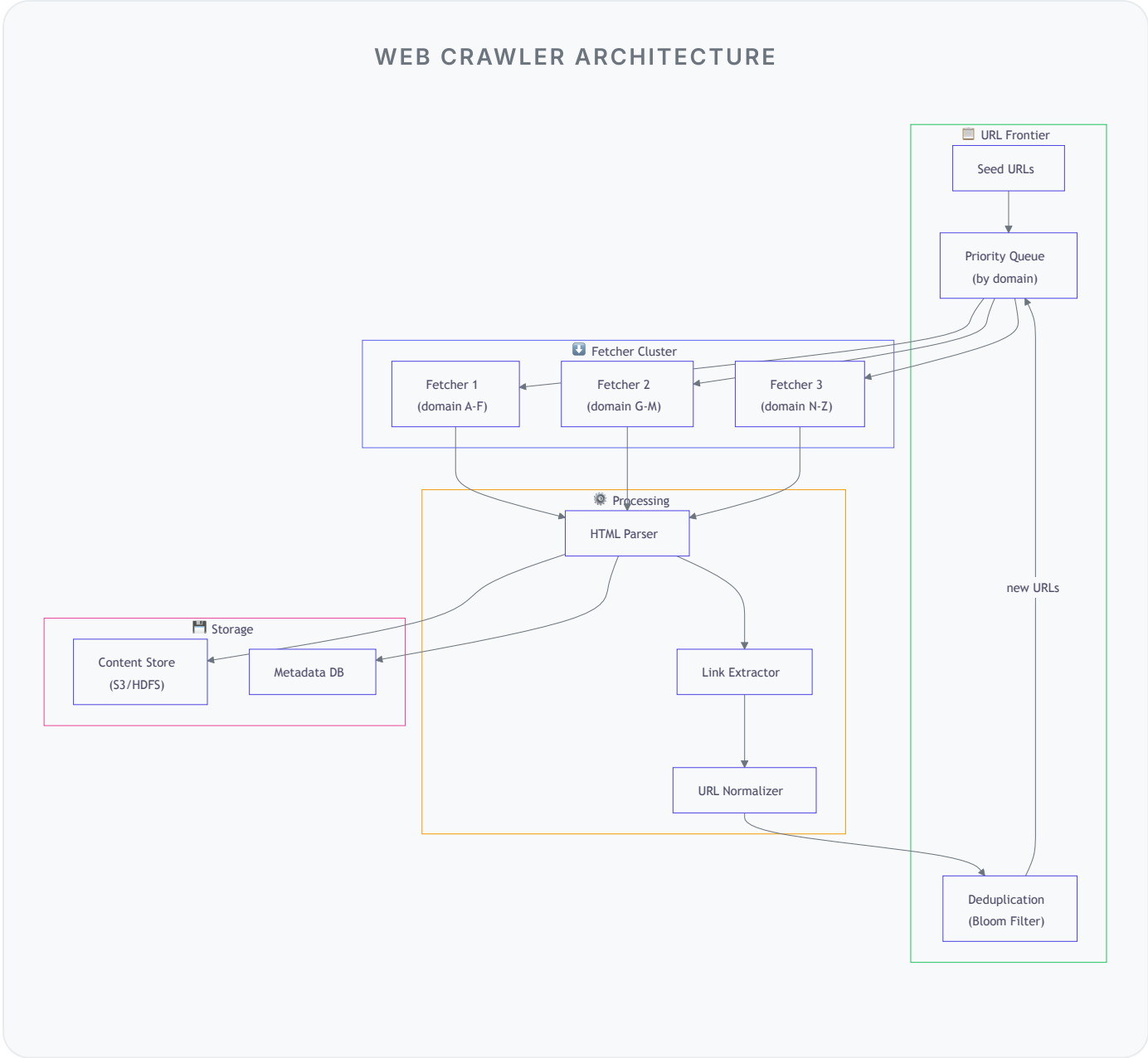
Sold to
@theskilledc
kumar.dodda@gmail.com

SUB-PROBLEM	QUESTION	SOLUTION CATEGORY
URL Frontier	Which URLs to crawl next?	Priority queue + scheduling
URL Deduplication	Have we seen this URL before?	Bloom filter + URL store
Politeness	How to avoid hammering sites?	Per-domain rate limiting
Fetching	How to download pages fast?	Distributed fetchers
Parsing	How to extract links?	HTML parser + URL normalization
Storage	Where to store content?	Distributed file system
Coordination	How do workers divide work?	Partition by domain

Each sub-problem is a building block. Solve each one, then compose them.

The Correct Architecture

Sold to
@theskilledc
kumar.dodda@gmail.com



Component 1: URL Frontier (The Brain)

The URL Frontier manages which URLs to crawl and in what order.

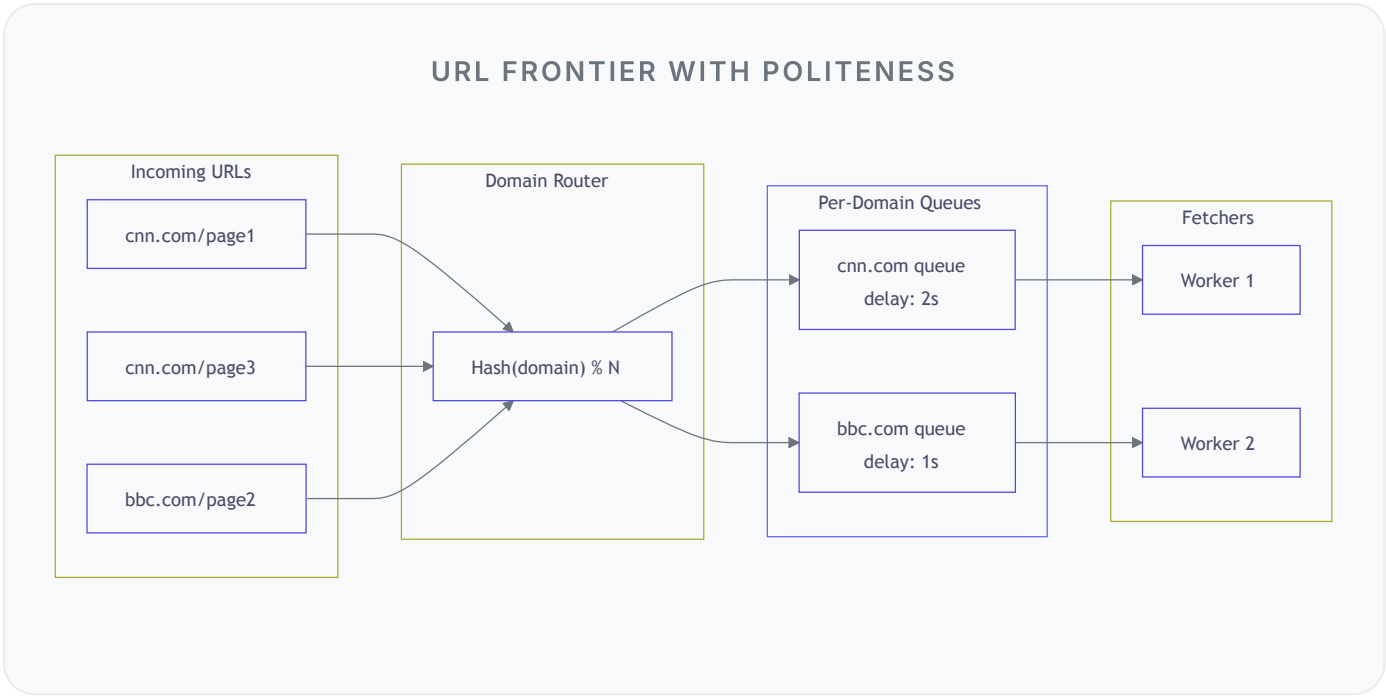
Key requirements:

- Prioritize important/fresh pages
- Ensure politeness (per-domain rate limits)
- Distribute URLs to appropriate workers

Design:

- Partition URLs by domain (hash of domain → worker)
- Each partition has a priority queue
- Priority = (importance × freshness)
- Delay between requests to same domain (1-10 seconds)

Sold to
@theskilledc
kumar.dodda@gmail.com



Component 2: URL Deduplication

How do you check if you've seen a URL among 1 billion URLs?

Challenge: 1B URLs × 100 bytes = 100GB. Too big for memory.

Solution: Bloom Filter + Persistent Store

- **Bloom Filter** - probabilistic; says "definitely not seen" or "maybe seen"
- 1B items with 1% false positive = ~1.2 GB memory
- If Bloom says "maybe seen," check persistent store
- Trade-off: tiny % of duplicates vs massive memory savings

PYTHON

```python
def should_crawl(url):
    normalized = normalize_url(url)

    # Fast check (in-memory)
    if not bloom_filter.might_contain(normalized):
        bloom_filter.add(normalized)
        return True  # Definitely new

    # Slow check (disk)
    if not url_store.exists(normalized):
        url_store.add(normalized)
        return True  # Actually new (false positive)

    return False  # Already crawled
```

Component 3: URL Normalization

The same page can have many URLs:

Sold to
@theskilledc
kumar.dodda@gmail.com

```text
https://example.com/page
https://example.com/page/
https://example.com/page?ref=twitter
https://EXAMPLE.COM/page
http://www.example.com/page
```

Normalization rules:

- Lowercase the domain
- Remove default ports (80, 443)
- Remove trailing slashes
- Remove tracking parameters (utm_*, ref, etc.)
- Resolve relative URLs to absolute
- Handle URL encoding consistently

Component 4: Politeness & robots.txt

Politeness means not overwhelming websites:

- Respect `robots.txt` - which pages are allowed
- Respect `Crawl-delay` directive
- Default: 1 request per domain per second
- Identify yourself: `User-Agent: MyBot/1.0`

robots.txt example:

```text
User-agent: *
Disallow: /admin/
Disallow: /private/
Crawl-delay: 2

User-agent: Googlebot
Allow: /
```

Implementation:

- Cache robots.txt per domain (refresh every 24h)
- Check before fetching each URL
- Track last request time per domain
- Wait if too soon since last request

Sold to
@theskilledc
kumar.dodda@gmail.com

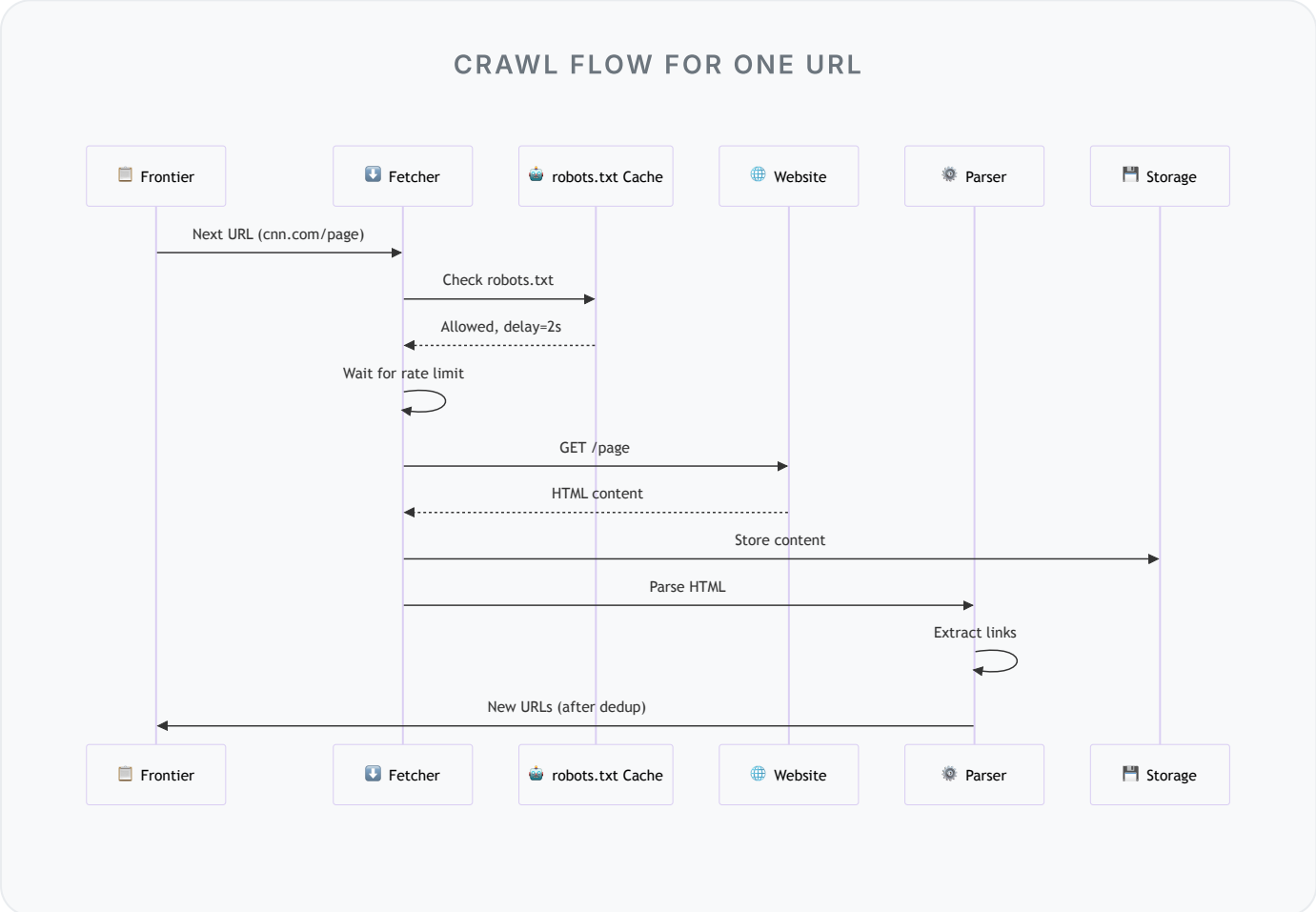
Component 5: Distributed Fetching

Key insight: Partition by domain, not by URL.

- Worker 1 handles domains starting with A-F
- Worker 2 handles domains starting with G-M
- etc.

Why partition by domain?

- Politeness is per-domain - one worker tracks one domain's rate limit
- No coordination needed between workers for politeness
- robots.txt cached once per worker



Component 6: Content Storage

100TB of content needs distributed storage:

Sold to
@theskilledc
kumar.dodda@gmail.com

OPTION	PROS	CONS
S3/GCS	Cheap, scalable, durable	Higher latency
HDFS	Good for batch processing	Complex to manage
Custom	Optimized for access patterns	Build/maintain burden

Storage schema:

```text
/crawl-data/
  /2024/01/15/
    /cnn.com/
      page1.html.gz
      page2.html.gz
    /bbc.com/
      ...

Metadata DB:
  url: string (primary key)
  content_hash: string
  last_crawled: timestamp
  content_location: string (S3 path)
  status_code: int
  content_type: string
```

Handling Re-Crawls (Freshness)

Pages change. How often to re-crawl?

PAGE TYPE	RE-CRAWL FREQUENCY	WHY
News homepage	Every 15 minutes	Changes constantly
Blog post	Every week	Rarely changes
Wikipedia article	Every day	Moderate changes
Product page	Every day	Prices change

Approach: Adaptive scheduling based on change history.

- Track how often each page actually changes
- Pages that change often → crawl more frequently
- Pages that never change → crawl less

Sold to
@theskilledc
kumar.dodda@gmail.com

- Use content hash to detect actual changes

Interview Pivot (How to Explain It)

If asked:

"How would you design a web crawler for 1 billion pages?"

> "Let me break this down from first principles. A web crawler is fundamentally a distributed graph traversal - fetching pages and extracting links.
>
> The key constraints are: network latency requires parallelism, politeness requires per-domain rate limiting, and 1B URLs requires distributed deduplication.
>
> I'd partition work by domain - each worker handles specific domains. This keeps politeness simple (one worker tracks one domain's rate). URLs go into a frontier with priority queues. Deduplication uses a Bloom filter for fast rejection, backed by persistent storage.
>
> For 1B pages in a month, I need ~400 pages/sec. With 500ms average fetch time, that's ~200 concurrent fetchers across 50-100 machines. Content goes to S3/HDFS, metadata to a database."

What's Next?

Web crawlers lead to deeper questions:

How do you handle JavaScript-rendered pages?

> **Headless browser (Puppeteer, Playwright):** Render JS before extracting content. Much slower and resource-intensive. Use selectively for JS-heavy sites.

How do you detect duplicate content (not just URLs)?

> **Content fingerprinting:** SimHash or MinHash of text content. Same content with different URLs = near-duplicate. Cluster and keep one.

Sold to
@theskilledc
kumar.dodda@gmail.com

How do you prioritize which pages to crawl first?

> **PageRank-like scoring:** Pages linked from important pages are important. Also consider domain authority, freshness needs, and user queries.

How do you handle crawler traps (infinite URLs)?

> **Depth limits:** Don't crawl beyond N links from seed. **URL pattern detection:** Detect calendars, session IDs generating infinite URLs. **Per-domain page limits:** Cap pages per domain.

The first-principles approach: Don't start with tools. Start with "What does this system fundamentally need to do?" Then identify constraints. Do the math. Break into sub-problems. Solve each. Compose.

Partition by domain for politeness. Bloom filter for deduplication. Priority queue for scheduling. Distributed fetchers for parallelism. That's the architecture.

Same Principles Apply To

- **API scrapers (data aggregation)** - same rate limiting and deduplication
- **Link validators (broken link checkers)** - same distributed fetching
- **Price monitoring systems** - same adaptive re-crawl scheduling
- **Search engine indexing (Googlebot)** - same URL frontier architecture

Sold to @theskilledc
kumar.dodda@gmail.com

How Would You Build a Rate Limiter for 1M Requests/Second

The Bouncer at the Door of Your System

It's 2:47 AM. Your phone buzzes. Then again. Then it doesn't stop.

You check PagerDuty: **"API latency spike. Database connections exhausted. Customer complaints incoming."**

What happened? A single customer's script gone rogue - 50,000 requests per second instead of 50. Their bug is now your outage.

Or maybe it's not a bug. Maybe someone is scraping your entire catalog. Or testing your API for vulnerabilities. Or running a DDoS attack disguised as legitimate traffic.

Without rate limiting, one bad actor can consume resources meant for millions.

Rate limiting is the **bouncer at the door** - deciding who gets in, how often, and telling the rest to wait or go home.

🎯 Why Rate Limiting Matters

- **Protection** - Prevent abuse and DDoS
- **Fairness** - No single user hogs resources
- **Cost control** - Limit expensive operations
- **Stability** - Keep the system responsive under load

⚡ The Engineering Complexity

'100 requests per minute' is simple math. Enforcing it across distributed servers isn't:

Sold to
@theskilledc
kumar.dodda@gmail.com

⏱️ Time Windows

100 requests per minute. But which minute? Sliding window? Fixed window? Different results.

🌐 Distributed State

User hits Server A, then Server B. How does B know A already counted 50 requests?

🎯 Granularity

Limit per user? Per IP? Per endpoint? All three? Each multiplies storage.

⚡ Latency Budget

Rate check on every request. If check takes 10ms, you've added 10ms to everything.

First Attempts (and Why They Break)

Attempt 1: Count Requests Per Second in Memory

The simplest idea: keep a counter for each user, reset every second.

PYTHON

```python
counters = {}  # user_id → count
last_reset = time.time()

def is_allowed(user_id, limit=100):
    global last_reset

    # Reset every second
    if time.time() - last_reset ≥ 1:
        counters.clear()
        last_reset = time.time()

    counters[user_id] = counters.get(user_id, 0) + 1
    return counters[user_id] ≤ limit
```

Why it feels right:

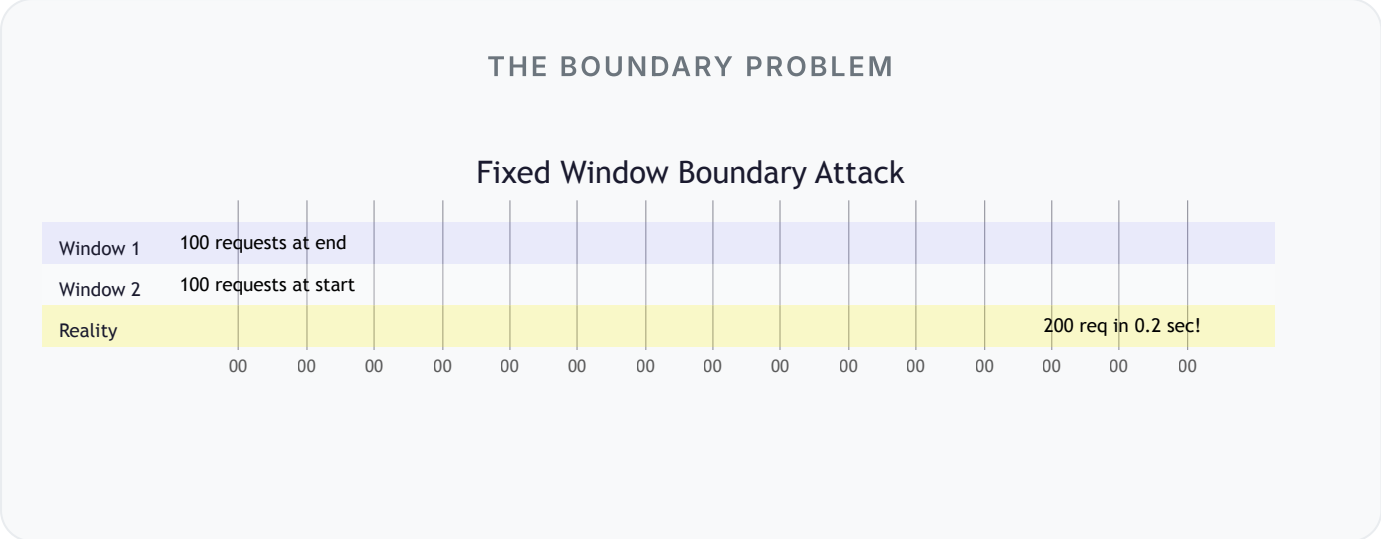
- Simple to understand
- Fast - O(1) lookups
- Works on a single server

Why it fails:

- **Boundary problem:** Send 100 requests at 0.99s, 100 more at 1.01s = 200 requests in 0.02s

Scribd @theskilledc kumar.dodda@gmail.com

- **Single machine:** Doesn't work across multiple servers
- **Memory:** Millions of users = millions of counters



> 💡 **Lesson**
>
> Fixed windows have edges. Traffic can spike at the boundary.

Attempt 2: Sliding Window Log

Track the timestamp of every request. Count requests in the last N seconds.

PYTHON

```python
request_logs = {}  # user_id → [timestamps]

def is_allowed(user_id, limit=100, window=60):
    now = time.time()

    # Get user's request log
    if user_id not in request_logs:
        request_logs[user_id] = []

    # Remove old timestamps
    request_logs[user_id] = [
        t for t in request_logs[user_id]
        if t > now - window
    ]

    # Check limit
    if len(request_logs[user_id]) < limit:
        request_logs[user_id].append(now)
        return True

    return False
```

Why it feels right:

Sold to
@theskilledc
kumar.dodda@gmail.com

- Perfectly accurate - no boundary issues
- True sliding window

Why it fails:

- **Memory explosion:** Store every timestamp
- 1M users × 100 requests × 8 bytes = 800 MB just for timestamps
- Cleanup is expensive (iterate through all timestamps)
- Still single machine

> 💡 **Lesson**
>
> Perfect accuracy comes at a cost. Sometimes approximate is good enough.

Attempt 3: Use Redis Across All Servers

Great, put the counters in Redis! All servers share the same state.

```python
def is_allowed(user_id, limit=100):
    key = f"ratelimit:{user_id}"

    count = redis.incr(key)

    if count == 1:
        redis.expire(key, 60)  # 1 minute window

    return count <= limit
```

Why it feels right:

- Distributed - all servers see same count
- Redis is fast
- Simple implementation

Why it fails at 1M req/sec:

- **Race condition:** INCR and EXPIRE are not atomic
- **Single Redis:** Becomes the bottleneck
- **Network latency:** Add 1-5ms to every request
- Still has fixed window boundary problem

Sold to
@theskilledc
kumar.dodda@gmail.com

⚠️ **The Race Condition**

If INCR succeeds but EXPIRE fails (crash, network issue), the key never expires. User is locked out forever.

💡 **Lesson**

Centralized state becomes a bottleneck. And distributed = more failure modes.

Attempt 4: Just Add More Redis Nodes

Shard Redis by user_id. Problem solved?

Why it fails:

- Each request still needs a network round-trip
- At 1M req/sec, that's 1M Redis operations per second
- Sharding adds complexity (consistent hashing, rebalancing)
- Still adds latency to the hot path

💡 **Lesson**

Throwing more hardware at a per-request operation doesn't scale linearly.

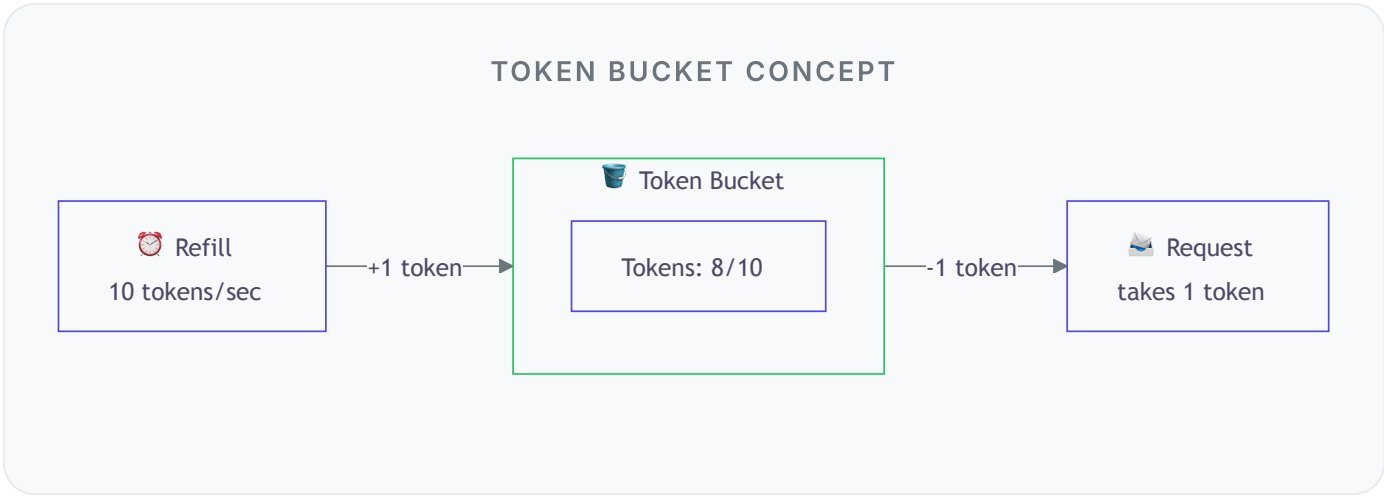
The Correct Approach: Algorithms + Architecture

The solution isn't just choosing the right tool - it's choosing the right **algorithm** and the right **architecture**.

Algorithm 1: Token Bucket

Imagine a bucket that holds tokens. Each request takes a token. Tokens refill at a steady rate.

Sold to
@theskilledc
kumar.dodda@gmail.com



How it works:

- Bucket has capacity (e.g., 10 tokens)
- Tokens refill at rate R (e.g., 10/sec)
- Each request consumes 1 token
- If bucket is empty → reject request
- Allows bursts up to bucket capacity

PYTHON

```python
class TokenBucket:
    def __init__(self, capacity, refill_rate):
        self.capacity = capacity
        self.tokens = capacity
        self.refill_rate = refill_rate  # tokens per second
        self.last_refill = time.time()

    def allow(self):
        now = time.time()

        # Refill tokens based on elapsed time
        elapsed = now - self.last_refill
        self.tokens = min(
            self.capacity,
            self.tokens + elapsed * self.refill_rate
        )
        self.last_refill = now

        # Try to consume a token
        if self.tokens >= 1:
            self.tokens -= 1
            return True

        return False
```

Why token bucket is great:

- ✅ Allows controlled bursts (good UX)

Sold to
@theskilledc
kumar.dodda@gmail.com

- ✅ Smooth rate limiting (no boundary issues)
- ✅ Memory-efficient (just 2 numbers per user)
- ✅ O(1) time complexity

Algorithm 2: Sliding Window Counter

Combine the accuracy of sliding windows with the efficiency of counters.

How it works:

- Divide time into windows (e.g., 1-minute windows)
- Track count in current window AND previous window
- Estimate current rate using weighted average

PYTHON

```python
def is_allowed(user_id, limit=100, window=60):
    now = time.time()
    current_window = int(now / window)

    # Get counts from Redis
    current_key = f"rate:{user_id}:{current_window}"
    prev_key = f"rate:{user_id}:{current_window - 1}"

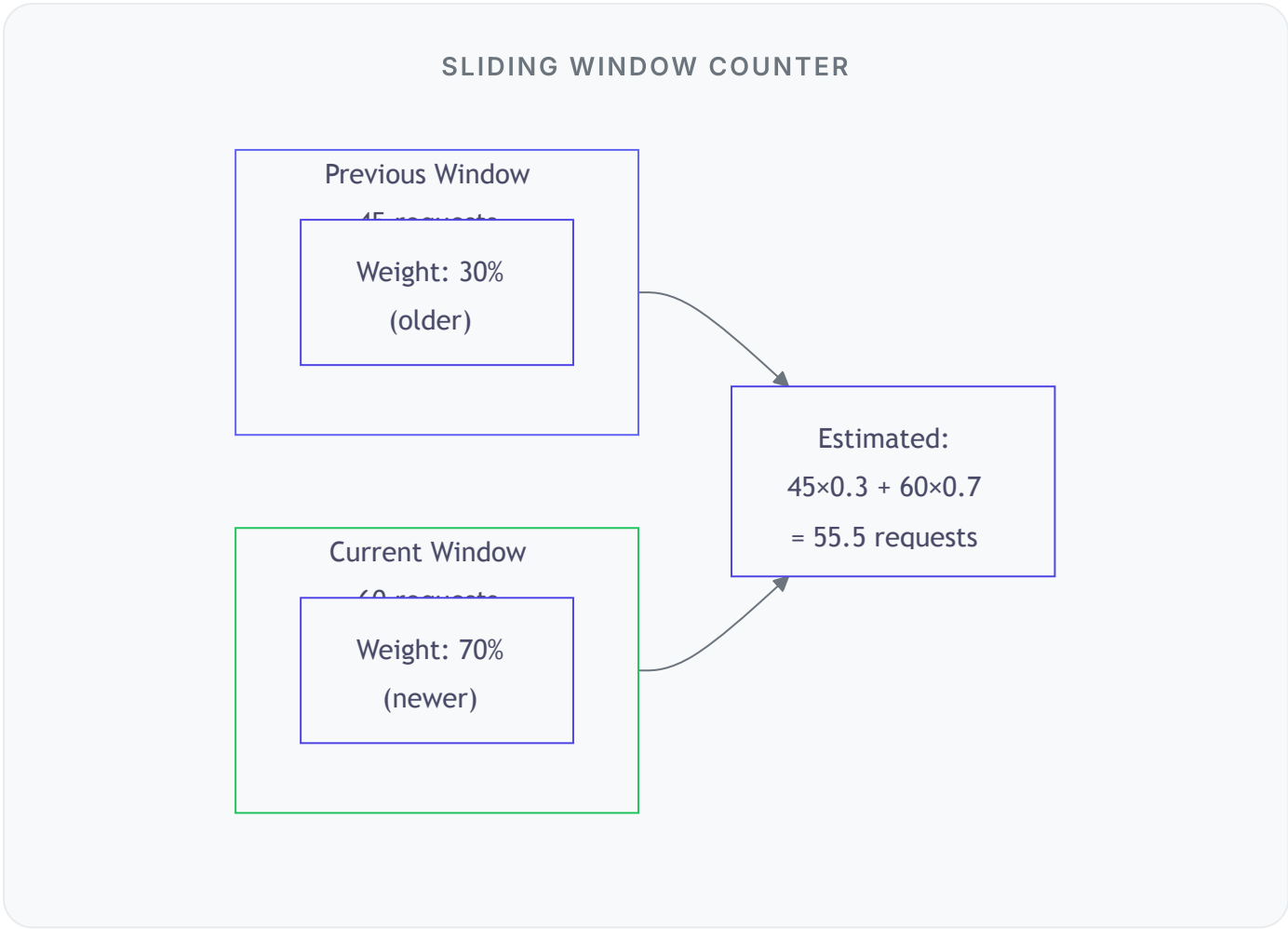
    current_count = redis.get(current_key) or 0
    prev_count = redis.get(prev_key) or 0

    # Weight previous window by how much of it is still relevant
    weight = 1 - (now % window) / window
    estimated = prev_count * weight + current_count

    if estimated < limit:
        redis.incr(current_key)
        redis.expire(current_key, window * 2)
        return True

    return False
```

Sold to
@theskilledc
kumar.dodda@gmail.com



Why sliding window counter is great:

- ✅ Smooths boundary issues
- ✅ Memory-efficient (just 2 counters per user)
- ✅ Accurate enough for most use cases

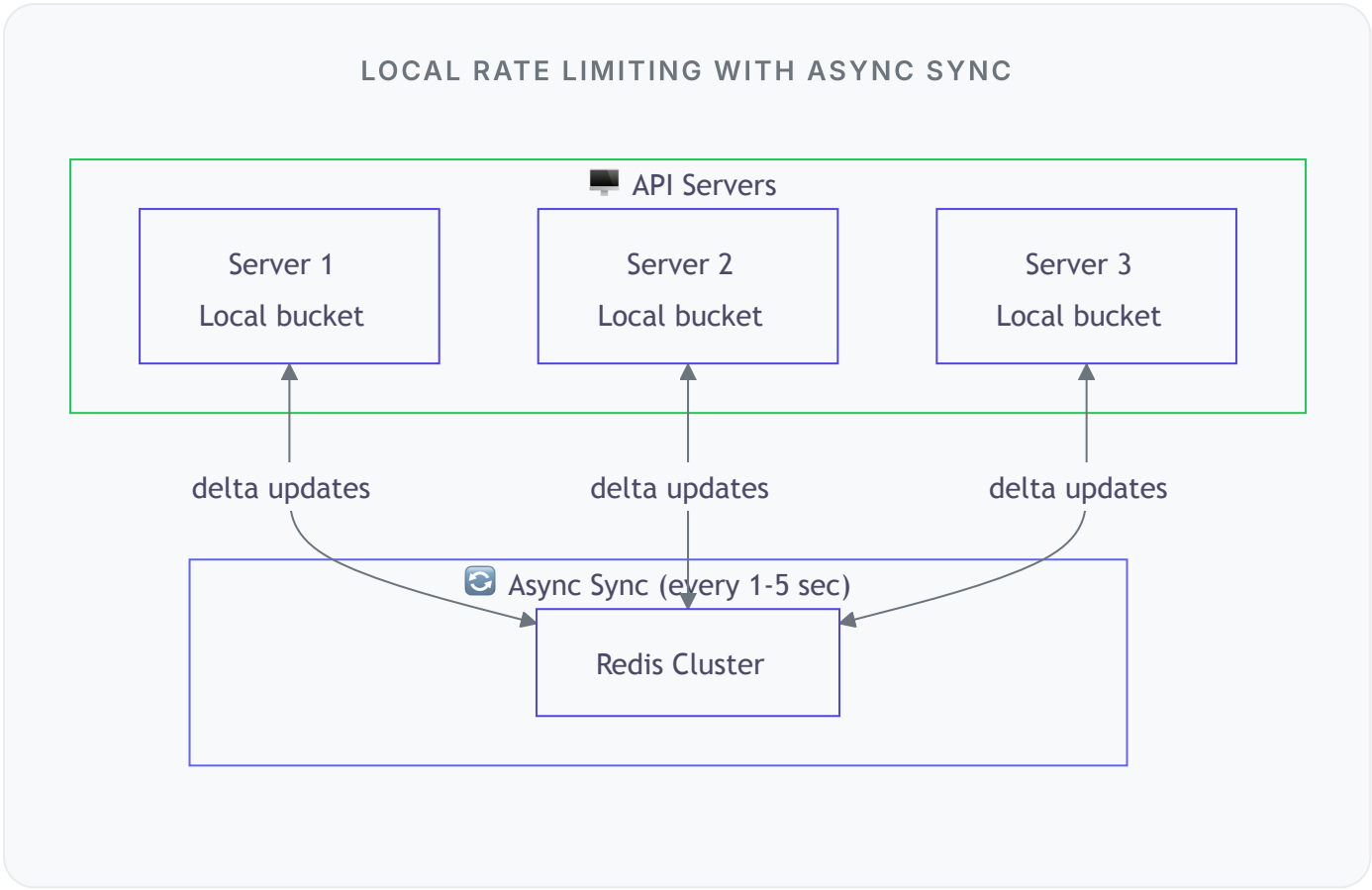
Architecture for 1M Requests/Second

The algorithm is only half the battle. At 1M req/sec, architecture matters more.

Approach 1: Local + Async Sync (Recommended)

Each server maintains **local rate limit state** and **periodically syncs** with central store.

Sold to
@theskilledc
kumar.dodda@gmail.com



How it works:

1. Each server keeps local token buckets in memory
2. Rate limit check is **in-memory** - no network call
3. Every 1-5 seconds, servers sync their counts to Redis
4. Servers read each other's counts and adjust local limits

TEXT

```
Server 1 local limit: 100 req/sec ÷ 3 servers = 33 req/sec
Server 2 local limit: 33 req/sec
Server 3 local limit: 33 req/sec

If load is uneven, sync redistributes:
- Server 1 is getting 50 req/sec → needs more quota
- Server 2 is getting 10 req/sec → can give up quota
- Redis coordinates the redistribution
```

Tradeoff:

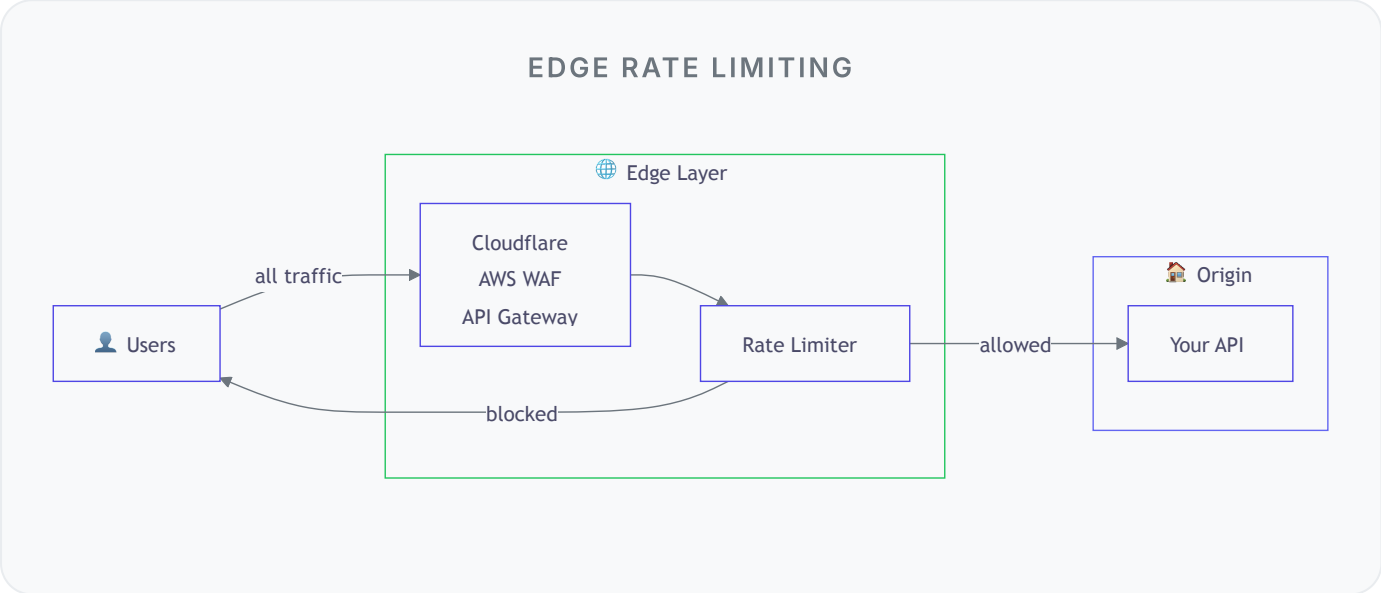
- ✅ No network call per request - blazing fast
- ✅ Works even if Redis is temporarily unavailable
- ⚠️ Not perfectly accurate (eventual consistency)
- ⚠️ Can briefly exceed limits during sync gaps

Sold to
@theskilledc
kumar.dodda@gmail.com

> **The 99% Rule**
>
> For most rate limiting use cases, "approximately 100 req/sec" is fine. You don't need atomic precision - you need protection from abuse.

Approach 2: Edge Rate Limiting (CDN/API Gateway)

Push rate limiting to the **edge** - before traffic even hits your servers.



Benefits:

- ✅ Blocked requests never reach your infrastructure
- ✅ DDoS protection built-in
- ✅ Globally distributed - low latency
- ✅ Managed service - less to build

When to use:

- IP-based rate limiting
- Basic API key rate limiting
- DDoS protection

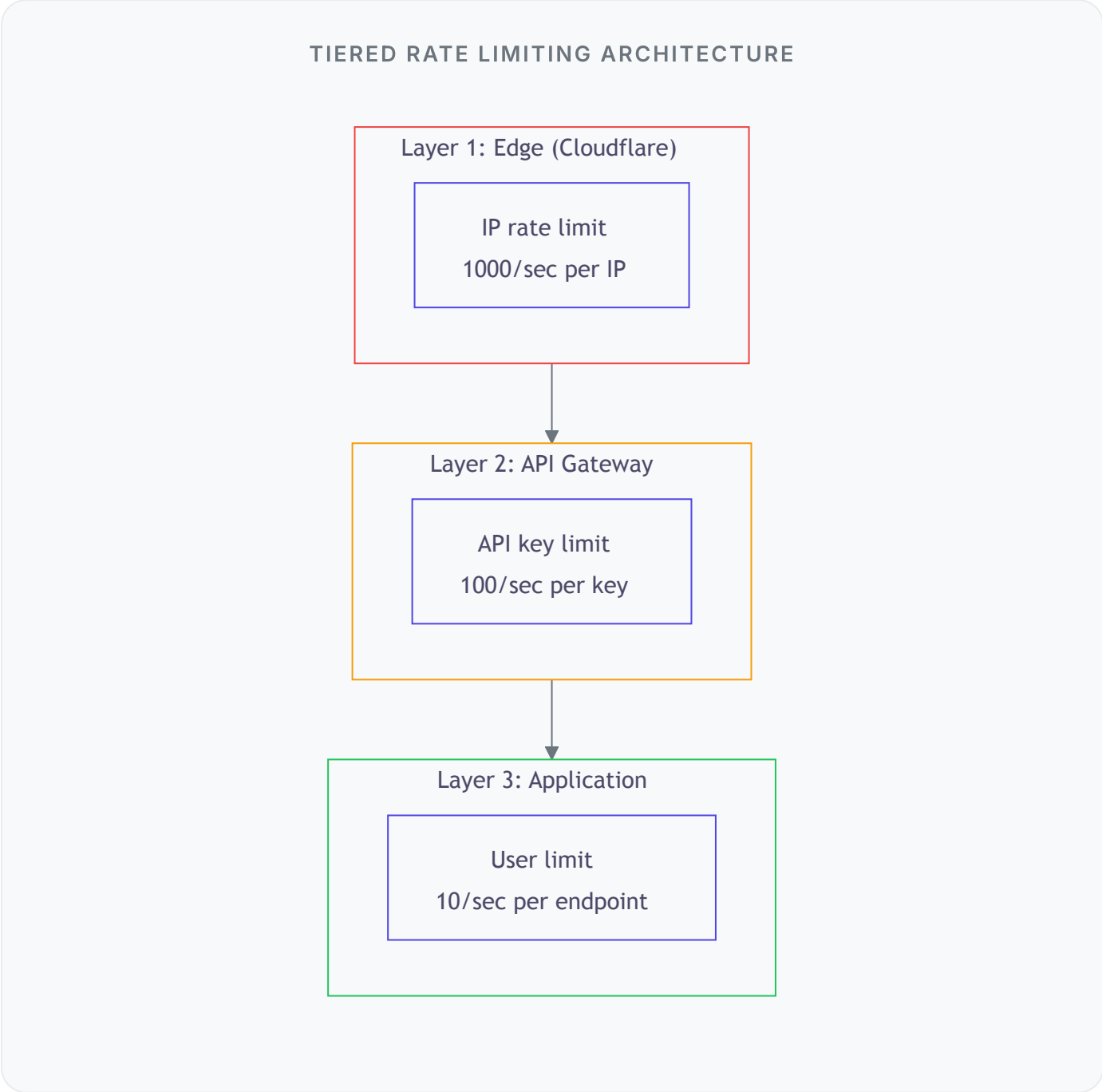
When it's not enough:

- User-based limits (requires authentication)
- Complex business rules
- Dynamic limits based on subscription tier

Approach 3: Tiered Rate Limiting

Sold to
@theskilledc
kumar.dodda@gmail.com

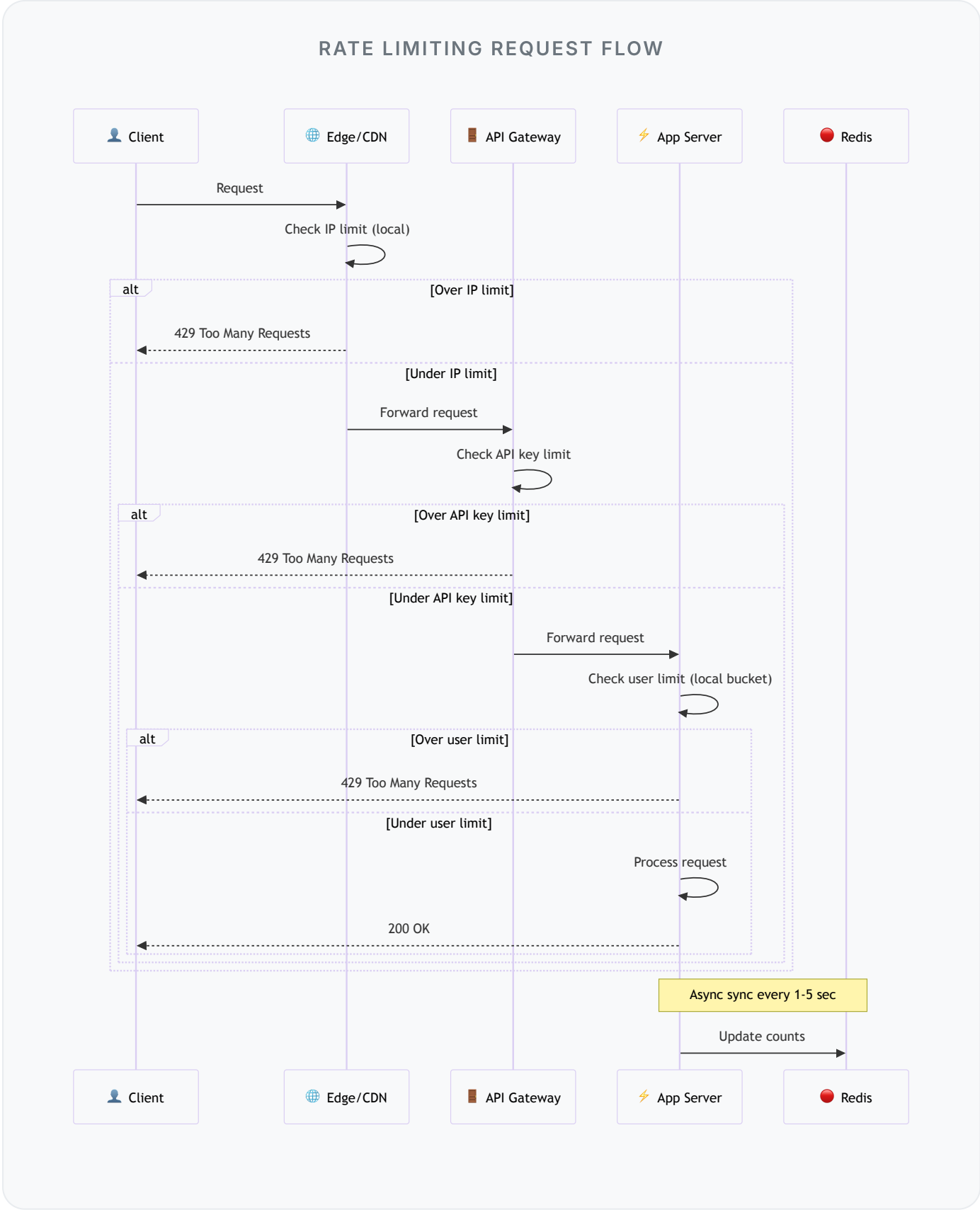
Combine multiple layers for defense in depth.



LAYER	WHAT IT LIMITS	THRESHOLD	PURPOSE
Edge (CDN)	IP address	1000/sec	Stop DDoS, bot attacks
API Gateway	API key	100/sec	Fair usage per customer
Application	User + endpoint	10/sec	Protect expensive operations

Complete Request Flow

Sold to
@theskilledc
kumar.dodda@gmail.com



The Right Response

When you rate limit, tell the client what's happening.

Sold to
@theskilledc
kumar.dodda@gmail.com

```http
HTTP/1.1 429 Too Many Requests
Content-Type: application/json
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1703012400
Retry-After: 30

{
    "error": "rate_limit_exceeded",
    "message": "Too many requests. Please retry after 30 seconds.",
    "retry_after": 30
}
```

Standard headers:

- `X-RateLimit-Limit` - Maximum requests allowed
- `X-RateLimit-Remaining` - Requests left in window
- `X-RateLimit-Reset` - Unix timestamp when limit resets
- `Retry-After` - Seconds to wait before retrying

Interview Pivot (How to Explain It)

If asked:

"How would you build a rate limiter for 1M requests/second?"

> "At 1M req/sec, I can't afford a network call per request. So I'd use local rate limiting with async sync.
>
> Each server maintains local token buckets in memory - that's O(1) with no network latency. Every few seconds, servers sync their counts to Redis and adjust quotas based on actual traffic distribution.
>
> I'd use token bucket as the algorithm - it allows bursts, has no boundary issues, and only needs two numbers per user. For a 1M users with 100 req/sec each, that's just 16 MB of memory.
>
> At the edge, I'd add IP-based rate limiting via the CDN to block obvious abuse before it hits my servers."

What's Next?

Rate limiting opens up deeper questions:

Sold to
@theskilledc
kumar.dodda@gmail.com

How do you handle rate limits for different subscription tiers?

> **Dynamic configuration:** Store limits per tier in config. Look up user's tier at request time. Token bucket capacity = tier limit. Update without code deploy.

How do you rate limit fairly when users share an API key?

> **Composite keys:** Rate limit by API key + user ID, or API key + IP. Prevents one user from consuming another's quota.

How do you handle distributed rate limiting without Redis?

> **Gossip protocol:** Servers periodically share their counts with each other. Eventually consistent. Works in peer-to-peer fashion without central store.

How do you prevent retry storms when you return 429?

> **Jittered backoff:** Return randomized Retry-After. Clients use exponential backoff with jitter. Prevents thundering herd when limit resets.

Here's the thing about rate limiting: it's not about being mean to users. It's about ensuring one user's enthusiasm doesn't ruin everyone else's experience.

Token bucket for flexibility. Local state for speed. Async sync for consistency. That's how you build a rate limiter for 1M requests per second.

Sold to
@theskilledc
kumar.dodda@gmail.com

Same Principles Apply To

- **API throttling (Stripe, Twitter API)** - same token bucket with tiered limits
- **Circuit breakers** - same pattern of limiting calls to failing services
- **Queue admission control** - same principle of controlling inflow
- **CDN bandwidth limiting** - same per-customer quotas

Sold to
@theskilledc
kumar.dodda@gmail.com

How Would You Design File Sync for 100M Files Across Devices

The Magic of "It Just Works"

You're on your laptop, editing a document. You save it. Walk to the kitchen, grab your phone, and there it is - the same document, already updated. No cables. No "send to myself" emails. It just... works.

That's the magic of Dropbox, Google Drive, iCloud, OneDrive. We take it for granted now. But behind that seamless experience is one of the most intricate distributed systems problems.

Think about what's actually happening:

- Files can be gigabytes in size
- Users can edit the same file from multiple devices
- Changes must sync in seconds, not minutes
- It must work offline - and merge when back online
- You can't lose data. Ever.

Now scale that to **100 million files** across **millions of users** on **billions of devices**.

File sync looks simple from the outside. Inside, it's a beast of chunking, deduplication, conflict resolution, and distributed consistency.

💾 THE CONFLICT

```
User: "I edited my resume on my laptop on the flight."

Dropbox: "Great, syncing now..."

User: "Wait, I also edited it on my phone in the terminal."

Dropbox: "Um... which one should I keep?"

User: "BOTH. They have different changes!"
```

Sold to @theskilledc
kumar.dodda@gmail.com

```
Dropbox: "Here's 'resume (conflicted copy - laptop).docx' and
'resume.docx'. You figure it out."

User: "...I thought this was supposed to 'just work'?" 😭
```

⚡ The Engineering Complexity

Edit on your laptop. Edit on your phone. Now merge them without losing work:

🔲 Conflict Detection

File edited on laptop AND phone while offline. Which version wins?

📊 Delta Sync

1GB file, 1 byte changed. Upload 1GB again? Or detect and send just the change?

🔄 Ordering Guarantees

Rename folder, then add file. Events arrive out of order. Chaos.

💾 Storage Deduplication

1000 users have the same movie.mp4. Store 1000 copies? Or 1 with 1000 pointers?

First Attempts (and Why They Break)

Attempt 1: Upload the Entire File on Every Change

The obvious approach: when a file changes, upload the whole thing.

Why it feels right:

- Simple to implement
- Files are the natural unit
- Works for small files

Why it fails:

- Edit one line in a 1GB video project → upload 1GB
- Slow over mobile networks

Sold to @theskilledc kumar.dodda@gmail.com

- Wastes bandwidth (and money)
- User adds a typo fix → waits 20 minutes to sync

> 💡 **Lesson**
>
> The unit of transfer shouldn't be the file - it should be the change.

Attempt 2: Poll the Server for Changes

Every 10 seconds, each device asks: "Anything new?"

Why it feels right:

- Simple HTTP requests
- No persistent connections
- Easy to implement

Why it fails:

- 1M devices × 6 polls/minute = 6M requests/minute = 100K req/sec
- 99% of polls return "nothing changed"
- Wastes server resources
- Max 10 second delay feels sluggish

> 💡 **Lesson**
>
> Polling doesn't scale. Push notifications do.

Attempt 3: rsync Everything

Use rsync - it's designed for exactly this! It only transfers differences.

Why it feels right:

- Battle-tested tool
- Efficient delta transfer
- Works great for backups

Why it fails at scale:

- rsync needs to compare entire file to find diffs

Sold to @theskilledc
kumar.dodda@gmail.com

- For large files, this comparison is expensive
- Doesn't handle concurrent edits from multiple devices
- No built-in conflict resolution
- Not designed for real-time sync

> 💡 **Lesson**
>
> Sync is not backup. Real-time multi-device sync has different requirements.

Attempt 4: Store Files in a Database

Put file contents in PostgreSQL or MongoDB. Let the database handle it.

Why it fails:

- Databases hate large blobs
- 100M files × 10MB average = 1 PB in your database
- Query performance degrades
- Backups become impossible
- Wrong tool for the job

> ⚠️ **Lesson**
>
> Databases store metadata. Object storage stores blobs. Don't mix them up.

The Core Insight

Let's think about what file sync *really* is:

1. **Detect** that a file changed locally
2. **Upload** only what changed (not the whole file)
3. **Notify** other devices about the change
4. **Download** changes on other devices
5. **Handle conflicts** when multiple devices edit simultaneously

The key challenges are:

Sold to
@theskilledc
kumar.dodda@gmail.com

CHALLENGE	SOLUTION
Large files are slow to upload	Chunking - split into blocks
Same file on many accounts	Deduplication - store once
Polling wastes resources	Push notifications (WebSocket/long-poll)
Concurrent edits cause conflicts	Conflict detection + resolution
Must work offline	Local database + sync queue

The Correct Architecture

Principle 1: Chunking

Don't store files. Store **chunks**.

Split every file into fixed-size blocks (e.g., 4MB). Each block gets a hash (SHA-256). The file becomes a list of block hashes.

```text
File: presentation.pptx (50 MB)

Chunks:
  Block 0: hash_abc123 (4 MB)
  Block 1: hash_def456 (4 MB)
  Block 2: hash_ghi789 (4 MB)
  ...
  Block 12: hash_xyz999 (2 MB)

File Metadata:
  filename: presentation.pptx
  size: 50 MB
  blocks: [hash_abc123, hash_def456, ..., hash_xyz999]
```

Why chunking is brilliant:

- Edit one slide? Only that 4MB chunk changes
- Upload 4MB instead of 50MB = **12x faster**
- Resume interrupted uploads (just retry failed chunks)
- Parallel uploads (multiple chunks at once)

Principle 2: Content-Addressable Storage

Store chunks by their hash, not by filename.

Sold to
@theskilledc
kumar.dodda@gmail.com

```text
Storage:
  /blocks/abc123 → [4MB of data]
  /blocks/def456 → [4MB of data]
  /blocks/ghi789 → [4MB of data]

Two different files with the same 4MB of data?
→ Same hash → Same block → Store once!
```

This enables **deduplication** - if 1000 users have the same PDF, store it once, not 1000 times.

> **Dropbox's Numbers**
>
> Dropbox reported that deduplication saves them 75% of storage. Popular files (like software installers) are stored once but referenced millions of times.

Principle 3: Metadata Service

Separate **metadata** (filenames, folders, versions) from **block storage**.

METADATA DB	BLOCK STORE
Small records	Large blobs
SQL/NoSQL database	Object storage (S3)
Fast queries	Optimized for throughput
Complex relationships	Simple key-value

```sql
-- Metadata schema
CREATE TABLE files (
    id UUID PRIMARY KEY,
    user_id UUID,
    path VARCHAR(4096),
    size BIGINT,
    block_list JSONB,  -- ["hash1", "hash2", ...]
    version BIGINT,
    modified_at TIMESTAMP,
    deleted BOOLEAN DEFAULT FALSE
);

CREATE TABLE blocks (
    hash VARCHAR(64) PRIMARY KEY,
    storage_location VARCHAR(255),  -- s3://bucket/path
    size INT,
    reference_count INT  -- for garbage collection
);
```

Sold to @theskilledc kumar.dodda@gmail.com

Principle 4: Sync Protocol

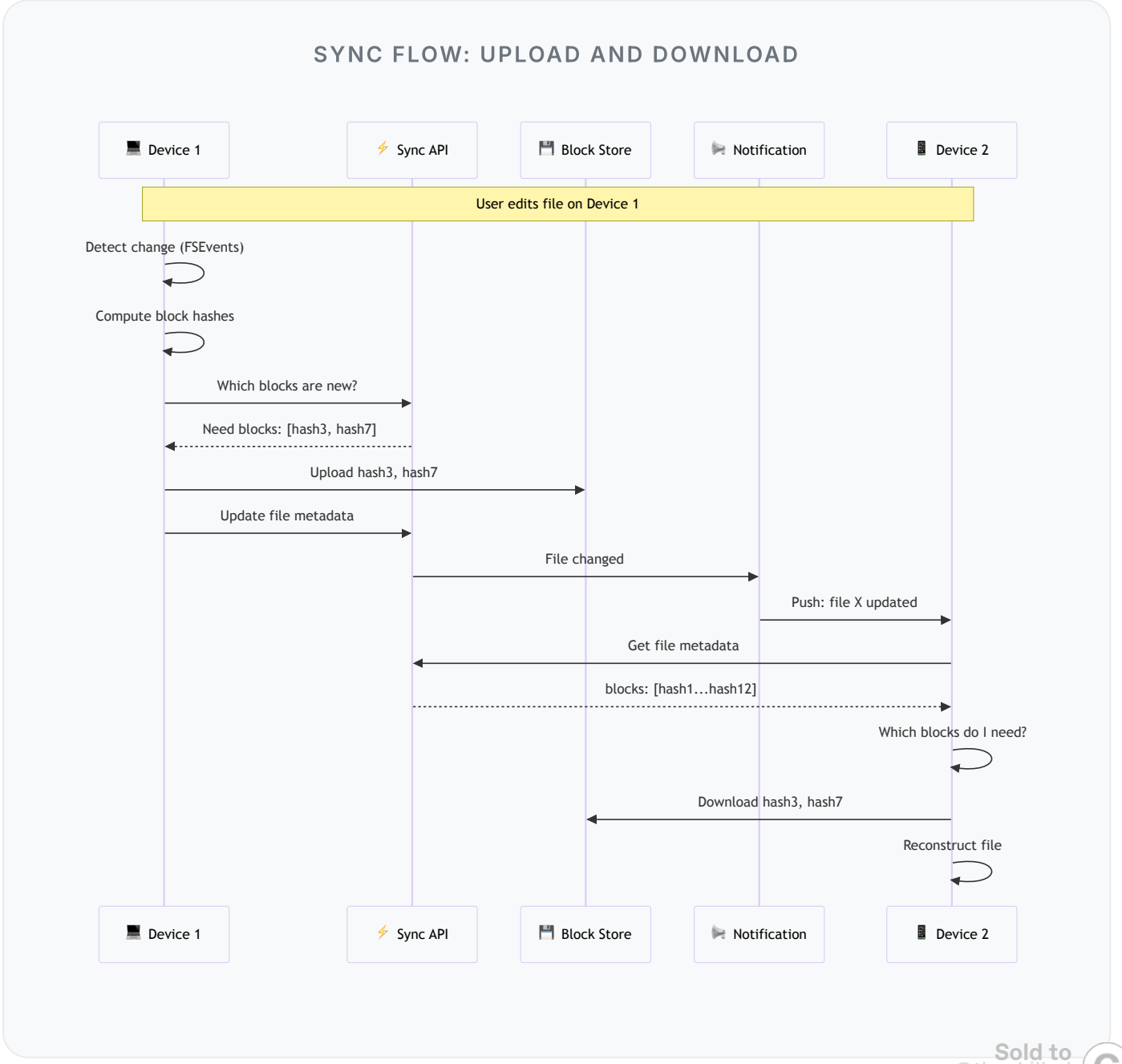
How does a device know something changed? Two mechanisms:

1. Change Detection (Local)

- File system watcher (inotify on Linux, FSEvents on macOS)
- Detects file creates, modifies, deletes
- Triggers sync immediately

2. Change Notification (Remote)

- Long-lived connection to notification service
- Server pushes: "File X changed, here's the new metadata"
- Client fetches new/changed blocks



Sold to
@theskilledc
kumar.dodda@gmail.com

Principle 5: Conflict Resolution

The hardest part: what happens when two devices edit the same file offline?

Scenario:

- Device A and Device B both have file.txt (version 5)
- Both go offline
- Device A edits → version 6a
- Device B edits → version 6b
- Both come online. Now what?

Options:

STRATEGY	HOW IT WORKS	USED BY
Last-write-wins	Latest timestamp wins, other is lost	Some simple systems
Conflicted copies	Save both as "file.txt" and "file (conflicted).txt"	Dropbox
Manual merge	Ask user to resolve	Git
Auto-merge	Try to merge changes (for supported formats)	Google Docs

Dropbox's approach (simple and safe):

- Detect conflict via version numbers
- Keep the first-to-sync version as the "winner"
- Save the other as "filename (conflicted copy - Device B).txt"
- User manually resolves

PYTHON

```python
def sync_file(local_version, server_version):
    if local_version.parent == server_version.parent:
        # Both derived from same version - conflict!
        save_conflicted_copy(local_version)
        download(server_version)
    elif local_version.parent == server_version.version:
        # Local is newer - upload it
        upload(local_version)
    else:
        # Server is newer - download it
        download(server_version)
```

Principle 6: Offline Support

Sold to
@theskilledc
kumar.dodda@gmail.com

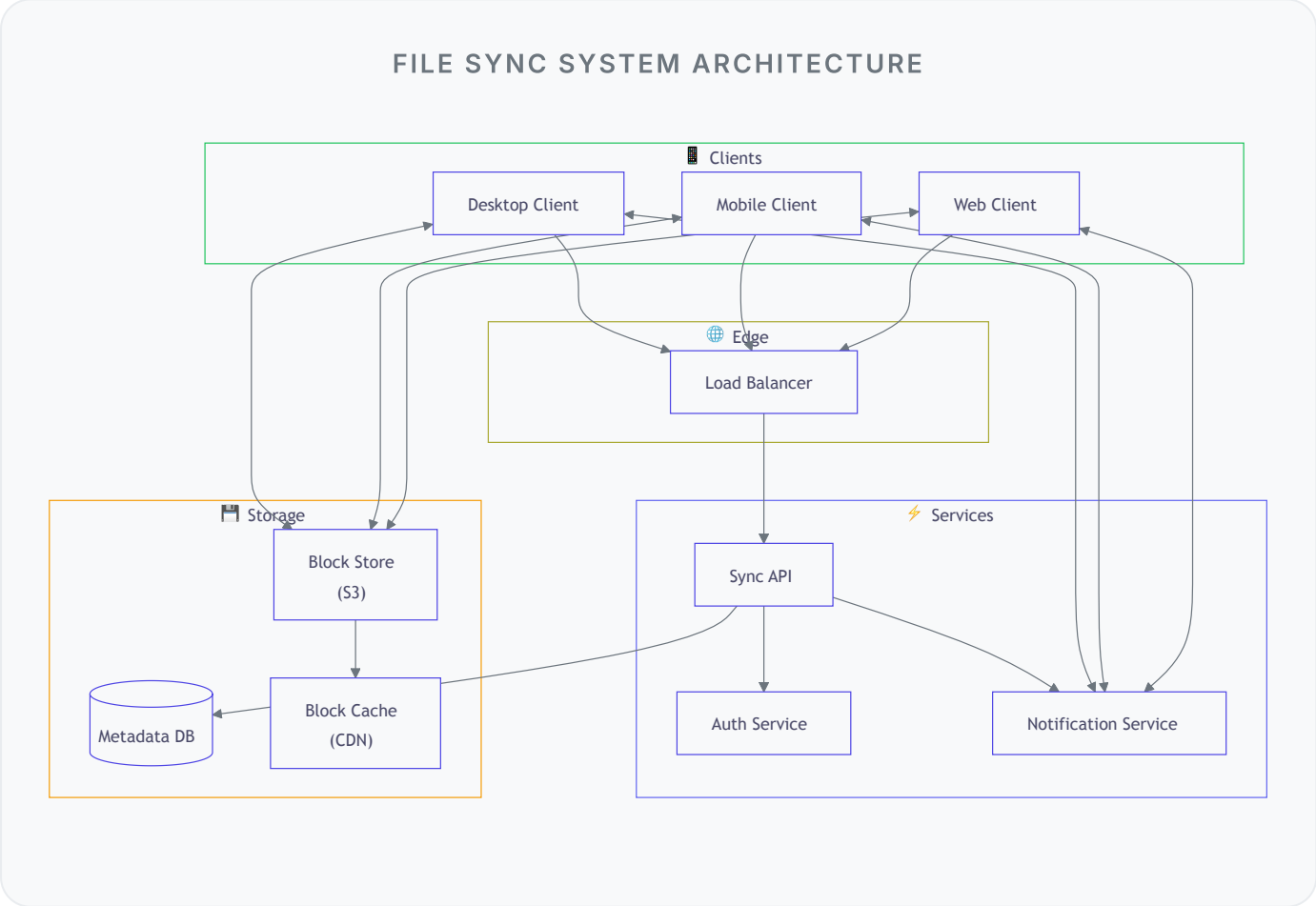
The client must work without internet:

- **Local database:** SQLite with all file metadata
- **Local block cache:** Recent blocks stored locally
- **Change queue:** Pending uploads stored until online
- **Sync state machine:** Track what's synced, pending, conflicted

When connectivity returns:

1. Pull remote changes first
2. Detect conflicts
3. Push local changes
4. Update local state

High-Level Architecture



Scale Considerations

Sold to
@theskilledc
kumar.dodda@gmail.com

COMPONENT	SCALE	STRATEGY
Metadata DB	100M files × 1KB = 100GB	Shard by user_id
Block Store	Petabytes (after dedup)	S3 / GCS - infinitely scalable
Notifications	Millions of connections	WebSocket servers, pub/sub
Block Cache	Hot blocks	CDN for popular files

Interview Pivot (How to Explain It)

If asked:

"How would you design a file sync system like Dropbox for 100M files?"

> "The key insight is: don't sync files, sync blocks. I'd split every file into fixed-size chunks (4MB), hash each chunk, and store them by hash - this gives me deduplication for free.
>
> When a file changes, the client computes new block hashes, uploads only the changed blocks, and updates the metadata. Other devices get a push notification, fetch the new metadata, and download only the blocks they're missing.
>
> For conflicts, I'd use Dropbox's approach: detect via version vectors, keep the first-to-sync version, and save the other as a conflicted copy. The client works offline by maintaining a local SQLite database and a queue of pending changes.
>
> At scale: metadata goes in a sharded database, blocks go in S3, popular blocks get cached at the CDN, and notifications use a WebSocket-based pub/sub system."

What's Next?

File sync leads to deeper questions:

How do you handle very large files (100GB+)? ?

> **Streaming upload:** Upload chunks as they're ready, don't wait for full file. Show progress per-chunk. Handle restarts gracefully.

Sold to @theskilledc
kumar.dodda@gmail.com

How do you optimize for mobile with limited bandwidth?

> **Selective sync:** Only sync thumbnails and metadata initially. Download full files on demand. Compress before transfer. Pause on metered networks.

How do you handle folder sharing between users?

> **ACL layer:** Folder has list of permitted users. Changes visible to all with access. Conflict handling becomes more complex with multiple editors.

How do you implement file versioning (undo/history)?

> **Keep old block lists:** Each file version is just a list of block hashes. Previous versions are cheap to store (blocks are shared). Time-based or count-based retention.

The magic of file sync is that users never think about it. The file is just... there. On every device. Updated. Consistent. That seamless experience comes from getting the hard parts right: chunking, deduplication, notifications, and conflict handling.

Don't sync files. Sync blocks. Don't poll. Push. Don't assume connectivity. Queue.

Same Principles Apply To

- **Note syncing (Notion, Evernote)** - same chunk + merge model
- **Photo backup (Google Photos)** - same deduplication + metadata separation
- **Database replication** - same change detection + conflict resolution
- **CDN cache invalidation** - same content-addressable storage

Sold to
@theskilledc
kumar.dodda@gmail.com

How Would You Design a Ticket Booking System for 100K Concurrent Users

When 100,000 People Want the Same 10,000 Seats - At the Same Second

It's 10:00:00 AM. Taylor Swift concert tickets go live.

In the next 60 seconds:

- 100,000 users hit the site simultaneously
- They're all fighting for 10,000 seats
- Everyone wants front row
- Everyone's credit card is ready

By 10:00:05, your system either handled the stampede gracefully - or it's on fire. Users are seeing "seat unavailable" after selecting. Payments are timing out. Twitter is exploding with complaints. And somewhere, two people have confirmation emails for the same seat.

Ticket booking is the ultimate test of handling concurrent writes to scarce resources.

The challenge isn't showing available seats. It's ensuring that when 500 people click on Seat A1 at the same millisecond, exactly ONE of them gets it - and the other 499 know immediately that they didn't.



THE TICKET DROP

PM: "Taylor Swift tickets go live at 10 AM. We're ready, right?"

Junior Dev: "Yeah, it's just a simple booking system. Check if seat available, book it."

9:59:59 AM: Load: 0 users

10:00:00 AM: Load: 147,000 users. Database: "I am become death."

Twitter: "I got 3 confirmation emails for Seat A1! Which one is real??"

Sold to @theskilledc
kumar.dodda@gmail.com

Twitter: "I selected a seat and now it says 'unavailable.' BRING IT BACK!"

Junior Dev: "...maybe it's not that simple." 😰

⚡ The Engineering Complexity

50,000 seats. 1 million fans. All clicking at exactly 10:00 AM:

🎫 One Seat, 10K Requests

BTS concert. 50K seats. 1M fans. All click at exactly 10:00 AM.

⏱️ Temporary Holds

User selects seat. Held for 10 minutes. Doesn't pay. Seat stuck unavailable.

🔄 Race Conditions

Two users click same seat at same millisecond. Both see 'available.' Both click 'book.'

💳 Payment Timeout

Seat held. Payment takes too long. Release seat? But payment might still go through.

First Attempts (and Why They Break)

Attempt 1: Check-Then-Book (Read-Modify-Write)

The obvious approach: check if seat is available, then book it.

```python
def book_seat(user_id, seat_id):
    seat = db.query("SELECT * FROM seats WHERE id = ?", seat_id)

    if seat.status == "available":
        db.execute("UPDATE seats SET status = 'booked', user_id = ? WHERE id = ?",
                   user_id, seat_id)
        return "Success!"
    else:
        return "Seat already taken"
```

Why it feels right:

- Simple and intuitive

Sold to @theskilledc kumar.dodda@gmail.com

- Works in development
- Works with low traffic

Why it fails:

- **Race condition!** Two users check at the same time, both see "available"
- Both proceed to update → one overwrites the other
- Congratulations, you've sold the same seat twice

```text
Time    User A                      User B
─────────────────────────────────────────────────
T1      SELECT → available
T2                                  SELECT → available
T3      UPDATE → booked
T4                                  UPDATE → booked (overwrites!)
T5      ✓ Success                   ✓ Success (WRONG!)
```

> ⚠️ **The Classic TOCTOU Bug**
>
> "Time-of-check to time-of-use" - the state can change between when you check it and when you act on it. This is the #1 bug in booking systems.

Attempt 2: Database Transactions

Wrap it in a transaction. That should fix it, right?

```python
def book_seat(user_id, seat_id):
    with db.transaction():
        seat = db.query("SELECT * FROM seats WHERE id = ?", seat_id)

        if seat.status == "available":
            db.execute("UPDATE seats SET status = 'booked', user_id = ?",
                       user_id, seat_id)
            return "Success!"
        else:
            return "Seat already taken"
```

Why it still fails:

- Default transaction isolation (Read Committed) doesn't prevent this
- Both transactions can still read "available"
- Need SERIALIZABLE isolation, which is slow
- Or explicit locking...

Sold to @theskilledc kumar.dodda@gmail.com

> 💡 **Lesson**
>
> Transactions alone don't prevent race conditions. You need the RIGHT isolation level or explicit locks.

Attempt 3: Pessimistic Locking (SELECT FOR UPDATE)

Lock the row when you read it. No one else can read or modify until you're done.

```python
def book_seat(user_id, seat_id):
    with db.transaction():
        # Lock this row - blocks other transactions
        seat = db.query("SELECT * FROM seats WHERE id = ? FOR UPDATE", seat_id)

        if seat.status == "available":
            db.execute("UPDATE seats SET status = 'booked', user_id = ?",
                       user_id, seat_id)
            return "Success!"
        else:
            return "Seat already taken"
```

Why it feels right:

- Actually prevents the race condition
- Correct behavior guaranteed

Why it breaks at scale:

- 500 users click Seat A1 → 499 wait in line for the lock
- Lock contention creates massive latency
- Database connections pile up
- Timeout errors everywhere
- Works for 10 concurrent users, not 100,000

> 💡 **Lesson**
>
> Pessimistic locking is correct but slow. At high concurrency, the database becomes the bottleneck.

Attempt 4: Optimistic Locking (Version Number)

Don't lock upfront. Check at update time if anyone else modified the row.

Sold to @theskilledc kumar.dodda@gmail.com

```python
def book_seat(user_id, seat_id):
    seat = db.query("SELECT * FROM seats WHERE id = ?", seat_id)

    if seat.status != "available":
        return "Seat already taken"

    # Update only if version hasn't changed
    result = db.execute("""
        UPDATE seats
        SET status = 'booked', user_id = ?, version = version + 1
        WHERE id = ? AND version = ?
    """, user_id, seat_id, seat.version)

    if result.rows_affected == 1:
        return "Success!"
    else:
        return "Seat was taken by someone else, please try another"
```

Why it's better:

- No locks held during read
- Conflict detection at write time
- Losers fail fast, can retry

Why it still has problems:

- High contention on popular seats = many failed attempts
- Users frustrated by "sorry, try again" loops
- Doesn't solve the UX problem of 500 people all selecting the same seat

> 💡 **Lesson**
>
> Optimistic locking is faster but doesn't prevent users from fighting over the same seat. We need to separate "I want this seat" from "I'm paying for this seat".

The Core Insight

The problem isn't just *booking*. It's the entire flow:

1. User browses available seats
2. User selects a seat (wants to think about it)
3. User enters payment details (takes 30-60 seconds)
4. User submits payment

Sold to
@theskilledc
kumar.dodda@gmail.com

5. Seat is booked

Between steps 2 and 5, the seat is in limbo. Is it taken? Is it available? What if the user abandons? What if they take 10 minutes?

The solution: Temporary holds. When you select a seat, you don't buy it - you RESERVE it for a limited time (e.g., 10 minutes). During that time, no one else can select it.

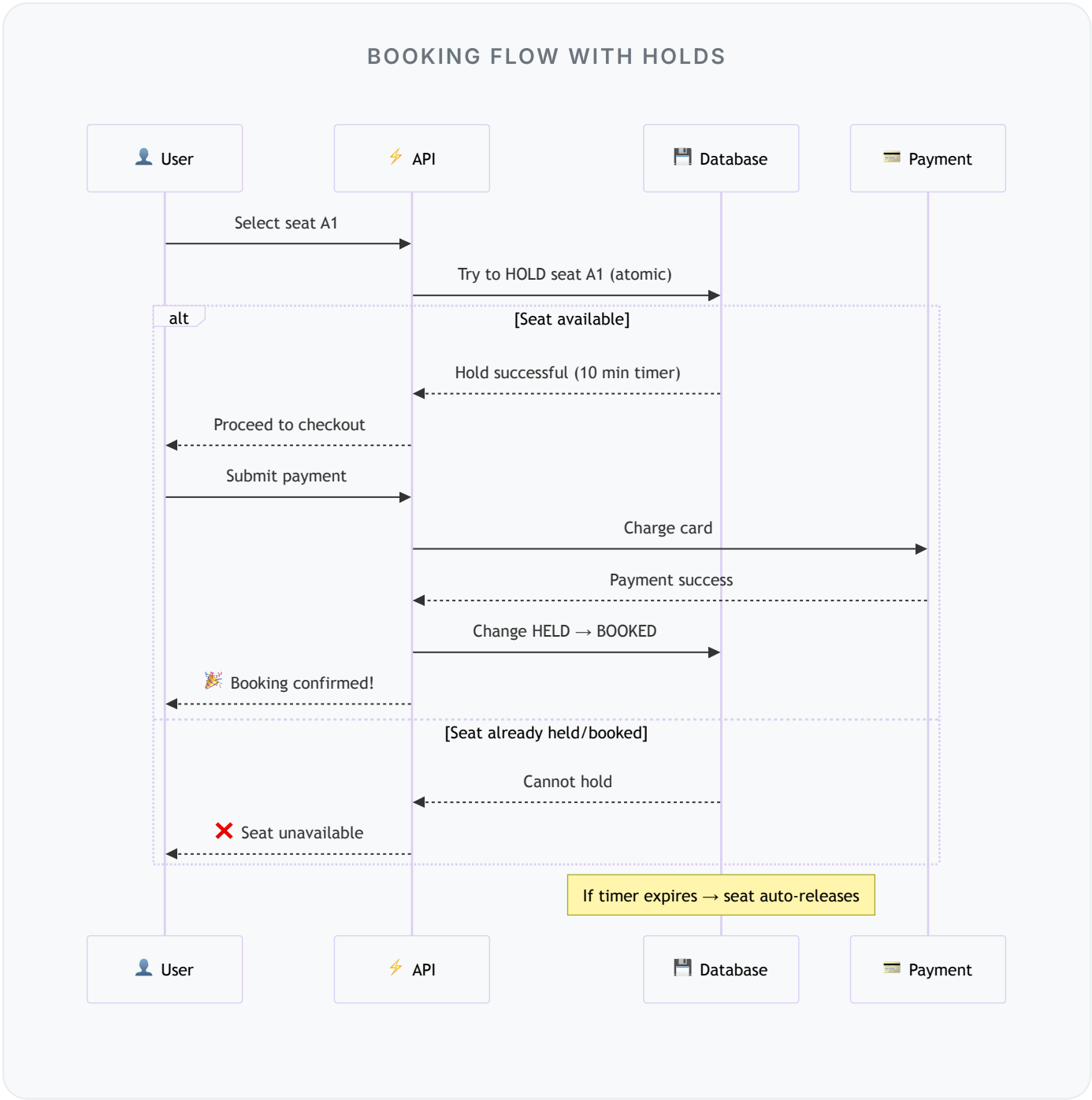
SEAT STATE	MEANING	DURATION
available	No one has it	-
held	Someone is in checkout	10 minutes
booked	Paid and confirmed	Permanent

The Correct Architecture

Principle 1: Hold → Pay → Confirm

The booking flow becomes three distinct phases:

Sold to
@theskilledc
kumar.dodda@gmail.com



The atomic hold operation:

```sql
-- Single atomic update - no race conditions
UPDATE seats
SET
    status = 'held',
    held_by = :user_id,
    held_until = NOW() + INTERVAL '10 minutes'
WHERE
    id = :seat_id
    AND (status = 'available'
        OR (status = 'held' AND held_until < NOW()))
RETURNING *;
```

If `rows_affected = 1`, you got the seat. If `0`, someone else has it. **No locks needed.** The WHERE clause makes it atomic.

@theskilledc
kumar.dodda@gmail.com

Principle 2: Automatic Hold Expiration

Holds must expire automatically. If a user abandons checkout:

- **Option A:** Background job that runs every minute
- **Option B:** Check expiration on every read (lazy expiration)
- **Option C:** Redis with TTL (holds as keys with auto-expire)

```python
# Background job (runs every 60 seconds)
def release_expired_holds():
    db.execute("""
        UPDATE seats
        SET status = 'available', held_by = NULL, held_until = NULL
        WHERE status = 'held' AND held_until < NOW()
    """)
```

Or use Redis for real-time expiration:

```python
# Hold seat with Redis TTL
def hold_seat(user_id, seat_id):
    key = f"hold:{seat_id}"

    # SET NX = Set if Not eXists (atomic!)
    success = redis.set(key, user_id, nx=True, ex=600)  # 10 min TTL

    if success:
        return "Hold acquired"
    else:
        return "Seat already held"
```

Principle 3: Handle the 100K User Stampede

When 100K users hit at once, you need more than correct locking:

A. Virtual Waiting Room (Queue)

- Don't let all 100K hit the booking system simultaneously
- Put users in a queue: "You are #45,234 in line"
- Release users in batches (1000 per minute)
- Fair for users, controlled load for servers

B. Show Real-Time Availability

- As seats get held, update the seat map via WebSocket
- Users see seats disappearing in real-time

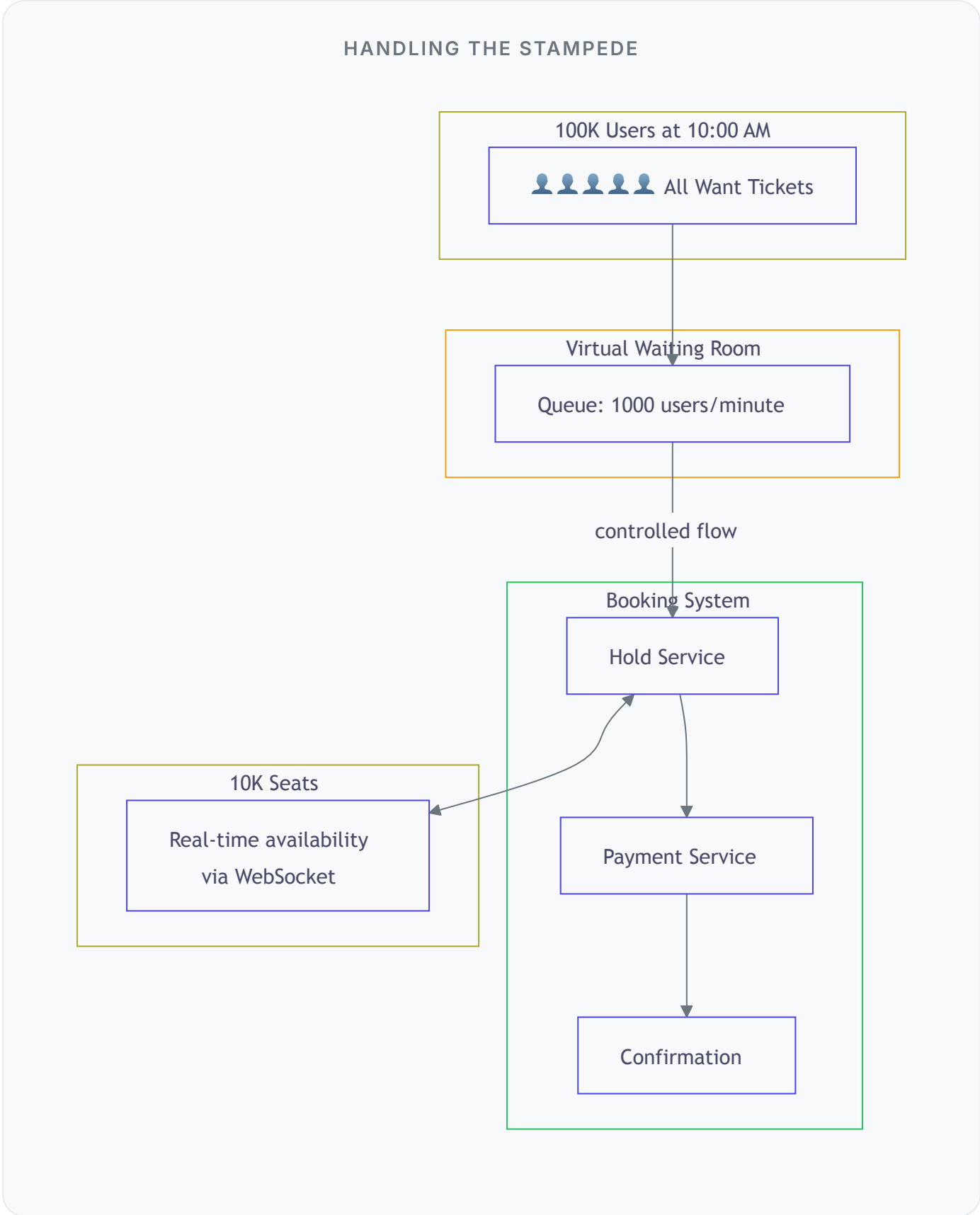
Sold to
@theskilledc
kumar.dodda@gmail.com

- Reduces contention - users pick different seats

C. Tiered Inventory

- Release seats in batches, not all at once
- 10:00 AM: First 3000 seats
- 10:05 AM: Next 3000 seats
- Spreads the load, creates multiple "drops"

Sold to
@theskilledc
kumar.dodda@gmail.com



Principle 4: Payment Integration

The hold-to-book transition involves payment. This is tricky:

1. User holds seat
2. User submits payment → call Stripe/PayPal
3. Payment succeeds → convert hold to booking

Sold to
@theskilledc
kumar.dodda@gmail.com

4. Payment fails → release hold immediately

Edge cases:

SCENARIO	PROBLEM	SOLUTION
Payment times out	Did it charge?	Use idempotency key, check status
Payment succeeds, DB update fails	Money taken, no ticket	Retry DB update, or refund
Hold expires during payment	Payment succeeds, but seat gone	Extend hold before charging, or refund

PYTHON

```python
def complete_booking(user_id, seat_id, payment_token):
    # Extend hold to cover payment time
    extended = extend_hold(seat_id, user_id, minutes=5)
    if not extended:
        return "Hold expired, please try again"

    # Charge with idempotency key
    idempotency_key = f"booking:{seat_id}:{user_id}"
    payment = stripe.charge(
        amount=150_00,  # $150
        token=payment_token,
        idempotency_key=idempotency_key
    )

    if payment.success:
        # Convert hold to booking
        confirm_booking(seat_id, user_id, payment.id)
        return "🎉 Booking confirmed!"
    else:
        release_hold(seat_id)
        return "Payment failed, seat released"
```

Principle 5: Inventory Accuracy

The seat map must reflect reality. Stale data = user frustration.

- **Cache carefully:** Seat status changes every second during sales
- **Push updates:** WebSocket to update seat map in real-time
- **Expect failures:** Even with real-time updates, race is possible - handle gracefully

Sold to
@theskilledc
kumar.dodda@gmail.com

```python
# When seat status changes, notify all viewers
def on_seat_held(event_id, seat_id):
    # Publish to WebSocket channel for this event
    websocket.broadcast(
        channel=f"event:{event_id}",
        message={"type": "seat_held", "seat": seat_id}
    )
```

Data Model

```sql
CREATE TABLE events (
    id UUID PRIMARY KEY,
    name VARCHAR(255),
    venue_id UUID,
    date TIMESTAMP,
    total_seats INT
);

CREATE TABLE seats (
    id UUID PRIMARY KEY,
    event_id UUID REFERENCES events(id),
    section VARCHAR(50),
    row VARCHAR(10),
    number INT,
    price DECIMAL(10,2),
    status VARCHAR(20) DEFAULT 'available',  -- available, held, booked
    held_by UUID,
    held_until TIMESTAMP,
    booked_by UUID,
    booking_id UUID,
    version INT DEFAULT 0,

    INDEX idx_event_status (event_id, status)
);

CREATE TABLE bookings (
    id UUID PRIMARY KEY,
    user_id UUID,
    event_id UUID,
    seat_ids UUID[],
    total_amount DECIMAL(10,2),
    payment_id VARCHAR(255),
    status VARCHAR(20),  -- confirmed, cancelled, refunded
    created_at TIMESTAMP DEFAULT NOW()
);
```

Interview Pivot (How to Explain It)

If asked:

Sold to
@theskilledc
kumar.dodda@gmail.com

"How would you design a ticket booking system for 10K seats with 100K concurrent users?"

> "The core challenge is preventing overselling while handling massive concurrency. I'd use a three-phase approach: hold → pay → confirm.
>
> When a user selects a seat, I atomically UPDATE the row to 'held' with a TTL - the WHERE clause includes status = 'available', so only one user can get it. No locks needed, just a conditional update.
>
> The hold lasts 10 minutes. If payment completes, I convert to 'booked'. If they abandon or payment fails, the seat auto-releases. A background job or Redis TTL handles expired holds.
>
> To handle the stampede, I'd add a virtual waiting room - queue users and release them in batches. Plus real-time seat map updates via WebSocket so users see availability changing and pick different seats, reducing contention."

What's Next?

Ticket booking leads to deeper questions:

How do you handle multi-seat bookings atomically?

> **All-or-nothing:** Lock all seats in a single transaction. If any seat is unavailable, release all. Use SELECT FOR UPDATE on the set, or batch the atomic UPDATE with seat_id IN (...).

How do you prevent bots from hoarding tickets?

> **Defense in depth:** CAPTCHA, rate limiting per IP/user, device fingerprinting, verified accounts, limit tickets per user, queue randomization, post-purchase verification.

How do you implement the waiting room queue fairly?

> **Position tracking:** Assign queue position at join time (randomized within the first second to prevent refresh attacks). Store in Redis sorted set. Release in batches. JWT token proves your position.

Sold to @theskilledc
kumar.dodda@gmail.com

How do you handle refunds and cancellations?

> **State machine:** booked → cancelled → refunded. Release seats back to inventory. Handle partial refunds for multi-seat bookings. Refunds are async (payment processors are slow).

Here's the thing about ticket booking: users don't care about your distributed systems challenges. They care that when they click "Book," they either get the seat or get told immediately that they didn't. No double-bookings. No mysterious errors. No stale seat maps.

Hold first. Pay second. Book third. Release on failure. That's how you sell 10,000 seats to 100,000 people without selling any seat twice.

Same Principles Apply To

- **Hotel reservations** - same hold-then-book model
- **Restaurant reservations (OpenTable)** - same time-limited holds
- **Appointment scheduling** - same slot locking with expiration
- **Limited-edition product drops** - same queue + inventory control

Sold to
@theskilledc
kumar.dodda@gmail.com

How Would You Design Proximity Search for 100M Points of Interest

Finding a Needle in a Haystack - When the Haystack is the Entire Planet

You open Google Maps. Type "pizza." In 200 milliseconds, you see 20 pizza places sorted by distance, with ratings, photos, and hours. You don't think twice about it.

But here's what happened behind the scenes:

- Your phone sent your GPS coordinates (latitude, longitude)
- Google searched through **200+ million businesses** worldwide
- Found all pizza places within, say, 5 km of you
- Ranked them by relevance, distance, and ratings
- Returned results before you finished reading "pizza"

Proximity search is the backbone of every map app, food delivery service, ride-sharing platform, and "near me" feature on the internet.

The challenge: latitude and longitude are just numbers. How do you quickly find all points within 5 km when you have 100 million points scattered across the globe?

📍 THE LATITUDE PROBLEM

Interviewer: "Design 'pizza places near me' for 100M restaurants."

Candidate: "Easy. SELECT * WHERE lat BETWEEN x AND y AND lon BETWEEN a AND b."

Interviewer: "Two separate B-tree indexes. Which one do you use first?"

Candidate: "Uh... both? Intersect the results?"

Interviewer: "10M lat matches × 10M lon matches. That's 100M rows to intersect. In 200ms."

Sold to @theskilledc kumar.dodda@gmail.com

Candidate: "Maybe I could... precompute... something?"

Interviewer: "Have you heard of Geohash?" 😏

⚡ The Engineering Complexity

'Find drivers within 5km' is a 2D search problem. B-trees don't help here:

📍 2D Indexing

Latitude AND longitude. B-trees work for 1D. 2D needs geospatial indexes.

🔄 Moving Objects

10M drivers, updating location every 3 seconds. 3M index updates/sec.

📏 Distance Calculation

Haversine formula for every pair? That's $O(n^2)$. 10M objects = 10^14 calculations.

🎯 Dynamic Radius

'Show restaurants within 5km' - but in Manhattan, 5km = 500 options. In rural? Maybe 2.

First Attempts (and Why They Break)

Attempt 1: Brute Force Distance Calculation

For every POI in the database, calculate the distance. Keep the ones within range.

```python
def find_nearby(user_lat, user_lon, radius_km):
    results = []

    for poi in all_pois:  # 100 million POIs...
        distance = haversine(user_lat, user_lon, poi.lat, poi.lon)
        if distance ≤ radius_km:
            results.append(poi)

    return sorted(results, key=lambda p: p.distance)
```

Why it feels right:

- Simple and correct
- Uses the actual distance formula

Sold to @theskilledc kumar.dodda@gmail.com

- Works perfectly for 1000 POIs

Why it fails:

- O(n) - must check every single POI
- 100M POIs × expensive Haversine calculation = minutes, not milliseconds
- Can't use indexes - lat/lon are two separate columns

> 💡 **Lesson**
>
> Linear scan doesn't scale. We need to eliminate candidates BEFORE calculating distances.

Attempt 2: Bounding Box Filter

Before calculating distances, filter by a bounding box (min/max lat/lon).

```SQL
SELECT * FROM pois
WHERE lat BETWEEN :min_lat AND :max_lat
  AND lon BETWEEN :min_lon AND :max_lon;
```

Why it's better:

- Uses database indexes on lat and lon
- Eliminates most candidates quickly

Why it still has problems:

- Bounding box is a square, but radius search is a circle
- Corners include points outside the radius
- Two separate indexes (lat, lon) aren't efficient together
- Near poles or date line, calculations break

> 💡 **Lesson**
>
> Bounding box is a good first filter, but we need a smarter spatial index.

Attempt 3: Just Use PostGIS

PostgreSQL has PostGIS extension with proper geospatial indexes!

Sold to
@theskilledc
kumar.dodda@gmail.com

```sql
SELECT * FROM pois
WHERE ST_DWithin(
    location,
    ST_MakePoint(:user_lon, :user_lat)::geography,
    5000  -- 5 km in meters
);
```

Why it feels right:

- Proper R-tree spatial index
- Handles edge cases (poles, date line)
- Battle-tested, works great

Why it's not enough for 100M POIs:

- Single database becomes the bottleneck
- Can't easily shard geospatial indexes
- Read replicas help, but have limits
- Need to distribute the spatial index itself

> 💡 **Lesson**
>
> PostGIS is perfect for small-to-medium scale. At 100M+ POIs with high QPS, we need a distributed solution.

The Core Insight

The fundamental problem with lat/lon is that they're two dimensions. Regular database indexes work on one dimension. We need to convert 2D space into 1D.

The solution: Divide the world into a grid. Assign each cell an ID. Now "find nearby" becomes "find cells nearby, then filter."

Three main approaches:

Sold to
@theskilledc
kumar.dodda@gmail.com

APPROACH	HOW IT WORKS	USED BY
Geohash	Divide world into rectangles, encode as string	Elasticsearch, MongoDB
QuadTree	Recursively divide into 4 quadrants	Many custom implementations
S2 / H3	Divide sphere into hierarchical cells	Google Maps, Uber

Approach 1: Geohash

Geohash converts a latitude/longitude pair into a short string like `9q8yyk`.

How it works:

1. Divide the world into 32 cells (5 bits)
2. Which cell is the point in? That's the first character
3. Divide that cell into 32 sub-cells
4. Repeat for desired precision

GEOHASH LENGTH	CELL SIZE	EXAMPLE
1	~5,000 km	`9`
3	~150 km	`9q8`
5	~5 km	`9q8yy`
6	~1.2 km	`9q8yyk`
8	~40 m	`9q8yykbh`

The magic property: Points that are close together share the same prefix!

```text
San Francisco: 9q8yy...
Nearby coffee shop: 9q8yy...  ✓ Same prefix!
New York: dr5ru...            ✗ Completely different
```

Searching with Geohash:

Sold to
@theskilledc
kumar.dodda@gmail.com

```python
def find_nearby(user_lat, user_lon, radius_km):
    # Get geohash of user's location (precision based on radius)
    user_geohash = geohash.encode(user_lat, user_lon, precision=5)

    # Get this cell + 8 neighboring cells
    cells = get_neighbors(user_geohash) + [user_geohash]

    # Query database with prefix match
    candidates = db.query("""
        SELECT * FROM pois
        WHERE geohash LIKE ANY(:prefixes)
    """, prefixes=[c + '%' for c in cells])

    # Filter by actual distance (in-memory, small set now)
    return [p for p in candidates
            if haversine(user_lat, user_lon, p.lat, p.lon) ≤ radius_km]
```

Why we need neighboring cells:

A point near the edge of cell `9q8yy` might have neighbors in `9q8yz`. Geohash boundaries don't respect distance - points across a boundary can be very close but have different prefixes.

> ⚠️ **Geohash Edge Case**
>
> Two points can be 1 meter apart but have completely different geohashes (if they're on opposite sides of a cell boundary). Always query neighbors!

Approach 2: S2 Geometry (Google's Choice)

S2 projects the Earth onto a cube, then divides each face into a hierarchy of cells.

Why S2 is better than Geohash:

- Cells are more uniform in size (Geohash rectangles vary)
- Handles the sphere properly (no date line issues)
- Hierarchical - easy to go coarser or finer
- Efficient "covering" - can find exactly which cells a region touches

S2 Cell IDs:

```text
Level 0:  6 cells (one per cube face)
Level 10: ~50,000 cells (~1 km² each)
Level 14: ~800,000 cells (~100 m² each)
Level 20: ~6 billion cells (~1 m² each)
```

Sold to
@theskilledc
kumar.dodda@gmail.com

Searching with S2:

```python
import s2geometry as s2

def find_nearby(user_lat, user_lon, radius_km):
    # Create a circle region
    center = s2.S2LatLng.FromDegrees(user_lat, user_lon)
    cap = s2.S2Cap.FromCenterAngle(
        center.ToPoint(),
        s2.S1Angle.Radians(radius_km / 6371)  # Earth radius
    )

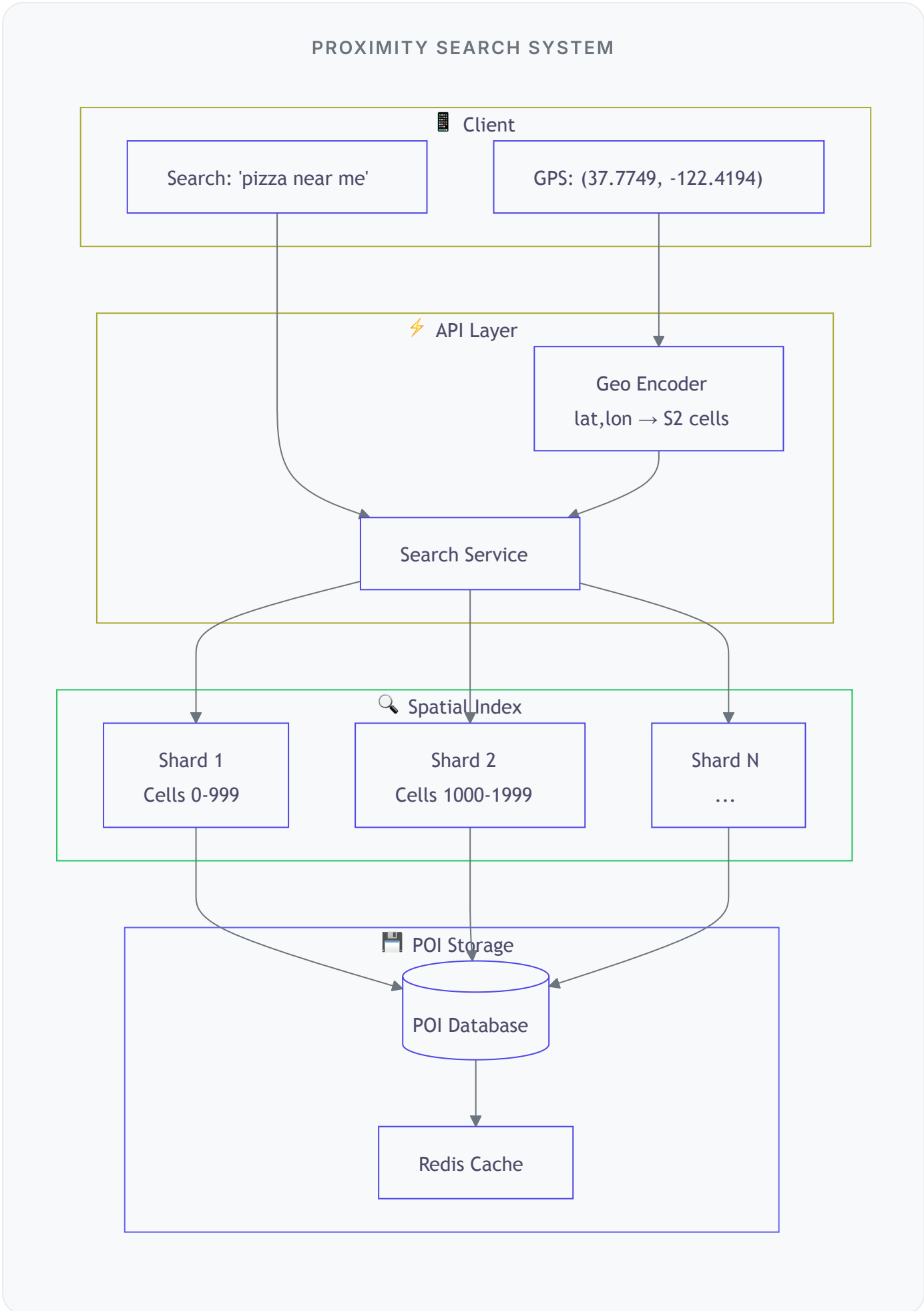
    # Get all S2 cells that cover this circle
    coverer = s2.S2RegionCoverer()
    coverer.set_max_cells(20)
    covering = coverer.GetCovering(cap)

    # Query for POIs in these cells
    cell_ids = [cell.id() for cell in covering]
    candidates = db.query("SELECT * FROM pois WHERE s2_cell IN :cells",
                          cells=cell_ids)

    # Filter by actual distance
    return [p for p in candidates
            if haversine(user_lat, user_lon, p.lat, p.lon) ⩽ radius_km]
```

The Complete Architecture

Sold to
@theskilledc
kumar.dodda@gmail.com



Sharding Strategy

How do you shard 100M POIs across multiple machines?

Sold to
@theskilledc
kumar.dodda@gmail.com

Option A: Shard by Geohash/S2 Cell Prefix

- All POIs starting with `9q` go to Shard 1
- All POIs starting with `dr` go to Shard 2
- Queries only hit 1-2 shards (cells + neighbors)

Option B: Shard by Region

- US West → Shard 1
- US East → Shard 2
- Europe → Shard 3
- Uneven distribution (cities vs. rural)

Option C: Shard by POI ID, Query All (Scatter-Gather)

- POIs distributed randomly by ID
- Every query hits all shards
- Simple, but doesn't scale as well

> 💡 **Google's Approach**
>
> Google uses S2 cells for sharding. Each shard handles a contiguous range of S2 cell IDs. Queries hit only the shards that contain the covering cells.

Caching Strategy

Location queries have high locality - many users query similar areas.

- Cache POIs by S2 cell ID
- Cache popular cells (Manhattan, downtown SF)
- TTL based on how often POIs change (hours to days)

```python
def get_pois_in_cell(cell_id):
    cache_key = f"cell:{cell_id}"

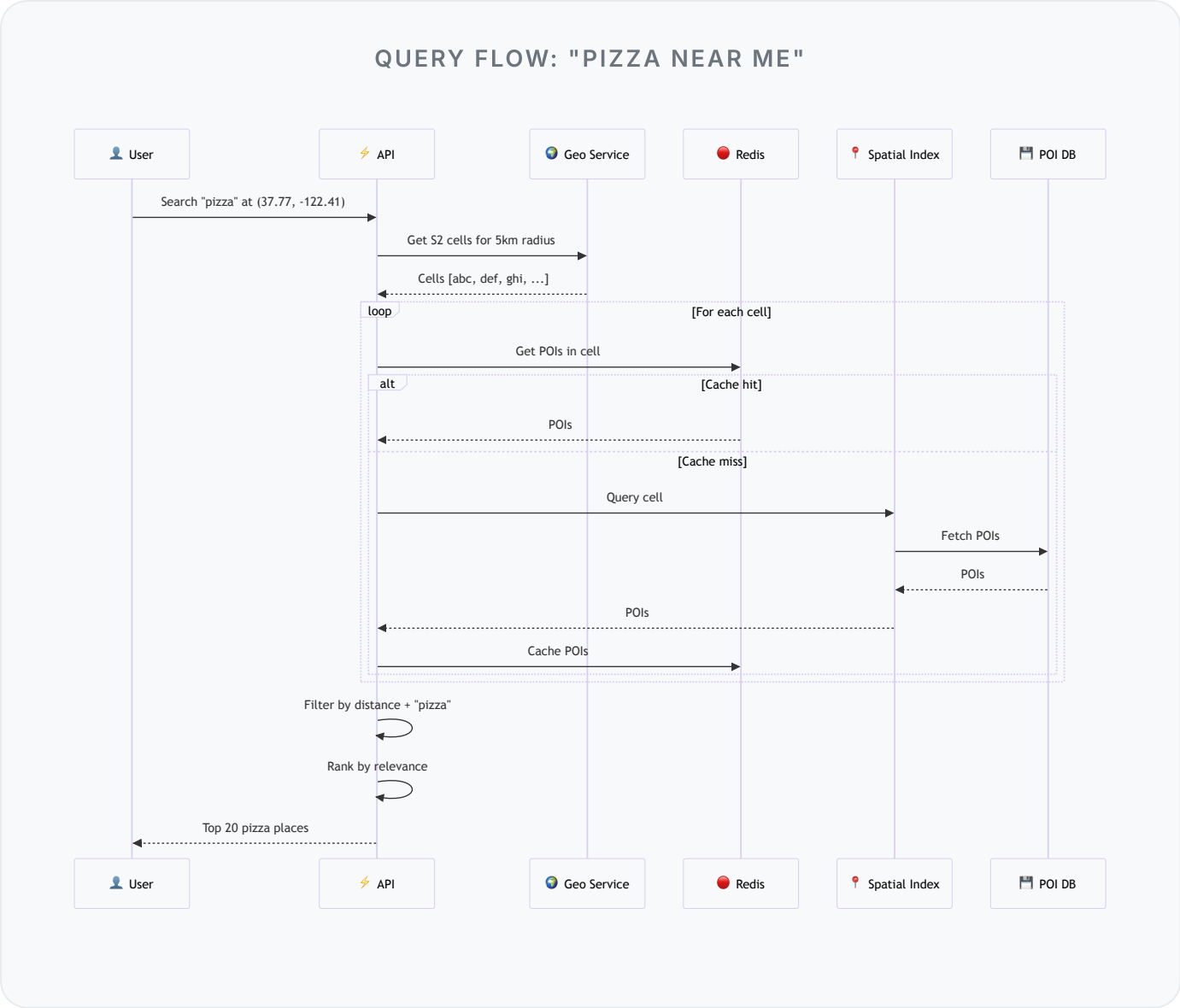
    cached = redis.get(cache_key)
    if cached:
        return json.loads(cached)

    pois = db.query("SELECT * FROM pois WHERE s2_cell = :cell", cell=cell_id)
    redis.setex(cache_key, 3600, json.dumps(pois))  # 1 hour TTL

    return pois
```

Sold to @theskilledc kumar.dodda@gmail.com

The Complete Query Flow



Ranking Results

Finding nearby POIs is step 1. Ranking them is step 2.

SIGNAL	WHY IT MATTERS	WEIGHT
Distance	Closer is usually better	High
Rating	4.8 stars vs 3.2 stars	High
Relevance	Name/category matches query	High
Popularity	Number of reviews, check-ins	Medium
Open now	Closed restaurant is useless	Medium
Price	User's budget preference	Low

Sold to @theskilledc
kumar.dodda@gmail.com

```python
def rank_pois(pois, user_lat, user_lon, query):
    for poi in pois:
        dist_score = 1 / (1 + poi.distance)  # Closer = higher
        rating_score = poi.rating / 5.0
        relevance = text_match(query, poi.name, poi.category)
        popularity = log(poi.review_count + 1) / 10
        open_now = 1.0 if poi.is_open() else 0.3

        poi.score = (
            dist_score * 0.3 +
            rating_score * 0.25 +
            relevance * 0.25 +
            popularity * 0.1 +
            open_now * 0.1
        )

    return sorted(pois, key=lambda p: p.score, reverse=True)
```

Interview Pivot (How to Explain It)

If asked:

"How would you design a proximity search system for 100M POIs?"

> "The core challenge is that lat/lon are two dimensions, but database indexes are one-dimensional. We need to convert 2D space into 1D.
>
> I'd use Geohash or S2 cells. They divide the world into a grid where each cell has an ID. Points that are close share similar IDs. For a 5km radius search, I compute which cells the search area covers (maybe 10-20 cells), query those cells via a simple index lookup, then filter by actual distance.
>
> For 100M POIs, I'd shard by cell ID prefix - all POIs in cells starting with '9q' go to one shard. Queries only hit shards that contain the covering cells. Hot cells (Manhattan, downtown) get cached in Redis.
>
> Finally, I'd rank results by distance, rating, relevance to the query, and whether they're currently open."

What's Next?

Proximity search leads to deeper questions:

Sold to
@theskilledc
kumar.dodda@gmail.com

How do you handle real-time location updates (Uber drivers)?

> **Write-heavy geospatial:** Use Redis GEOADD for real-time positions. Don't persist every update - batch writes to database. Drivers update every 5 seconds, query against in-memory spatial index.

How do you search along a route ("gas stations on my way")?

> **Polyline covering:** Convert the route to a series of S2 cells. Query POIs in those cells. Filter by distance from route segment.

How do you handle "restaurants in this area" (polygon search)?

> **Polygon covering:** Find S2 cells that cover the polygon. Query those cells. Post-filter with point-in-polygon test.

How do you handle POI updates (new restaurants, closed ones)?

> **Eventually consistent:** Updates go to write path. Cache invalidation by cell ID. Search index rebuilds periodically or via CDC.

The insight: Proximity search is a problem of dimensional reduction. Lat/lon are 2D. Indexes are 1D. Geohash and S2 bridge that gap by encoding location as a hierarchical string that preserves locality.

Encode location as cells. Query cells that cover the area. Filter by actual distance. That's how you find a pizza place in 200 milliseconds among 100 million options.

Sold to
@theskilledc
kumar.dodda@gmail.com

Same Principles Apply To

- **Ride matching (Uber, Lyft)** - same S2 cells for driver lookup
- **Food delivery (DoorDash)** - same geospatial indexing for restaurants
- **Real estate search (Zillow)** - same polygon/radius queries
- **Friend finder ("people nearby")** - same cell-based matching

Sold to
@theskilledc
kumar.dodda@gmail.com

How Would You Design a Video Upload Pipeline for 500 Hours/Minute

From Raw Upload to "Your Video Is Ready" - At YouTube Scale

A creator finishes editing their 10-minute video. They click "Upload" on YouTube. The file is 2GB. Their internet is mediocre. After 15 minutes, they see:

"Upload complete. Processing..."

30 minutes later:

"Your video is ready. Available in 1080p, 720p, 480p, 360p. Auto-generated captions ready. Thumbnail options generated."

This happens **500 hours every minute** on YouTube. That's 30,000 hours of video uploaded every hour. 720,000 hours per day. Each video needs to be:

- Uploaded reliably (even with flaky connections)
- Transcoded into multiple resolutions and formats
- Distributed to CDN edge servers globally
- Made searchable (metadata, captions)
- Available to play within minutes

Video upload is a study in massive data movement, async processing, and graceful handling of partial failures.

The Scale

- 500 hours/minute = 8.3 hours/second
- Average video: 10 minutes, 1GB file
- That's ~50 videos/second, ~50 GB/second upload bandwidth
- Each video → 5+ transcoded versions = 250+ transcoding jobs/second

Sold to
@theskilledc
kumar.dodda@gmail.com

🎬 THE UPLOAD

Creator: *uploads 2GB video* "Finally uploading my masterpiece!"

Progress Bar: 47%... 78%... 95%...

WiFi: *drops for 3 seconds*

Upload: "Connection lost. Starting over from 0%..."

Creator: "NOOOOOO!!! I was at 95%!!"

YouTube: "Maybe we should implement resumable uploads..."

2007 YouTube: "Way ahead of you. We learned this the hard way." 😤

⚡ The Engineering Complexity

A 1GB upload fails at 900MB. Do you start over? Here's why resumable isn't trivial:

📤 Large Files

1GB video. Connection drops at 900MB. Start over? Resume? How do you resume?

🎬 Transcoding

One upload → 10 resolutions × 3 formats = 30 versions. Each takes CPU hours.

📊 Processing Pipeline

Upload → virus scan → transcode → thumbnail → metadata → CDN. Any step can fail.

⏱️ Progress Feedback

User uploaded 500MB. Show progress? But you don't know final size until done.

First Attempts (and Why They Break)

Attempt 1: Single POST Request Upload

The obvious approach: send the whole file in one HTTP POST.

Sold to
@theskilledc
kumar.dodda@gmail.com

```http
POST /upload HTTP/1.1
Content-Type: video/mp4
Content-Length: 2147483648

[2GB of binary data...]
```

Why it feels right:

- Simple HTTP
- Works for small files
- Standard file upload pattern

Why it fails:

- 2GB over flaky connection = guaranteed failure
- Network drops at 1.9GB → start over from 0
- Server timeout after 30 minutes
- No progress indicator
- Memory: server needs to buffer entire file

> 💡 **Lesson**
>
> Large files need to be chunked. Network failures are guaranteed at scale.

Attempt 2: Transcode Synchronously

Wait for upload, then transcode, then respond "success."

Why it fails:

- Transcoding a 10-minute video takes 5-30 minutes
- HTTP connection stays open → timeouts
- User sees spinner for 45 minutes
- Server resources tied up during transcoding
- One failure = entire retry

> 💡 **Lesson**
>
> Transcoding is slow. It must be async. Separate upload from processing.

Sold to
@theskilledc
kumar.dodda@gmail.com

Attempt 3: Process Videos Sequentially

Queue videos, process one at a time.

Why it fails:

- 50 videos/second incoming, 1 video/5 minutes processing
- Queue grows infinitely
- Videos uploaded today processed next week

> 💡 **Lesson**
>
> Processing must be massively parallel. One transcoder isn't enough - you need thousands.

The Correct Approach

The video upload pipeline has three phases, each with different characteristics:

PHASE	CHALLENGE	SOLUTION
Upload	Reliability over flaky networks	Chunked, resumable uploads
Process	CPU-intensive transcoding	Distributed job queue
Distribute	Global availability	CDN edge caching

Phase 1: Chunked, Resumable Uploads

Split the file into chunks (e.g., 5MB each). Upload each chunk separately. Resume from last successful chunk on failure.

The flow:

1. Client requests upload session → Server returns `uploadId`
2. Client splits file into 5MB chunks
3. Client uploads each chunk with `uploadId` + chunk number
4. Server stores chunks temporarily
5. On last chunk, server assembles the file

Sold to
@theskilledc
kumar.dodda@gmail.com

```HTTP
// Step 1: Initialize upload
POST /uploads/init
Content-Type: application/json

{"filename": "video.mp4", "size": 2147483648, "chunks": 430}

Response: {"uploadId": "abc123", "chunkSize": 5242880}

// Step 2-N: Upload each chunk
PUT /uploads/abc123/chunks/0
Content-Type: application/octet-stream
Content-Range: bytes 0-5242879/2147483648

[5MB of data]

Response: {"chunk": 0, "status": "received"}

// On failure, client retries just that chunk
// On resume, client asks: "Which chunks do you have?"
GET /uploads/abc123/status

Response: {"receivedChunks": [0, 1, 2, 3, 4], "missing": [5, 6, ...]}
```

Why chunked upload is essential:

- Network fails at chunk 50? Resume from chunk 50, not chunk 0
- Progress bar: chunks completed / total chunks
- Parallel upload: send multiple chunks simultaneously
- Server doesn't buffer entire file in memory



Real-World Standard

Google uses the **Resumable Upload Protocol**. Amazon uses **Multipart Upload** for S3. Tus.io is an open protocol for resumable uploads.

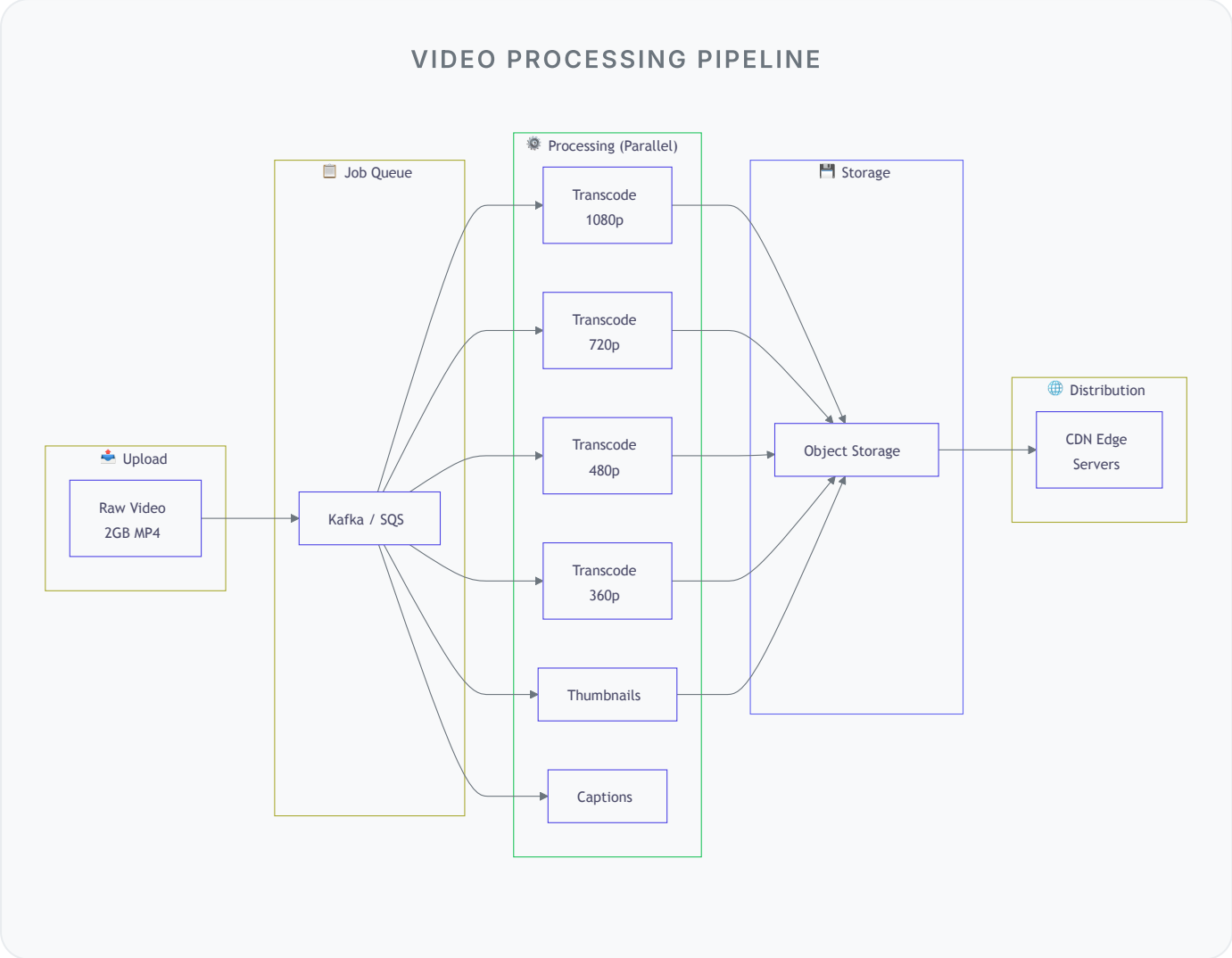
Phase 2: Video Processing Pipeline

Once the video is uploaded, it enters the processing pipeline:

1. **Validation** - Is it a valid video file? Not corrupted?
2. **Transcoding** - Convert to multiple resolutions and formats
3. **Thumbnail generation** - Extract frames for preview
4. **Audio extraction** - For captions, audio-only mode

Sold to
@theskilledc
kumar.dodda@gmail.com

5. **Caption generation** - Speech-to-text AI
6. **Content moderation** - Check for policy violations



Transcoding: The Heavy Lift

Transcoding converts video from one format/resolution to another. It's CPU-intensive.

Why multiple formats?

- User on 4G → serve 480p
- User on fiber → serve 1080p or 4K
- Different devices need different codecs
- Adaptive bitrate streaming switches quality mid-stream

Output for a single 10-minute video:

Sold to
@theskilledc
kumar.dodda@gmail.com

RESOLUTION	BITRATE	FILE SIZE
2160p (4K)	20 Mbps	~1.5 GB
1080p	8 Mbps	~600 MB
720p	5 Mbps	~375 MB
480p	2.5 Mbps	~190 MB
360p	1 Mbps	~75 MB

1 upload → 5+ output files. And each needs to be transcoded.

Parallel Processing

The key insight: **split the video into segments** and transcode in parallel.

```text
10-minute video → split into 60 segments (10 seconds each)

Segment 1 → Worker A → 1080p segment 1
Segment 2 → Worker B → 1080p segment 2
Segment 3 → Worker C → 1080p segment 3
...

All segments done → Concatenate → Final 1080p video
```

Result: A 30-minute transcoding job becomes 30 concurrent 30-second jobs.

```python
def process_video(video_id, input_path):
    # Split into segments
    segments = split_video(input_path, segment_duration=10)

    # Create jobs for each resolution + segment
    jobs = []
    for resolution in ["1080p", "720p", "480p", "360p"]:
        for segment in segments:
            jobs.append({
                "video_id": video_id,
                "segment": segment,
                "resolution": resolution
            })

    # Submit to job queue (workers pick up in parallel)
    queue.submit_batch(jobs)

    # When all segments done, concatenate
    wait_for_completion(video_id)
    concatenate_segments(video_id)
```

Sold to
@theskilledc
kumar.dodda@gmail.com

Job Queue Architecture

At 250+ transcoding jobs per second, you need a robust job queue:

- **Kafka** - For high-throughput job streaming
- **SQS** - Managed, auto-scaling queue
- **Worker pool** - Thousands of transcoding workers
- **Auto-scaling** - Scale workers based on queue depth

```text
Queue depth = 10,000 jobs → Scale to 500 workers
Queue depth = 100 jobs → Scale to 50 workers
```

Phase 3: Storage and Distribution

Once transcoded, videos need to be:

- Stored durably (never lose a video)
- Available globally with low latency
- Served efficiently (adaptive bitrate)

Storage

- **Object storage (S3/GCS)** - Cheap, durable, scalable to exabytes
- **Tiered storage** - Hot (SSD) for new videos, cold for old
- **Replication** - 3+ copies across regions

CDN Distribution

A viewer in Tokyo shouldn't fetch video from a US server. CDNs solve this.

- Popular videos cached at edge locations worldwide
- 50+ CDN points of presence
- First viewer in Tokyo → cache miss → fetch from origin
- Second viewer in Tokyo → cache hit → served from Tokyo edge

Adaptive Bitrate Streaming (HLS/DASH)

Modern video players switch quality based on network conditions.

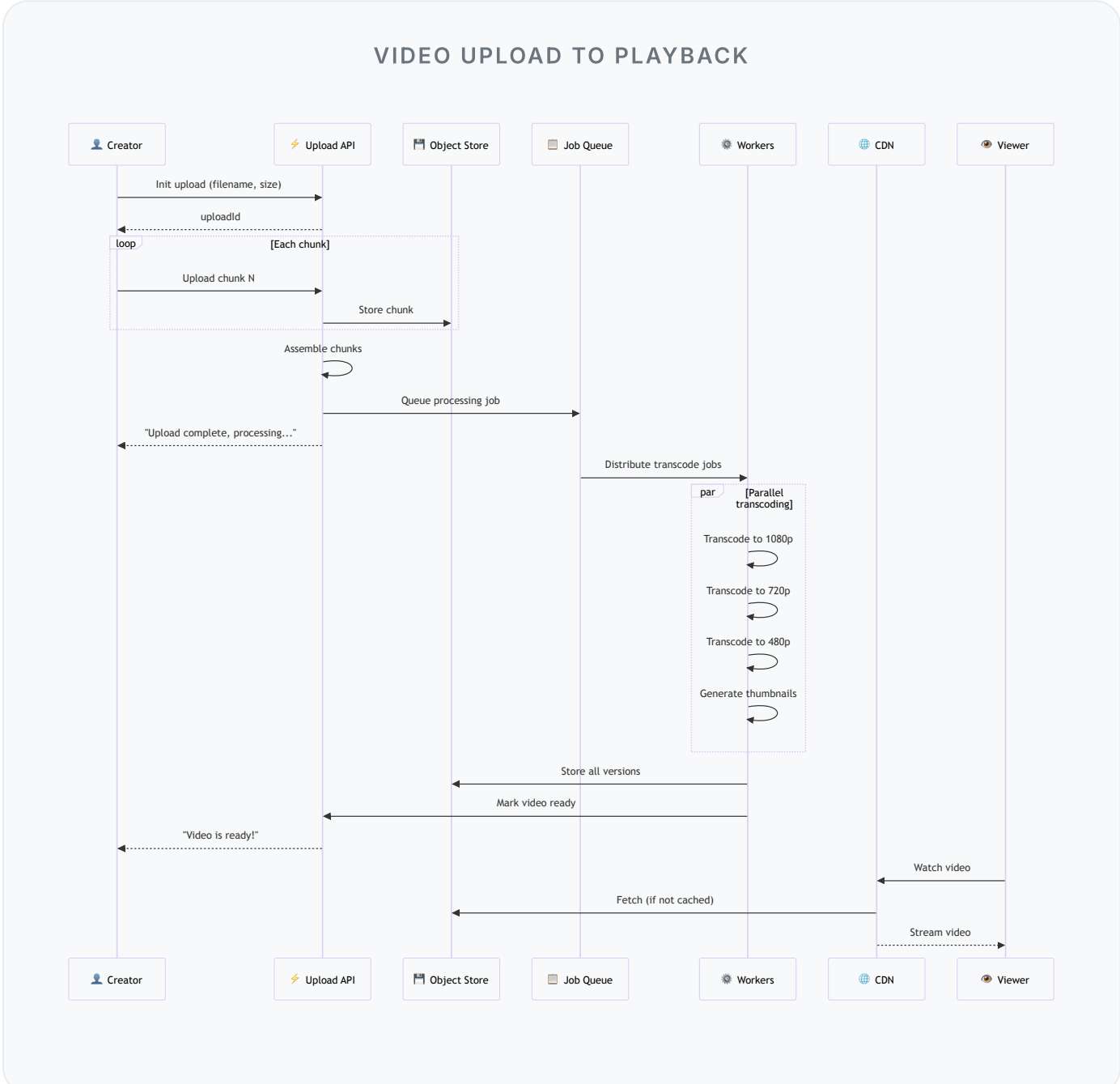
Sold to
@theskilledc
kumar.dodda@gmail.com

```
Manifest file (playlist.m3u8):
#EXTM3U
#EXT-X-STREAM-INF:BANDWIDTH=8000000,RESOLUTION=1920x1080
1080p/playlist.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=5000000,RESOLUTION=1280x720
720p/playlist.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=2500000,RESOLUTION=854x480
480p/playlist.m3u8

Player: "Network is fast → fetch 1080p segments"
Player: "Network degraded → switch to 480p mid-stream"
```

TEXT

Complete Architecture



Handling Failures

Sold to
@theskilledc
kumar.dodda@gmail.com

At this scale, failures are constant. Design for them:

FAILURE	IMPACT	SOLUTION
Upload interrupted	Partial chunks	Resumable upload, retry from last chunk
Transcoding worker dies	Segment not processed	Job timeout + retry, idempotent processing
Corrupted video file	Transcode fails	Validation step, notify creator
Storage failure	Video lost	Multi-region replication
CDN edge down	Slow playback	Route to next nearest edge

PYTHON

```python
# Idempotent job processing (safe to retry)
def transcode_segment(job):
    output_key = f"{job.video_id}/{job.resolution}/segment_{job.segment}.ts"

    # Check if already done (idempotent)
    if s3.exists(output_key):
        return "already_done"

    # Do the work
    output = ffmpeg.transcode(job.input_path, job.resolution)
    s3.upload(output_key, output)

    return "success"
```

Interview Pivot (How to Explain It)

If asked:

"How would you design a video upload pipeline for 500 hours/minute?"

"I'd split this into three phases: upload, process, and distribute.

For upload, I'd use chunked, resumable uploads. The client splits the file into 5MB chunks, uploads each with an upload ID. On network failure, we resume from the last successful chunk, not restart.

For processing, transcoding is CPU-intensive - one video takes 30 minutes. So I'd split the video into 10-second segments and transcode in parallel across a worker pool. A 10-minute video becomes 60 parallel jobs. I'd use Kafka or SQS for the job queue with auto-scaling workers.

Sold to
@theskilledc
kumar.dodda@gmail.com

For distribution, transcoded videos go to object storage (S3), then CDN caches them at edge locations worldwide. I'd use adaptive bitrate streaming (HLS) so players can switch quality based on network conditions."

What's Next?

Video upload leads to deeper questions:

How do you show processing progress to the user?

> **Progress tracking:** Track completed segments vs total. Push updates via WebSocket or poll an API. Show "Processing: 45%".

How do you handle copyright detection?

> **Content ID:** Audio/video fingerprinting. Compare against database of copyrighted content. Flag or block before publishing.

How do you prioritize processing for different creators?

> **Priority queues:** Verified creators get high-priority queue. Viral video re-requested → priority boost. Time-sensitive content (live) first.

How do you handle live streaming vs. on-demand?

> **Different pipeline:** Live = real-time encoding, no retries, segment-by-segment broadcast. On-demand = can take time, quality over speed.

Sold to
@theskilledc
kumar.dodda@gmail.com

The insight: Video upload is a pipeline, not a request-response. Upload, processing, and distribution are separate concerns with different scaling characteristics. Chunk everything: chunks for upload, segments for transcoding, edge servers for distribution.

Resume on failure. Process in parallel. Cache everywhere. That's how you handle 500 hours of video every minute.

🔁 **Same Principles Apply To**

- **Image processing (Instagram, Pinterest)** - same chunked upload + parallel resize
- **Audio processing (Spotify uploads)** - same transcode to multiple formats
- **Document processing (Google Docs PDF)** - same async conversion pipeline
- **Large file sharing (WeTransfer)** - same resumable upload protocol

Sold to
@theskilledc
kumar.dodda@gmail.com

How Would You Design a Distributed Cache for 1M Hits/Second

The Invisible Layer That Makes Everything Fast

Here's a number that should terrify you: **10 milliseconds**.

That's how long a typical database query takes. Seems fast, right? Now multiply by a million requests per second:

```text
1,000,000 requests × 10ms = 10,000,000 ms of database work per second
                          = 10,000 seconds of CPU time per second

You'd need 10,000 database servers just to keep up.
At $1000/month each = $10 million/month in database costs.
```

Now consider this: What if 90% of those requests are asking for the same data over and over?

- The homepage - same for everyone
- User profile #12345 - viewed 50,000 times today
- Product details - same product, millions of views
- Session data - same session, every single request

What if we could answer those requests in **0.1ms** instead of 10ms? What if we could answer them *without touching the database at all*?

That's what caching does. It's the invisible layer that makes your application feel instant. Without it, most high-traffic websites would be either impossibly slow or impossibly expensive.

But here's the twist: caching *seems* simple. Just store stuff in memory, right? At 1 million hits per second, every assumption breaks. Memory fills up. Servers fail. Data goes stale. Keys get hot. And suddenly, your "optimization" becomes your biggest operational nightmare.

Sold to
@theskilledc
kumar.dodda@gmail.com

The Famous Quote

"There are only two hard things in Computer Science: cache invalidation and naming things." - Phil Karlton

After this chapter, you'll understand why cache invalidation earned its place on that list.

The Engineering Complexity

'Just cache it' is the first solution. Cache invalidation is the eternal regret:

Invalidation

'There are only two hard things: cache invalidation and naming things.'

Cache Stampede

Cache expires. 10K requests hit DB simultaneously for same key. DB melts.

Consistency

Cache says one thing. DB says another. Which is truth? For how long?

Memory Pressure

Cache is full. Which key to evict? LRU? LFU? Random? Each has trade-offs.

First Attempts (and Why They Break)

Attempt 1: In-Process Cache (HashMap)

The simplest cache: a HashMap in your application's memory.

```python
cache = {}

def get_user(user_id):
    if user_id in cache:
        return cache[user_id]  # Cache hit!

    user = database.query("SELECT * FROM users WHERE id = ?", user_id)
    cache[user_id] = user
    return user
```

Why it feels right:

Sold to @theskilledc kumar.dodda@gmail.com

- Dead simple
- Zero network latency (same process)
- Works great on a single server

Why it fails:

- **No sharing:** Server A's cache doesn't help Server B
- **Memory limit:** Application OOMs when cache grows too big
- **Restart = empty:** Deploy a new version → cache is gone → database tsunami
- **Inconsistency:** User updates profile on Server A, Server B shows stale data

With 10 application servers, you effectively have 10 separate caches, each with different data, each limited to a fraction of total memory. **Cache hit rates plummet.**

> 💡 **Lesson**
>
> In-process caches work for single-server applications. At scale, you need a shared, external cache that all servers can access.

Attempt 2: Single Dedicated Cache Server

Move the cache to a dedicated server. All application servers connect to it.

```python
cache_client = CacheClient(host="cache.example.com")

def get_user(user_id):
    cached = cache_client.get(f"user:{user_id}")
    if cached:
        return json.loads(cached)

    user = database.query("SELECT * FROM users WHERE id = ?", user_id)
    cache_client.set(f"user:{user_id}", json.dumps(user), ttl=3600)  # 1 hour TTL
    return user
```

Why it's better:

- Shared across all servers - higher hit rate
- Persists across application deployments
- In-memory = fast (~100K ops/sec)

Why it fails at 1M hits/sec:

- **Single point of failure:** Cache server dies → entire site slows to a crawl

Sold to @theskilledc kumar.dodda@gmail.com

- **Single machine limit:** ~100K ops/sec max, you need 10x that
- **Memory ceiling:** One machine max ~500GB RAM
- **Network bottleneck:** All traffic through one NIC

> 💡 **Lesson**
>
> A single cache server is a bottleneck and a SPOF. For real scale, you need to distribute the cache across multiple machines.

Attempt 3: Multiple Servers with Random Routing

Add more cache servers! Route each key to a random server.

Why it fails:

- Key "user:123" goes to Server 2 on the first request
- Next request for same key goes to Server 7
- Cache miss on every request → no benefit

> 💡 **Lesson**
>
> The same key must ALWAYS go to the same server. Randomness defeats caching.

Attempt 4: Modulo Hashing

Hash the key, mod by number of servers.

```python
def get_server(key, num_servers=10):
    return hash(key) % num_servers
```

`user:123` always goes to server `hash("user:123") % 10 = 7`. Same key, same server. Problem solved?

Why it works... until it doesn't:

You have 10 servers. Everything is humming. Then Server 3 dies. You now have 9 servers.

Sold to
@theskilledc
kumar.dodda@gmail.com

```
Before (10 servers):
  hash("user:123") % 10 = 7 → Server 7
  hash("user:456") % 10 = 3 → Server 3

After (9 servers):
  hash("user:123") % 9 = 5 → Server 5 (DIFFERENT!)
  hash("user:456") % 9 = 0 → Server 0 (DIFFERENT!)

ALMOST EVERY KEY MOVED TO A DIFFERENT SERVER.
All cached data is now on the "wrong" server.
100% cache miss rate until cache warms up again.
```

TEXT

The same disaster happens when you add a server. Growing from 10 to 11 servers remaps 90% of your keys. Every scale event = cache stampede.

> ⚠️ **The Problem**
>
> With modulo hashing, adding or removing a server invalidates almost the entire cache. At 1M requests/sec, that means 1M requests suddenly hit your database. That's called a **cache stampede**, and it can take down your entire infrastructure.

The Solution: Consistent Hashing

Consistent hashing is the algorithm that makes distributed caching possible. The idea is deceptively simple but incredibly powerful.

Imagine a circle - a ring from 0 to 2^32. Both servers and keys get hashed onto this ring.

1. Hash each server name: `hash("server-1") = 12345` → place at position 12345 on ring
2. Hash each key: `hash("user:123") = 67890` → place at position 67890
3. To find which server handles a key: walk clockwise from key's position until you hit a server

Now the magic: when you add or remove a server, only the keys between that server and its predecessor are affected. All other keys stay on the same server!

Sold to
@theskilledc
kumar.dodda@gmail.com

```text
Ring with 4 servers: A at 0°, B at 90°, C at 180°, D at 270°

Keys 0-89° → Server A
Keys 90-179° → Server B
Keys 180-269° → Server C
Keys 270-359° → Server D

Server B dies. Now:
Keys 0-89° → Server A (unchanged!)
Keys 90-179° → Server C (only these moved!)
Keys 180-269° → Server C (unchanged!)
Keys 270-359° → Server D (unchanged!)

Only 25% of keys moved, not 90%!
```

Virtual nodes improve distribution further. Instead of one point per server, each server gets 100-200 virtual positions on the ring. This smooths out imbalances.

```python
class ConsistentHash:
    def __init__(self, servers, virtual_nodes=150):
        self.ring = {}
        self.sorted_keys = []

        for server in servers:
            for i in range(virtual_nodes):
                virtual_key = hash(f"{server}:{i}")
                self.ring[virtual_key] = server
                self.sorted_keys.append(virtual_key)

        self.sorted_keys.sort()

    def get_server(self, key):
        if not self.ring:
            return None

        key_hash = hash(key)

        # Find first server position ≥ key_hash
        for server_hash in self.sorted_keys:
            if server_hash ≥ key_hash:
                return self.ring[server_hash]

        # Wrap around to first server
        return self.ring[self.sorted_keys[0]]
```

Why It Matters

With consistent hashing, adding a cache server moves ~1/N of keys (where N is number of servers). With 10 servers, that's 10% of keys, not 90%. Your database survives.

Sold to
@theskilledc
kumar.dodda@gmail.com

Cache Patterns: How to Keep Data Fresh

Caching is easy. **Keeping the cache consistent with the database is the hard part.**

When data changes in the database, what happens to the cached copy? You have three choices:

Pattern 1: Cache-Aside (Lazy Loading)

The application manages the cache explicitly. Most common pattern.

```python
def get_user(user_id):
    # Try cache first
    cached = cache.get(f"user:{user_id}")
    if cached:
        return cached

    # Cache miss → load from DB
    user = db.get_user(user_id)
    cache.set(f"user:{user_id}", user, ttl=3600)
    return user

def update_user(user_id, data):
    db.update_user(user_id, data)
    cache.delete(f"user:{user_id}")  # Invalidate cache
```

Pros: Simple, application controls everything, resilient to cache failures

Cons: First request after invalidation is slow (cache miss)

Pattern 2: Write-Through

Write to cache and database together. Cache is always fresh.

```python
def update_user(user_id, data):
    db.update_user(user_id, data)
    cache.set(f"user:{user_id}", data)  # Write to cache too
```

Pros: Cache is always up-to-date, no cache misses after writes

Cons: Write latency increases (two writes), cache fills with data that may never be read

Pattern 3: Write-Behind (Write-Back)

Write to cache immediately, async persist to database later.

Sold to
@theskilledc
kumar.dodda@gmail.com

```python
def update_user(user_id, data):
    cache.set(f"user:{user_id}", data)
    queue.push({"user_id": user_id, "data": data})  # Async DB write
```

Pros: Fastest writes, can batch DB updates

Cons: Data loss risk if cache fails before DB write, complex to implement

Pattern 4: Read-Through

Cache handles loading from database automatically. Application just asks cache.

Pros: Simple application code, cache handles data loading

Cons: Cache library must understand your data access patterns

> 📝 **Most Common Choice**
>
> **Cache-aside** is the most widely used pattern because it's simple, gives you control, and your system keeps working if the cache dies. Start here.

The Hardest Problem: Cache Invalidation

User updates their profile. You update the database. You delete the cached version. Simple, right? Now consider:

Scenario 1: Race Condition

```text
Time    Thread 1 (read)              Thread 2 (write)
─────────────────────────────────────────────────────
T1      Cache miss for user:123
T2      Read from DB: {name: "Alice"}
T3                                   Update DB: {name: "Bob"}
T4                                   Delete cache key
T5      Write to cache: {name: "Alice"}  ← STALE DATA!
```

Thread 1's cache write happens AFTER Thread 2's invalidation. The cache now has stale data until TTL expires. User sees "Alice" when they should see "Bob."

Scenario 2: Distributed Invalidation

You have 3 data centers. User updates profile in US-East. How do caches in US-West and EU-Central know to invalidate?

Sold to
@theskilledc
kumar.dodda@gmail.com

Scenario 3: Complex Invalidation

User changes their name. You invalidate `user:123`. But what about:

- `timeline:123` - which shows the user's name
- `comments:456` - where user left a comment
- `search:alice` - search results containing the user

Invalidating one entity can cascade to dozens of cache keys.

Strategies for Cache Invalidation

1. TTL (Time-To-Live)

- Set expiration on every cache entry
- Trade-off: shorter TTL = fresher data, more cache misses
- Works well when "eventually consistent" is acceptable

```python
cache.setex("user:123", 300, data)  # 5 minute TTL
```

2. Explicit Invalidation

- When data changes, explicitly delete related cache keys
- Challenge: knowing ALL the keys to invalidate

```python
def update_user(user_id, data):
    db.update_user(user_id, data)
    cache.delete(f"user:{user_id}")
    cache.delete(f"timeline:{user_id}")
    cache.delete_pattern(f"comments:*:author:{user_id}")
    # ... how many more?
```

3. Versioning / Generational Keys

- Include a version number in the cache key
- Changing version effectively invalidates all old entries

```python
version = db.get_user_version(user_id)  # Stored in DB
cache_key = f"user:{user_id}:v{version}"

# On update: increment version in DB, old cache keys become orphans
```

4. Pub/Sub Invalidation

Sold to
@theskilledc
kumar.dodda@gmail.com

- Publish invalidation events to a message bus
- All cache nodes subscribe and invalidate locally
- Works across data centers

The Hot Key Problem

Taylor Swift announces a concert. Her profile page gets 10 million views in an hour. The key `user:taylor` is requested 3000 times per second.

With consistent hashing, that key maps to ONE server. That server is now handling 3000 requests/second for a single key while other servers are idle.

This is the **hot key problem**, and it can melt a cache server.

Solutions for Hot Keys

1. Replication

- Replicate hot keys across multiple servers
- Append random suffix: `user:taylor:1`, `user:taylor:2`, etc.
- Read from random replica

```python
def get_hot_key(key):
    replica = random.randint(0, 9)  # 10 replicas
    return cache.get(f"{key}:replica:{replica}")
```

2. Local Cache + Distributed Cache

- Two-tier caching: local in-process cache + shared distributed cache
- Hot keys cached locally on every app server
- Reduces network hops for the hottest data

```python
local_cache = {}  # In-process, small (1000 items)

def get(key):
    if key in local_cache:
        return local_cache[key]

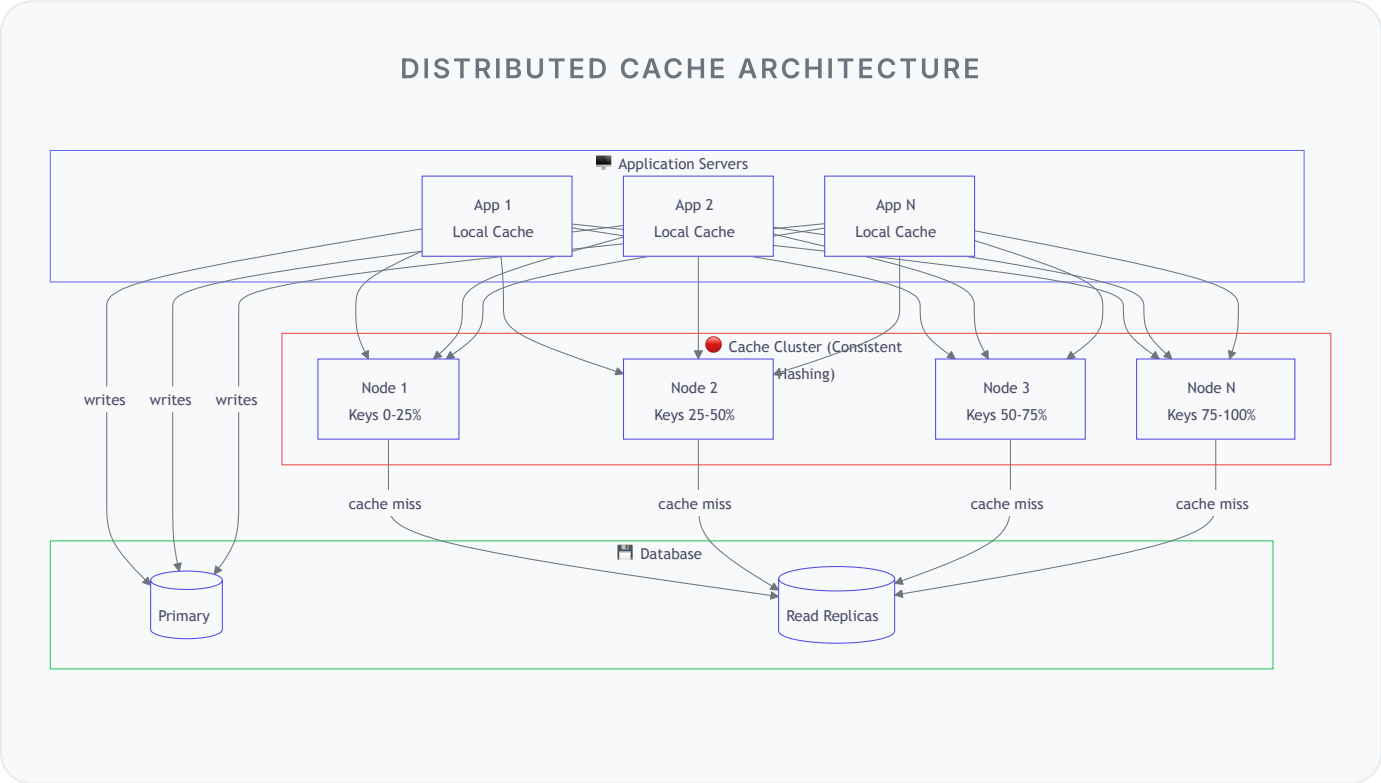
    value = distributed_cache.get(key)
    if is_hot(key):  # Accessed frequently
        local_cache[key] = value
    return value
```

Sold to
@theskilledc
kumar.dodda@gmail.com

3. Request Coalescing

- If 100 requests come for the same key simultaneously, only ONE fetches from source
- Others wait and share the result

Complete Architecture



Putting It All Together

COMPONENT	PURPOSE	TECHNOLOGY
Local Cache	Fastest access for hot data	In-process HashMap, Caffeine
Distributed Cache	Shared cache across servers	Redis Cluster, Memcached
Consistent Hashing	Route keys to correct shard	Client library (redis-cluster)
TTL + Invalidation	Keep cache fresh	Application logic + pub/sub

Interview Pivot (How to Explain It)

If asked:

"How would you design a distributed cache for 1M hits/second?"

Sold to
@theskilledc
kumar.dodda@gmail.com

"At 1M hits/sec, a single cache server isn't enough. I'd use a cluster of Redis or Memcached nodes with consistent hashing to distribute keys.

Consistent hashing ensures that when a node fails or we add capacity, only 1/N of keys move - not 90% like with modulo hashing. This prevents cache stampedes from melting the database.

For consistency, I'd use cache-aside pattern with TTLs. On writes, invalidate the cache and let it reload on next read. For hot keys that would overload a single node, I'd replicate them across multiple shards with random suffixes.

I'd also add a small local in-process cache on each app server for the absolute hottest keys - eliminates network round-trips for the most frequent accesses."

What's Next?

Distributed caching leads to deeper questions:

How do you handle cache stampede (thundering herd)?

> **Lock on cache miss:** First request acquires lock, fetches from DB, populates cache. Other requests wait for lock, then read from cache. Or use probabilistic early expiration - refresh before TTL hits.

How do you warm up a cold cache after deployment?

> **Cache warming:** Pre-populate cache from DB before directing traffic. Or use shadow traffic to warm cache. Gradual traffic ramp-up with fallback to DB.

Redis vs Memcached - when to use which?

> **Memcached:** Simple key-value, multi-threaded, pure cache. **Redis:** Rich data structures (lists, sets, sorted sets), persistence, pub/sub, Lua scripting. Use Redis unless you need extreme simplicity.

Sold to
@theskilledc
kumar.dodda@gmail.com

How do you monitor cache health?

?

> **Key metrics:** Hit rate (should be >95%), memory usage, eviction rate, latency percentiles, connection count. Alert on hit rate drops - they often indicate bugs.

The paradox of caching: it looks simple but hides enormous complexity. The cache that speeds up your application can also take it down - through stampedes, hot keys, inconsistency, and operational surprises.

Consistent hashing for distribution. TTL for freshness. Cache-aside for simplicity. Replication for hot keys. That's how you handle 1M hits per second without breaking your database or your sanity.

Same Principles Apply To

- **CDN (content caching)** - same consistent hashing, same invalidation challenges
- **DNS caching** - same TTL-based freshness model
- **Browser caching** - same cache-control and versioning strategies
- **CPU caches (L1/L2/L3)** - same hierarchy, same cache coherence problems

Sold to
@theskilledc
kumar.dodda@gmail.com

How Would You Design a Fraud Detection System for 10K Transactions/Second

The 50-Millisecond Decision That Saves Billions

A user in New York just swiped their card at a coffee shop. $4.50 for a latte.

In the next **50 milliseconds**, your system must answer one question:

> **Is this transaction legitimate, or is someone stealing money?**

If you say "yes" to a fraudster, you lose money. If you say "no" to a legitimate customer, you lose trust (and maybe the customer). And you have 50ms to decide - because the customer is standing at the counter, waiting.

Now multiply that by 10,000 transactions every second. That's the scale of a major payment processor. **864 million decisions per day**. Every single one needs to be accurate, fast, and explainable.

The Stakes

- **Global card fraud:** $32+ billion annually
- **False positives:** 30% of legitimate transactions wrongly declined
- **Customer abandonment:** 33% never retry after a false decline
- **Latency requirement:** Sub-100ms for card-present transactions

This is the paradox of fraud detection: you're fighting an adversary who constantly adapts, using incomplete information, under extreme time pressure, where both types of errors are costly.

The Engineering Complexity

You have 50 milliseconds to decide: fraud or legitimate. No pressure:

Sold to @theskilledc kumar.dodda@gmail.com

⏱️ Real-Time Decision

Transaction in progress. You have 50ms to say 'fraud' or 'legit.' No pressure.

🎯 False Positives

Block legitimate user = lost sale + angry customer. 1% false positive = thousands blocked.

📊 Evolving Patterns

Fraudsters adapt. Yesterday's rules don't catch today's tricks. Models go stale.

🔍 Feature Explosion

IP, device, amount, time, location, history... 100+ features per transaction to evaluate.

First Attempts (and Why They Break)

Attempt 1: Simple Rules

The obvious approach: write rules based on known fraud patterns.

```python
def is_fraud(transaction):
    # Rule 1: Large transactions are suspicious
    if transaction.amount > 5000:
        return True

    # Rule 2: International transactions are risky
    if transaction.country ≠ user.home_country:
        return True

    # Rule 3: Night transactions are suspicious
    if transaction.hour < 6 or transaction.hour > 23:
        return True

    return False
```

Why it feels right:

- Easy to understand and explain
- Fast to execute
- Catches obvious fraud

Why it fails:

- **Too many false positives:** Blocks legitimate travelers, night owls, big purchases

Sold to @theskilledc gunaltaneja@gmail.com

- **Easy to evade:** Fraudsters learn the rules and stay under thresholds
- **No context:** "$5000 is suspicious" - but not for someone who regularly buys expensive items
- **Rule explosion:** Hundreds of rules, conflicting logic, unmaintainable

Real story: A major bank blocked all transactions from Nigeria after a fraud spike. They blocked thousands of legitimate customers - including their own employees traveling there.

> 💡 **Lesson**
>
> Static rules can't capture the nuance of human behavior. What's "normal" varies by person, time, location, and context.

Attempt 2: Batch Machine Learning

Train a machine learning model on historical fraud data. Run predictions nightly.

```python
# Nightly batch job
def score_users():
    for user in all_users:
        features = compute_features(user)  # Last 30 days of activity
        user.fraud_score = model.predict(features)
        db.save(user)

# At transaction time
def is_fraud(transaction):
    return transaction.user.fraud_score > 0.8
```

Why it's better:

- ML captures complex patterns
- Learns from historical data
- Transaction-time lookup is fast

Why it fails:

- **Stale data:** Score computed last night doesn't know about today's activity
- **No transaction context:** Can't see that THIS transaction is unusual
- **Slow to adapt:** New fraud patterns take days to detect
- **Account takeover:** If account is stolen today, batch job won't know until tomorrow

Imagine: A thief steals a card at 9 AM. The batch job ran at midnight. For the next 15 hours, every fraudulent transaction sails through with yesterday's "low risk" score.

Sold to
@theskilledc
kumar.dodda@gmail.com

> **Lesson**
>
> Fraud is real-time. Your detection must be real-time too. Yesterday's features can't catch today's fraud.

Attempt 3: Real-Time ML But Slow Features

Run ML scoring in real-time, but compute features by querying the database.

```python
def is_fraud(transaction):
    # Compute features in real-time (SLOW!)
    tx_count_24h = db.query("""
        SELECT COUNT(*) FROM transactions
        WHERE user_id = ? AND created_at > NOW() - INTERVAL 24 HOUR
    """, transaction.user_id)

    total_spent_24h = db.query("""
        SELECT SUM(amount) FROM transactions
        WHERE user_id = ? AND created_at > NOW() - INTERVAL 24 HOUR
    """, transaction.user_id)

    # ... 20 more queries for other features ...

    features = [tx_count_24h, total_spent_24h, ...]
    return model.predict(features) > 0.8
```

Why it fails:

- **Too slow:** 20 database queries × 10ms each = 200ms latency
- **Database overload:** 10K transactions × 20 queries = 200K queries/second
- **Doesn't scale:** Adding features means adding more queries

> **Lesson**
>
> Feature computation is the bottleneck. You can't query the database 20 times per transaction and still hit 50ms latency.

The Core Insight

Real-time fraud detection requires solving two problems simultaneously:

1. **Fast feature computation** - Can't query the database per transaction
2. **Fast model inference** - Must score in single-digit milliseconds

Sold to @theskilledc kumar.dodda@gmail.com

The solution: **pre-compute and maintain features continuously**, so at transaction time, you just look them up and score.

COMPONENT	PURPOSE	LATENCY
Feature Store	Pre-computed features, continuously updated	< 5ms lookup
Streaming Pipeline	Update features in real-time as events flow	< 1 second lag
Model Server	Low-latency ML inference	< 10ms
Rule Engine	Hard rules + soft signals	< 5ms

The Architecture

Component 1: Feature Store

The feature store maintains up-to-date features for every user and entity. Features are updated in real-time as events flow through the system.

Example features for fraud detection:

FEATURE	WHAT IT CAPTURES	WHY IT MATTERS
`tx_count_1h`	Transactions in last hour	Velocity spike = card testing
`tx_count_24h`	Transactions in last day	Unusual activity pattern
`unique_merchants_24h`	Different merchants today	Fraudsters hit many stores quickly
`avg_amount_30d`	Typical spend amount	This transaction vs. normal behavior
`max_amount_ever`	Largest past transaction	Is this unusually large?
`last_location`	Previous transaction location	Impossible travel detection
`device_age_days`	How long using this device	New device = higher risk
`failed_tx_1h`	Failed transactions recently	Card testing pattern

Storage: Low-latency key-value store (Redis, Cassandra) keyed by user_id.

Sold to @theskilledc
kumar.dodda@gmail.com

```json
{
  "user_id": "12345",
  "features": {
    "tx_count_1h": 3,
    "tx_count_24h": 12,
    "unique_merchants_24h": 5,
    "avg_amount_30d": 45.67,
    "max_amount_ever": 1200.00,
    "last_location": {"lat": 40.7128, "lng": -74.0060},
    "last_tx_time": "2024-01-15T14:30:00Z",
    "device_age_days": 180,
    "failed_tx_1h": 0
  },
  "updated_at": "2024-01-15T14:32:00Z"
}
```

Component 2: Streaming Pipeline

Every transaction, login, and event updates the feature store in real-time.

```python
# Streaming processor (Kafka Streams / Flink / Spark Streaming)
def process_transaction_event(event):
    user_id = event.user_id

    # Update velocity features
    features = feature_store.get(user_id)

    features["tx_count_1h"] += 1
    features["tx_count_24h"] += 1
    features["last_location"] = event.location
    features["last_tx_time"] = event.timestamp

    # Recalculate aggregates
    features["avg_amount_30d"] = recalculate_rolling_avg(
        features["avg_amount_30d"],
        event.amount
    )

    if event.amount > features["max_amount_ever"]:
        features["max_amount_ever"] = event.amount

    feature_store.set(user_id, features)
```

Windowed aggregations:

- `tx_count_1h` - Sliding window, decays over time
- `tx_count_24h` - Tumbling windows or sliding
- Use approximate algorithms (HyperLogLog, Count-Min Sketch) for cardinality

Sold to @theskilledc
kumar.dodda@gmail.com

Component 3: Scoring Service

At transaction time, fetch features and score in real-time.

```python
def score_transaction(transaction):
    # Step 1: Fetch pre-computed features (< 5ms)
    user_features = feature_store.get(transaction.user_id)
    merchant_features = feature_store.get(transaction.merchant_id)

    # Step 2: Compute transaction-specific features (< 1ms)
    tx_features = {
        "amount": transaction.amount,
        "amount_vs_avg": transaction.amount / user_features["avg_amount_30d"],
        "distance_from_last": haversine(
            transaction.location,
            user_features["last_location"]
        ),
        "time_since_last_tx": now() - user_features["last_tx_time"],
        "is_new_merchant": transaction.merchant_id not in user_features["known_merchants"],
    }

    # Step 3: Combine all features
    all_features = {**user_features, **merchant_features, **tx_features}

    # Step 4: Run ML model (< 10ms)
    fraud_score = model.predict(all_features)

    # Step 5: Apply rules + thresholds
    decision = make_decision(fraud_score, transaction, user_features)

    return decision
```

Component 4: Decision Engine

The ML score is just a number. The decision engine turns it into action.

SCORE RANGE	ACTION	USER EXPERIENCE
0.0 - 0.3	Approve	Transaction succeeds
0.3 - 0.7	Step-up verification	SMS code, biometric, etc.
0.7 - 0.9	Soft decline + review	Ask to verify, human review
0.9 - 1.0	Hard decline	Block transaction

Plus hard rules that override the model:

Sold to
@theskilledc
kumar.dodda@gmail.com

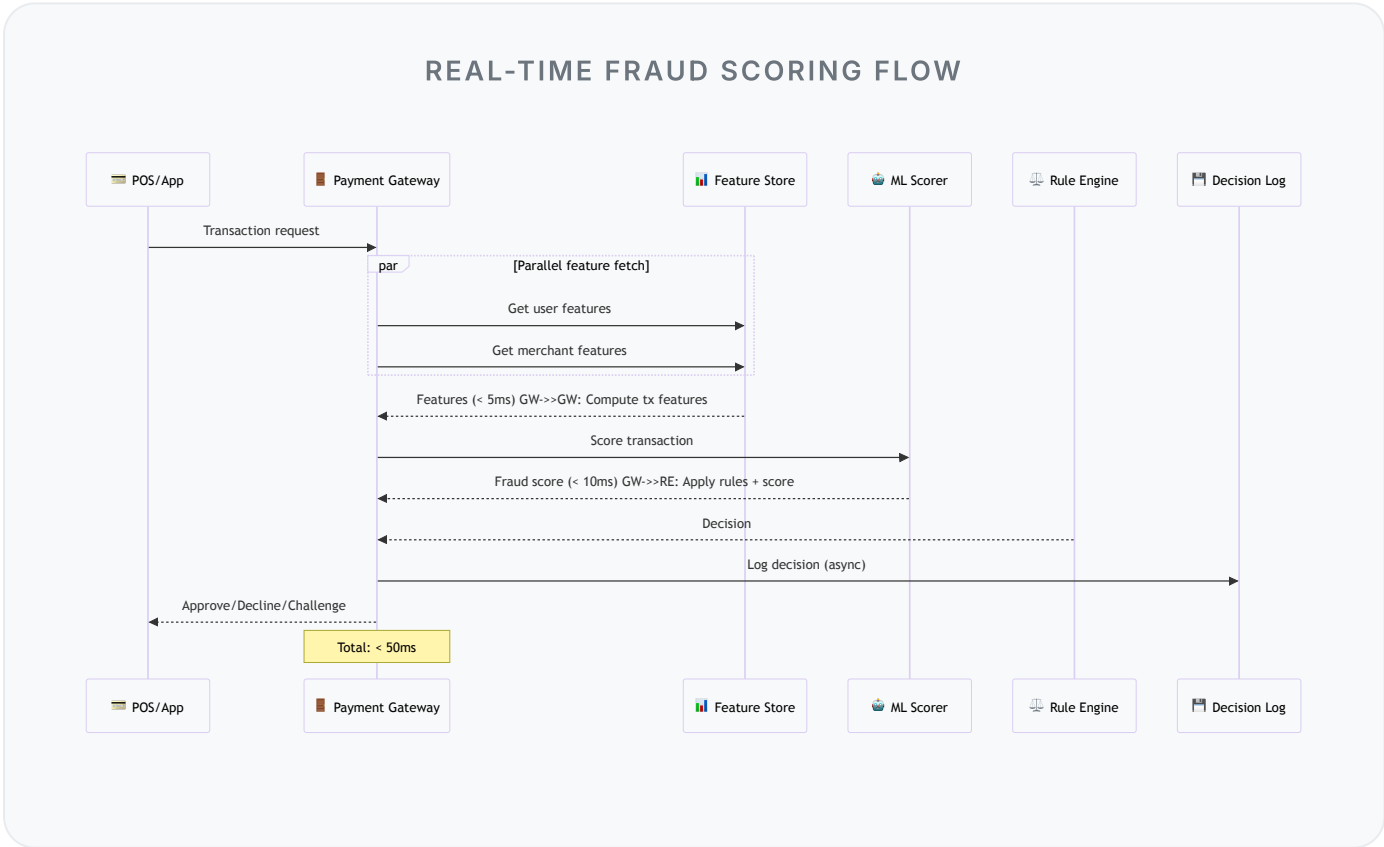
```python
def make_decision(fraud_score, transaction, features):
    # Hard rules (always block)
    if transaction.card in stolen_cards_list:
        return "BLOCK"

    if impossible_travel(transaction.location, features["last_location"],
features["last_tx_time"]):
        return "BLOCK"  # NYC 5 min ago, now in Tokyo?

    # Velocity rules (always challenge)
    if features["tx_count_1h"] > 20:
        return "CHALLENGE"

    # ML-based decision
    if fraud_score > 0.9:
        return "BLOCK"
    elif fraud_score > 0.5:
        return "CHALLENGE"
    else:
        return "APPROVE"
```

Complete Request Flow

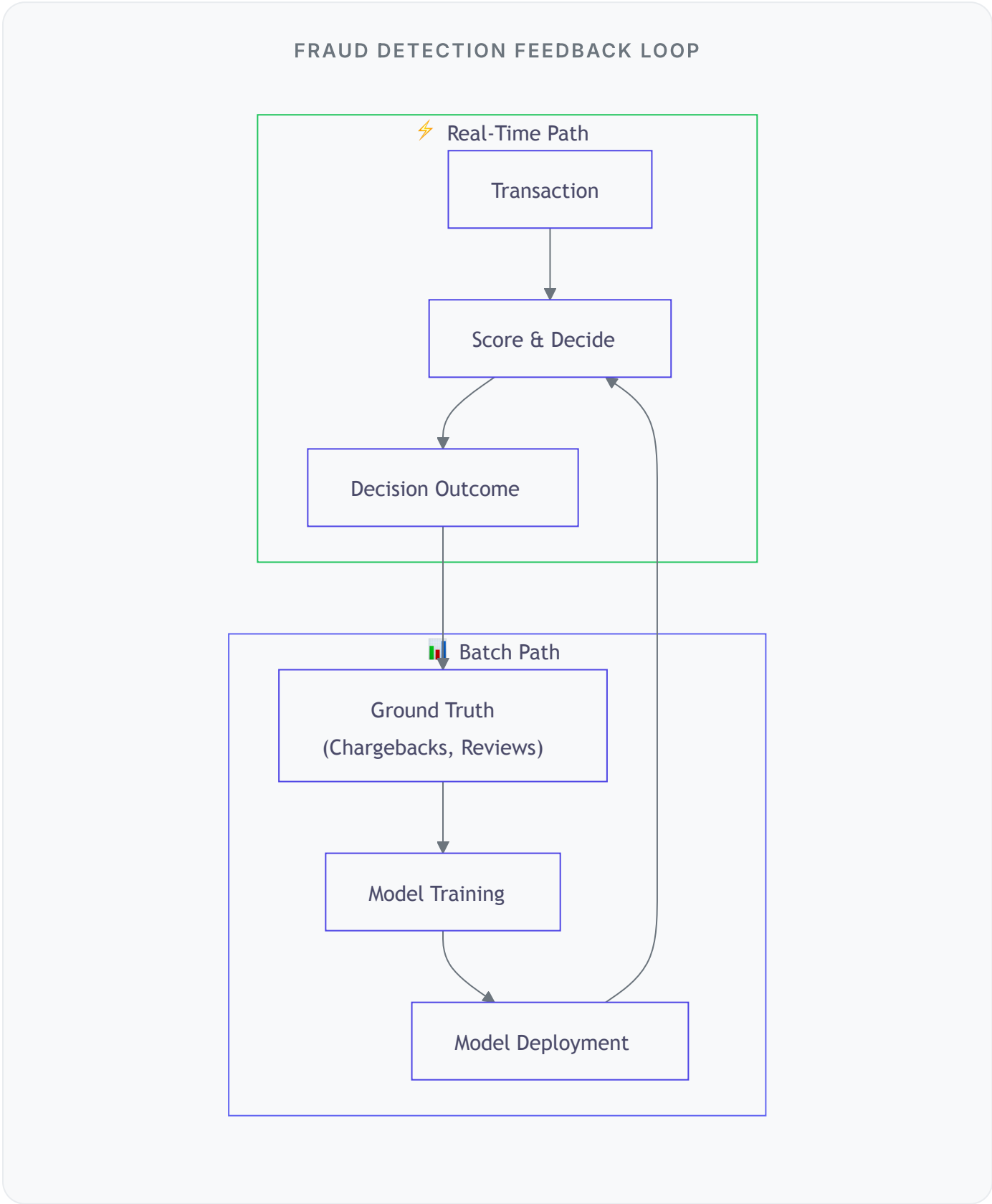


The Feedback Loop

A fraud detection system is only as good as its learning loop.

Sold to
@theskilledc
kumar.dodda@gmail.com

1. **Transaction happens** → Score generated → Decision made
2. **Outcome known** → Was it actually fraud? (chargebacks, disputes, manual review)
3. **Labels collected** → Ground truth dataset updated
4. **Model retrained** → New patterns incorporated
5. **Model deployed** → Better decisions



Time to feedback:

Sold to
@theskilledc
kumar.dodda@gmail.com

- Chargebacks: 30-60 days (slow, but definitive)
- Customer disputes: 1-7 days
- Manual review: Same day
- Challenge pass/fail: Immediate

> ⚠️ **The Label Delay Problem**
>
> A new fraud pattern starts today. You won't see chargebacks for 30-60 days. By the time your model learns, you've lost millions.
>
> **Solution:** Use early signals (manual reports, velocity anomalies) and unsupervised anomaly detection alongside supervised models.

Model Considerations

Common model types:

MODEL	PROS	CONS
Gradient Boosting (XGBoost, LightGBM)	Accurate, handles mixed features, fast inference	Less interpretable, needs feature engineering
Logistic Regression	Interpretable, fast, explainable	Misses complex patterns
Neural Networks	Can learn complex patterns	Slow inference, black box, needs lots of data
Isolation Forest (Anomaly Detection)	Catches unknown patterns	High false positives

Most common approach: Ensemble of XGBoost/LightGBM for primary scoring + anomaly detection for catching novel patterns + rules for known fraud types.

Class imbalance: Fraud is rare (~0.1% of transactions). Handle with:

- Oversampling fraud cases (SMOTE)
- Undersampling legitimate
- Class weights in loss function
- Precision-recall tradeoff (not accuracy!)

Sold to @theskilledc kumar.dodda@gmail.com

Interview Pivot (How to Explain It)

If asked:

"How would you design a fraud detection system for 10K transactions/second?"

> "The key insight is that we can't compute features at transaction time - that's too slow. Instead, I'd maintain a feature store with pre-computed, continuously updated features for every user and entity.
>
> A streaming pipeline (Kafka + Flink) processes every transaction and updates features in real-time - things like transaction count in the last hour, average amount, last location. These are stored in a low-latency store like Redis.
>
> At transaction time, I fetch the pre-computed features (5ms), compute transaction-specific features like 'is this amount unusual' (1ms), and run ML inference (10ms). Total latency under 50ms.
>
> I'd use a gradient boosting model for the main scoring, plus a rule engine for hard rules like impossible travel detection. The system needs a feedback loop - chargebacks and disputes become training labels. And I'd run shadow scoring on new models before deploying to avoid breaking production."

What's Next?

Fraud detection leads to deeper questions:

How do you detect collusion fraud (multiple accounts working together)?

> **Graph analysis:** Build a graph of relationships (shared devices, addresses, payment methods). Connected components reveal fraud rings.

How do you A/B test fraud models without losing money?

> **Shadow scoring:** Run new model in parallel, log predictions, but don't affect decisions. Compare outcomes. Or use multi-armed bandits to gradually shift traffic.

Sold to
@theskilledc
kumar.dodda@gmail.com

How do you explain a decline to regulators/customers?

> **Explainability:** Use SHAP/LIME to show feature contributions. "Declined because: unusual location (30%), velocity spike (25%), new device (20%)."

How do you handle adversarial attacks on the model?

> **Robustness:** Monitor for distribution shift. Ensemble multiple models. Don't expose score. Rate limit challenge attempts. Humans review edge cases.

The art of fraud detection: You're not just building a system, you're playing chess against an adversary who learns and adapts. Every model you deploy, they'll probe its weaknesses. Every rule you write, they'll find the edge.

Pre-compute features. Score in real-time. Combine rules with ML. Build a feedback loop. That's how you make 864 million decisions a day - each one in 50 milliseconds, each one balancing fraud prevention against customer experience.

Same Principles Apply To

- **Spam detection** - same real-time scoring, velocity features
- **Content moderation** - same ML + rules hybrid
- **Credit risk scoring** - same feature store, similar models
- **Anomaly detection in monitoring** - same streaming + ML approach

Sold to
@theskilledc
kumar.dodda@gmail.com

How Do You Design a System Knowing Requirements Will Change Completely in a Year?

The Art of Building for a Future You Can't Predict

Picture this: It's 2015, and you're designing a ride-sharing app. The PM says: "We match riders with drivers. That's it."

So you build it. Clean database schema. Nice, tight API. Everything optimized for that one use case.

Fast forward to 2024. The same system now handles:

- Ride-sharing (cars, bikes, scooters)
- Food delivery
- Package delivery
- Grocery delivery
- Freight logistics
- Autonomous vehicles
- Electric vehicle charging

If you designed it tightly for "match riders with drivers," you've now rewritten it seven times. If you designed it with change in mind, it grew gracefully.

The question isn't whether requirements will change - they will. The question is: how do you build systems that embrace change instead of fighting it?

💬 **The Paradox**

"We can't predict the future, so let's just build for today's requirements."

This seems practical, but it's a trap. The cost of changing a brittle system later is 10-100x higher than building with flexibility from the start.

Sold to
@theskilledc
kumar.dodda@gmail.com

The Patterns That Break Under Change

Let's look at what makes systems rigid - and then fix each one.

Anti-Pattern 1: Tightly Coupled Schemas

You design a `users` table for your MVP:

```sql
CREATE TABLE users (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100),
    phone VARCHAR(20),
    address VARCHAR(500)
);
```

Six months later: "We need to support multiple addresses per user." "We need to track which phone is verified." "We need to store metadata about each email."

That simple table now needs a complete rewrite. Every query breaks. Every service that reads this table breaks.

Anti-Pattern 2: Implicit Contracts

Service A calls Service B. Nobody wrote down the contract. Service B changes a field name. Service A breaks in production.

```json
// Service B's response (before)
{
  "user_name": "Alice",
  "created": "2024-01-15"
}

// Service B's response (after "cleanup")
{
  "userName": "Alice",          // camelCase now
  "createdAt": "2024-01-15"     // renamed
}

// Service A: crashes
```

Anti-Pattern 3: Big Bang Deploys

You can't deploy the new schema until all services are updated. You can't update all services until the new schema is deployed. Deadlock.

Sold to
@theskilledc
kumar.dodda@gmail.com

Result: Six-month projects to change a single field. Migrations that require downtime. Fear of making any changes.

Anti-Pattern 4: Knowing Too Much

Service A doesn't just call Service B - it knows exactly how B stores data, which database it uses, where the files live. When B changes internals, A breaks.

Principles for Building Evolvable Systems

Principle 1: Contracts First, Implementation Second

Before writing a line of code, define the contract: the API, the schema, the events. Write it down. Version it. Make it explicit.

```yaml
# OpenAPI contract
paths:
  /users/{id}:
    get:
      summary: Get user by ID
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                required:
                  - id
                  - email
                properties:
                  id:
                    type: string
                  email:
                    type: string
                  name:
                    type: string
                  addresses:
                    type: array
                    items:
                      $ref: '#/components/schemas/Address'
```

Now everyone who consumes this API can trust it. Changes to the contract are versioned and communicated. Implementation details are hidden.

Principle 2: Backwards-Compatible Schema Evolution

When schemas must change, do it in a way that doesn't break existing readers.

Sold to @theskilledc
kumar.dodda@gmail.com

The Rules:

CHANGE TYPE	BACKWARDS COMPATIBLE?	APPROACH
Add optional field	✅ Yes	Just add it, old readers ignore it
Add required field	❌ No	Add as optional first, then require
Remove field	⚠️ Careful	Deprecate, wait, then remove
Rename field	❌ No	Add new, populate both, migrate, remove old
Change field type	❌ No	Add new field, migrate readers, remove old

Example: Renaming a field safely

```text
Phase 1: Add new field, write to both
{
  "user_name": "Alice",     // old
  "userName": "Alice"       // new (duplicate data)
}

Phase 2: Migrate all readers to use new field
Phase 3: Stop writing old field
Phase 4: Remove old field
```

This is tedious. But it means zero downtime, zero broken clients, zero surprises.

Principle 3: API Versioning

When you must make breaking changes, version the API so old clients keep working.

Three strategies:

STRATEGY	EXAMPLE	TRADE-OFFS
URL Path	`/api/v1/users`, `/api/v2/users`	Clear, but URL changes with each version
Header	`Accept: application/vnd.api.v2+json`	Clean URLs, but less discoverable
Query param	`/users?version=2`	Simple, but pollutes query string

Most common: URL path versioning (`/v1/`). It's explicit and cache-friendly.

Sold to @theskilledc kumar.dodda@gmail.com

```python
# Running multiple API versions simultaneously
@app.route("/api/v1/users/")
def get_user_v1(id):
    user = db.get_user(id)
    return {
        "user_name": user.name,  # old format
        "email": user.email
    }

@app.route("/api/v2/users/")
def get_user_v2(id):
    user = db.get_user(id)
    return {
        "userName": user.name,   # new format
        "email": user.email,
        "addresses": user.addresses  # new field
    }
```

Principle 4: Design for Extension, Not Prediction

You can't predict the future. But you can design for extension.

Instead of this:

```python
class Order:
    def calculate_shipping(self):
        if self.type == "standard":
            return 5.99
        elif self.type == "express":
            return 15.99
        elif self.type == "overnight":
            return 25.99
        # Adding a new shipping type = modifying this code
```

Do this:

Sold to
@theskilledc
kumar.dodda@gmail.com

```python
class ShippingCalculator:
    calculators = {}

    @classmethod
    def register(cls, type_name, calculator):
        cls.calculators[type_name] = calculator

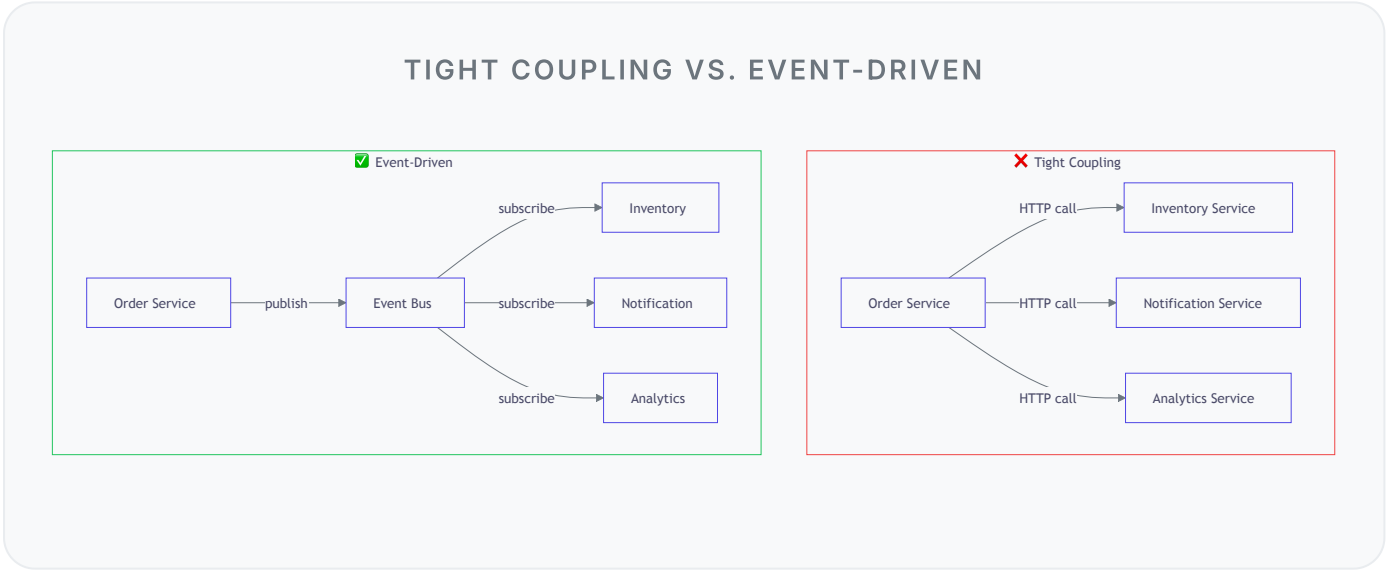
    @classmethod
    def calculate(cls, order):
        calculator = cls.calculators.get(order.type)
        if not calculator:
            raise ValueError(f"Unknown type: {order.type}")
        return calculator(order)

# Each shipping type is independently addable
ShippingCalculator.register("standard", lambda o: 5.99)
ShippingCalculator.register("express", lambda o: 15.99)
ShippingCalculator.register("drone", lambda o: 35.99)  # New! No core code changes
```

Principle 5: Loose Coupling with Events

When services talk directly to each other, they're tightly coupled. Change one, break the other.

Events decouple services. A service publishes what happened; others react if they care.



With events:

- Order service doesn't know who's listening
- Add new consumers without changing Order service
- Consumers can be added, removed, or rewritten independently
- If Notification service is down, Order service still works

Sold to
@theskilledc
kumar.dodda@gmail.com

```json
// Event: OrderPlaced
{
  "event_type": "order.placed",
  "version": "1.0",
  "timestamp": "2024-01-15T14:30:00Z",
  "data": {
    "order_id": "12345",
    "user_id": "67890",
    "total": 99.99,
    "items": [...]
  }
}
```

Principle 6: Feature Flags for Gradual Change

Deploy code that isn't "live" yet. Enable features for specific users, regions, or percentages. Roll back instantly if problems arise.

```python
def get_shipping_cost(order):
    if feature_flags.is_enabled("new_shipping_calculator", user=order.user):
        return new_calculate_shipping(order)  # New implementation
    else:
        return old_calculate_shipping(order)  # Original

# Feature flag config
{
    "new_shipping_calculator": {
        "enabled": True,
        "rollout_percentage": 10,  # 10% of users
        "allowlist": ["internal_testers"]
    }
}
```

Benefits:

- Deploy anytime - feature is "off" until ready
- Test with real traffic on subset of users
- Instant rollback - just toggle the flag
- Decouple deployment from release

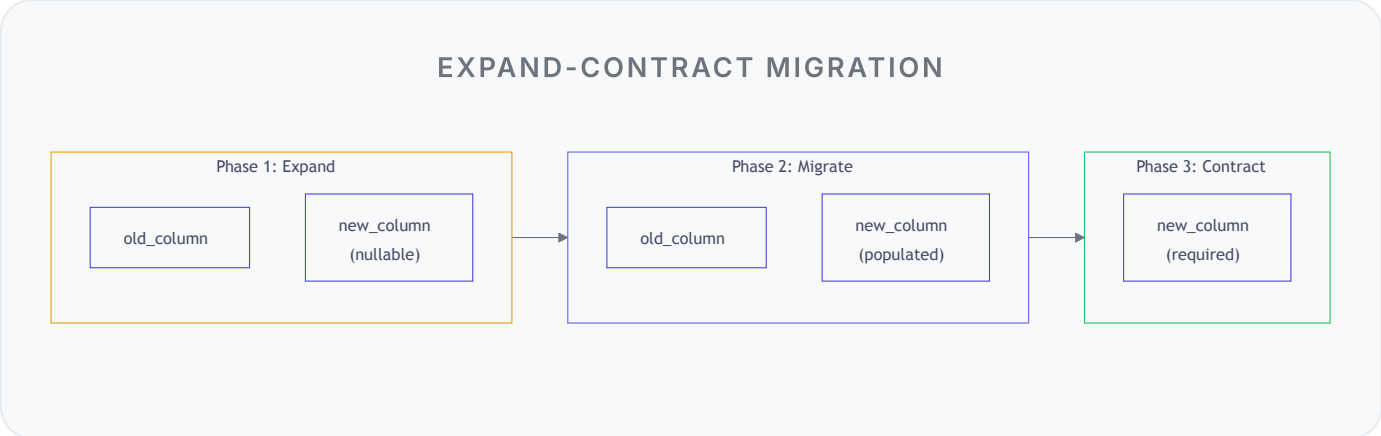
Evolving Databases Without Downtime

Database changes are the scariest. Here's how to do them safely.

The Expand-Contract Pattern

Sold to
@theskilledc
kumar.dodda@gmail.com

Never make breaking changes in one step. Expand first, then contract.



Example: Renaming a column

```sql
-- Phase 1: EXPAND - Add new column
ALTER TABLE users ADD COLUMN user_name VARCHAR(100);

-- Application writes to BOTH columns
UPDATE users SET name = :value, user_name = :value WHERE id = :id;

-- Phase 2: MIGRATE - Backfill existing data
UPDATE users SET user_name = name WHERE user_name IS NULL;

-- Application reads from new column, writes to both

-- Phase 3: CONTRACT - Remove old column (after all readers migrated)
ALTER TABLE users DROP COLUMN name;
```

Each phase is a separate deploy. At no point do you have downtime or broken code.

Data Format Evolution

Use serialization formats that support schema evolution:

FORMAT	SCHEMA EVOLUTION	BEST FOR
JSON	Flexible, no schema	APIs, configs, flexible data
Protobuf	Excellent - field numbers	Service-to-service, gRPC
Avro	Schema registry, evolution rules	Event streaming, Kafka
Thrift	Good - field IDs	RPC frameworks

Protobuf example:

Sold to
@theskilledc
kumar.dodda@gmail.com

```protobuf
message User {
  int32 id = 1;
  string name = 2;
  string email = 3;

  // Added later - doesn't break old readers
  optional string phone = 4;
  repeated Address addresses = 5;

  // Removed fields - just reserve the number
  reserved 6, 7;
}
```

Field numbers (1, 2, 3...) are the contract. Add new fields with new numbers. Never reuse old numbers. Old readers ignore unknown fields.

The Meta-Strategy: What to Build Today

You can't predict the future, but you can follow a simple heuristic:

> **Build 80% for what you know today. Build 20% for what might change.**

What to get right from day one:

- **Explicit contracts** - APIs, events, schemas documented and versioned
- **Service boundaries** - Draw the lines where change is likely
- **Data model flexibility** - Use extensible formats, avoid over-normalization
- **Deployment pipeline** - Fast, safe deploys make change cheap

What to defer:

- Optimizations for scale you don't have yet
- Features that "might be needed someday"
- Abstractions for problems you haven't encountered

> 💡 **The Real Insight**
>
> It's not about predicting the future. It's about making change cheap. If you can change your system in hours instead of months, you don't need to predict anything.

Sold to
@theskilledc
kumar.dodda@gmail.com

Interview Pivot (How to Explain It)

If asked:

"How do you design a system knowing requirements will change completely?"

> "I focus on making change cheap rather than predicting the future. Three main strategies:
>
> First, explicit contracts. Every API and event has a versioned schema. Consumers depend on contracts, not implementations. This lets us change internals without breaking clients.
>
> Second, backwards-compatible evolution. When schemas must change, I follow the expand-contract pattern: add new fields as optional, migrate data, update readers, then remove old fields. No big bang migrations.
>
> Third, loose coupling through events. Services publish events rather than calling each other directly. New features can subscribe to existing events without touching the producers.
>
> The goal isn't to build for every possible future - that's impossible. It's to make changes fast and safe when requirements inevitably shift."

What's Next?

Designing for change leads to deeper questions:

How do you deprecate an API version? ?

> **Sunset headers:** Add `Deprecation` header with sunset date. Monitor usage of old versions. Reach out to heavy users. Set a deadline and stick to it.

How do you migrate a million rows in production? ?

> **Online migrations:** Migrate in batches during low traffic. Dual-write during migration. Use tools like pt-online-schema-change or gh-ost for MySQL. Never lock the table.

Sold to
@theskilledc
kumar.dodda@gmail.com

How do you handle breaking changes in internal services?

> **Consumer-driven contracts:** Consumers define what they need. Providers are tested against consumer expectations. Pact or similar tools automate this.

When do you say "no" to backwards compatibility?

> **Major versions:** Some changes are too expensive to do backwards-compatibly. In that case, create a new major version (v2), run both, and give a long migration window. But this is expensive - only for fundamental shifts.

The paradox of designing for change: The more you try to predict the future, the more brittle your system becomes. The more you embrace uncertainty, the more adaptable you are.

Explicit contracts. Backwards compatibility. Expand-contract migrations. Event-driven decoupling. Feature flags. These aren't about predicting change - they're about making change cheap. And when change is cheap, you stop fearing it.

Same Principles Apply To

- **Mobile apps** - can't force updates; must be backwards-compatible
- **Public APIs** - external clients you don't control rely on stability
- **Microservices** - each service evolves independently
- **Data pipelines** - schema changes ripple through downstream systems

Sold to
@theskilledc
kumar.dodda@gmail.com

How Do You Design Systems Where Data Must Disappear Everywhere Instantly?

The Right to Be Forgotten - In a System That Never Forgets

A user in Germany clicks "Delete my account."

Under GDPR, you have **30 days** to erase all their personal data. Not just the main database. *Everything*:

- Their profile in the users database
- Their orders spread across 5 services
- Their messages in the chat system
- Their photos on object storage
- Their events in analytics pipelines
- Their data in search indexes
- Their logs in the logging system
- Their data in data warehouses
- Their data in ML training datasets
- Their data in backups from the last 90 days
- Their data at third-party vendors

Miss even one place, and you're facing **€20 million or 4% of global revenue** in fines. Whichever is higher.

Deletion is not the opposite of creation. In distributed systems, data spreads like water - into caches, replicas, logs, backups, analytics, ML models. Getting it all out is exponentially harder than putting it in.

Sold to
@theskilledc
kumar.dodda@gmail.com

📊 **The Scale of the Problem**

- Average enterprise has data in **400+ SaaS applications**
- One user's data can be in **50+ internal systems**
- GDPR deletion requests up **72% year-over-year**
- Average time to fully delete: **45 days** (often over the 30-day limit)

First Attempts (and Why They Break)

Attempt 1: Just DELETE FROM users

The obvious approach: delete the row from the main database.

```sql
DELETE FROM users WHERE id = 12345;
```

Why it feels right:

- Simple
- User is gone from the main table
- Most queries won't find them

Why it fails:

- **Foreign keys:** Orders, messages, reviews all reference this user
- **Other services:** Analytics, search, recommendations all have copies
- **Backups:** User still exists in last night's backup
- **Logs:** User's IP, email appear in application logs
- **Caches:** User profile cached in Redis for 24 hours

You deleted the user from one table. They still exist in 47 other places.

💡 **Lesson**

Deletion from one database isn't deletion. It's hiding. The data is still everywhere else.

Attempt 2: Soft Delete (is_deleted Flag)

Don't actually delete. Just mark as deleted.

Sold to
@theskilledc
kumar.dodda@gmail.com

```sql
UPDATE users
SET is_deleted = TRUE, deleted_at = NOW()
WHERE id = 12345;
```

Why it's tempting:

- No foreign key issues
- Easy to "undo" if needed
- Less risk of breaking things

Why it fails for GDPR:

- **Data still exists.** "Deleted" flag doesn't delete data - regulators check.
- **Personal data must be erased,** not just hidden
- **Name, email, IP - all still in the row**

> ⚠ **GDPR Requirement**
>
> "The data subject shall have the right to obtain from the controller the **erasure** of personal data." Soft delete is not erasure.

Attempt 3: Cascade Delete Everywhere

Add `ON DELETE CASCADE` to all foreign keys. Call every service with a delete.

```python
def delete_user(user_id):
    users_service.delete(user_id)
    orders_service.delete_by_user(user_id)
    messages_service.delete_by_user(user_id)
    analytics_service.delete_by_user(user_id)
    search_service.delete_by_user(user_id)
    # ... 20 more services ...
    backup_service.delete_from_all_backups(user_id)  # How?!
```

Why it fails:

- **Discovery problem:** Which services have this user's data?
- **Partial failures:** 15 services succeed, 5 fail. Now what?
- **Backups:** Can't surgically remove one user from a backup
- **Third parties:** Data sent to vendors, analytics providers
- **Immutable logs:** Event sourcing systems can't delete events

Sold to @theskilledc
kumar.dodda@gmail.com

> 💡 **Lesson**
>
> You can't delete what you can't find. And in most organizations, nobody knows everywhere a user's data exists.

The Core Problem: Data Lineage

Before you can delete data, you need to answer:

1. **Where is this user's data?** Every system, service, database, file
2. **What counts as their data?** PII vs. aggregated/anonymized
3. **What must we keep?** Legal retention requirements (financial records)
4. **Who else has it?** Third-party vendors, partners

You cannot delete what you cannot find. Data lineage - knowing where data flows - is the foundation of compliant deletion.

The Solution: Design for Deletion From Day One

Strategy 1: User ID as the Anchor

Every piece of user data, in every system, must be linked to a user identifier. No orphaned data with emails or names but no user ID.

```json
// ✅ Good: Always include user_id
{
  "order_id": "ORD-123",
  "user_id": "USR-456",     // Anchor for deletion
  "items": [...]
}

// ❌ Bad: User info without user_id
{
  "order_id": "ORD-123",
  "customer_email": "alice@example.com",  // Can't find all of Alice's orders easily
  "items": [...]
}
```

Strategy 2: Data Catalog / Registry

Sold to
@theskilledc
kumar.dodda@gmail.com

Maintain a central registry of every system that stores user data. When deletion happens, you know exactly where to look.

SYSTEM	DATA TYPES	RETENTION	DELETION METHOD
users-db	profile, email, phone	Until deletion	Hard delete
orders-db	order history	7 years (legal)	Anonymize
analytics	events, pageviews	2 years	Crypto shredding
search-index	name, products	Realtime	Delete from index
S3 photos	profile pics	Until deletion	Delete objects

Strategy 3: Cryptographic Erasure (Crypto Shredding)

Here's the elegant solution for data that's hard to delete (backups, logs, event streams):

Encrypt user data with a per-user encryption key. To "delete" the data, delete the key. The encrypted data becomes unreadable garbage.

PYTHON

```python
# When storing user data
def store_user_data(user_id, data):
    key = get_or_create_user_key(user_id)  # Per-user encryption key
    encrypted = encrypt(data, key)
    database.store(user_id, encrypted)

# When reading user data
def read_user_data(user_id):
    key = get_user_key(user_id)
    if not key:
        return None  # Key deleted = data unreadable
    encrypted = database.get(user_id)
    return decrypt(encrypted, key)

# GDPR deletion request
def gdpr_delete(user_id):
    delete_user_key(user_id)  # That's it. All data is now cryptographic garbage.
```

Why this is powerful:

- Backups still contain encrypted data, but it's unreadable
- Event logs still have events, but payloads are unreadable
- No need to find and delete every copy

Sold to @theskilledc kumar.dodda@gmail.com

- Deletion is instant - just delete one key

> 💡 **Key Management**
>
> You need a secure key management system. Store keys separately from data. Use a service like AWS KMS, HashiCorp Vault, or similar. Never store keys in the same database as the encrypted data.

Strategy 4: Anonymization vs. Deletion

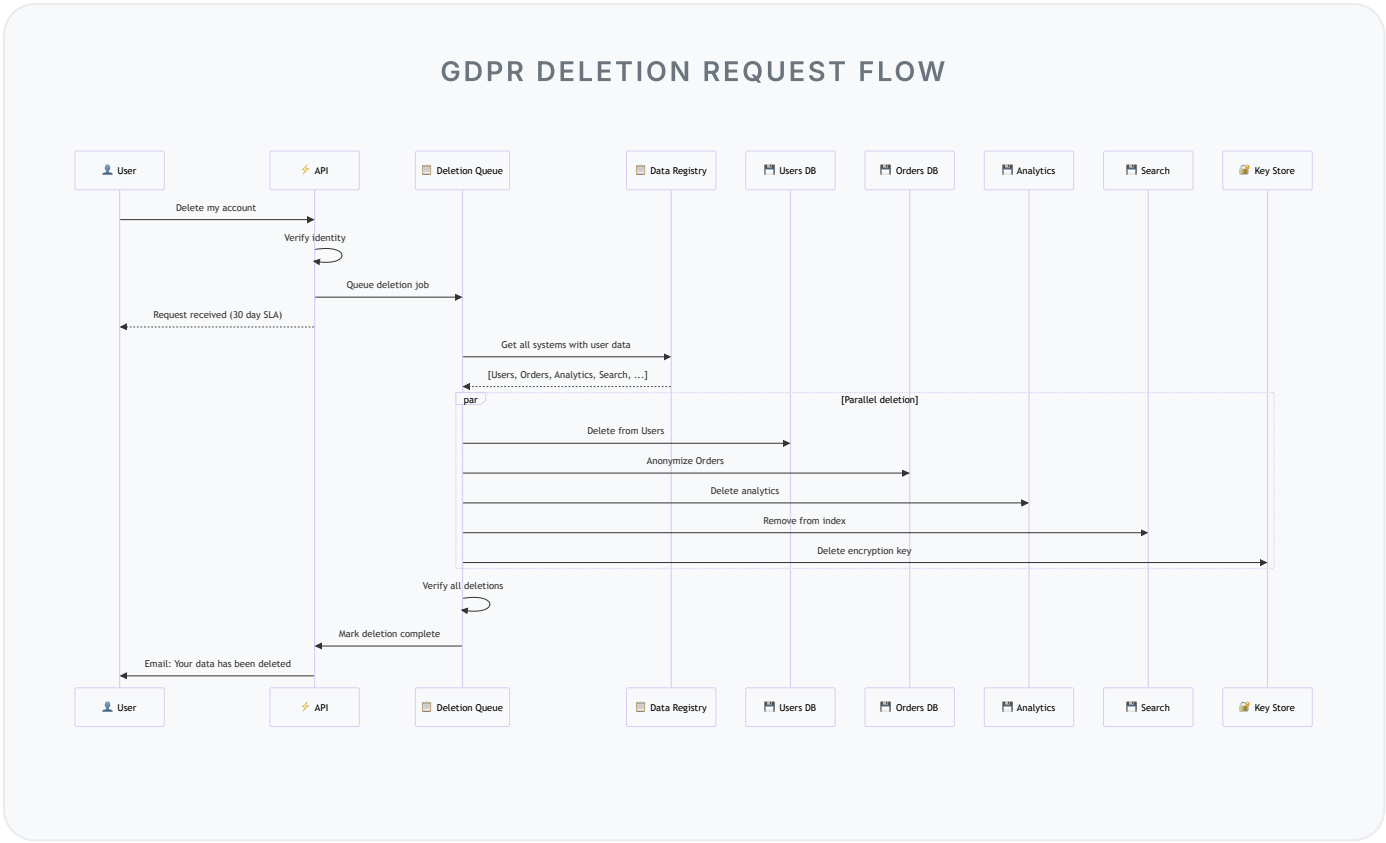
Sometimes you can't delete data (legal requirements). But you can anonymize it.

BEFORE	AFTER ANONYMIZATION
`name: "Alice Smith"`	`name: "[DELETED]"`
`email: "alice@example.com"`	`email: null`
`user_id: "USR-456"`	`user_id: "anon-abc123"`
`total: $150.00`	`total: $150.00` (keep for accounting)

Key principle: Anonymized data is no longer "personal data" under GDPR. If you can't re-identify the person, it's not covered.

The Deletion Pipeline

Sold to
@theskilledc
kumar.dodda@gmail.com



Key Components

COMPONENT	PURPOSE
Deletion Queue	Reliable processing, retries, tracking
Data Registry	Central catalog of where data lives
Per-system handlers	Each system knows how to delete its data
Verification	Confirm deletion actually happened
Audit log	Prove to regulators what you deleted and when

The Hard Problems

Problem 1: Backups

You can't surgically remove one user from a backup. Options:

- **Crypto shredding:** Data in backup is encrypted with deleted key
- **Backup rotation:** Wait for old backups to age out (90 days typical)
- **Document the gap:** "Data may exist in backups for up to 90 days"

Problem 2: Event Sourcing / Immutable Logs

Sold to
@theskilledc
kumar.dodda@gmail.com

Event sourced systems are designed to never delete. Events are facts that happened.

- **Crypto shredding:** Event payloads encrypted per-user
- **Tombstone events:** "UserDeleted" event, readers skip that user
- **Log compaction:** Kafka can delete messages for a key
- **Cold storage rotation:** Delete when logs age out

Problem 3: Third-Party Data

You sent data to Stripe, Segment, Mixpanel, Salesforce, Google Analytics...

- **Vendor audit:** List all vendors with user data
- **Contractual requirements:** Vendors must support deletion requests
- **API calls:** Programmatically trigger deletion at each vendor
- **Minimize data sharing:** Don't send PII if you don't need to

Problem 4: Derived Data

User's data was used to train an ML model. Is the model "their data"?

- **Generally no:** Aggregated models don't contain individual data
- **But:** If you can extract individual data from the model, it's covered
- **Best practice:** Document that models are aggregated, not personal

When You Can't Delete: Retention Requirements

Sometimes law requires you to KEEP data:

REQUIREMENT	RETENTION PERIOD	RESOLUTION
Financial records	7 years	Anonymize name/email, keep transaction
Tax records	7 years	Keep minimum required fields
Legal holds	Until resolved	Freeze deletion for that user
Audit logs	Varies	Keep anonymized record of actions

The principle: Delete what you can. Anonymize what you must keep. Document everything.

Interview Pivot (How to Explain It)

Sold to
@theskilledc
kumar.dodda@gmail.com

If asked:

"How do you design a system where data must disappear everywhere?"

> "The core challenge is that data spreads - into caches, replicas, backups, logs, analytics, third parties. You can't delete what you can't find.
>
> I'd design for deletion from day one with three strategies:
>
> First, data lineage. Maintain a registry of every system that stores user data, keyed by user ID. When deletion happens, I know exactly where to look.
>
> Second, cryptographic erasure. For hard-to-delete data like backups and event logs, encrypt user data with a per-user key. To 'delete,' I just delete the key. The encrypted data becomes unreadable garbage.
>
> Third, a deletion pipeline. Queued, reliable, with verification. Each system has a handler that knows how to delete or anonymize its data. The pipeline tracks completion and provides an audit trail for regulators."

What's Next?

Data deletion leads to deeper questions:

How do you verify deletion actually happened? ?

> **Verification queries:** After deletion, query each system to confirm user data is gone. Log the verification. Some companies do random audits - request deletion, then search all systems to verify.

How do you handle deletion in a multi-tenant system? ?

> **Tenant isolation:** If data is properly partitioned by tenant, deletion is easier. If data is mixed, you need to filter by user_id across shared tables - more complex.

Sold to
@theskilledc
kumar.dodda@gmail.com

What about data the user shared with others?

> **Context matters:** A message sent to another user - delete the sender's copy but the recipient may retain their copy. Shared photos - complex. Document your policy.

How do you handle deletion requests at massive scale?

> **Batch processing:** Queue deletions, process in batches during off-peak hours. Prioritize by deadline. Alert if backlog threatens SLA.

The irony of deletion: In systems designed to never lose data, the hardest feature is losing data on purpose. Replication, backups, caching - everything that makes systems durable fights against deletion.

Design for deletion from day one. Track where data flows. Use crypto shredding for the hard cases. Build a pipeline with verification. And remember: the €20 million fine is for data you forgot existed.

Same Principles Apply To

- **CCPA (California)** - similar right to deletion
- **Data breach response** - knowing where data lives
- **Customer offboarding (B2B)** - deleting an entire company's data
- **Test data cleanup** - removing PII from dev/staging environments

Sold to
@theskilledc
kumar.dodda@gmail.com

How Do You Debug a Bug That Only Appears at Peak Traffic?

The Art of Finding the Needle When the Haystack is on Fire

It's Black Friday. Traffic is 10x normal. Everything worked fine in staging. Everything worked fine this morning. Now, at peak load:

- 5% of checkout requests are failing
- API latency spikes to 30 seconds
- Users are refreshing, making it worse
- Your phone is blowing up
- Nobody can reproduce the issue locally

Welcome to the special hell of **load-dependent bugs**.

These bugs don't exist in staging. They can't be reproduced at your desk. They appear only when thousands of users hit your system simultaneously. And they disappear the moment traffic drops, taking all evidence with them.

🔥 Why Load-Dependent Bugs Are Different

- **Race conditions** need many threads competing
- **Resource exhaustion** needs many connections
- **Cascading timeouts** need backed-up queues
- **Memory leaks** need hours of traffic to manifest
- **Cache coherence** needs many invalidations simultaneously

⚡ The Engineering Complexity

The bug exists at 10K requests/second. It disappears the moment you look:

Sold to @theskilledc
kumar.dodda@gmail.com

🔍 Needle in Haystack

Error happened once. In 1B requests. On 1 of 1000 servers. Find it.

📡 Distributed Tracing

Request touched 12 services. Which one added 500ms latency? Trace them all.

⏱️ Heisenbug

Bug disappears when you add logging. Timing-dependent issues are the worst.

📊 Data Volume

Capture everything = petabytes of debug data. Capture nothing = blind debugging.

The Six Categories of Load-Dependent Bugs

1. Resource Exhaustion

You have a pool of 20 database connections. At low load, requests come and go, connections are available. At peak load:

```text
Request 1: acquires connection 1... processing...
Request 2: acquires connection 2... processing...
...
Request 20: acquires connection 20... processing...
Request 21: waits for connection... waits... waits... timeout!
Request 22-500: all waiting, all timing out
```

What to look for:

- Connection pool exhaustion (database, HTTP clients)
- Thread pool starvation
- File descriptor limits
- Memory exhaustion (OOM kills)
- Disk I/O saturation

```bash
# Commands that save your life at 3 AM
$ netstat -an | grep ESTABLISHED | wc -l      # connection count
$ cat /proc/sys/fs/file-nr                    # file descriptors used
$ free -m                                     # memory usage
$ iostat -x 1                                 # disk I/O
```

Sold to @theskilledc kumar.dodda@gmail.com

2. Race Conditions

Two requests hit the same resource at the exact same millisecond.

```python
# The classic race condition
def increment_counter():
    count = redis.get("counter")    # Thread A reads: 100
                                    # Thread B reads: 100
    count += 1                      # Thread A: 101
                                    # Thread B: 101
    redis.set("counter", count)     # Thread A writes: 101
                                    # Thread B writes: 101
                                    # Result: 101 (should be 102)
```

At 10 requests/second, threads rarely overlap. At 10,000/second, they collide constantly.

What to look for:

- Counters that don't add up
- Duplicate records created
- Inventory going negative
- Read-modify-write without locking

3. Cascading Failures

One service slows down. Callers queue up waiting. Their callers queue up. Suddenly everything is frozen.

```text
Database: 200ms/query (normally 10ms)
↓
API Server: Threads blocked waiting for DB
↓
API Server: Thread pool exhausted
↓
Load Balancer: All backends unhealthy
↓
504 Gateway Timeout for everyone
```

What to look for:

- One slow dependency taking down everything
- No timeouts configured
- No circuit breakers
- Retry storms making things worse

Sold to
@theskilledc
kumar.dodda@gmail.com

4. Hot Spots

One key, one record, one partition gets all the traffic.

```text
At low traffic:
  - User A reads product 123
  - User B reads product 456
  - Even distribution

At peak (flash sale):
  - 50,000 users read product 789
  - That shard melts
  - Other shards are idle
```

What to look for:

- One database shard at 100% CPU while others idle
- One cache key causing hot partition
- One message queue partition backed up

5. Memory Leaks (Under Load)

A slow memory leak is invisible at normal traffic. At 10x traffic, it accelerates 10x.

```text
Normal: 10MB leaked/hour → noticeable in days
Peak:   100MB leaked/hour → OOM in 2 hours
```

What to look for:

- RSS memory climbing steadily
- GC pauses getting longer
- Heap dumps showing accumulating objects
- Connections or handles not closed

6. Timeout Storms

Everything has a 30-second timeout. Downstream is slow. Every request waits 30 seconds, then fails. Users retry. Now you have 3x the load, all waiting.

```text
Second 0:   1000 requests start, all waiting
Second 30:  1000 requests timeout, users retry → 2000 in flight
Second 60:  2000 requests timeout, users retry → 4000 in flight
→ Exponential collapse
```

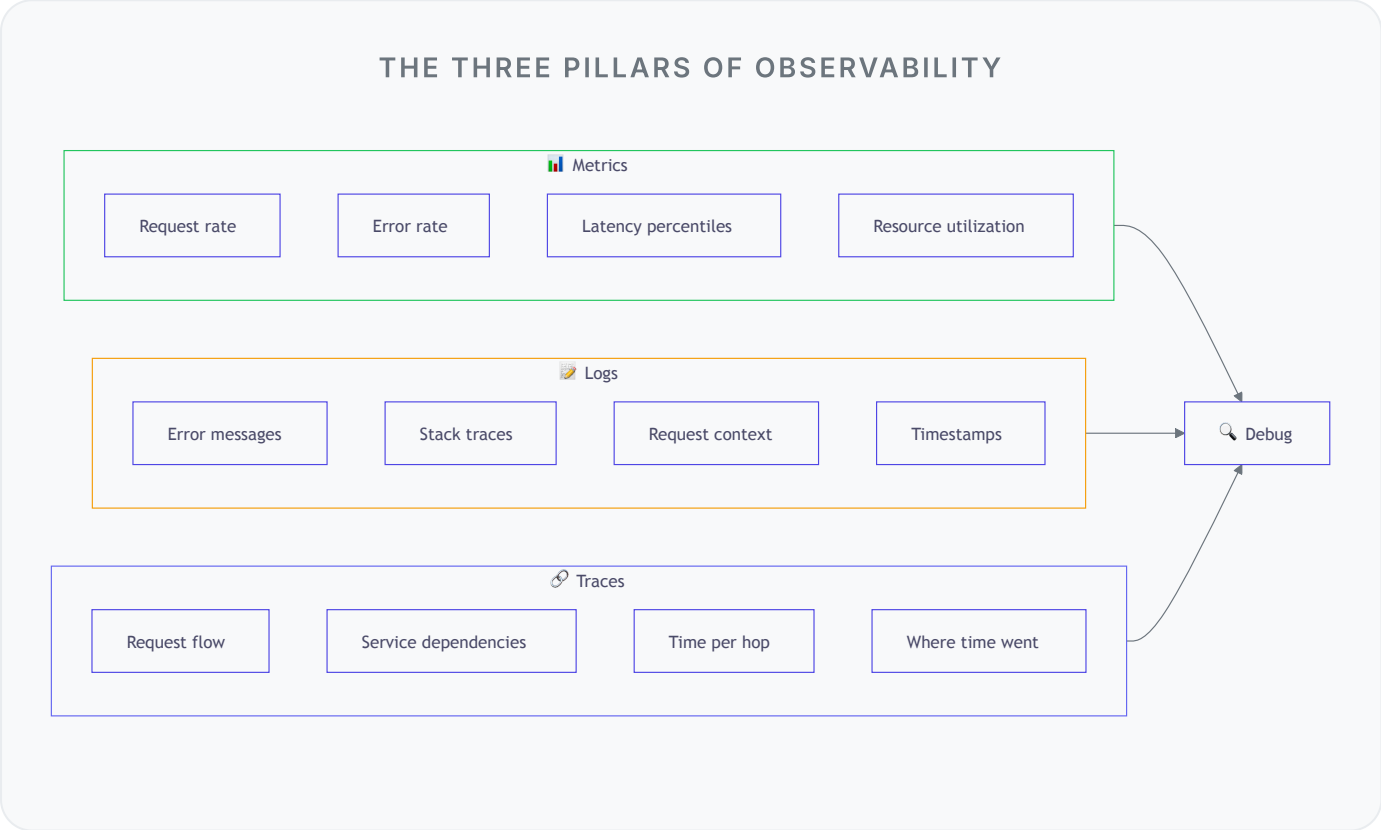
Sold to
@theskilledc
kumar.dodda@gmail.com

What to look for:

- Timeouts matching default values (30s, 60s)
- Request queue growing faster than draining
- P99 latency >> P50 latency

The Observability Toolkit

You can't debug what you can't see. At peak traffic, you need:



Metrics: What's Happening Right Now

Metrics tell you what's happening at aggregate level.

Key metrics for debugging load issues:

Sold to
@theskilledc
kumar.dodda@gmail.com

METRIC	WHAT IT TELLS YOU
`request_rate`	Is traffic actually elevated?
`error_rate`	What % is failing?
`latency_p99`	Slowest 1% of requests (more useful than avg)
`connection_pool_used`	Are we running out of connections?
`queue_depth`	Is work backing up?
`cpu_usage`	Are we compute-bound?
`memory_usage`	Memory pressure?
`gc_pause_time`	GC causing latency spikes?

Logs: What Happened

Logs give you detail on individual requests.

What good logs look like:

```json
{
  "timestamp": "2024-01-15T14:30:45.123Z",
  "level": "ERROR",
  "service": "checkout",
  "request_id": "abc-123",
  "user_id": "usr-456",
  "message": "Failed to acquire database connection",
  "error": "ConnectionPoolExhausted: waited 30000ms",
  "connection_pool_size": 20,
  "connections_in_use": 20,
  "waiting_requests": 347
}
```

Key logging practices:

- `request_id` on every log line (correlate a request's journey)
- Structured logging (JSON) for search
- Include relevant context (pool sizes, queue depths)
- Don't log at DEBUG level in production - or make it toggleable

Traces: Where Did Time Go?

A trace follows a single request across services, showing time spent at each hop.

Sold to @theskilledc kamal.dadda@gmail.com

```
Request: POST /checkout (total: 45000ms)
├── api-gateway: 2ms
├── checkout-service: 44995ms
│   ├── validate-cart: 5ms
│   ├── calculate-tax: 10ms
│   ├── charge-payment: 44975ms  ← HERE'S YOUR PROBLEM
│   │   └── (waiting for payment-service)
│   └── send-confirmation: 5ms
└── response: 3ms
```

Without tracing, you'd see "checkout is slow." With tracing, you see "checkout is slow because payment-service is slow."

> 🛠️ **Distributed Tracing Tools**
>
> **Jaeger, Zipkin, Datadog APM, AWS X-Ray, Honeycomb** - pick one and instrument your services. It's the single most valuable tool for debugging distributed systems.

Reproducing the Issue

The hardest part: the bug disappears when traffic drops.

Strategy 1: Load Testing

Simulate peak traffic before it happens.

```bash
# Using k6 to simulate peak load
$ k6 run --vus 1000 --duration 30m load-test.js

# Using Apache Bench for quick tests
$ ab -n 10000 -c 500 https://api.example.com/checkout
```

Key practices:

- Use realistic traffic patterns (not just one endpoint)
- Test against production-like data volumes
- Run in a production-like environment (not your laptop)
- Gradually ramp up load to find the breaking point

Strategy 2: Traffic Replay

Capture production traffic, replay it in staging.

Sold to
@theskilledc
kumar.dodda@gmail.com

```
Production → capture requests → replay at 10x speed → staging
```

Tools like **GoReplay**, **tcpreplay** can capture and replay traffic.

Strategy 3: Chaos Engineering

Intentionally break things to see what happens.

- Kill a database node - does the app failover?
- Add 500ms latency to a dependency - do timeouts work?
- Exhaust the connection pool - does it degrade gracefully?

```python
# Chaos middleware - randomly slow down requests
def chaos_middleware(request):
    if random.random() < 0.01:  # 1% of requests
        time.sleep(5)  # Simulate slow dependency
    return next(request)
```

The 3 AM Investigation Playbook

When you're paged at 3 AM, follow this systematic approach:

Step 1: Establish the Symptoms (2 minutes)

- What's the error rate? (5%? 50%? 100%?)
- What's the latency? (P50, P99)
- When did it start? (correlate with deployments, traffic spike)
- What's the impact? (which users, which endpoints)

Step 2: Check the Usual Suspects (5 minutes)

Sold to
@theskilledc
kumar.dodda@gmail.com

CHECK	COMMAND / DASHBOARD
Recent deployments	Git log / deployment dashboard
Traffic spike	Request rate graph
Database CPU/connections	DB dashboard
Memory / OOM kills	`dmesg | grep -i oom`
Disk full	`df -h`
Dependency health	Status pages, health checks

Step 3: Narrow Down (10 minutes)

- **Which service?** Error rate per service
- **Which endpoint?** Error rate per endpoint
- **Which users?** Affected user segment
- **Which requests?** Sample failed request traces

Step 4: Mitigate First, Debug Later

Don't spend 2 hours debugging while the site is down.

- **Roll back** if recent deploy
- **Scale up** if resource exhaustion
- **Feature flag off** if new feature
- **Shed load** (reject some requests to save others)
- **Fail over** to backup region

Get the bleeding stopped. Root cause analysis can happen after.

Prevention: Find Bugs Before Peak

1. Regular Load Testing

- Test to 2x your expected peak
- Run before every major release
- Run before every major traffic event (Black Friday, launch)

2. Dashboards That Answer Questions

Sold to @theskilledc kumar.dodda@gmail.com

Build dashboards for "is the system healthy" before you need them.

DASHBOARD	QUESTIONS IT ANSWERS
Overview	Is anything on fire right now?
Service Health	Which service is the problem?
Dependencies	Is it our code or a dependency?
Resources	CPU, memory, disk, connections

3. Alerts on Leading Indicators

- **Error rate > 1%** - something is wrong
- **P99 latency > 2s** - degradation starting
- **Connection pool > 80%** - about to run out
- **Memory > 85%** - OOM incoming
- **Queue depth growing** - falling behind

4. Graceful Degradation

Design systems that fail partially instead of completely.

- Circuit breakers on all external calls
- Timeouts on everything (and shorter than you think)
- Fallback responses when dependencies fail
- Load shedding under extreme pressure

Interview Pivot (How to Explain It)

If asked:

"How do you debug a bug that only appears at peak traffic?"

> "Load-dependent bugs typically fall into a few categories: resource exhaustion, race conditions, cascading failures, or hot spots. The approach differs.
>
> First, I need observability. Metrics to see aggregate behavior - error rates, latency percentiles, resource utilization. Traces to see where time goes in individual requests. Logs with request IDs to correlate events.

Sold to
@theskilledc
kumar.dodda@gmail.com

To reproduce, I'd use load testing tools like k6 to simulate peak traffic in a staging environment. Or replay captured production traffic. Chaos engineering can expose failure modes proactively.

In an incident, my priority is mitigate first: roll back, scale up, feature flag off. Then narrow down: which service, which endpoint, which requests. Check the usual suspects: connection pools, memory, recent deploys, dependency health.

For prevention: regular load tests to 2x peak, dashboards built before you need them, alerts on leading indicators like connection pool usage, and designing for graceful degradation with timeouts and circuit breakers."

What's Next?

Debugging at scale leads to deeper questions:

How do you handle observability costs at scale?

> **Sampling:** Trace 1% of requests, not 100%. Store metrics at lower resolution over time. Use log levels wisely. Aggregate before storing.

How do you debug race conditions specifically?

> **Stress testing + logging:** Increase concurrency until race appears. Log every state change with timestamps. Look for interleaved operations. Static analysis tools can find some races in code.

How do you ensure logs/traces are available during an outage?

> **Separate failure domains:** Observability infrastructure shouldn't go down with your app. Use managed services or separate clusters. Local fallback logging when central system is down.

Sold to
@theskilledc
kumar.dodda@gmail.com

How do you load test without affecting production?

> **Isolated environments:** Production-like staging with production-like data volumes. Shadow traffic (replay prod traffic to staging). Isolated "load test" cluster that can be stressed without affecting real users.

The paradox of load-dependent bugs: They're invisible when you have time to investigate and catastrophic when you don't. The solution isn't better debugging - it's better preparation.

Build observability before you need it. Load test regularly. Design for graceful degradation. And when the 3 AM page comes, mitigate first, debug later. The root cause can wait; your users can't.

Same Principles Apply To

- **Performance regressions** - same load testing approach
- **Memory leaks** - same observability + stress testing
- **Intermittent failures** - same tracing + correlation
- **Dependency outages** - same circuit breaker patterns

Sold to
@theskilledc
kumar.dodda@gmail.com

How Do You Design a System Where Correctness is Optional But Cost is Not?

The Art of "Good Enough" at Massive Scale

Your payment system processed exactly **1,847,293** transactions today. You need that number to be exact. Every cent accounted for. Every transaction auditable.

Your analytics dashboard says there were approximately **~47 million** pageviews today. Was it 47,000,000 or 47,123,456? *Does it matter?*

For payments: Yes. For analytics: No.

This is the fundamental insight: Not all data has the same value. Some data must be perfect. Other data just needs to be "close enough." And when you stop demanding perfection, you unlock massive cost savings.

💰 **The Cost of Perfection**

- Logging everything at a large company: **$50 million/year**
- Sampling 10% of logs: **$5 million/year**
- Exact unique visitors per second: **Impossible at scale**
- 99.9% accurate unique visitors: **12 KB of memory**

This chapter is about systems where we intentionally trade correctness for cost: analytics, logging, metrics, telemetry. Systems where losing 1% of data is fine, but spending 10x the budget is not.

The Correctness Spectrum

Sold to @theskilledc kumar.dodda@gmail.com

SYSTEM	CORRECTNESS REQUIREMENT	WHY
Payments	100% - Every cent	Legal, financial, trust
Inventory	100% - Every item	Can't oversell
Medical records	100% - Every record	Patient safety
Analytics	~95% is fine	Trends matter, not exact counts
Metrics	~99% is fine	Patterns matter, not every point
Debug logs	~90% is fine	Looking for patterns, not auditing
A/B test clicks	~99% is fine	Statistically robust anyway

When you recognize that 95% accuracy at 10% of the cost is a great trade, you unlock entirely different architectural choices.

First Attempts (and Why They're Expensive)

Attempt 1: Store Everything, Forever

```python
# Log everything!
logger.info(f"User {user_id} viewed page {page_id} at {timestamp}")
logger.info(f"User {user_id} clicked button {button_id}")
logger.info(f"User {user_id} scrolled to {scroll_position}")
logger.info(f"User {user_id} mouse moved to ({x}, {y})")  # REALLY?
```

Why it seems right:

- More data = better insights
- "We might need it later"
- Storage is cheap (right?)

Why it fails:

- **Storage isn't cheap at scale:** 1TB/day × 365 days × $0.023/GB = $8,400/year for storage alone
- **Processing costs more:** Ingestion, indexing, querying all cost money
- **95% is never used:** You're paying to store data nobody looks at

Attempt 2: Exactly-Once Everything

Sold to @theskilledc kumar.dodda@gmail.com

```python
# Analytics event with guaranteed delivery
def track_event(event):
    try:
        kafka.send(event, acks="all")  # Wait for all replicas
        await confirmation                  # Wait for ack
        db.mark_as_sent(event.id)          # Persist state
    except:
        retry_queue.add(event)             # Retry later
```

Why it fails:

- **High latency:** Every event waits for confirmation (adds 10-50ms)
- **Backpressure:** If downstream is slow, upstream backs up
- **Complexity:** Retry queues, deduplication, state management
- **Cost:** All this infrastructure for... pageview counts?

> 💡 **The Question to Ask**
>
> Would I pay 10x more to have this data be 100% accurate instead of 99%? For analytics and logging, the answer is almost always "no."

Tools for "Good Enough" Systems

Tool 1: Fire-and-Forget

Send data without waiting for confirmation. If it gets lost, so be it.

```python
# Fire-and-forget analytics
def track_event(event):
    try:
        udp_socket.send(event)  # No ack, no retry
    except:
        pass  # Silently drop on failure
```

When to use: Debug logs, mouse movements, scroll events - high volume, low value per event.

Tool 2: Sampling

Instead of logging everything, log a statistically representative subset.

Sold to
@theskilledc
kumar.dodda@gmail.com

```python
# Sample 10% of events
def track_event(event):
    if hash(event.request_id) % 100 < 10:  # Deterministic sampling
        analytics.send(event)
        event.sample_rate = 0.1  # Mark so we can multiply by 10 later
```

Why hash-based sampling?

- Deterministic - same request always sampled or not
- All events for a request stay together (trace continuity)
- Can adjust rate on the fly

SAMPLE RATE	COST SAVINGS	STATISTICAL ERROR
100%	0%	0%
10%	90%	~3% at 1000 samples
1%	99%	~10% at 100 samples

Tool 3: Approximate Algorithms

Some questions can be answered with tiny memory using probabilistic data structures.

ALGORITHM	PURPOSE	MEMORY	ACCURACY
HyperLogLog	Count unique items	12 KB	~0.81% error
Count-Min Sketch	Frequency estimation	Variable	Never underestimates
Bloom Filter	Set membership	1 bit/item	False positives, no false negatives
T-Digest	Percentiles	~10 KB	Accurate at tails

Example: Counting unique visitors

Sold to
@theskilledc
kumar.dodda@gmail.com

```python
# Exact counting: O(n) memory
unique_visitors = set()  # 1 billion users = ~50GB memory
for visitor in events:
    unique_visitors.add(visitor.id)
count = len(unique_visitors)

# HyperLogLog: O(1) memory - 12 KB for any cardinality
hll = HyperLogLog()
for visitor in events:
    hll.add(visitor.id)
count = hll.count()  # ~0.81% error, 12 KB memory
```

Tool 4: Batching and Buffering

Don't send one event per network call. Batch them.

```python
buffer = []

def track_event(event):
    buffer.append(event)
    if len(buffer) ⩾ 100 or time_since_last_flush() > 5:
        flush()

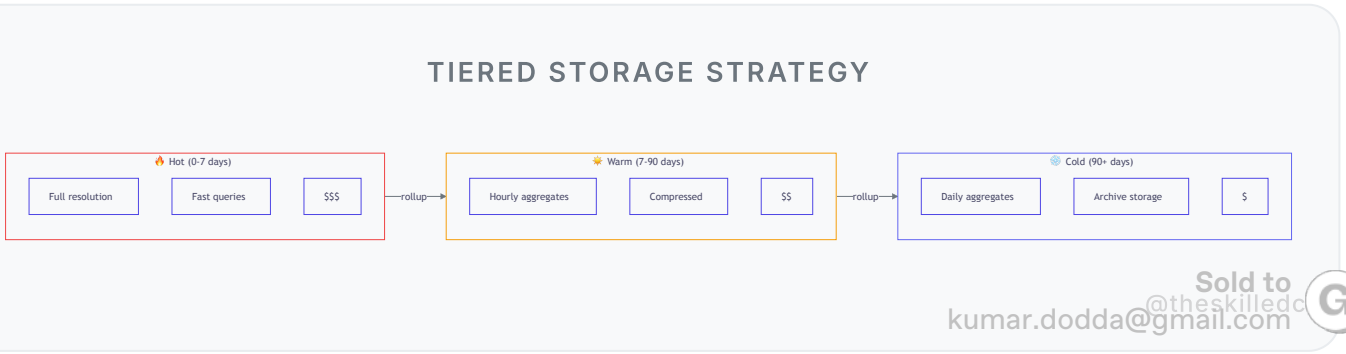
def flush():
    try:
        analytics.send_batch(buffer)  # One network call for 100 events
        buffer.clear()
    except:
        buffer.clear()  # Drop on failure - it's just analytics
```

Benefits:

- Fewer network calls (100x reduction)
- Better compression (batches compress well)
- Amortize connection overhead

Tool 5: Tiered Storage

Not all data needs the same storage tier.



Sold to
@theskilledc
kumar.dodda@gmail.com

TIER	DATA	COST
Hot	Last 7 days, full resolution	SSD, high IOPS - expensive
Warm	7-90 days, hourly aggregates	HDD, slower - cheaper
Cold	90+ days, daily aggregates	S3 Glacier - very cheap

After 7 days, roll up minute-resolution metrics to hourly. After 90 days, aggregate to daily. The questions you ask about old data don't need minute precision.

Tool 6: Drop on Backpressure

When the system is overloaded, drop data gracefully rather than crash.

```python
def track_event(event):
    if queue.full():
        metrics.increment("events.dropped")
        return  # Drop rather than block

    queue.put_nowait(event)
```

The philosophy: It's better to lose some analytics than to slow down the application that generates them. The primary service (checkout) must never be blocked by the secondary service (analytics).

Pre-Aggregation: Compute Once, Query Often

Don't store every event and query across billions of rows. Compute aggregates as data arrives.

```python
# Instead of storing every pageview:
# {user: 123, page: "/home", timestamp: "..."}
# {user: 456, page: "/home", timestamp: "..."}
# {user: 789, page: "/home", timestamp: "..."}
# ... 47 million rows

# Store pre-aggregated:
# {page: "/home", date: "2024-01-15", hour: 14, count: 15234, unique_users: 12890}

def process_pageview(event):
    key = (event.page, event.date, event.hour)
    aggregates[key]["count"] += 1
    aggregates[key]["users"].add(event.user_id)  # Or HyperLogLog
```

Storage difference:

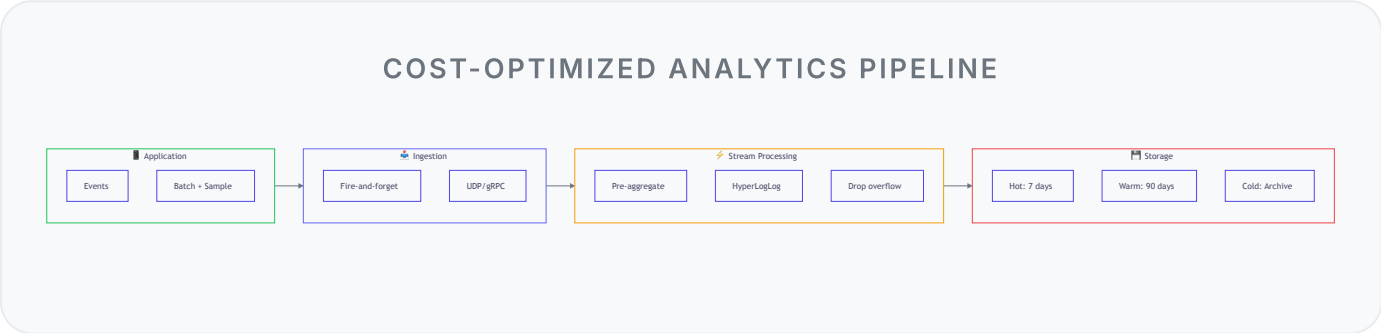
- Raw events: 47 million rows × 200 bytes = ~9 GB per day

Sold to @theskilledc kumar.dodda@gmail.com

- Hourly aggregates: 1000 pages × 24 hours × 100 bytes = ~2.4 MB per day
- **3,750x smaller**

The trade-off: You can't ask "which specific users visited /home at 2:47 PM." But if your question is "how many users visited /home today," aggregates are enough.

Architecture for Best-Effort Systems



Design Principles

1. **Batch at the edge:** Don't send one event at a time
2. **Sample before sending:** Reduce volume at source
3. **Fire-and-forget:** Don't block on analytics
4. **Aggregate in-flight:** Fewer writes, smaller storage
5. **Tier by age:** Hot data is expensive, cold is cheap
6. **Drop on overload:** Never let analytics hurt the app

Interview Pivot (How to Explain It)

If asked:

"How do you design a system where correctness is optional but cost is not?"

> "The key insight is that not all data has equal value. Payments need 100% accuracy, but analytics just needs to be 'close enough.' Once you accept that, you unlock powerful cost optimizations.
>
> First, I'd sample at the source. Log 10% of events instead of 100% - 90% cost savings, statistically valid results. Use deterministic sampling (hash-based) so all events for a request stay together.
>
> Second, use approximate algorithms. HyperLogLog for unique counts uses 12KB to count billions of items with 0.8% error. Count-Min Sketch for frequency. T-Digest for percentiles. These let you answer questions that would otherwise require storing every event.

Sold to
@theskilledc
kumar.dodda@gmail.com

Third, pre-aggregate as data arrives. Store hourly counts instead of individual events. 3000x smaller, answers most questions.

Finally, tier storage by age - full resolution for 7 days, aggregated for 90 days, archived after that. And always drop on backpressure - never let analytics slow down the main application."

What's Next?

Best-effort systems lead to deeper questions:

How do you know if your sampling rate is high enough?

> **Statistical significance:** For common events, 1% sampling is fine. For rare events (errors), you might need 100%. Solution: adaptive sampling - higher rates for rare event types.

How do you debug an issue that only appeared in dropped data?

> **Accept the trade-off:** This is the cost of sampling. Mitigate by sampling traces 100% when errors occur. And keep enough signal that patterns are visible even in sampled data.

How do you handle exact-count requirements (billing, compliance)?

> **Separate pipelines:** Run two systems - best-effort for analytics, exactly-once for billing. Tag events by category. Apply different reliability requirements to each.

How do you communicate uncertainty to stakeholders?

> **Error bars:** Show "47M ± 500K pageviews" not just "47M". Educate that sampled data is statistically valid. Compare against known ground truth periodically.

Sold to
@theskilledc
kumar.dodda@gmail.com

The liberating truth: Most of the data we obsess over storing perfectly will never be looked at. We pay to store it, index it, back it up, and replicate it - and then nobody queries it.

For analytics, logging, and metrics: sample aggressively, aggregate early, tier by age, and drop on pressure. Accept that 99% accuracy at 10% of the cost is almost always the right trade-off.

Same Principles Apply To

- **Application logging** - sample, batch, drop on overload
- **Distributed tracing** - sample traces head-based or tail-based
- **IoT sensor data** - aggregate and downsample
- **ML feature stores** - approximate features often good enough

Sold to
@theskilledc
kumar.dodda@gmail.com

How Would You Design a Stock Trading System?

Where Microseconds Are Worth Millions

At 9:30:00.000 AM, the market opens. In the next millisecond:

- 50,000 orders flood the system
- Each order must be validated, risk-checked, and matched
- Matches must execute in **price-time priority** - exact fairness
- Every trade must be recorded for regulators
- One bug, one delay, one out-of-order execution = lawsuits

This isn't your typical "eventually consistent" system. In stock trading:

> **Every microsecond matters. Every order matters. Every cent matters. There is no "retry later." There is no "close enough." If your system is 1 millisecond slower than the competition, you lose every trade.**



The Scale

- **NYSE:** 6+ billion shares traded daily
- **Peak rate:** 350,000 orders per second
- **Latency target:** < 10 microseconds for matching
- **Uptime requirement:** 99.999% during market hours
- **Audit trail:** Every order, forever, nanosecond precision

Core Concepts

The Order Book

The order book is the heart of the exchange. It's a data structure that holds all outstanding buy orders (bids) and sell orders (asks) for a stock.

Sold to @theskilledc kumar.dodda@gmail.com

```
Order Book for AAPL:

    BIDS (Buy Orders)                ASKS (Sell Orders)
    ─────────────────                ──────────────────
    $150.02 × 500 shares             $150.05 × 300 shares
    $150.01 × 1,200 shares           $150.06 × 800 shares
    $150.00 × 3,000 shares           $150.07 × 150 shares
    $149.99 × 800 shares             $150.10 × 2,000 shares

          ↑ Best Bid                       ↑ Best Ask

    Spread = $150.05 - $150.02 = $0.03
```

The **spread** is the gap between the highest bid and lowest ask. When a trade executes, one side "crosses" the spread.

Order Types

ORDER TYPE	MEANING	EXAMPLE
Market Order	Execute immediately at best available price	"Buy 100 shares now at any price"
Limit Order	Execute only at specified price or better	"Buy 100 shares only at $150 or less"
Stop Order	Becomes market order when price hits trigger	"Sell if price drops to $140"
Stop-Limit	Becomes limit order when price hits trigger	"Sell at $139 if price drops to $140"
IOC	Immediate or Cancel - fill what you can, cancel rest	For urgent partial executions
FOK	Fill or Kill - all or nothing	Must execute entire order or cancel

Price-Time Priority

When multiple orders exist at the same price, they execute in the order received. This is **FIFO** (First In, First Out) and is legally mandated for fairness.

Sold to
@theskilledc
kumar.dodda@gmail.com

```text
Two orders arrive at $150.00:
  Order A: 10:00:00.001234 - Buy 100 shares @ $150.00
  Order B: 10:00:00.001235 - Buy 200 shares @ $150.00

Seller arrives: Sell 150 shares @ $150.00

Match:
  → Order A gets filled first (100 shares) - arrived 1 microsecond earlier
  → Order B gets partial fill (50 shares)
  → Order B has 150 shares remaining in book
```

This is why microseconds matter. High-frequency traders spend millions to be physically closer to the exchange to gain microseconds.

⚡ The Engineering Complexity

A millisecond late is a missed trade. Welcome to microsecond engineering:

⚡ Microsecond Latency

In trading, 1 millisecond late = missed opportunity. Latency measured in microseconds.

📊 Order Matching

1M orders in queue. Match buyers to sellers. FIFO by price-time priority. Fast.

🔒 Consistency

Two people buy last share simultaneously. Exactly one gets it. No partial fills on this.

📈 Audit Trail

Every order, every match, every cancellation. Regulators want nanosecond-level logs.

First Attempts (and Why They Fail)

Attempt 1: Traditional Database

```sql
INSERT INTO orders (symbol, side, price, quantity, user_id, timestamp);
-- Find matching orders
SELECT * FROM orders
WHERE symbol = 'AAPL' AND side = 'sell' AND price ⩽ 150.00
ORDER BY price ASC, timestamp ASC;
```

Sold to @theskilledc kumar.dodda@gmail.com

Why it fails:

- **Too slow:** Database query = milliseconds, need microseconds
- **Locking:** Concurrent matching causes contention
- **Disk I/O:** Disk access is 1000x slower than memory
- **Network hop:** App → DB is network latency

Attempt 2: Distributed Matching

```text
Order for AAPL → Router → Matching Engine 1 or 2 or 3?
```

Why it fails:

- **Ordering problem:** How do you ensure FIFO across machines?
- **Split book:** Some bids on node 1, some on node 2 = partial view
- **Coordination overhead:** Distributed consensus is slow

> 💡 **The Core Insight**
>
> The order book for each symbol must live on **one machine** to guarantee FIFO ordering without coordination. You can partition by symbol, but each symbol's book is single-threaded, in-memory, deterministic.

The Architecture

Sold to
@theskilledc
kumar.dodda@gmail.com



Component 1: Order Gateway

The front door. Accepts orders from brokers, validates format, authenticates, rate limits, and routes to the correct matching engine.

PYTHON

```python
def receive_order(order):
    # 1. Validate format
    if not validate_order(order):
        return reject("Invalid order format")

    # 2. Authenticate broker
    if not authenticate(order.broker_id, order.signature):
        return reject("Authentication failed")

    # 3. Assign sequence number (global ordering)
    order.sequence = sequencer.next()

    # 4. Route to matching engine for this symbol
    engine = get_engine(order.symbol)
    engine.submit(order)
```

Key design points:

- Binary protocols (FIX, custom) - not JSON/HTTP
- Kernel bypass (DPDK/RDMA) for sub-microsecond networking
- Sequence number assigned here - single point of ordering

Component 2: Pre-Trade Risk Check

Sold to
@theskilledc
kumar.dodda@gmail.com

Before an order reaches the matching engine, real-time risk checks.

CHECK	PURPOSE	EXAMPLE
Position Limit	Max shares in one stock	Can't hold > 1M shares of AAPL
Credit Limit	Max $ exposure	Can't have > $50M open orders
Fat Finger	Obviously wrong orders	Buying $1B of stock? Reject.
Order Rate	Orders per second cap	Max 10,000 orders/second per firm

These checks must be **in-memory** and take microseconds. No database lookups.

Component 3: Matching Engine

The core. A single-threaded, in-memory engine per symbol partition.

Sold to
@theskilledc
kumar.dodda@gmail.com

```python
class MatchingEngine:
    def __init__(self):
        self.bids = SortedDict()  # Price → Queue of orders (highest first)
        self.asks = SortedDict()  # Price → Queue of orders (lowest first)

    def process_order(self, order):
        if order.side == "buy":
            return self.match_buy_order(order)
        else:
            return self.match_sell_order(order)

    def match_buy_order(self, order):
        trades = []

        while order.remaining > 0 and self.asks:
            best_ask_price = self.asks.peekitem(0)[0]  # Lowest ask

            # Market order matches any price
            # Limit order matches if ask ≤ limit price
            if order.type == "limit" and best_ask_price > order.price:
                break  # No match at this price

            # Execute against the order queue at this price (FIFO)
            queue = self.asks[best_ask_price]
            while queue and order.remaining > 0:
                resting_order = queue[0]
                trade = self.execute(order, resting_order, best_ask_price)
                trades.append(trade)

                if resting_order.remaining == 0:
                    queue.popleft()

            if not queue:
                del self.asks[best_ask_price]

        # If order not fully filled, add to bid book
        if order.remaining > 0 and order.type == "limit":
            self.bids.setdefault(order.price, deque()).append(order)

        return trades
```

Key design points:

- **Single-threaded:** No locks, no contention, deterministic
- **In-memory:** SortedDict for O(log n) price lookup, deque for O(1) FIFO
- **No I/O:** All data in memory, writes are async to journal
- **Partitioned by symbol:** AAPL on engine 1, GOOG on engine 2

Component 4: Market Data Feed

Every trade and order book change is published to subscribers in real-time.

Sold to @theskilledc kumar.dodda@gmail.com

```json
// Trade execution message
{
  "type": "trade",
  "symbol": "AAPL",
  "price": 150.05,
  "quantity": 100,
  "timestamp": "2024-01-15T10:30:00.123456789Z",
  "buyer_order_id": "A123",
  "seller_order_id": "B456"
}

// Order book update
{
  "type": "book_update",
  "symbol": "AAPL",
  "side": "bid",
  "price": 150.02,
  "new_quantity": 600,  // Total at this price level
  "timestamp": "..."
}
```

Multicast: Market data is often published via IP multicast - one packet reaches all subscribers simultaneously.

Latency Optimization

In trading, microseconds = money. Here's how exchanges achieve sub-10µs latency:

TECHNIQUE	LATENCY SAVED
Kernel bypass (DPDK)	Skip OS network stack - 10-50µs
RDMA	Direct memory access over network - 1-5µs
FPGA acceleration	Hardware matching - sub-1µs
Co-location	Servers in same datacenter - 0.05ms per mile saved
Single-threaded	No lock contention - 1-10µs
Pre-allocated memory	No GC pauses, no malloc - unpredictable → 0
Cache-friendly data	Sequential memory access - 10-100x faster

Reliability and Recovery

Write-Ahead Journal

Sold to
@theskilledc
kumar.dodda@gmail.com

Every order/trade is written to a persistent journal before acknowledgment.

```
Order arrives → Write to journal → Process → Ack to client

If matching engine crashes:
  1. Start new engine
  2. Replay journal from last checkpoint
  3. Engine returns to exact state before crash
```

The journal is **append-only** (fast), sequential writes to disk, and can use battery-backed RAM for durability without disk latency.

Failover

```
Primary Matching Engine ──→ Journal ←── Secondary (Hot Standby)
                            (shared)

Primary fails:
  1. Secondary takes over
  2. Replays any missed journal entries
  3. Resumes matching in < 100ms
```

Circuit Breakers

When markets move too fast, trading halts automatically.

TRIGGER	ACTION
Stock moves > 10% in 5 minutes	Halt trading for 5 minutes
S&P 500 drops > 7%	Market-wide 15-minute halt
S&P 500 drops > 20%	Close market for the day

Interview Pivot (How to Explain It)

If asked:

"How would you design a stock trading system?"

> "The core is the matching engine and order book. The order book holds all outstanding bids and asks, sorted by price. Matching happens in price-time priority - best price first, then FIFO at the same price.

Sold to
@theskilledc
kumar.dodda@gmail.com

For each symbol, the matching engine must be single-threaded and in-memory to guarantee FIFO ordering without locks. I'd partition by symbol - AAPL on engine 1, GOOG on engine 2 - so each engine handles a subset.

Before orders hit the engine, a pre-trade risk layer checks position limits, credit limits, and rejects obviously bad orders. After matching, trades are published to a market data feed using multicast.

For reliability, every order is written to a persistent journal before processing. If the engine crashes, we replay the journal to recover exact state. A hot standby engine can take over in under 100ms.

Latency is critical - exchanges target sub-10 microsecond matching. This requires kernel bypass networking, pre-allocated memory to avoid GC, and sometimes FPGA hardware acceleration."

What's Next?

Stock trading leads to deeper questions:

How do you handle market manipulation detection?

> **Surveillance systems:** Analyze order patterns for spoofing (fake orders to move price), layering, wash trading. Machine learning on order flow data. Real-time alerts for suspicious patterns.

How do you handle different asset classes?

> **Same principles, different rules:** Options have expiration. Futures have margin requirements. Crypto trades 24/7. FX has currency pairs. The matching engine is similar, but risk and settlement differ.

How do you handle clearing and settlement?

> **T+1/T+2 settlement:** Trades execute instantly, but actual transfer of shares/money happens in 1-2 days. Clearinghouses (DTCC) net trades and handle counterparty risk.

Sold to
@theskilledc
kumar.dodda@gmail.com

How do you test a trading system?

> **Deterministic replay:** Capture production order flow, replay in test environment, verify exact same trades. No randomness anywhere. Extensive conformance testing against exchange specs.

The paradox of trading systems: They appear simple - match buyers with sellers. But the requirements of microsecond latency, perfect fairness, and absolute reliability create one of the most demanding software engineering problems in existence.

Single-threaded in-memory matching. Partitioned by symbol. Write-ahead journaling for recovery. Pre-trade risk checks. Multicast market data. And relentless optimization of every microsecond - because in trading, speed is money.

Same Principles Apply To

- **Cryptocurrency exchanges** - same order book, different settlement
- **Sports betting** - same matching, odds instead of prices
- **Ad exchanges** - real-time bidding, similar auction mechanics
- **Ride matching** - different domain, similar priority queues

Sold to
@theskilledc
kumar.dodda@gmail.com

System Design Quick Reference

Four essential reference guides to bookmark, print, and keep at your desk.

🎯 10 Questions to Ask Before Designing Any System

Before drawing a single box or arrow, answer these questions. They'll save you from redesigning halfway through.

1. What's the Goal?

What exactly are you building? Who's using it? What's the main use case?

2. Read-Heavy or Write-Heavy?

TYPE	CLUES	DESIGN BIAS	EXAMPLE
Read-heavy	Lots of users viewing/searching	Caching, indexing, read replicas	Netflix catalog, Amazon product pages
Write-heavy	Tons of incoming data, uploads	Queues, batch writes, eventual consistency	WhatsApp messages, IoT sensors

3. Consistency vs Availability?

Strict consistency → Banking, bookings ($100 must be $100 everywhere)

@theskilledcoder
kumar.dodda@gmail.com

Eventual consistency → Instagram feeds, analytics (can show stale data)

4. Latency Requirements?

- **Real-time?** → Optimize for low-latency (cache, precompute) - Gaming, Zoom, payments
- **Async okay?** → Queue + background workers - Email delivery, report generation

5. Expected Scale?

How many users? Requests/sec? Storage/day? Design a simple MVP, but **think ahead: can this scale 10×?**

Use stateless services and horizontal scaling when needed.

6. Access Patterns?

PATTERN	DESIGN APPROACH	EXAMPLE
Search by geo/location	Geohash + ElasticSearch	Booking.com, Zomato
Frequently accessed IDs	Redis/Memcached cache	Twitter profiles, trending posts

7. Data Growth & Partitioning?

- Pre-plan sharding strategy (by user, time, geo) e.g. YouTube comments (shard by video ID)
- Use compression or archiving for cold data (Gmail, Google Drive)

@thesolidc
kumar.dodda@gmail.com

8. Failure Handling?

What happens if a service/db/cache goes down? Use retries, fallbacks, circuit breakers.

SCENARIO	STRATEGY	EXAMPLE
API timeout	Retry + timeout config	Amazon's payment retry on failure
Cache miss/failure	Fallback to DB	Reddit comments fallback to DB

9. Security & Auth?

Public system? Private accounts? Add rate limiting, JWTs, OAuth, IP blocklists if needed.

10. Build vs Buy?

Can you use a 3rd party for auth, payments, media, etc.? Offload non-core features to SaaS or APIs.

FEATURE	USE 3RD PARTY	EXAMPLE
Payments	Stripe, PayPal	Doordash, BookMyShow
Media Upload	Cloudinary, S3	Instagram, Amazon
Notifications	Firebase, SendGrid	Dominos, Uber

⚡ Backend Tech Concepts - One-Liner Explanations

Quick mental models for the building blocks of every system design.

API Gateway

@thesoulless_c
kumar.dodda@gmail.com

Manage, route, secure, and monitor API calls centrally in microservice architectures.

Load Balancer

Distribute traffic evenly across multiple servers for scalability and fault tolerance.

Reverse Proxy

Forward client requests to backend servers, often adding SSL, caching, or compression.

Cache (Redis/Memcached)

Speed up responses by storing frequently accessed data in memory.

Message Queue (Kafka/RabbitMQ)

Decouple services and ensure asynchronous communication.

Object Storage (S3/MinIO)

Store large, unstructured data like images, videos, or backups.

CDN (Cloudflare/Akamai)

Deliver static assets faster by caching them closer to users.

Job Scheduler (Celery/Quartz)

Run background tasks or scheduled jobs like cron.

Serverless (AWS Lambda)

Run backend code without managing servers, ideal for low-traffic or event-driven tasks.

@thesolidarc
kumar.dodda@gmail.com

Container Orchestration (Kubernetes)

Manage large-scale containerized applications needing auto-scaling and resilience.

Service Mesh (Istio/Envoy)

Handle service-to-service communication, observability, and security in microservices.

Web Server (Nginx/Apache)

Serve HTTP requests, static content, or act as a reverse proxy.

🔥 25 Concepts You'll Only Learn After a Year in Production

These lessons usually come from countless incidents. We're giving them to you for free.

1. Production readiness involves more than code correctness

It requires monitoring, alerting, security, reliability, and performance awareness.

2. Structured logging is essential for debugging

Without consistent formats and trace IDs, logs become hard to search and correlate.

3. Every network call should have a timeout

Leaving out timeouts can lead to resource exhaustion and stalled systems.

4. Rate limiting is critical, even for internal services

Without it, cascading failures or misuse can take down entire systems.

5. High latency impacts user experience as much as downtime

Systems must be optimized not just to stay up, but to respond quickly.

@thesolidc
kumar.dodda@gmail.com

6. Retries should be bounded and controlled

Blind retries can amplify load and worsen outages.

7. Observability is more than just logs

You need metrics, traces, and dashboards to diagnose issues quickly.

8. Idempotency is essential for safe retries

Reprocessing the same request should not lead to incorrect behavior or duplication.

9. Feature flags must be maintained and cleaned up

Stale or unmanaged flags can introduce bugs and reduce clarity in the codebase.

10. Backpressure mechanisms prevent overload

They protect systems under stress by rejecting or slowing input appropriately.

11. Every system has a performance bottleneck

Identifying and addressing the limiting factor is key to scaling.

12. Safe deployments require rollback mechanisms

Being able to revert quickly minimizes impact during faulty releases.

13. Real-world failures differ from test failures

Edge cases, degraded networks, and unanticipated interactions often only surface in production.

14. API versioning is necessary for evolution

Even internal consumers need contract stability as services evolve.

@thesolidc
kumar.dodda@gmail.com

15. Avoid exposing internal error messages to users

It reveals implementation details and can pose security risks.

16. Databases often become the scalability bottleneck

You must plan for indexing, replication, partitioning, and archiving early.

17. Cumulative latency across services adds up

Optimize each layer, even if individually it seems fast.

18. High availability requires planning for failure modes

Redundancy, failover, and disaster recovery must be part of the system design.

19. Scheduled tasks must be monitored

Jobs that silently fail or stop running can cause hidden data issues.

20. Rolling back code doesn't undo external effects

Changes to data, emails sent, or messages queued require compensating actions.

21. Caching systems and CDNs can become points of failure

Cache invalidation, propagation delays, or outages must be handled gracefully.

22. Test coverage must include real-world data scenarios

Tests on synthetic data often miss edge cases found in production.

23. Configuration changes can cause as many issues as code changes

Config should be version-controlled, validated, and reviewed.

@thesolidc
kumar.dodda@gmail.com

24. Scalability issues often appear only under sustained load

Load testing should simulate realistic traffic patterns and durations.

25. System behavior can change unexpectedly due to external dependencies

Third-party APIs, infrastructure updates, or expired certificates can introduce regressions.

📊 System Design Patterns - Quick Reference

A cheat sheet of common patterns, when to use them, and real-world examples.

@thesolidtech
kumar.dodda@gmail.com

PATTERN	CORE IDEA	WHEN TO USE	CLASSIC PROBLEMS
Caching	Keep hot data close to the app	Read-heavy workloads, expensive queries	Product pages, session stores
CDN	Push static assets to edge	Global users, large media	Video streaming, downloads
Load Balancing	Spread traffic across instances	Scale stateless services, HA	Web/API tier scaling
Rate Limiting	Control request volume per key/client	Protect downstream services	Public APIs, login abuse protection
Circuit Breaker	Fail fast when dependency is unhealthy	Prevent cascades, degrade gracefully	Payment gateway outage
Retry + Idempotency	Safe replays of failed ops	Unreliable networks, async workflows	Order creation, webhook delivery
Read Replicas	Offload reads from primary DB	Read-heavy, reporting, geo-reads	Analytics pages, timelines
Sharding	Split data across nodes	Data > single node, parallelism	Multi-TB user tables, geo data
CQRS	Separate read/write models	Complex reads + high write throughput	Event feeds, denormalized dashboards
Event Sourcing	State = log of events	Full audit, rebuild state, temporal queries	Ledger systems, order timelines
Message Queue	Async decoupling via durable logs	Spikes, fan-out, ordered pipelines	Email/SMS, ETL, clickstream
Saga Pattern	Distributed transaction via steps + compensation	Cross-service workflows without 2PC	Book-pay-reserve flows, refunds
Search Index	Inverted index for fast text/filters	Full-text, aggregations, relevance	Product search, logs explorer
Blue-Green / Canary	Shift traffic gradually	Safer releases, quick rollback	API rollout, config changes
Feature Flags	Runtime on/off & % rollouts	Experimentation, kill-switches	A/B tests, dark launches

@thesolidtech
kumar.dodda@gmail.com

Pro Tip

Bookmark this section. These patterns will come up in every system design interview you ever do.

@thesolidtech
kumar.dodda@gmail.com

Thank You for Reading! 🙏

We hope this book helps you ace your system design interviews and build better systems in production.

📬 Stay Connected

Newsletter

Get weekly system design insights:

newsletter.theskilledcoder.com

(https://newsletter.theskilledcoder.com)

Need Help?

Questions, feedback, or just want to say hi:

mdb.businesshelp@gmail.com

Built with ❤️ by @theskilledcoder

Sold to
kumar.dodda@gmail.com